An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Shaojie Bai [1] **J. Zico Kolter** [2] **Vladlen Koltun** [3]

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at http://github.com/locuslab/TCN.

1. Introduction

Deep learning practitioners commonly regard recurrent architectures as the default starting point for sequence modeling tasks. The sequence modeling chapter in the canonical textbook on deep learning is titled "Sequence Modeling: Recurrent and Recursive Nets" (Goodfellow et al., 2016), capturing the common association of sequence modeling and recurrent architectures. A well-regarded recent online course on "Sequence Models" focuses exclusively on recurrent architectures (Ng, 2018).

[1]Machine Learning Department, Carnegie Mellon University, Pittsburgh, PA, USA [2]Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, USA [3]Intel Labs, Santa Clara, CA, USA. Correspondence to: Shaojie Bai <shaojieb@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>, Vladlen Koltun <vkoltun@gmail.edu>.

On the other hand, recent research indicates that certain convolutional architectures can reach state-of-the-art accuracy in audio synthesis, word-level language modeling, and machine translation (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b). This raises the question of whether these successes of convolutional sequence modeling are confined to specific application domains or whether a broader reconsideration of the association between sequence processing and recurrent networks is in order.

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks' "home turf".

To represent convolutional networks, we describe a generic temporal convolutional network (TCN) architecture that is applied across all tasks. This architecture is informed by recent research, but is deliberately kept simple, combining some of the best practices of modern convolutional architectures. It is compared to canonical recurrent architectures such as LSTMs and GRUs.

The results suggest that TCNs convincingly outperform baseline recurrent architectures across a broad range of sequence modeling tasks. This is particularly notable because the tasks include diverse benchmarks that have commonly been used to evaluate recurrent network designs (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Zhang et al., 2016). This indicates that the recent successes of convolutional architectures in applications such as audio processing are not confined to these domains.

To further understand these results, we analyze more deeply the memory retention characteristics of recurrent networks. We show that despite the theoretical ability of recurrent architectures to capture infinitely long history, TCNs exhibit

substantially longer memory, and are thus more suitable for domains where a long history is required.

To our knowledge, the presented study is the most extensive systematic comparison of convolutional and recurrent architectures on sequence modeling tasks. The results suggest that the common association between sequence modeling and recurrent networks should be reconsidered. The TCN architecture appears not only more accurate than canonical recurrent networks such as LSTMs and GRUs, but also simpler and clearer. It may therefore be a more appropriate starting point in the application of deep networks to sequences.

2. Background

Convolutional networks (LeCun et al., 1989) have been applied to sequences for decades (Sejnowski & Rosenberg, 1987; Hinton, 1989). They were used prominently for speech recognition in the 80s and 90s (Waibel et al., 1989; Bottou et al., 1990). ConvNets were subsequently applied to NLP tasks such as part-of-speech tagging and semantic role labelling (Collobert & Weston, 2008; Collobert et al., 2011; dos Santos & Zadrozny, 2014). More recently, convolutional networks were applied to sentence classification (Kalchbrenner et al., 2014; Kim, 2014) and document classification (Zhang et al., 2015; Conneau et al., 2017; Johnson & Zhang, 2015; 2017). Particularly inspiring for our work are the recent applications of convolutional architectures to machine translation (Kalchbrenner et al., 2016; Gehring et al., 2017a;b), audio synthesis (van den Oord et al., 2016), and language modeling (Dauphin et al., 2017).

Recurrent networks are dedicated sequence models that maintain a vector of hidden activations that are propagated through time (Elman, 1990; Werbos, 1990; Graves, 2012). This family of architectures has gained tremendous popularity due to prominent applications to language modeling (Sutskever et al., 2011; Graves, 2013; Hermans & Schrauwen, 2013) and machine translation (Sutskever et al., 2014; Bahdanau et al., 2015). The intuitive appeal of recurrent modeling is that the hidden state can act as a representation of everything that has been seen so far in the sequence. Basic RNN architectures are notoriously difficult to train (Bengio et al., 1994; Pascanu et al., 2013) and more elaborate architectures are commonly used instead, such as the LSTM (Hochreiter & Schmidhuber, 1997) and the GRU (Cho et al., 2014). Many other architectural innovations and training techniques for recurrent networks have been introduced and continue to be actively explored (El Hihi & Bengio, 1995; Schuster & Paliwal, 1997; Gers et al., 2002; Koutnik et al., 2014; Le et al., 2015; Ba et al., 2016; Wu et al., 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018).

Multiple empirical studies have been conducted to evaluate the effectiveness of different recurrent architectures. These studies have been motivated in part by the many degrees of freedom in the design of such architectures. Chung et al. (2014) compared different types of recurrent units (LSTM vs. GRU) on the task of polyphonic music modeling. Pascanu et al. (2014) explored different ways to construct deep RNNs and evaluated the performance of different architectures on polyphonic music modeling, character-level language modeling, and word-level language modeling. Jozefowicz et al. (2015) searched through more than ten thousand different RNN architectures and evaluated their performance on various tasks. They concluded that if there were "architectures much better than the LSTM", then they were "not trivial to find". Greff et al. (2017) benchmarked the performance of eight LSTM variants on speech recognition, handwriting recognition, and polyphonic music modeling. They also found that "none of the variants can improve upon the standard LSTM architecture significantly". Zhang et al. (2016) systematically analyzed the connecting architectures of RNNs and evaluated different architectures on character-level language modeling and on synthetic stress tests. Melis et al. (2018) benchmarked LSTM-based architectures on word-level and character-level language modeling, and concluded that "LSTMs outperform the more recent models".

Other recent works have aimed to combine aspects of RNN and CNN architectures. This includes the Convolutional LSTM (Shi et al., 2015), which replaces the fully-connected layers in an LSTM with convolutional layers to allow for additional structure in the recurrent layers; the Quasi-RNN model (Bradbury et al., 2017) that interleaves convolutional layers with simple recurrent layers; and the dilated RNN (Chang et al., 2017), which adds dilations to recurrent architectures. While these combinations show promise in combining the desirable aspects of both types of architectures, our study here focuses on a comparison of generic convolutional and recurrent architectures.

While there have been multiple thorough evaluations of RNN architectures on representative sequence modeling tasks, we are not aware of a similarly thorough comparison of convolutional and recurrent approaches to sequence modeling. (Yin et al. (2017) have reported a comparison of convolutional and recurrent networks for sentence-level and document-level classification tasks. In contrast, sequence modeling calls for architectures that can synthesize whole sequences, element by element.) Such comparison is particularly intriguing in light of the aforementioned recent success of convolutional architectures in this domain. Our work aims to compare generic convolutional and recurrent architectures on typical sequence modeling tasks that are commonly used to benchmark RNN variants themselves (Hermans & Schrauwen, 2013; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016).

3. Temporal Convolutional Networks

We begin by describing a generic architecture for convolutional sequence prediction. Our aim is to distill the best practices in convolutional network design into a simple architecture that can serve as a convenient but powerful starting point. We refer to the presented architecture as a temporal convolutional network (TCN), emphasizing that we adopt this term not as a label for a truly new architecture, but as a simple descriptive term for a family of architectures. (Note that the term has been used before (Lea et al., 2017).) The distinguishing characteristics of TCNs are: 1) the convolutions in the architecture are causal, meaning that there is no information "leakage" from future to past; 2) the architecture can take a sequence of any length and map it to an output sequence of the same length, just as with an RNN. Beyond this, we emphasize how to build very long effective history sizes (i.e., the ability for the networks to look very far into the past to make a prediction) using a combination of very deep networks (augmented with residual layers) and dilated convolutions.

Our architecture is informed by recent convolutional architectures for sequential data (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b), but is distinct from all of them and was designed from first principles to combine simplicity, autoregressive prediction, and very long memory. For example, the TCN is much simpler than WaveNet (van den Oord et al., 2016) (no skip connections across layers, conditioning, context stacking, or gated activations).

Compared to the language modeling architecture of Dauphin et al. (2017), TCNs do not use gating mechanisms and have much longer memory.

3.1. Sequence Modeling

Before defining the network structure, we highlight the nature of the sequence modeling task. Suppose that we are given an input sequence $x_0, \ldots, x_T$, and wish to predict some corresponding outputs $y_0, \ldots, y_T$ at each time. The key constraint is that to predict the output y_t for some time t, we are constrained to only use those inputs that have been previously observed: $x_0, \ldots, x_t$. Formally, a sequence modeling network is any function $f : \mathcal{X}^{T+1} \to \mathcal{Y}^{T+1}$ that produces the mapping

$$\hat{y}_0, \ldots, \hat{y}_T = f(x_0, \ldots, x_T) \tag{1}$$

if it satisfies the causal constraint that y_t depends only on $x_0, \ldots, x_t$ and not on any "future" inputs $x_{t+1}, \ldots, x_T$. The goal of learning in the sequence modeling setting is to find a network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0, \ldots, y_T, f(x_0, \ldots, x_T))$, where the sequences and outputs are drawn according to some distribution.

This formalism encompasses many settings such as autoregressive prediction (where we try to predict some signal given its past) by setting the target output to be simply the input shifted by one time step. It does not, however, directly capture domains such as machine translation, or sequence-to-sequence prediction in general, since in these cases the entire input sequence (including "future" states) can be used to predict each output (though the techniques can naturally be extended to work in such settings).

3.2. Causal Convolutions

As mentioned above, the TCN is based upon two principles: the fact that the network produces an output of the same length as the input, and the fact that there can be no leakage from the future into the past. To accomplish the first point, the TCN uses a 1D fully-convolutional network (FCN) architecture (Long et al., 2015), where each hidden layer is the same length as the input layer, and zero padding of length $(\text{kernel size} - 1)$ is added to keep subsequent layers the same length as previous ones. To achieve the second point, the TCN uses *causal convolutions*, convolutions where an output at time t is convolved only with elements from time t and earlier in the previous layer.

To put it simply: TCN = 1D FCN + causal convolutions.

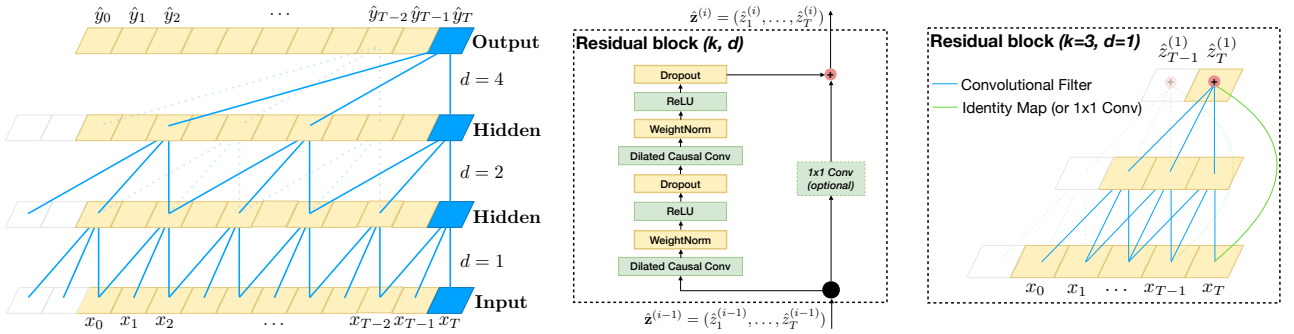
Note that this is essentially the same architecture as the time delay neural network proposed nearly 30 years ago by Waibel et al. (1989), with the sole tweak of zero padding to ensure equal sizes of all layers.

A major disadvantage of this basic design is that in order to achieve a long effective history size, we need an extremely deep network or very large filters, neither of which were particularly feasible when the methods were first introduced. Thus, in the following sections, we describe how techniques from modern convolutional architectures can be integrated into a TCN to allow for both very deep networks and very long effective history.

3.3. Dilated Convolutions

A simple causal convolution is only able to look back at a history with size linear in the depth of the network. This makes it challenging to apply the aforementioned causal convolution on sequence tasks, especially those requiring longer history. Our solution here, following the work of van den Oord et al. (2016), is to employ dilated convolutions that enable an exponentially large receptive field (Yu & Koltun, 2016). More formally, for a 1-D sequence input $\mathbf{x} \in \mathbb{R}^n$ and a filter $f : \{0, \ldots, k-1\} \to \mathbb{R}$, the dilated convolution operation F on element s of the sequence is defined as

$$F(s) = (\mathbf{x} *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i} \tag{2}$$



(a) (b) (c)

Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1x1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

where d is the dilation factor, k is the filter size, and $s - d \cdot i$ accounts for the direction of the past. Dilation is thus equivalent to introducing a fixed step between every two adjacent filter taps. When $d = 1$, a dilated convolution reduces to a regular convolution. Using larger dilation enables an output at the top level to represent a wider range of inputs, thus effectively expanding the receptive field of a ConvNet.

This gives us two ways to increase the receptive field of the TCN: choosing larger filter sizes k and increasing the dilation factor d, where the effective history of one such layer is $(k-1)d$. As is common when using dilated convolutions, we increase d exponentially with the depth of the network (i.e., $d = O(2^i)$ at level i of the network). This ensures that there is some filter that hits each input within the effective history, while also allowing for an extremely large effective history using deep networks. We provide an illustration in Figure 1(a).

3.4. Residual Connections

A residual block (He et al., 2016) contains a branch leading out to a series of transformations $\mathcal{F}$, whose outputs are added to the input $\mathbf{x}$ of the block:

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x})) \tag{3}$$

This effectively allows layers to learn modifications to the identity mapping rather than the entire transformation, which has repeatedly been shown to benefit very deep networks.

Since a TCN's receptive field depends on the network depth n as well as filter size k and dilation factor d, stabilization of deeper and larger TCNs becomes important. For example, in a case where the prediction could depend on a history of size 2^{12} and a high-dimensional input sequence, a network of up to 12 layers could be needed. Each layer, more specifically, consists of multiple filters for feature extraction. In our design of the generic TCN model, we therefore employ a generic residual module in place of a convolutional layer.

The residual block for our baseline TCN is shown in Figure 1(b). Within a residual block, the TCN has two layers of dilated causal convolution and non-linearity, for which we used the rectified linear unit (ReLU) (Nair & Hinton, 2010). For normalization, we applied weight normalization (Salimans & Kingma, 2016) to the convolutional filters. In addition, a spatial dropout (Srivastava et al., 2014) was added after each dilated convolution for regularization: at each training step, a whole channel is zeroed out.

However, whereas in standard ResNet the input is added directly to the output of the residual function, in TCN (and ConvNets in general) the input and output could have different widths. To account for discrepant input-output widths, we use an additional 1x1 convolution to ensure that element-wise addition $\oplus$ receives tensors of the same shape (see Figure 1(b,c)).

3.5. Discussion

We conclude this section by listing several advantages and disadvantages of using TCNs for sequence modeling.

- **Parallelism**. Unlike in RNNs where the predictions for later timesteps must wait for their predecessors to complete, convolutions can be done in parallel since the same filter is used in each layer. Therefore, in both training and evaluation, a long input sequence can be processed as a whole in TCN, instead of sequentially as in RNN.
- **Flexible receptive field size**. A TCN can change its receptive field size in multiple ways. For instance, stacking more dilated (causal) convolutional layers, using larger dilation factors, or increasing the filter size are all viable options (with possibly different interpretations). TCNs thus afford better control of the model's memory size, and are easy to adapt to different domains.
- **Stable gradients**. Unlike recurrent architectures, TCN has a backpropagation path different from the temporal direction of the sequence. TCN thus avoids the problem

of exploding/vanishing gradients, which is a major issue for RNNs (and which led to the development of LSTM, GRU, HF-RNN (Martens & Sutskever, 2011), etc.).

- **Low memory requirement for training**. Especially in the case of a long input sequence, LSTMs and GRUs can easily use up a lot of memory to store the partial results for their multiple cell gates. However, in a TCN the filters are shared across a layer, with the backpropagation path depending only on network depth. Therefore in practice, we found gated RNNs likely to use up to a multiplicative factor more memory than TCNs.
- **Variable length inputs**. Just like RNNs, which model inputs with variable lengths in a recurrent way, TCNs can also take in inputs of arbitrary lengths by sliding the 1D convolutional kernels. This means that TCNs can be adopted as drop-in replacements for RNNs for sequential data of arbitrary length.

There are also two notable disadvantages to using TCNs.

- **Data storage during evaluation**. In evaluation/testing, RNNs only need to maintain a hidden state and take in a current input x_t in order to generate a prediction. In other words, a "summary" of the entire history is provided by the fixed-length set of vectors h_t, and the actual observed sequence can be discarded. In contrast, TCNs need to take in the raw sequence up to the effective history length, thus possibly requiring more memory during evaluation.
- **Potential parameter change for a transfer of domain**. Different domains can have different requirements on the amount of history the model needs in order to predict. Therefore, when transferring a model from a domain where only little memory is needed (i.e., small k and d) to a domain where much longer memory is required (i.e., much larger k and d), TCN may perform poorly for not having a sufficiently large receptive field.

4. Sequence Modeling Tasks

We evaluate TCNs and RNNs on tasks that have been commonly used to benchmark the performance of different RNN sequence modeling architectures (Hermans & Schrauwen, 2013; Chung et al., 2014; Pascanu et al., 2014; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016). The intention is to conduct the evaluation on the "home turf" of RNN sequence models. We use a comprehensive set of synthetic stress tests along with real-world datasets from multiple domains.

The adding problem. In this task, each input consists of a length-n sequence of depth 2, with all values randomly chosen in $[0, 1]$, and the second dimension being all zeros except for two elements that are marked by 1. The objective is to sum the two random values whose second dimensions are marked by 1. Simply predicting the sum to be 1 should give an MSE of about 0.1767. First introduced by Hochreiter & Schmidhuber (1997), the adding problem has been used repeatedly as a stress test for sequence models (Martens & Sutskever, 2011; Pascanu et al., 2013; Le et al., 2015; Arjovsky et al., 2016; Zhang et al., 2016).

Sequential MNIST and P-MNIST. Sequential MNIST is frequently used to test a recurrent network's ability to retain information from the distant past (Le et al., 2015; Zhang et al., 2016; Wisdom et al., 2016; Cooijmans et al., 2016; Krueger et al., 2017; Jing et al., 2017). In this task, MNIST images (LeCun et al., 1998) are presented to the model as a 784×1 sequence for digit classification. In the more challenging P-MNIST setting, the order of the sequence is permuted at random (Le et al., 2015; Arjovsky et al., 2016; Wisdom et al., 2016; Krueger et al., 2017).

Copy memory. In this task, each input sequence has length $T + 20$. The first 10 values are chosen randomly among the digits $1, \ldots, 8$, with the rest being all zeros, except for the last 11 entries that are filled with the digit '9' (the first '9' is a delimiter). The goal is to generate an output of the same length that is zero everywhere except the last 10 values after the delimiter, where the model is expected to repeat the 10 values it encountered at the start of the input. This task was used in prior works such as Zhang et al. (2016); Arjovsky et al. (2016); Wisdom et al. (2016); Jing et al. (2017).

JSB Chorales and Nottingham. JSB Chorales (Allan & Williams, 2005) is a polyphonic music dataset consisting of the entire corpus of 382 four-part harmonized chorales by J. S. Bach. Each input is a sequence of elements. Each element is an 88-bit binary code that corresponds to the 88 keys on a piano, with 1 indicating a key that is pressed at a given time. Nottingham is a polyphonic music dataset based on a collection of 1,200 British and American folk tunes, and is much larger than JSB Chorales. JSB Chorales and Nottingham have been used in numerous empirical investigations of recurrent sequence modeling (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Greff et al., 2017). The performance on both tasks is measured in terms of negative log-likelihood (NLL).

PennTreebank. We used the PennTreebank (PTB) (Marcus et al., 1993) for both character-level and word-level language modeling. When used as a character-level language corpus, PTB contains 5,059K characters for training, 396K for validation, and 446K for testing, with an alphabet size of 50. When used as a word-level language corpus, PTB contains 888K words for training, 70K for validation, and 79K for testing, with a vocabulary size of 10K. This is a highly studied but relatively small language modeling dataset (Miyamoto & Cho, 2016; Krueger et al., 2017; Merity et al., 2017).

Wikitext-103. Wikitext-103 (Merity et al., 2016) is almost

Table 1. Evaluation of TCNs and recurrent architectures on synthetic stress tests, polyphonic music modeling, character-level language modeling, and word-level language modeling. The generic TCN architecture outperforms canonical recurrent networks across a comprehensive suite of tasks and datasets. Current state-of-the-art results are listed in the supplement. ^{h} means that higher is better. $^\ell$ means that lower is better.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	**TCN**
Seq. MNIST (accuracy^h)	70K	87.2	96.2	21.5	**99.0**
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	**97.2**
Adding problem T=600 (loss$^\ell$)	70K	0.164	**5.3e-5**	0.177	**5.8e-5**
Copy memory T=1000 (loss)	16K	0.0204	0.0197	0.0202	**3.5e-5**
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	**8.10**
Music Nottingham (loss)	1M	3.29	3.46	4.05	**3.07**
Word-level PTB (perplexity$^\ell$)	13M	**78.93**	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	**45.19**
Word-level LAMBADA (perplexity)	-	4186	-	14725	**1279**
Char-level PTB (bpc$^\ell$)	3M	1.36	1.37	1.48	**1.31**
Char-level text8 (bpc)	5M	1.50	1.53	1.69	**1.45**

110 times as large as PTB, featuring a vocabulary size of about 268K. The dataset contains 28K Wikipedia articles (about 103 million words) for training, 60 articles (about 218K words) for validation, and 60 articles (246K words) for testing. This is a more representative and realistic dataset than PTB, with a much larger vocabulary that includes many rare words, and has been used in Merity et al. (2016); Grave et al. (2017); Dauphin et al. (2017).

LAMBADA. Introduced by Paperno et al. (2016), LAMBADA is a dataset comprising 10K passages extracted from novels, with an average of 4.6 sentences as context, and 1 target sentence the last word of which is to be predicted. This dataset was built so that a person can easily guess the missing word when given the context sentences, but not when given only the target sentence without the context sentences. Most of the existing models fail on LAMBADA (Paperno et al., 2016; Grave et al., 2017). In general, better results on LAMBADA indicate that a model is better at capturing information from longer and broader context. The training data for LAMBADA is the full text of 2,662 novels with more than 200M words. The vocabulary size is about 93K.

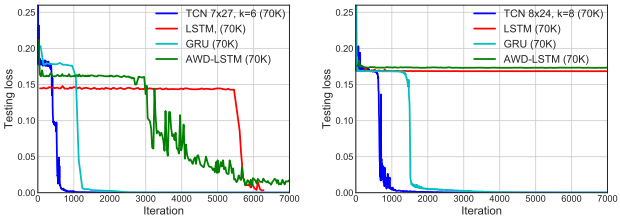
text8. We also used the text8 dataset for character-level language modeling (Mikolov et al., 2012). text8 is about 20 times larger than PTB, with about 100M characters from Wikipedia (90M for training, 5M for validation, and 5M for testing). The corpus contains 27 unique alphabets.

5. Experiments

We compare the generic TCN architecture described in Section 3 to canonical recurrent architectures, namely LSTM, GRU, and vanilla RNN, with standard regularizations. All experiments reported in this section used exactly the same TCN architecture, just varying the depth of the network n and occasionally the kernel size k so that the receptive field covers enough context for predictions. We use an exponential dilation $d = 2^i$ for layer i in the network, and the Adam optimizer (Kingma & Ba, 2015) with learning rate 0.002 for TCN, unless otherwise noted. We also empirically find that gradient clipping helped convergence, and we pick the maximum norm for clipping from $[0.3, 1]$. When training recurrent models, we use grid search to find a good set of hyperparameters (in particular, optimizer, recurrent drop $p \in [0.05, 0.5]$, learning rate, gradient clipping, and initial forget-gate bias), while keeping the network around the same size as TCN. No other architectural elaborations, such as gating mechanisms or skip connections, were added to either TCNs or RNNs. Additional details and controlled experiments are provided in the supplementary material.

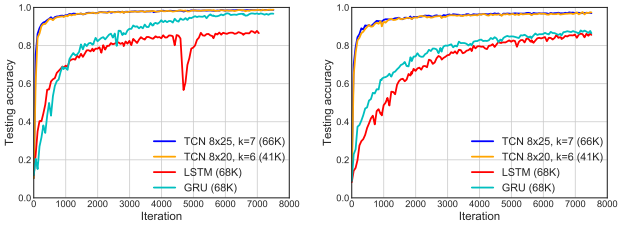
5.1. Synopsis of Results

A synopsis of the results is shown in Table 1. Note that on several of these tasks, the generic, canonical recurrent architectures we study (e.g., LSTM, GRU) are not the state-of-the-art. (See the supplement for more details.) With this caveat, the results strongly suggest that the generic TCN architecture *with minimal tuning* outperforms canonical recurrent architectures across a broad variety of sequence modeling tasks that are commonly used to benchmark the performance of recurrent architectures themselves. We now analyze these results in more detail.



(a) $T = 200$ (b) $T = 600$

Figure 2. Results on the adding problem for different sequence lengths T. TCNs outperform recurrent architectures.



(a) Sequential MNIST (b) P-MNIST

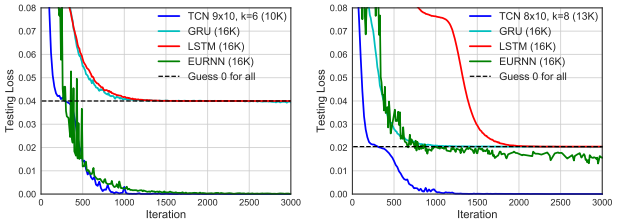
Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

5.2. Synthetic Stress Tests

The adding problem. Convergence results for the adding problem, for problem sizes $T = 200$ and 600, are shown in Figure 2. All models were chosen to have roughly 70K parameters. TCNs quickly converged to a virtually perfect solution (i.e., MSE near 0). GRUs also performed quite well, albeit slower to converge than TCNs. LSTMs and vanilla RNNs performed significantly worse.

Sequential MNIST and P-MNIST. Convergence results on sequential and permuted MNIST, run over 10 epochs, are shown in Figure 3. All models were configured to have roughly 70K parameters. For both problems, TCNs substantially outperform the recurrent architectures, both in terms of convergence and in final accuracy on the task. For P-MNIST, TCNs outperform state-of-the-art results (95.9%) based on recurrent networks with Zoneout and Recurrent BatchNorm (Cooijmans et al., 2016; Krueger et al., 2017).

Copy memory. Convergence results on the copy memory task are shown in Figure 4. TCNs quickly converge to correct answers, while LSTMs and GRUs simply converge to the same loss as predicting all zeros. In this case we also compare to the recently-proposed EURNN (Jing et al., 2017), which was highlighted to perform well on this task. While both TCN and EURNN perform well for sequence length $T = 500$, the TCN has a clear advantage for $T = 1000$ and longer (in terms of both loss and rate of convergence).



(a) $T = 500$ (b) $T = 1000$

Figure 4. Result on the copy memory task for different sequence lengths T. TCNs outperform recurrent architectures.

5.3. Polyphonic Music and Language Modeling

We now discuss the results on polyphonic music modeling, character-level language modeling, and word-level language modeling. These domains are dominated by recurrent architectures, with many specialized designs developed for these tasks (Zhang et al., 2016; Ha et al., 2017; Krueger et al., 2017; Grave et al., 2017; Greff et al., 2017; Merity et al., 2017). We mention some of these specialized architectures when useful, but our primary goal is to compare the generic TCN model to similarly generic recurrent architectures, before domain-specific tuning. The results are summarized in Table 1.

Polyphonic music. On Nottingham and JSB Chorales, the TCN with virtually no tuning outperforms the recurrent models by a considerable margin, and even outperforms some enhanced recurrent architectures for this task such as HF-RNN (Boulanger-Lewandowski et al., 2012) and Diagonal RNN (Subakan & Smaragdis, 2017). Note however that other models such as the Deep Belief Net LSTM perform better still (Vohra et al., 2015); we believe this is likely due to the fact that the datasets are relatively small, and thus the right regularization method or generative modeling procedure can improve performance significantly. This is largely orthogonal to the RNN/TCN distinction, as a similar variant of TCN may well be possible.

Word-level language modeling. Language modeling remains one of the primary applications of recurrent networks and many recent works have focused on optimizing LSTMs for this task (Krueger et al., 2017; Merity et al., 2017). Our implementation follows standard practice that ties the weights of encoder and decoder layers for both TCN and RNNs (Press & Wolf, 2016), which significantly reduces the number of parameters in the model. For training, we use SGD and anneal the learning rate by a factor of 0.5 for both TCN and RNNs when validation accuracy plateaus.

On the smaller PTB corpus, an optimized LSTM architecture (with recurrent and embedding dropout, etc.) outperforms the TCN, while the TCN outperforms both GRU and vanilla RNN. However, on the much larger Wikitext-103 corpus and the LAMBADA dataset (Paperno et al., 2016), without any hyperparameter search, the TCN outperforms

the LSTM results of Grave et al. (2017), achieving much lower perplexities.

Character-level language modeling. On character-level language modeling (PTB and text8, accuracy measured in bits per character), the generic TCN outperforms regularized LSTMs and GRUs as well as methods such as Norm-stabilized LSTMs (Krueger & Memisevic, 2015). (Specialized architectures exist that outperform all of these, see the supplement.)

5.4. Memory Size of TCN and RNNs

One of the theoretical advantages of recurrent architectures is their unlimited memory: the theoretical ability to retain information through sequences of unlimited length. We now examine specifically how long the different architectures can retain information in practice. We focus on 1) the copy memory task, which is a stress test designed to evaluate long-term, distant information propagation in recurrent networks, and 2) the LAMBADA task, which tests both local and non-local textual understanding.

The copy memory task is perfectly set up to examine a model's ability to retain information for different lengths of time. The requisite retention time can be controlled by varying the sequence length T. In contrast to Section 5.2, we now focus on the accuracy on the last 10 elements of the output sequence (which are the nontrivial elements that must be recalled). We used models of size 10K for both TCN and RNNs.

The results of this focused study are shown in Figure 5. TCNs consistently converge to 100% accuracy for all sequence lengths, whereas LSTMs and GRUs of the same size quickly degenerate to random guessing as the sequence length T grows. The accuracy of the LSTM falls below 20% for $T < 50$, while the GRU falls below 20% for $T < 200$. These results indicate that TCNs are able to maintain a much longer effective history than their recurrent counterparts.

This observation is backed up on real data by experiments on the large-scale LAMBADA dataset, which is specifically designed to test a model's ability to utilize broad context (Paperno et al., 2016). As shown in Table 1, TCN outperforms LSTMs and vanilla RNNs by a significant margin in perplexity on LAMBADA, with a substantially smaller network and virtually no tuning. (State-of-the-art results on this dataset are even better, but only with the help of additional memory mechanisms (Grave et al., 2017).)

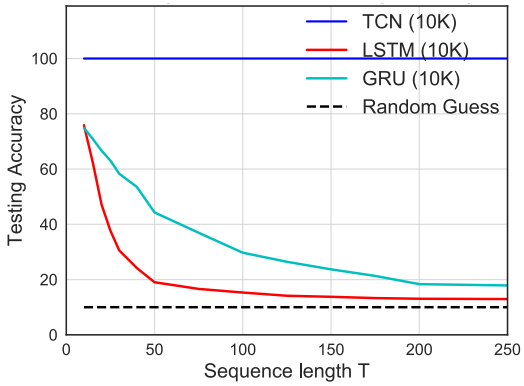


Figure 5. Accuracy on the copy memory task for sequences of different lengths T. While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

6. Conclusion

We have presented an empirical evaluation of generic convolutional and recurrent architectures across a comprehensive suite of sequence modeling tasks. To this end, we have described a simple temporal convolutional network (TCN) that combines best practices such as dilations and residual connections with the causal convolutions needed for autoregressive prediction. The experimental results indicate that TCN models substantially outperform generic recurrent architectures such as LSTMs and GRUs. We further studied long-range information propagation in convolutional and recurrent networks, and showed that the "infinite memory" advantage of RNNs is largely absent in practice. TCNs exhibit longer memory than recurrent architectures with the same capacity.

Numerous advanced schemes for regularizing and optimizing LSTMs have been proposed (Press & Wolf, 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018). These schemes have significantly advanced the accuracy achieved by LSTM-based architectures on some datasets. The TCN has not yet benefitted from this concerted community-wide investment into architectural and algorithmic elaborations. We see such investment as desirable and expect it to yield advances in TCN performance that are commensurate with the advances seen in recent years in LSTM performance. We will release the code for our project to encourage this exploration.

The preeminence enjoyed by recurrent networks in sequence modeling may be largely a vestige of history. Until recently, before the introduction of architectural elements such as dilated convolutions and residual connections, convolutional architectures were indeed weaker. Our results indicate that with these elements, a simple convolutional architecture is more effective across diverse sequence modeling tasks than recurrent architectures such as LSTMs. Due to the comparable clarity and simplicity of TCNs, we conclude that convolutional networks should be regarded as a natural starting point and a powerful toolkit for sequence modeling.

References

Allan, Moray and Williams, Christopher. Harmonising chorales by probabilistic inference. In *NIPS*, 2005.

Arjovsky, Martin, Shah, Amar, and Bengio, Yoshua. Unitary evolution recurrent neural networks. In *ICML*, 2016.

Ba, Lei Jimmy, Kiros, Ryan, and Hinton, Geoffrey E. Layer normalization. *arXiv:1607.06450*, 2016.

Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

Bengio, Yoshua, Simard, Patrice, and Frasconi, Paolo. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 1994.

Bottou, Léon, Soulie, F Fogelman, Blanchet, Pascal, and Liénard, Jean-Sylvain. Speaker-independent isolated digit recognition: Multilayer perceptrons vs. dynamic time warping. *Neural Networks*, 3(4), 1990.

Boulanger-Lewandowski, Nicolas, Bengio, Yoshua, and Vincent, Pascal. Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription. *arXiv:1206.6392*, 2012.

Bradbury, James, Merity, Stephen, Xiong, Caiming, and Socher, Richard. Quasi-recurrent neural networks. In *ICLR*, 2017.

Campos, Victor, Jou, Brendan, Giró i Nieto, Xavier, Torres, Jordi, and Chang, Shih-Fu. Skip RNN: Learning to skip state updates in recurrent neural networks. In *ICLR*, 2018.

Chang, Shiyu, Zhang, Yang, Han, Wei, Yu, Mo, Guo, Xiaoxiao, Tan, Wei, Cui, Xiaodong, Witbrock, Michael J., Hasegawa-Johnson, Mark A., and Huang, Thomas S. Dilated recurrent neural networks. In *NIPS*, 2017.

Cho, Kyunghyun, Van Merriënboer, Bart, Bahdanau, Dzmitry, and Bengio, Yoshua. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv:1409.1259*, 2014.

Chung, Junyoung, Gulcehre, Caglar, Cho, KyungHyun, and Bengio, Yoshua. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv:1412.3555*, 2014.

Chung, Junyoung, Ahn, Sungjin, and Bengio, Yoshua. Hierarchical multiscale recurrent neural networks. *arXiv:1609.01704*, 2016.

Collobert, Ronan and Weston, Jason. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *ICML*, 2008.

Collobert, Ronan, Weston, Jason, Bottou, Léon, Karlen, Michael, Kavukcuoglu, Koray, and Kuksa, Pavel P. Natural language processing (almost) from scratch. *JMLR*, 12, 2011.

Conneau, Alexis, Schwenk, Holger, LeCun, Yann, and Barrault, Loïc. Very deep convolutional networks for text classification. In *European Chapter of the Association for Computational Linguistics (EACL)*, 2017.

Cooijmans, Tim, Ballas, Nicolas, Laurent, César, Gülçehre, Çağlar, and Courville, Aaron. Recurrent batch normalization. In *ICLR*, 2016.

Dauphin, Yann N., Fan, Angela, Auli, Michael, and Grangier, David. Language modeling with gated convolutional networks. In *ICML*, 2017.

dos Santos, Cícero Nogueira and Zadrozny, Bianca. Learning character-level representations for part-of-speech tagging. In *ICML*, 2014.

El Hihi, Salah and Bengio, Yoshua. Hierarchical recurrent neural networks for long-term dependencies. In *NIPS*, 1995.

Elman, Jeffrey L. Finding structure in time. *Cognitive Science*, 14 (2), 1990.

Gehring, Jonas, Auli, Michael, Grangier, David, and Dauphin, Yann. A convolutional encoder model for neural machine translation. In *ACL*, 2017a.

Gehring, Jonas, Auli, Michael, Grangier, David, Yarats, Denis, and Dauphin, Yann N. Convolutional sequence to sequence learning. In *ICML*, 2017b.

Gers, Felix A, Schraudolph, Nicol N, and Schmidhuber, Jürgen. Learning precise timing with lstm recurrent networks. *JMLR*, 3, 2002.

Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016.

Grave, Edouard, Joulin, Armand, and Usunier, Nicolas. Improving neural language models with a continuous cache. In *ICLR*, 2017.

Graves, Alex. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.

Graves, Alex. Generating sequences with recurrent neural networks. *arXiv:1308.0850*, 2013.

Greff, Klaus, Srivastava, Rupesh Kumar, Koutník, Jan, Steunebrink, Bas R., and Schmidhuber, Jürgen. LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10), 2017.

Ha, David, Dai, Andrew, and Le, Quoc V. HyperNetworks. In *ICLR*, 2017.

He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, and Sun, Jian. Deep residual learning for image recognition. In *CVPR*, 2016.

Hermans, Michiel and Schrauwen, Benjamin. Training and analysing deep recurrent neural networks. In *NIPS*, 2013.

Hinton, Geoffrey E. Connectionist learning procedures. *Artificial Intelligence*, 40(1-3), 1989.

Hochreiter, Sepp and Schmidhuber, Jürgen. Long short-term memory. *Neural Computation*, 9(8), 1997.

Jing, Li, Shen, Yichen, Dubcek, Tena, Peurifoy, John, Skirlo, Scott, LeCun, Yann, Tegmark, Max, and Soljačić, Marin. Tunable efficient unitary neural networks (EUNN) and their application to RNNs. In *ICML*, 2017.

Johnson, Rie and Zhang, Tong. Effective use of word order for text categorization with convolutional neural networks. In *HLT-NAACL*, 2015.

Johnson, Rie and Zhang, Tong. Deep pyramid convolutional neural networks for text categorization. In *ACL*, 2017.

Jozefowicz, Rafal, Zaremba, Wojciech, and Sutskever, Ilya. An empirical exploration of recurrent network architectures. In *ICML*, 2015.

Kalchbrenner, Nal, Grefenstette, Edward, and Blunsom, Phil. A convolutional neural network for modelling sentences. In *ACL*, 2014.

Kalchbrenner, Nal, Espeholt, Lasse, Simonyan, Karen, van den Oord, Aäron, Graves, Alex, and Kavukcuoglu, Koray. Neural machine translation in linear time. *arXiv:1610.10099*, 2016.

Kim, Yoon. Convolutional neural networks for sentence classification. In *EMNLP*, 2014.

Kingma, Diederik and Ba, Jimmy. Adam: A method for stochastic optimization. In *ICLR*, 2015.

Koutnik, Jan, Greff, Klaus, Gomez, Faustino, and Schmidhuber, Juergen. A clockwork RNN. In *ICML*, 2014.

Krueger, David and Memisevic, Roland. Regularizing RNNs by stabilizing activations. *arXiv:1511.08400*, 2015.

Krueger, David, Maharaj, Tegan, Kramár, János, Pezeshki, Mohammad, Ballas, Nicolas, Ke, Nan Rosemary, Goyal, Anirudh, Bengio, Yoshua, Larochelle, Hugo, Courville, Aaron C., and Pal, Chris. Zoneout: Regularizing RNNs by randomly preserving hidden activations. In *ICLR*, 2017.

Le, Quoc V, Jaitly, Navdeep, and Hinton, Geoffrey E. A simple way to initialize recurrent networks of rectified linear units. *arXiv:1504.00941*, 2015.

Lea, Colin, Flynn, Michael D., Vidal, René, Reiter, Austin, and Hager, Gregory D. Temporal convolutional networks for action segmentation and detection. In *CVPR*, 2017.

LeCun, Yann, Boser, Bernhard, Denker, John S., Henderson, Donnie, Howard, Richard E., Hubbard, Wayne, and Jackel, Lawrence D. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 1989.

LeCun, Yann, Bottou, Léon, Bengio, Yoshua, and Haffner, Patrick. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 1998.

Long, Jonathan, Shelhamer, Evan, and Darrell, Trevor. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

Marcus, Mitchell P, Marcinkiewicz, Mary Ann, and Santorini, Beatrice. Building a large annotated corpus of English: The Penn treebank. *Computational Linguistics*, 19(2), 1993.

Martens, James and Sutskever, Ilya. Learning recurrent neural networks with Hessian-free optimization. In *ICML*, 2011.

Melis, Gábor, Dyer, Chris, and Blunsom, Phil. On the state of the art of evaluation in neural language models. In *ICLR*, 2018.

Merity, Stephen, Xiong, Caiming, Bradbury, James, and Socher, Richard. Pointer sentinel mixture models. *arXiv:1609.07843*, 2016.

Merity, Stephen, Keskar, Nitish Shirish, and Socher, Richard. Regularizing and optimizing LSTM language models. *arXiv:1708.02182*, 2017.

Mikolov, Tomáš, Sutskever, Ilya, Deoras, Anoop, Le, Hai-Son, Kombrink, Stefan, and Cernocky, Jan. Subword language modeling with neural networks. *Preprint*, 2012.

Miyamoto, Yasumasa and Cho, Kyunghyun. Gated word-character recurrent language model. *arXiv:1606.01700*, 2016.

Nair, Vinod and Hinton, Geoffrey E. Rectified linear units improve restricted Boltzmann machines. In *ICML*, 2010.

Ng, Andrew. Sequence Models (Course 5 of Deep Learning Specialization). *Coursera*, 2018.

Paperno, Denis, Kruszewski, Germán, Lazaridou, Angeliki, Pham, Quan Ngoc, Bernardi, Raffaella, Pezzelle, Sandro, Baroni, Marco, Boleda, Gemma, and Fernández, Raquel. The LAMBADA dataset: Word prediction requiring a broad discourse context. *arXiv:1606.06031*, 2016.

Pascanu, Razvan, Mikolov, Tomas, and Bengio, Yoshua. On the difficulty of training recurrent neural networks. In *ICML*, 2013.

Pascanu, Razvan, Gülçehre, Çaglar, Cho, Kyunghyun, and Bengio, Yoshua. How to construct deep recurrent neural networks. In *ICLR*, 2014.

Press, Ofir and Wolf, Lior. Using the output embedding to improve language models. *arXiv:1608.05859*, 2016.

Salimans, Tim and Kingma, Diederik P. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *NIPS*, 2016.

Schuster, Mike and Paliwal, Kuldip K. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45 (11), 1997.

Sejnowski, Terrence J. and Rosenberg, Charles R. Parallel networks that learn to pronounce English text. *Complex Systems*, 1, 1987.

Shi, Xingjian, Chen, Zhourong, Wang, Hao, Yeung, Dit-Yan, Wong, Wai-Kin, and Woo, Wang-chun. Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In *NIPS*, 2015.

Srivastava, Nitish, Hinton, Geoffrey E, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(1), 2014.

Subakan, Y Cem and Smaragdis, Paris. Diagonal RNNs in symbolic music modeling. *arXiv:1704.05420*, 2017.

Sutskever, Ilya, Martens, James, and Hinton, Geoffrey E. Generating text with recurrent neural networks. In *ICML*, 2011.

Sutskever, Ilya, Vinyals, Oriol, and Le, Quoc V. Sequence to sequence learning with neural networks. In *NIPS*, 2014.

van den Oord, Aäron, Dieleman, Sander, Zen, Heiga, Simonyan, Karen, Vinyals, Oriol, Graves, Alex, Kalchbrenner, Nal, Senior, Andrew W., and Kavukcuoglu, Koray. WaveNet: A generative model for raw audio. *arXiv:1609.03499*, 2016.

Vohra, Raunaq, Goel, Kratarth, and Sahoo, JK. Modeling temporal dependencies in data using a DBN-LSTM. In *Data Science and Advanced Analytics (DSAA)*, 2015.

Waibel, Alex, Hanazawa, Toshiyuki, Hinton, Geoffrey, Shikano, Kiyohiro, and Lang, Kevin J. Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(3), 1989.

Werbos, Paul J. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10), 1990.

Wisdom, Scott, Powers, Thomas, Hershey, John, Le Roux, Jonathan, and Atlas, Les. Full-capacity unitary recurrent neural networks. In *NIPS*, 2016.

Wu, Yuhuai, Zhang, Saizheng, Zhang, Ying, Bengio, Yoshua, and Salakhutdinov, Ruslan R. On multiplicative integration with recurrent neural networks. In *NIPS*, 2016.

Yang, Zhilin, Dai, Zihang, Salakhutdinov, Ruslan, and Cohen, William W. Breaking the softmax bottleneck: A high-rank RNN language model. *ICLR*, 2018.

Yin, Wenpeng, Kann, Katharina, Yu, Mo, and Schütze, Hinrich. Comparative study of CNN and RNN for natural language processing. *arXiv:1702.01923*, 2017.

Yu, Fisher and Koltun, Vladlen. Multi-scale context aggregation by dilated convolutions. In *ICLR*, 2016.

Zhang, Saizheng, Wu, Yuhuai, Che, Tong, Lin, Zhouhan, Memisevic, Roland, Salakhutdinov, Ruslan R, and Bengio, Yoshua. Architectural complexity measures of recurrent neural networks. In *NIPS*, 2016.

Zhang, Xiang, Zhao, Junbo Jake, and LeCun, Yann. Character-level convolutional networks for text classification. In *NIPS*, 2015.

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Supplementary Material

A. Hyperparameters Settings

A.1. Hyperparameters for TCN

Table 2 lists the hyperparameters we used when applying the generic TCN model on various tasks and datasets. The most important factor for picking parameters is to make sure that the TCN has a sufficiently large receptive field by choosing k and d that can cover the amount of context needed for the task.

As discussed in Section 5, the number of hidden units was chosen so that the model size is approximately at the same level as the recurrent models with which we are comparing. In Table 2, a gradient clip of N/A means no gradient clipping was applied. In larger tasks (e.g., language modeling), we empirically found that gradient clipping (we randomly picked a threshold from $[0.3, 1]$) helps with regularizing TCN and accelerating convergence.

All weights were initialized from a Gaussian disitribution $\mathcal{N}(0, 0.01)$. In general, we found TCN to be relatively insensitive to hyperparameter changes, as long as the effective history (i.e., receptive field) size is sufficient.

A.2. Hyperparameters for LSTM/GRU

Table 3 reports hyperparameter settings that were used for the LSTM. These values are picked from hyperparameter search for LSTMs that have up to 3 layers, and the optimizers are chosen from {SGD, Adam, RMSprop, Adagrad}. For certain larger datasets, we adopted the settings used in prior work (e.g., Grave et al. (2017) on Wikitext-103). GRU hyperparameters were chosen in a similar fashion, but typically with more hidden units than in LSTM to keep the total network size approximately the same (since a GRU cell is more compact).

B. State-of-the-Art Results

As previously noted, the generic TCN and LSTM/GRU models we used can be outperformed by more specialized architectures on some tasks. State-of-the-art results are summarized in Table 4. The same TCN architecture is used across all tasks. Note that the size of the state-of-the-art model may be different from the size of the TCN.

C. Effect of Filter Size and Residual Block

In this section we briefly study the effects of different components of a TCN layer. Overall, we believe dilation is required for modeling long-term dependencies, and so we mainly focus on two other factors here: the filter size k used by each layer, and the effect of residual blocks.

We perform a series of controlled experiments, with the results of the ablative analysis shown in Figure 6. As before, we kept the model size and depth exactly the same for different models, so that the dilation factor is strictly controlled. The experiments were conducted on three different tasks: copy memory, permuted MNIST (P-MNIST), and Penn Treebank word-level language modeling. These experiments confirm that both factors (filter size and residual connections) contribute to sequence modeling performance.

Filter size k. In both the copy memory and the P-MNIST tasks, we observed faster convergence and better accuracy for larger filter sizes. In particular, looking at Figure 6a, a TCN with filter size ≤ 3 only converges to the same level as random guessing. In contrast, on word-level language modeling, a smaller kernel with filter size of $k = 3$ works best. We believe this is because a smaller kernel (along with fixed dilation) tends to focus more on the local context, which is especially important for PTB language modeling (in fact, the very success of n-gram models suggests that only a relatively short memory is needed for modeling language).

Residual block. In all three scenarios that we compared here, we observed that the residual function stabilized training and brought faster convergence with better final results. Especially in language modeling, we found that residual connections contribute substantially to performance (See Figure 6f).

D. Gating Mechanisms

One component that had been used in prior work on convolutional architectures for language modeling is the gated activation (van den Oord et al., 2016; Dauphin et al., 2017). We have chosen not to use gating in the generic TCN model. We now examine this choice more closely.

Table 2. TCN parameter settings for experiments in Section 5.

TCN SETTINGS							
Dataset/Task	**Subtask**	k	n	**Hidden**	**Dropout**	**Grad Clip**	**Note**
The Adding Problem	$T = 200$	6	7	27	0.0	N/A	
	$T = 400$	7	7	27			
	$T = 600$	8	8	24			
Seq. MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Permuted MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Copy Memory Task	$T = 500$	6	9	10	0.05	1.0	RMSprop 5e-4
	$T = 1000$	8	8	10			
	$T = 2000$	8	9	10			
Music JSB Chorales	-	3	2	150	0.5	0.4	
Music Nottingham	-	6	4	150	0.2	0.4	
Word-level LM	PTB	3	4	600	0.5	0.4	Embed. size 600
	Wiki-103	3	5	1000	0.4		Embed. size 400
	LAMBADA	4	5	500			Embed. size 500
Char-level LM	PTB	3	3	450	0.1	0.15	Embed. size 100
	text8	2	5	520			

Dauphin et al. (2017) compared the effects of gated linear units (GLU) and gated tanh units (GTU), and adopted GLU in their non-dilated gated ConvNet. Following the same choice, we now compare TCNs using ReLU and TCNs with gating (GLU), represented by an elementwise product between two convolutional layers, with one of them also passing through a sigmoid function $\sigma(x)$. Note that the gates architecture uses approximately twice as many convolutional layers as the ReLU-TCN.

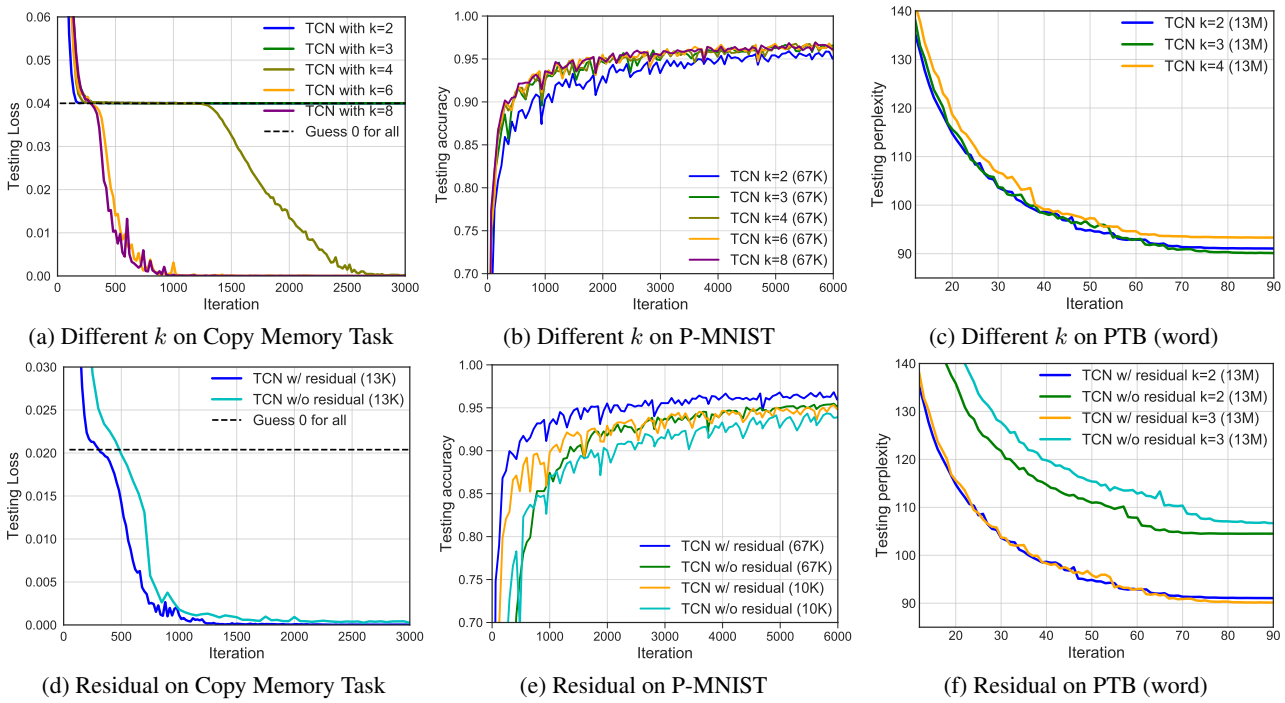
The results are shown in Table 5, where we kept the number of model parameters at about the same size. The GLU does further improve TCN accuracy on certain language modeling datasets like PTB, which agrees with prior work. However, we do not observe comparable benefits on other tasks, such as polyphonic music modeling or synthetic stress tests that require longer information retention. On the copy memory task with $T = 1000$, we found that TCN with gating converged to a worse result than TCN with ReLU (though still better than recurrent models).

Table 3. LSTM parameter settings for experiments in Section 5.

LSTM SETTINGS (KEY PARAMETERS)							
Dataset/Task	**Subtask**	n	**Hidden**	**Dropout**	**Grad Clip**	**Bias**	**Note**
The Adding Problem	$T = 200$	2	77	0.0	50	5.0	SGD 1e-3
	$T = 400$	2	77		50	10.0	Adam 2e-3
	$T = 600$	1	130		5	1.0	-
Seq. MNIST	-	1	130	0.0	1	1.0	RMSprop 1e-3
Permuted MNIST	-	1	130	0.0	1	10.0	RMSprop 1e-3
Copy Memory Task	$T = 500$	1	50	0.05	0.25	-	RMSprop/Adam
	$T = 1000$	1	50		1		
	$T = 2000$	3	28		1		
Music JSB Chorales	-	2	200	0.2	1	10.0	SGD/Adam
Music Nottingham	-	3	280	0.1	0.5	-	Adam 4e-3
		1	500		1	-	
Word-level LM	PTB	3	700	0.4	0.3	1.0	SGD 30, Emb. 700, etc.
	Wiki-103	-	-	-	-	-	Grave et al. (2017)
	LAMBADA	-	-	-	-	-	Grave et al. (2017)
Char-level LM	PTB	2	600	0.1	0.5	-	Emb. size 120
	text8	1	1024	0.15	0.5	-	Adam 1e-2

Table 4. State-of-the-art (SoTA) results for tasks in Section 5.

TCN VS. SOTA RESULTS					
Task	**TCN Result**	**Size**	**SoTA**	**Size**	**Model**
Seq. MNIST (acc.)	99.0	21K	99.0	21K	Dilated GRU (Chang et al., 2017)
P-MNIST (acc.)	97.2	42K	95.9	42K	Zoneout (Krueger et al., 2017)
Adding Prob. 600 (loss)	5.8e-5	70K	5.3e-5	70K	Regularized GRU
Copy Memory 1000 (loss)	3.5e-5	70K	0.011	70K	EURNN (Jing et al., 2017)
JSB Chorales (loss)	8.10	300K	3.47	-	DBN+LSTM (Vohra et al., 2015)
Nottingham (loss)	3.07	1M	1.32	-	DBN+LSTM (Vohra et al., 2015)
Word PTB (ppl)	88.68	13M	47.7	22M	AWD-LSTM-MoS + Dynamic Eval. (Yang et al., 2018)
Word Wiki-103 (ppl)	45.19	148M	40.4	>300M	Neural Cache Model (Large) (Grave et al., 2017)
Word LAMBADA (ppl)	1279	56M	138	>100M	Neural Cache Model (Large) (Grave et al., 2017)
Char PTB (bpc)	1.31	3M	1.22	14M	2-LayerNorm HyperLSTM (Ha et al., 2017)
Char text8 (bpc)	1.45	4.6M	1.29	>12M	HM-LSTM (Chung et al., 2016)



(a) Different k on Copy Memory Task (b) Different k on P-MNIST (c) Different k on PTB (word)

(d) Residual on Copy Memory Task (e) Residual on P-MNIST (f) Residual on PTB (word)

Figure 6. Controlled experiments that study the effect of different components of the TCN model.

Table 5. An evaluation of gating in TCN. A plain TCN is compared to a TCN that uses gated activations.

Task	**TCN**	**TCN + Gating**
Sequential MNIST (acc.)	**99.0**	**99.0**
Permuted MNIST (acc.)	**97.2**	96.9
Adding Problem $T = 600$ (loss)	**5.8e-5**	**5.6e-5**
Copy Memory $T = 1000$ (loss)	**3.5e-5**	0.00508
JSB Chorales (loss)	**8.10**	8.13
Nottingham (loss)	**3.07**	3.12
Word-level PTB (ppl)	88.68	**87.94**
Char-level PTB (bpc)	1.31	**1.306**
Char text8 (bpc)	**1.45**	1.485

CBAM: Convolutional Block Attention Module

Sanghyun Woo*[1], Jongchan Park*†[2], Joon-Young Lee[3], and In So Kweon[1]

[1] Korea Advanced Institute of Science and Technology, Daejeon, Korea
{shwoo93, iskweon77}@kaist.ac.kr
[2] Lunit Inc., Seoul, Korea
jcpark@lunit.io
[3] Adobe Research, San Jose, CA, USA
jolee@adobe.com

Abstract. We propose Convolutional Block Attention Module (CBAM), a simple yet effective attention module for feed-forward convolutional neural networks. Given an intermediate feature map, our module sequentially infers attention maps along two separate dimensions, channel and spatial, then the attention maps are multiplied to the input feature map for adaptive feature refinement. Because CBAM is a lightweight and general module, it can be integrated into any CNN architectures seamlessly with negligible overheads and is end-to-end trainable along with base CNNs. We validate our CBAM through extensive experiments on ImageNet-1K, MS COCO detection, and VOC 2007 detection datasets. Our experiments show consistent improvements in classification and detection performances with various models, demonstrating the wide applicability of CBAM. The code and models will be publicly available.

Keywords: Object recognition, attention mechanism, gated convolution

1 Introduction

Convolutional neural networks (CNNs) have significantly pushed the performance of vision tasks [1–3] based on their rich representation power. To enhance performance of CNNs, recent researches have mainly investigated three important factors of networks: *depth*, *width*, and *cardinality*.

From the LeNet architecture [4] to Residual-style Networks [5–8] so far, the network has become deeper for rich representation. VGGNet [9] shows that stacking blocks with the same shape gives fair results. Following the same spirit, ResNet [5] stacks the same topology of residual blocks along with skip connection to build an extremely deep architecture. GoogLeNet [10] shows that width is another important factor to improve the performance of a model. Zagoruyko and Komodakis [6] propose to increase the width of a network based on the ResNet architecture. They have shown that a 28-layer ResNet with increased

*Both authors have equally contributed.
†The work was done while the author was at KAIST.

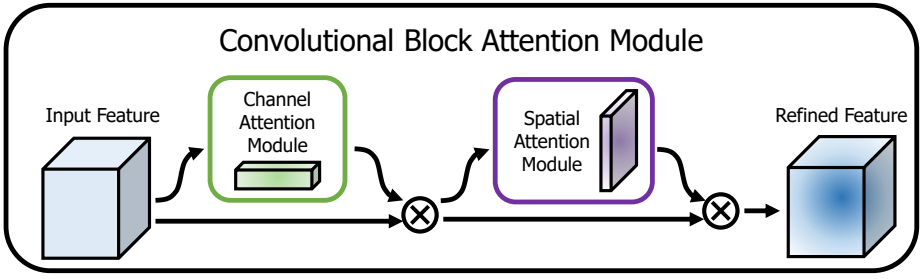


Fig. 1: **The overview of CBAM**. The module has two sequential sub-modules: *channel* and *spatial*. The intermediate feature map is adaptively refined through our module (CBAM) at every convolutional block of deep networks.

width can outperform an extremely deep ResNet with 1001 layers on the CIFAR benchmarks. Xception [11] and ResNeXt [7] come up with to increase the cardinality of a network. They empirically show that cardinality not only saves the total number of parameters but also results in stronger representation power than the other two factors: depth and width.

Apart from these factors, we investigate a different aspect of the architecture design, *attention*. The significance of attention has been studied extensively in the previous literature [12–17]. Attention not only tells where to focus, it also improves the representation of interests. Our goal is to increase representation power by using attention mechanism: focusing on important features and suppressing unnecessary ones. In this paper, we propose a new network module, named "Convolutional Block Attention Module". Since convolution operations extract informative features by blending cross-channel and spatial information together, we adopt our module to emphasize meaningful features along those two principal dimensions: channel and spatial axes. To achieve this, we sequentially apply channel and spatial attention modules (as shown in Fig. 1), so that each of the branches can learn 'what' and 'where' to attend in the channel and spatial axes respectively. As a result, our module efficiently helps the information flow within the network by learning which information to emphasize or suppress.

In the ImageNet-1K dataset, we obtain accuracy improvement from various baseline networks by plugging our tiny module, revealing the efficacy of CBAM. We visualize trained models using the grad-CAM [18] and observe that CBAM-enhanced networks focus on target objects more properly than their baseline networks. Taking this into account, we conjecture that the performance boost comes from accurate attention and noise reduction of irrelevant clutters. Finally, we validate performance improvement of object detection on the MS COCO and the VOC 2007 datasets, demonstrating a wide applicability of CBAM. Since we have carefully designed our module to be light-weight, the overhead of parameters and computation is negligible in most cases.

Contribution. Our main contribution is three-fold.

1. We propose a simple yet effective attention module (CBAM) that can be widely applied to boost representation power of CNNs.

2. We validate the effectiveness of our attention module through extensive ablation studies.
3. We verify that performance of various networks is greatly improved on the multiple benchmarks (ImageNet-1K, MS COCO, and VOC 2007) by plugging our light-weight module.

2 Related Work

Network engineering. "Network engineering" has been one of the most important vision research, because well-designed networks ensure remarkable performance improvement in various applications. A wide range of architectures has been proposed since the successful implementation of a large-scale CNN [19]. An intuitive and simple way of extension is to increase the depth of neural networks [9]. Szegedy *et al.* [10] introduce a deep Inception network using a multi-branch architecture where each branch is customized carefully. While a naive increase in depth comes to saturation due to the difficulty of gradient propagation, ResNet [5] proposes a simple identity skip-connection to ease the optimization issues of deep networks. Based on the ResNet architecture, various models such as WideResNet [6], Inception-ResNet [8], and ResNeXt [7] have been developed. WideResNet [6] proposes a residual network with a larger number of convolutional filters and reduced depth. PyramidNet [20] is a strict generalization of WideResNet where the width of the network gradually increases. ResNeXt [7] suggests to use grouped convolutions and shows that increasing the cardinality leads to better classification accuracy. More recently, Huang *et al.* [21] propose a new architecture, DenseNet. It iteratively concatenates the input features with the output features, enabling each convolution block to receive raw information from all the previous blocks. While most of recent network engineering methods mainly target on three factors *depth* [19, 9, 10, 5], *width* [10, 22, 6, 8], and *cardinality* [7, 11], we focus on the other aspect, '*attention*', one of the curious facets of a human visual system.

Attention mechanism. It is well known that attention plays an important role in human perception [23–25]. One important property of a human visual system is that one does not attempt to process a whole scene at once. Instead, humans exploit a sequence of partial glimpses and selectively focus on salient parts in order to capture visual structure better [26].

Recently, there have been several attempts [27, 28] to incorporate attention processing to improve the performance of CNNs in large-scale classification tasks. Wang *et al.* [27] propose *Residual Attention Network* which uses an encoder-decoder style attention module. By refining the feature maps, the network not only performs well but is also robust to noisy inputs. Instead of directly computing the 3d attention map, we decompose the process that learns channel attention and spatial attention separately. The separate attention generation process for 3D feature map has much less computational and parameter over-

head, and therefore can be used as a plug-and-play module for pre-existing base CNN architectures.

More close to our work, Hu *et al.* [28] introduce a compact module to exploit the inter-channel relationship. In their *Squeeze-and-Excitation* module, they use global average-pooled features to compute channel-wise attention. However, we show that those are suboptimal features in order to infer fine channel attention, and we suggest to use max-pooled features as well. They also miss the spatial attention, which plays an important role in deciding 'where' to focus as shown in [29]. In our CBAM, we exploit both spatial and channel-wise attention based on an efficient architecture and empirically verify that exploiting both is superior to using only the channel-wise attention as [28]. Moreover, we empirically show that our module is effective in detection tasks (MS-COCO and VOC). Especially, we achieve state-of-the-art performance just by placing our module on top of the existing one-shot detector [30] in the VOC2007 test set.

3 Convolutional Block Attention Module

Given an intermediate feature map $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$ as input, CBAM sequentially infers a 1D channel attention map $\mathbf{M_c} \in \mathbb{R}^{C \times 1 \times 1}$ and a 2D spatial attention map $\mathbf{M_s} \in \mathbb{R}^{1 \times H \times W}$ as illustrated in Fig. 1. The overall attention process can be summarized as:

$$\begin{aligned} \mathbf{F}' &= \mathbf{M_c}(\mathbf{F}) \otimes \mathbf{F}, \\ \mathbf{F}'' &= \mathbf{M_s}(\mathbf{F}') \otimes \mathbf{F}', \end{aligned} \tag{1}$$

where $\otimes$ denotes element-wise multiplication. During multiplication, the attention values are broadcasted (copied) accordingly: channel attention values are broadcasted along the spatial dimension, and vice versa. $\mathbf{F}''$ is the final refined output. Fig. 2 depicts the computation process of each attention map. The following describes the details of each attention module.

Channel attention module. We produce a channel attention map by exploiting the inter-channel relationship of features. As each channel of a feature map is considered as a feature detector [31], channel attention focuses on 'what' is meaningful given an input image. To compute the channel attention efficiently, we squeeze the spatial dimension of the input feature map. For aggregating spatial information, average-pooling has been commonly adopted so far. Zhou *et al.* [32] suggest to use it to learn the extent of the target object effectively and Hu *et al.* [28] adopt it in their attention module to compute spatial statistics. Beyond the previous works, we argue that max-pooling gathers another important clue about distinctive object features to infer finer channel-wise attention. Thus, we use both average-pooled and max-pooled features simultaneously. We empirically confirmed that exploiting both features greatly improves representation power of networks rather than using each independently (see Sec. 4.1), showing the effectiveness of our design choice. We describe the detailed operation below.

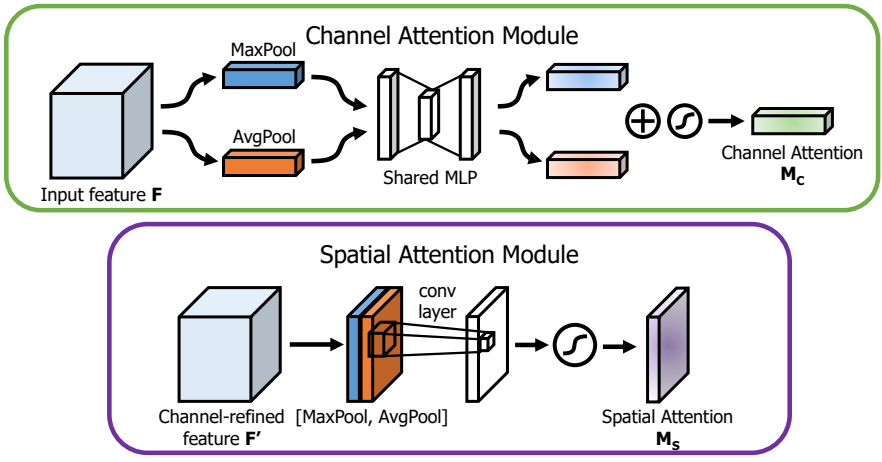


Fig. 2: **Diagram of each attention sub-module.** As illustrated, the channel sub-module utilizes both max-pooling outputs and average-pooling outputs with a shared network; the spatial sub-module utilizes similar two outputs that are pooled along the channel axis and forward them to a convolution layer.

We first aggregate spatial information of a feature map by using both average-pooling and max-pooling operations, generating two different spatial context descriptors: $\mathbf{F^c_{avg}}$ and $\mathbf{F^c_{max}}$, which denote average-pooled features and max-pooled features respectively. Both descriptors are then forwarded to a shared network to produce our channel attention map $\mathbf{M_c} \in \mathbb{R}^{C\times1\times1}$. The shared network is composed of multi-layer perceptron (MLP) with one hidden layer. To reduce parameter overhead, the hidden activation size is set to $\mathbb{R}^{C/r\times1\times1}$, where r is the reduction ratio. After the shared network is applied to each descriptor, we merge the output feature vectors using element-wise summation. In short, the channel attention is computed as:

$$\begin{aligned}\mathbf{M_c}(\mathbf{F}) &= \sigma(MLP(AvgPool(\mathbf{F})) + MLP(MaxPool(\mathbf{F}))) \\ &= \sigma(\mathbf{W_1}(\mathbf{W_0}(\mathbf{F^c_{avg}})) + \mathbf{W_1}(\mathbf{W_0}(\mathbf{F^c_{max}}))),\end{aligned} \tag{2}$$

where σ denotes the sigmoid function, $\mathbf{W_0} \in \mathbb{R}^{C/r\times C}$, and $\mathbf{W_1} \in \mathbb{R}^{C\times C/r}$. Note that the MLP weights, $\mathbf{W_0}$ and $\mathbf{W_1}$, are shared for both inputs and the ReLU activation function is followed by $\mathbf{W_0}$.

Spatial attention module. We generate a spatial attention map by utilizing the inter-spatial relationship of features. Different from the channel attention, the spatial attention focuses on 'where' is an informative part, which is complementary to the channel attention. To compute the spatial attention, we first apply average-pooling and max-pooling operations along the channel axis and concatenate them to generate an efficient feature descriptor. Applying pooling operations along the channel axis is shown to be effective in highlighting informative regions [33]. On the concatenated feature descriptor, we apply a convolution

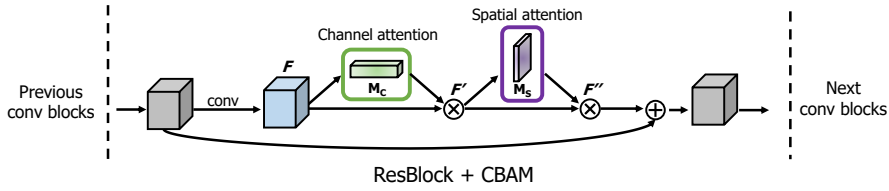


Fig. 3: **CBAM integrated with a ResBlock in ResNet[5].** This figure shows the exact position of our module when integrated within a ResBlock. We apply CBAM on the convolution outputs in each block.

layer to generate a spatial attention map $\mathbf{M_s}(\mathbf{F}) \in \mathbf{R}^{H \times W}$ which encodes where to emphasize or suppress. We describe the detailed operation below.

We aggregate channel information of a feature map by using two pooling operations, generating two 2D maps: $\mathbf{F^s_{avg}} \in \mathbb{R}^{1 \times H \times W}$ and $\mathbf{F^s_{max}} \in \mathbb{R}^{1 \times H \times W}$. Each denotes average-pooled features and max-pooled features across the channel. Those are then concatenated and convolved by a standard convolution layer, producing our 2D spatial attention map. In short, the spatial attention is computed as:

$$\begin{aligned} \mathbf{M_s}(\mathbf{F}) &= \sigma(f^{7\times7}([AvgPool(\mathbf{F}); MaxPool(\mathbf{F})])) \\ &= \sigma(f^{7\times7}([\mathbf{F^s_{avg}}; \mathbf{F^s_{max}}])), \end{aligned} \tag{3}$$

where σ denotes the sigmoid function and $f^{7\times7}$ represents a convolution operation with the filter size of 7×7.

Arrangement of attention modules. Given an input image, two attention modules, channel and spatial, compute complementary attention, focusing on 'what' and 'where' respectively. Considering this, two modules can be placed in a parallel or sequential manner. We found that the sequential arrangement gives a better result than a parallel arrangement. For the arrangement of the sequential process, our experimental result shows that the channel-first order is slightly better than the spatial-first. We will discuss experimental results on network engineering in Sec. 4.1.

4 Experiments

We evaluate CBAM on the standard benchmarks: ImageNet-1K for image classification; MS COCO and VOC 2007 for object detection. In order to perform better apple-to-apple comparisons, we reproduced all the evaluated networks [5–7, 34, 28] in the PyTorch framework [35] and report our reproduced results in the whole experiments.

To thoroughly evaluate the effectiveness of our final module, we first perform extensive ablation experiments. Then, we verify that CBAM outperforms all the

baselines without bells and whistles, demonstrating the general applicability of CBAM across different architectures as well as different tasks. One can seamlessly integrate CBAM in any CNN architectures and jointly train the combined CBAM-enhanced networks. Fig. 3 shows a diagram of CBAM integrated with a ResBlock in ResNet [5] as an example.

4.1 Ablation studies

In this subsection, we empirically show the effectiveness of our design choice. For this ablation study, we use the ImageNet-1K dataset and adopt ResNet-50 [5] as the base architecture. The ImageNet-1K classification dataset [1] consists of 1.2 million images for training and 50,000 for validation with 1,000 object classes. We adopt the same data augmentation scheme with [5, 36] for training and apply a single-crop evaluation with the size of 224×224 at test time. The learning rate starts from 0.1 and drops every 30 epochs. We train the networks for 90 epochs. Following [5, 36, 37], we report classification errors on the validation set.

Our module design process is split into three parts. We first search for the effective approach to computing the channel attention, then the spatial attention. Finally, we consider how to combine both channel and spatial attention modules. We explain the details of each experiment below.

Channel attention. We experimentally verify that using both average-pooled and max-pooled features enables finer attention inference. We compare 3 variants of channel attention: average pooling, max pooling, and joint use of both poolings. Note that the channel attention module with an average pooling is the same as the SE [28] module. Also, when using both poolings, we use a shared MLP for attention inference to save parameters, as both of aggregated channel features lie in the same semantic embedding space. We only use channel attention modules in this experiment and we fix the reduction ratio to 16.

Experimental results with various pooling methods are shown in Table 1. We observe that max-pooled features are as meaningful as average-pooled features, comparing the accuracy improvement from the baseline. In the work of SE [28], however, they only exploit the average-pooled features, missing the importance

Description	Parameters	GFLOPs	Top-1 Error(%)	Top-5 Error(%)
ResNet50 (baseline)	25.56M	3.86	24.56	7.50
ResNet50 + AvgPool (SE [28])	25.92M	3.94	23.14	6.70
ResNet50 + MaxPool	25.92M	3.94	23.20	6.83
ResNet50 + AvgPool & MaxPool	25.92M	4.02	**22.80**	**6.52**

Table 1: **Comparison of different channel attention methods**. We observe that using our proposed method outperforms recently suggested Squeeze and Excitation method [28].

Description	Param.	GFLOPs	Top-1 Error(%)	Top-5 Error(%)
ResNet50 + channel (SE [28])	28.09M	3.860	23.14	6.70
ResNet50 + channel	28.09M	3.860	22.80	6.52
ResNet50 + channel + spatial (1x1 conv, k=3)	28.10M	3.862	22.96	6.64
ResNet50 + channel + spatial (1x1 conv, k=7)	28.10M	3.869	22.90	6.47
ResNet50 + channel + spatial (avg&max, k=3)	28.09M	3.863	22.68	6.41
ResNet50 + channel + spatial (avg&max, k=7)	28.09M	3.864	**22.66**	**6.31**

Table 2: **Comparison of different spatial attention methods**. Using the proposed channel-pooling (*i.e.* average- and max-pooling along the channel axis) along with the large kernel size of 7 for the following convolution operation performs best.

Description	Top-1 Error(%)	Top-5 Error(%)
ResNet50 + channel (SE [28])	23.14	6.70
ResNet50 + channel + spatial	**22.66**	**6.31**
ResNet50 + spatial + channel	22.78	6.42
ResNet50 + channel & spatial in parallel	22.95	6.59

Table 3: **Combining methods of channel and spatial attention**. Using both attention is critical while the best-combining strategy (*i.e.* sequential, channel-first) further improves the accuracy.

of max-pooled features. We argue that max-pooled features which encode the degree of the most salient part can compensate the average-pooled features which encode global statistics softly. Thus, we suggest to use both features simultaneously and apply a shared network to those features. The outputs of a shared network are then merged by element-wise summation. We empirically show that our channel attention method is an effective way to push performance further from SE [28] without additional learnable parameters. As a brief conclusion, we use both average- and max-pooled features in our channel attention module with the reduction ratio of 16 in the following experiments.

Spatial attention. Given the channel-wise refined features, we explore an effective method to compute the spatial attention. The design philosophy is symmetric with the channel attention branch. To generate a 2D spatial attention map, we first compute a 2D descriptor that encodes channel information at each pixel over all spatial locations. We then apply one convolution layer to the 2D descriptor, obtaining the raw attention map. The final attention map is normalized by the sigmoid function.

We compare two methods of generating the 2D descriptor: *channel pooling* using average- and max-pooling across the channel axis and *standard* 1×1 *convolution* reducing the channel dimension into 1. In addition, we investigate the effect of a kernel size at the following convolution layer: kernel sizes of 3 and 7. In the experiment, we place the spatial attention module after the previously

designed channel attention module, as the final goal is to use both modules together.

Table 2 shows the experimental results. We can observe that the channel pooling produces better accuracy, indicating that explicitly modeled pooling leads to finer attention inference rather than learnable weighted channel pooling (implemented as 1×1 convolution). In the comparison of different convolution kernel sizes, we find that adopting a larger kernel size generates better accuracy in both cases. It implies that a broad view (*i.e.* large receptive field) is needed for deciding spatially important regions. Considering this, we adopt the channel-pooling method and the convolution layer with a large kernel size to compute spatial attention. In a brief conclusion, we use the average- and max-pooled features across the channel axis with a convolution kernel size of 7 as our spatial attention module.

Arrangement of the channel and spatial attention. In this experiment, we compare three different ways of arranging the channel and spatial attention submodules: sequential channel-spatial, sequential spatial-channel, and parallel use of both attention modules. As each module has different functions, the order may affect the overall performance. For example, from a spatial viewpoint, the channel attention is globally applied, while the spatial attention works locally. Also, it is natural to think that we may combine two attention outputs to build a 3D attention map. In the case, both attentions can be applied in parallel, then the outputs of the two attention modules are added and normalized with the sigmoid function.

Table 3 summarizes the experimental results on different attention arranging methods. From the results, we can find that generating an attention map sequentially infers a finer attention map than doing in parallel. In addition, the channel-first order performs slightly better than the spatial-first order. Note that all the arranging methods outperform using only the channel attention independently, showing that utilizing both attentions is crucial while the best-arranging strategy further pushes performance.

Final module design. Throughout the ablation studies, we have designed the channel attention module, the spatial attention module, and the arrangement of the two modules. Our final module is as shown in Fig. 1 and Fig. 2: we choose average- and max-pooling for both channel and spatial attention module; we use convolution with a kernel size of 7 in the spatial attention module; we arrange the channel and spatial submodules sequentially. Our final module(*i.e.* ResNet50 + CBAM) achieves top-1 error of 22.66%, which is much lower than SE [28](*i.e.* ResNet50 + SE), as shown in Table 4.

4.2 Image Classification on ImageNet-1K

We perform ImageNet-1K classification experiments to rigorously evaluate our module. We follow the same protocol as specified in Sec. 4.1 and evaluate our

Architecture	Param.	GFLOPs	Top-1 Error (%)	Top-5 Error (%)
ResNet18 [5]	11.69M	1.814	29.60	10.55
ResNet18 [5] + SE [28]	11.78M	1.814	29.41	10.22
ResNet18 [5] + CBAM	11.78M	1.815	**29.27**	**10.09**
ResNet34 [5]	21.80M	3.664	26.69	8.60
ResNet34 [5] + SE [28]	21.96M	3.664	26.13	8.35
ResNet34 [5] + CBAM	21.96M	3.665	**25.99**	**8.24**
ResNet50 [5]	25.56M	3.858	24.56	7.50
ResNet50 [5] + SE [28]	28.09M	3.860	23.14	6.70
ResNet50 [5] + CBAM	28.09M	3.864	**22.66**	**6.31**
ResNet101 [5]	44.55M	7.570	23.38	6.88
ResNet101 [5] + SE [28]	49.33M	7.575	22.35	6.19
ResNet101 [5] + CBAM	49.33M	7.581	**21.51**	**5.69**
WideResNet18 [6] (widen=1.5)	25.88M	3.866	26.85	8.88
WideResNet18 [6] (widen=1.5) + SE [28]	26.07M	3.867	26.21	8.47
WideResNet18 [6] (widen=1.5) + CBAM	26.08M	3.868	**26.10**	**8.43**
WideResNet18 [6] (widen=2.0)	45.62M	6.696	25.63	8.20
WideResNet18 [6] (widen=2.0) + SE [28]	45.97M	6.696	24.93	7.65
WideResNet18 [6] (widen=2.0) + CBAM	45.97M	6.697	**24.84**	**7.63**
ResNeXt50 [7] (32x4d)	25.03M	3.768	22.85	6.48
ResNeXt50 [7] (32x4d) + SE [28]	27.56M	3.771	**21.91**	6.04
ResNeXt50 [7] (32x4d) + CBAM	27.56M	3.774	21.92	**5.91**
ResNeXt101 [7] (32x4d)	44.18M	7.508	21.54	5.75
ResNeXt101 [7] (32x4d) + SE [28]	48.96M	7.512	21.17	5.66
ResNeXt101 [7] (32x4d) + CBAM	48.96M	7.519	**21.07**	**5.59**

* all results are reproduced in the PyTorch framework.

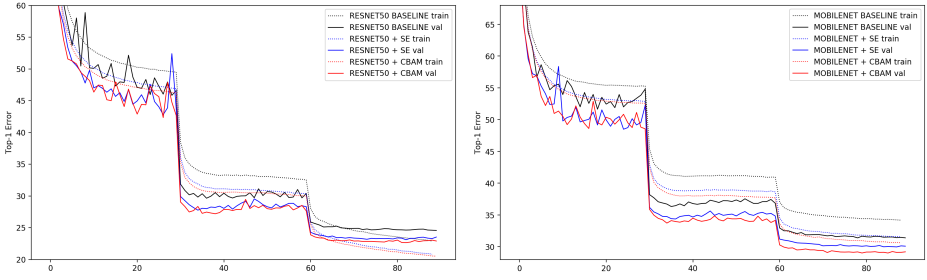
Table 4: **Classification results on ImageNet-1K.** Single-crop validation errors are reported.

module in various network architectures including ResNet [5], WideResNet [6], and ResNext [7].

Table 4 summarizes the experimental results. The networks with CBAM outperform all the baselines significantly, demonstrating that the CBAM can generalize well on various models in the large-scale dataset. Moreover, the models with CBAM improve the accuracy upon the one of the strongest method – SE [28] which is the winning approach of the ILSVRC 2017 classification task. It implies that our proposed approach is powerful, showing the efficacy of *new pooling method* that generates richer descriptor and *spatial attention* that complements the channel attention effectively.

Fig. 4 depicts the error curves of various networks during ImageNet-1K training. We can clearly see that our method exhibits lowest training and validation error in both error plots. It shows that CBAM has greater ability to improve generalization power of baseline models compared to SE [28].

We also find that the overall overhead of CBAM is quite small in terms of both parameters and computation. This motivates us to apply our proposed module CBAM to the light-weight network, MobileNet [34]. Table 5 summarizes the experimental results that we conducted based on the MobileNet architecture. We have placed CBAM to two models, basic and capacity-reduced model(*i.e.* adjusting width multiplier(α) to 0.7). We observe similar phenomenon as shown



(a) ResNet50 [5] (b) MobileNet [34]

Fig. 4: **Error curves during ImageNet-1K training**. Best viewed in color.

Architecture	Parameters	GFLOPs	Top-1 Error (%)	Top-5 Error (%)
MobileNet [34] $\alpha = 0.7$	2.30M	0.283	34.86	13.69
MobileNet [34] $\alpha = 0.7$ + SE [28]	2.71M	0.283	32.50	12.49
MobileNet [34] $\alpha = 0.7$ + CBAM	2.71M	0.289	**31.51**	**11.48**
MobileNet [34]	4.23M	0.569	31.39	11.51
MobileNet [34] + SE [28]	5.07M	0.570	29.97	10.63
MobileNet [34] + CBAM	5.07M	0.576	**29.01**	**9.99**

***** all results are reproduced in the PyTorch framework.

Table 5: **Classification results on ImageNet-1K using the light-weight network, MobileNet [34].** Single-crop validation errors are reported.

in Table 4. CBAM not only boosts the accuracy of baselines significantly but also favorably improves the performance of SE [28]. This shows the great potential of CBAM for applications on low-end devices.

4.3 Network Visualization with Grad-CAM [18]

For the qualitative analysis, we apply the Grad-CAM [18] to different networks using images from the ImageNet validation set. Grad-CAM is a recently proposed visualization method which uses gradients in order to calculate the importance of the spatial locations in convolutional layers. As the gradients are calculated with respect to a unique class, Grad-CAM result shows attended regions clearly. By observing the regions that network has considered as important for predicting a class, we attempt to look at how this network is making good use of features. We compare the visualization results of CBAM-integrated network (ResNet50 + CBAM) with baseline (ResNet50) and SE-integrated network (ResNet50 + SE). Fig. 5 illustrate the visualization results. The softmax scores for a target class are also shown in the figure.

In Fig. 5, we can clearly see that the Grad-CAM masks of the CBAM-integrated network cover the target object regions better than other methods. That is, the CBAM-integrated network learns well to exploit information in target object regions and aggregate features from them. Note that target class

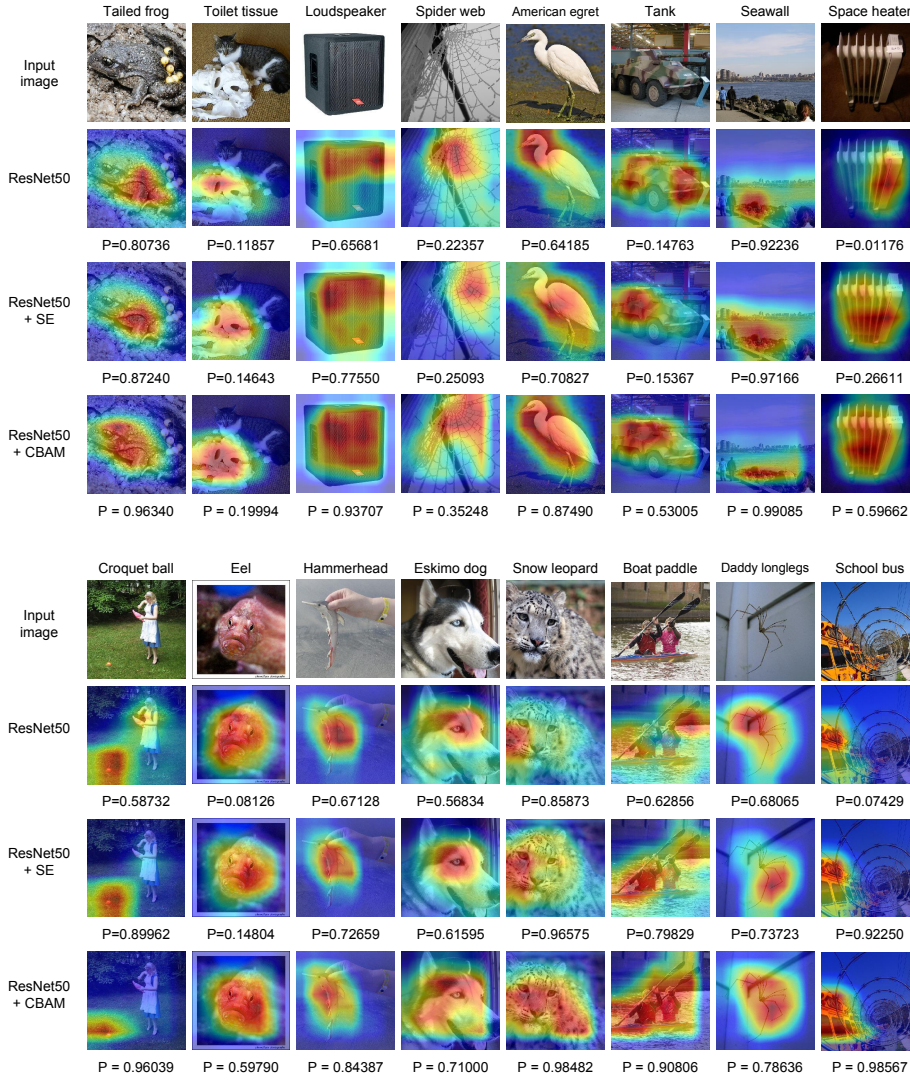


Fig. 5: **Grad-CAM [18] visualization results.** We compare the visualization results of CBAM-integrated network (ResNet50 + CBAM) with baseline (ResNet50) and SE-integrated network (ResNet50 + SE). The grad-CAM visualization is calculated for the last convolutional outputs. The ground-truth label is shown on the top of each input image and P denotes the softmax score of each network for the ground-truth class.

Backbone	Detector	mAP@.5	mAP@.75	mAP@[.5, .95]
ResNet50 [5]	Faster-RCNN [41]	46.2	28.1	27.0
ResNet50 [5] + CBAM	Faster-RCNN [41]	**48.2**	**29.2**	**28.1**
ResNet101 [5]	Faster-RCNN [41]	48.4	30.7	29.1
ResNet101 [5] + CBAM	Faster-RCNN [41]	**50.5**	**32.6**	**30.8**

* all results are reproduced in the PyTorch framework.

Table 6: **Object detection mAP(%) on the MS COCO validation set**. We adopt the Faster R-CNN [41] detection framework and apply our module to the base networks. CBAM boosts mAP@[.5, .95] by 0.9 for both baseline networks.

Backbone	Detector	mAP@.5	Parameters (M)
VGG16 [9]	SSD [39]	77.8	26.5
VGG16 [9]	StairNet [30]	78.9	32.0
VGG16 [9]	StairNet [30] + SE [28]	79.1	32.1
VGG16 [9]	StairNet [30] + CBAM	**79.3**	32.1
MobileNet [34]	SSD [39]	68.1	5.81
MobileNet [34]	StairNet [30]	70.1	5.98
MobileNet [34]	StairNet [30] + SE [28]	70.0	5.99
MobileNet [34]	StairNet [30] + CBAM	**70.5**	6.00

* all results are reproduced in the PyTorch framework.

Table 7: **Object detection mAP(%) on the VOC 2007 test set**. We adopt the StairNet [30] detection framework and apply SE and CBAM to the detectors. CBAM favorably improves all the strong baselines with negligible additional parameters.

scores also increase accordingly. From the observations, we conjecture that the feature refinement process of CBAM eventually leads the networks to utilize given features well.

4.4 MS COCO Object Detection

We conduct object detection on the Microsoft COCO dataset [3]. This dataset involves 80k training images ("2014 train") and 40k validation images ("2014 val"). The average mAP over different IoU thresholds from 0.5 to 0.95 is used for evaluation. According to [38, 39], we trained our model using all the training images as well as a subset of validation images, holding out 5,000 examples for validation. Our training code is based on [40] and we train the network for 490K iterations for fast performance validation. We adopt *Faster-RCNN* [41] as our detection method and ImageNet pre-trained ResNet50 and ResNet101 [5] as our baseline networks. Here we are interested in performance improvement by plugging CBAM to the baseline networks. Since we use the same detection method in all the models, the gains can only be attributed to the enhanced representation power, given by our module CBAM. As shown in the Table 6, we observe significant improvements from the baseline, demonstrating generalization performance of CBAM on other recognition tasks.

4.5 VOC 2007 Object Detection

We further perform experiments on the PASCAL VOC 2007 test set. In this experiment, we apply CBAM to the detectors, while the previous experiments (Table 6) apply our module to the base networks. We adopt the StairNet [30] framework, which is one of the strongest multi-scale method based on the SSD [39]. For the experiment, we reproduce SSD and StairNet in our PyTorch platform in order to estimate performance improvement of CBAM accurately and achieve 77.8% and 78.9% mAP@.5 respectively, which are higher than the original accuracy reported in the original papers. We then place SE [28] and CBAM right before every classifier, refining the final features which are composed of up-sampled global features and corresponding local features before the prediction, enforcing model to adaptively select only the meaningful features. We train all the models on the union set of VOC 2007 trainval and VOC 2012 trainval ("07+12"), and evaluate on the VOC 2007 test set. The total number of training epochs is 250. We use a weight decay of 0.0005 and a momentum of 0.9. In all the experiments, the size of the input image is fixed to 300 for the simplicity.

The experimental results are summarized in Table 7. We can clearly see that CBAM improves the accuracy of all strong baselines with two backbone networks. Note that accuracy improvement of CBAM comes with a negligible parameter overhead, indicating that enhancement is not due to a naive capacity-increment but because of our effective feature refinement. In addition, the result using the light-weight backbone network [34] again shows that CBAM can be an interesting method to low-end devices.

5 Conclusion

We have presented the convolutional bottleneck attention module (CBAM), a new approach to improve representation power of CNN networks. We apply attention-based feature refinement with two distinctive modules, channel and spatial, and achieve considerable performance improvement while keeping the overhead small. For the channel attention, we suggest to use the max-pooled features along with the average-pooled features, leading to produce finer attention than SE [28]. We further push the performance by exploiting the spatial attention. Our final module (CBAM) learns what and where to emphasize or suppress and refines intermediate features effectively. To verify its efficacy, we conducted extensive experiments with various state-of-the-art models and confirmed that CBAM outperforms all the baselines on three different benchmark datasets: ImageNet-1K, MS COCO, and VOC 2007. In addition, we visualize how the module exactly infers given an input image. Interestingly, we observed that our module induces the network to focus on target object properly. We hope CBAM become an important component of various network architectures.

References

1. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2009)
2. Krizhevsky, A., Hinton, G.: Learning multiple layers of features from tiny images
3. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: Proc. of European Conf. on Computer Vision (ECCV). (2014)
4. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11) (1998) 2278–2324
5. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2016)
6. Zagoruyko, S., Komodakis, N.: Wide residual networks. arXiv preprint arXiv:1605.07146 (2016)
7. Xie, S., Girshick, R., Dollár, P., Tu, Z., He, K.: Aggregated residual transformations for deep neural networks. arXiv preprint arXiv:1611.05431 (2016)
8. Szegedy, C., Ioffe, S., Vanhoucke, V., Alemi, A.A.: Inception-v4, inception-resnet and the impact of residual connections on learning. In: Proc. of Association for the Advancement of Artificial Intelligence (AAAI). (2017)
9. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)
10. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2015)
11. Chollet, F.: Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357 (2016)
12. Mnih, V., Heess, N., Graves, A., et al.: Recurrent models of visual attention." advances in neural information processing systems. In: Proc. of Neural Information Processing Systems (NIPS). (2014)
13. Ba, J., Mnih, V., Kavukcuoglu, K.: Multiple object recognition with visual attention. (2014)
14. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. (2014)
15. Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhudinov, R., Zemel, R., Bengio, Y.: Show, attend and tell: Neural image caption generation with visual attention. (2015)
16. Gregor, K., Danihelka, I., Graves, A., Rezende, D.J., Wierstra, D.: Draw: A recurrent neural network for image generation. (2015)
17. Jaderberg, M., Simonyan, K., Zisserman, A., et al.: Spatial transformer networks. In: Proc. of Neural Information Processing Systems (NIPS). (2015)
18. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D.: Grad-cam: Visual explanations from deep networks via gradient-based localization. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. (2017) 618–626
19. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Proc. of Neural Information Processing Systems (NIPS). (2012)
20. Han, D., Kim, J., Kim, J.: Deep pyramidal residual networks. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2017)

21. Huang, G., Liu, Z., Weinberger, K.Q., van der Maaten, L.: Densely connected convolutional networks. arXiv preprint arXiv:1608.06993 (2016)
22. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2016)
23. Itti, L., Koch, C., Niebur, E.: A model of saliency-based visual attention for rapid scene analysis. In: IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI). (1998)
24. Rensink, R.A.: The dynamic representation of scenes. In: Visual cognition 7.1-3. (2000)
25. Corbetta, M., Shulman, G.L.: Control of goal-directed and stimulus-driven attention in the brain. In: Nature reviews neuroscience 3.3. (2002)
26. Larochelle, H., Hinton, G.E.: Learning to combine foveal glimpses with a third-order boltzmann machine. In: Proc. of Neural Information Processing Systems (NIPS). (2010)
27. Wang, F., Jiang, M., Qian, C., Yang, S., Li, C., Zhang, H., Wang, X., Tang, X.: Residual attention network for image classification. arXiv preprint arXiv:1704.06904 (2017)
28. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. arXiv preprint arXiv:1709.01507 (2017)
29. Chen, L., Zhang, H., Xiao, J., Nie, L., Shao, J., Chua, T.S.: Sca-cnn: Spatial and channel-wise attention in convolutional networks for image captioning. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2017)
30. Sanghyun, W., Soonmin, H., So, K.I.: Stairnet: Top-down semantic aggregation for accurate one shot detection. In: Proc. of Winter Conference on Applications of Computer Vision (WACV). (2018)
31. Zeiler, M.D., Fergus, R.: Visualizing and understanding convolutional networks. In: Proc. of European Conf. on Computer Vision (ECCV). (2014)
32. Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., Torralba, A.: Learning deep features for discriminative localization. In: Computer Vision and Pattern Recognition (CVPR), 2016 IEEE Conference on, IEEE (2016) 2921–2929
33. Zagoruyko, S., Komodakis, N.: Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. In: ICLR. (2017)
34. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)
35. : Pytorch. http://pytorch.org/ Accessed: 2017-11-08.
36. He, K., Zhang, X., Ren, S., Sun, J.: Identity mappings in deep residual networks. In: Proc. of European Conf. on Computer Vision (ECCV). (2016)
37. Huang, G., Sun, Y., Liu, Z., Sedra, D., Weinberger, K.Q.: Deep networks with stochastic depth. In: Proc. of European Conf. on Computer Vision (ECCV). (2016)
38. Bell, S., Lawrence Zitnick, C., Bala, K., Girshick, R.: Inside-outside net: Detecting objects in context with skip pooling and recurrent neural networks. In: Proc. of Computer Vision and Pattern Recognition (CVPR). (2016)
39. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., Berg, A.C.: Ssd: Single shot multibox detector. In: Proc. of European Conf. on Computer Vision (ECCV). (2016)
40. Chen, X., Gupta, A.: An implementation of faster rcnn with study for region sampling. arXiv preprint arXiv:1702.02138 (2017)

41. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. In: Proc. of Neural Information Processing Systems (NIPS). (2015)

This is the author's version of an article published in Data Mining and Knowledge Discovery. The final authenticated version is available online at: https://doi.org/10.1007/s10618-019-00619-1.

Deep learning for time series classification: a review

Hassan Ismail Fawaz[1] · **Germain Forestier**[1,2] · **Jonathan Weber**[1] · **Lhassane Idoumghar**[1] · **Pierre-Alain Muller**[1]

Abstract Time Series Classification (TSC) is an important and challenging problem in data mining. With the increase of time series data availability, hundreds of TSC algorithms have been proposed. Among these methods, only a few have considered Deep Neural Networks (DNNs) to perform this task. This is surprising as deep learning has seen very successful applications in the last years. DNNs have indeed revolutionized the field of computer vision especially with the advent of novel deeper architectures such as Residual and Convolutional Neural Networks. Apart from images, sequential data such as text and audio can also be processed with DNNs to reach state-of-the-art performance for document classification and speech recognition. In this article, we study the current state-of-the-art performance of deep learning algorithms for TSC by presenting an empirical study of the most recent DNN architectures for TSC. We give an overview of the most successful deep learning applications in various time series domains under a unified taxonomy of DNNs for TSC. We also provide an open source deep learning framework to the TSC community where we implemented each of the compared approaches and evaluated them on a univariate TSC benchmark (the UCR/UEA archive) and 12 multivariate time series datasets. By training 8,730 deep learning models on 97 time series datasets, we propose the most exhaustive study of DNNs for TSC to date.

Keywords Deep learning · Time series · Classification · Review

1 Introduction

During the last two decades, Time Series Classification (TSC) has been considered as one of the most challenging problems in data mining (Yang and Wu, 2006; Esling and Agon, 2012). With the increase of temporal data availability (Silva et al., 2018), hundreds of TSC algorithms have been proposed since 2015 (Bagnall et al., 2017). Due to their natural temporal ordering, time series data are present in almost every task that requires some sort of human cognitive process (Längkvist et al., 2014). In fact, any classification problem, using data that is registered taking into account some notion of ordering, can be cast as a TSC problem (Cristian Borges Gamboa, 2017). Time series are encountered in many real-world applications ranging from electronic health records (Rajkomar et al., 2018) and human activity recognition (Nweke et al., 2018; Wang et al., 2018) to acoustic scene classification (Nwe et al., 2017) and cyber-security (Susto et al., 2018). In addition, the diversity of the datasets' types in the UCR/UEA archive (Chen et al., 2015b; Bagnall et al., 2017) (the largest repository of time series datasets) shows the different applications of the TSC problem.

✉ H. Ismail Fawaz
E-mail: hassan.ismail-fawaz@uha.fr

[1]IRIMAS, Université Haute Alsace, Mulhouse, France
[2]Faculty of IT, Monash University, Melbourne, Australia

Given the need to accurately classify time series data, researchers have proposed hundreds of methods to solve this task (Bagnall et al., 2017). One of the most popular and traditional TSC approaches is the use of a nearest neighbor (NN) classifier coupled with a distance function (Lines and Bagnall, 2015). Particularly, the Dynamic Time Warping (DTW) distance when used with a NN classifier has been shown to be a very strong baseline (Bagnall et al., 2017). Lines and Bagnall (2015) compared several distance measures and showed that there is no single distance measure that significantly outperforms DTW. They also showed that ensembling the individual NN classifiers (with different distance measures) outperforms all of the ensemble's individual components. Hence, recent contributions have focused on developing ensembling methods that significantly outperforms the NN coupled with DTW (NN-DTW) (Bagnall et al., 2016; Hills et al., 2014; Bostrom and Bagnall, 2015; Lines et al., 2016; Schäfer, 2015; Kate, 2016; Deng et al., 2013; Baydogan et al., 2013). These approaches use either an ensemble of decision trees (random forest) (Baydogan et al., 2013; Deng et al., 2013) or an ensemble of different types of discriminant classifiers (Support Vector Machine (SVM), NN with several distances) on one or several feature spaces (Bagnall et al., 2016; Bostrom and Bagnall, 2015; Schäfer, 2015; Kate, 2016). Most of these approaches significantly outperform the NN-DTW (Bagnall et al., 2017) and share one common property, which is the data transformation phase where time series are transformed into a new feature space (for example using shapelets transform (Bostrom and Bagnall, 2015) or DTW features (Kate, 2016)). This notion motivated the development of an ensemble of 35 classifiers named COTE (Collective Of Transformation-based Ensembles) (Bagnall et al., 2016) that does not only ensemble different classifiers over the same transformation, but instead ensembles different classifiers over different time series representations. Lines et al. (2016, 2018) extended COTE with a Hierarchical Vote system to become HIVE-COTE which has been shown to achieve a significant improvement over COTE by leveraging a new hierarchical structure with probabilistic voting, including two new classifiers and two additional representation transformation domains. HIVE-COTE is currently considered the state-of-the-art algorithm for time series classification (Bagnall et al., 2017) when evaluated over the 85 datasets from the UCR/UEA archive.

To achieve its high accuracy, HIVE-COTE becomes hugely computationally intensive and impractical to run on a real big data mining problem (Bagnall et al., 2017). The approach requires training 37 classifiers as well as cross-validating each hyperparameter of these algorithms, which makes the approach infeasible to train in some situations (Lucas et al., 2018). To emphasize on this infeasibility, note that one of these 37 classifiers is the Shapelet Transform (Hills et al., 2014) whose time complexity is $O(n^2 \cdot l^4)$ with n being the number of time series in the dataset and l being the length of a time series. Adding to the training time's complexity is the high *classification* time of one of the 37 classifiers: the nearest neighbor which needs to scan the training set before taking a decision at test time. Therefore since the nearest neighbor constitutes an essential component of HIVE-COTE, its deployment in a real-time setting is still limited if not impractical. Finally, adding to the huge runtime of HIVE-COTE, the decision taken by 37 classifiers cannot be interpreted easily by domain experts, since researchers already struggle with understanding the decisions taken by an individual classifier.

After having established the current state-of-the-art of non deep classifiers for TSC (Bagnall et al., 2017), we discuss the success of Deep Learning (LeCun et al., 2015) in various classification tasks which motivated the recent utilization of deep learning models for TSC (Wang et al., 2017b). Deep Convolutional Neural Networks (CNNs) have revolutionized the field of computer vision (Krizhevsky et al., 2012). For example, in 2015, CNNs were used to reach human level performance in image recognition tasks (Szegedy et al., 2015). Following the success of deep neural networks (DNNs) in computer vision, a huge amount of research proposed several DNN architectures to solve natural language processing (NLP) tasks such as machine translation (Sutskever

et al., 2014; Bahdanau et al., 2015), learning word embeddings (Mikolov et al., 2013; Mikolov et al., 2013) and document classification (Le and Mikolov, 2014; Goldberg, 2016). DNNs also had a huge impact on the speech recognition community (Hinton et al., 2012; Sainath et al., 2013). Interestingly, we should note that the intrinsic similarity between the NLP and speech recognition tasks is due to the sequential aspect of the data which is also one of the main characteristics of time series data.

In this context, this paper targets the following open questions: *What is the current state-of-the-art DNN for TSC*? *Is there a current DNN approach that reaches state-of-the-art performance for TSC and is less complex than HIVE-COTE*? *What type of DNN architectures works best for the TSC task*? *How does the random initialization affect the performance of deep learning classifiers*? And finally: *Could the black-box effect of DNNs be avoided to provide interpretability*? Given that the latter questions have not been addressed by the TSC community, it is surprising how much recent papers have neglected the possibility that TSC problems could be solved using a pure feature learning algorithm (Neamtu et al., 2018; Bagnall et al., 2017; Lines et al., 2016). In fact, a recent empirical study (Bagnall et al., 2017) evaluated 18 TSC algorithms on 85 time series datasets, none of which was a deep learning model. This shows how much the community lacks of an overview of the current performance of deep learning models for solving the TSC problem (Lines et al., 2018).

In this paper, we performed an empirical comparative study of the most recent deep learning approaches for TSC. With the rise of graphical processing units (GPUs), we show how deep architectures can be trained efficiently to learn hidden discriminative features from raw time series in an end-to-end manner. Similarly to Bagnall et al. (2017), in order to have a fair comparison between the tested approaches, we developed a common framework in Python, Keras (Chollet, 2015) and Tensorflow (Abadi et al., 2015) to train the deep learning models on a cluster of more than 60 GPUs.

In addition to the univariate datasets' evaluation, we tested the approaches on 12 Multivariate Time Series (MTS) datasets (Baydogan, 2015). The multivariate evaluation shows another benefit of deep learning models, which is the ability to handle the curse of dimensionality (Bellman, 2010; Keogh and Mueen, 2017) by leveraging different degrees of smoothness in compositional function (Poggio et al., 2017) as well as the parallel computations of the GPUs (Lu et al., 2015).

As for comparing the classifiers over multiple datasets, we followed the recommendations in Demšar (2006) and used the Friedman test (Friedman, 1940) to reject the null hypothesis. Once we have established that a statistical difference exists within the classifiers' performance, we followed the pairwise post-hoc analysis recommended by Benavoli et al. (2016) where the average rank comparison is replaced by a Wilcoxon signed-rank test (Wilcoxon, 1945) with Holm's alpha correction (Holm, 1979; Garcia and Herrera, 2008). See Section 5 for examples of critical difference diagrams (Demšar, 2006), where a thick horizontal line shows a group of classifiers (a clique) that are not significantly different in terms of accuracy.

In this study, we have trained about 1 billion parameters across 97 univariate and multivariate time series datasets. Despite the fact that a huge number of parameters risks overfitting (Zhang et al., 2017) the relatively small train set in the UCR/UEA archive, our experiments showed that not only DNNs are able to significantly outperform the NN-DTW, but are also able to achieve results that are *not significantly* different than COTE and HIVE-COTE using a deep residual network architecture (He et al., 2016; Wang et al., 2017b). Finally, we analyze how poor random initializations can have a significant effect on a DNN's performance.

The rest of the paper is structured as follows. In Section 2, we provide some background materials concerning the main types of architectures that have been proposed for TSC. In Section 3, the tested architectures are individually presented in details. We describe our experimental open source framework in Section 4. The corresponding results and the discussions are presented in Section 5.

In Section 6, we describe in detail a couple of methods that mitigate the black-box effect of the deep learning models. Finally, we present a conclusion in Section 7 to summarize our findings and discuss future directions.

The main contributions of this paper can be summarized as follows:

- We explain with practical examples, how deep learning can be adapted to one dimensional time series data.
- We propose a unified taxonomy that regroups the recent applications of DNNs for TSC in various domains under two main categories: generative and discriminative models.
- We detail the architecture of nine end-to-end deep learning models designed specifically for TSC.
- We evaluate these models on the univariate UCR/UEA archive benchmark and 12 MTS classification datasets.
- We provide the community with an open source deep learning framework for TSC in which we have implemented all nine approaches.
- We investigate the use of Class Activation Map (CAM) in order to reduce DNNs' black-box effect and explain the different decisions taken by various models.

2 Background

In this section, we start by introducing the necessary definitions for ease of understanding. We then follow by an extensive theoretical background on training DNNs for the TSC task. Finally we present our proposed taxonomy of the different DNNs with examples of their application in various real world data mining problems.

2.1 Time series classification

Before introducing the different types of neural networks architectures, we go through some formal definitions for TSC.

Definition 1 A univariate time series $X = [x_1, x_2, \ldots, x_T]$ is an ordered set of real values. The length of X is equal to the number of real values T.

Definition 2 An M-dimensional MTS, $X = [X^1, X^2, \ldots, X^M]$ consists of M different univariate time series with $X^i \in \mathbb{R}^T$.

Definition 3 A dataset $D = \{(X_1, Y_1), (X_2, Y_2), \ldots, (X_N, Y_N)\}$ is a collection of pairs (X_i, Y_i) where X_i could either be a univariate or multivariate time series with Y_i as its corresponding one-hot label vector. For a dataset containing K classes, the one-hot label vector Y_i is a vector of length K where each element $j \in [1, K]$ is equal to 1 if the class of X_i is j and 0 otherwise.

The task of TSC consists of training a classifier on a dataset D in order to map from the space of possible inputs to a probability distribution over the class variable values (labels).

2.2 Deep learning for time series classification

In this review, we focus on the TSC task (Bagnall et al., 2017) using DNNs which are considered complex machine learning models (LeCun et al., 2015). A general deep learning framework for TSC is depicted in Figure 1. These networks are designed to learn hierarchical representations of the

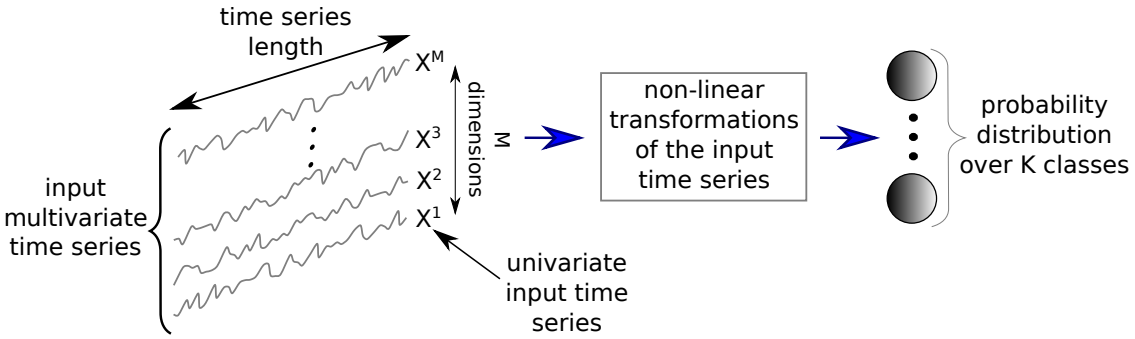


Fig. 1: A unified deep learning framework for time series classification.

data. A deep neural network is a composition of L parametric functions referred to as layers where each layer is considered a representation of the input domain (Papernot and McDaniel, 2018). One layer l_i, such as $i \in 1 \dots L$, contains neurons, which are small units that compute one element of the layer's output. The layer l_i takes as input the output of its previous layer l_{i-1} and applies a non-linearity (such as the sigmoid function) to compute its own output. The behavior of these non-linear transformations is controlled by a set of parameters θ_i for each layer. In the context of DNNs, these parameters are called weights which link the input of the previous layer to the output of the current layer. Hence, given an input x, a neural network performs the following computations to predict the class:

$$f_L(\theta_L, x) = f_{L-1}(\theta_{L-1}, f_{L-2}(\theta_{L-2}, \dots, f_1(\theta_1, x))) \tag{1}$$

where f_i corresponds to the non-linearity applied at layer l_i. For simplicity, we will omit the vector of parameters θ and use $f(x)$ instead of $f(\theta, x)$. This process is also referred to as *feed-forward* propagation in the deep learning literature.

During training, the network is presented with a certain number of known input-output (for example a dataset D). First, the weights are initialized randomly (LeCun et al., 1998b), although a robust alternative would be to take a pre-trained model on a source dataset and fine-tune it on the target dataset (Pan and Yang, 2010). This process is known as transfer learning which we do not study empirically, rather we discuss the transferability of each model with respect to the architecture in Section 3. After the weight's initialization, a forward pass through the model is applied: using the function f the output of an input x is computed. The output is a vector whose components are the estimated probabilities of x belonging to each class. The model's prediction loss is computed using a cost function, for example the negative log likelihood. Then, using gradient descent (LeCun et al., 1998b), the weights are updated in a backward pass to propagate the error. Thus, by iteratively taking a forward pass followed by backpropagation, the model's parameters are updated in a way that minimizes the loss on the training data.

During testing, the probabilistic classifier (the model) is tested on unseen data which is also referred to as the inference phase: a forward pass on this unseen input followed by a class prediction. The prediction corresponds to the class whose probability is maximum. To measure the performance of the model on the test data (generalization), we adopted the accuracy measure (similarly to Bagnall et al. (2017)). One advantage of DNNs over non-probabilistic classifiers (such as NN-DTW) is that a probabilistic decision is taken by the network (Large et al., 2017), thus allowing to measure the confidence of a certain prediction given by an algorithm.

Although there exist many types of DNNs, in this review we focus on three main DNN architectures used for the TSC task: Multi Layer Perceptron (MLP), Convolutional Neural Network

(CNN) and Echo State Network (ESN). These three types of architectures were chosen since they are widely adopted for end-to-end deep learning (LeCun et al., 2015) models for TSC.

2.2.1 Multi Layer Perceptrons

An MLP constitutes the simplest and most traditional architecture for deep learning models. This form of architecture is also known as a fully-connected (FC) network since the neurons in layer l_i are connected to every neuron in layer l_{i-1} with $i \in [1, L]$. These connections are modeled by the weights in a neural network. A general form of applying a non-linearity to an input time series X can be seen in the following equation:

$$A_{l_i} = f(\omega_{l_i} * X + b) \tag{2}$$

with ω_{l_i} being the set of weights with length and number of dimensions identical to X's, b the bias term and A_{l_i} the activation of the neurons in layer l_i. Note that the number of neurons in a layer is considered a hyperparameter.

One impediment from adopting MLPs for time series data is that they do not exhibit any spatial invariance. In other words, each time stamp has its own weight and the temporal information is lost: meaning time series elements are treated independently from each other. For example the set of weights w_d of neuron d contains $T \times M$ values denoting the weight of each time stamp t for each dimension of the M-dimensional input MTS of length T. Then by cascading the layers we obtain a computation graph similar to equation 1.

For TSC, the final layer is usually a discriminative layer that takes as input the activation of the previous layer and gives a probability distribution over the class variables in the dataset. Most deep learning approaches for TSC employ a softmax layer which corresponds to an FC layer with softmax as activation function f and a number of neurons equal to the number of classes in the dataset. Three main useful properties motivate the use of the softmax activation function: the sum of probabilities is guaranteed to be equal to 1, the function is differentiable and it is an adaptation of logistic regression to the multinomial case. The result of a softmax function can be defined as follows:

$$\hat{Y}_j(X) = \frac{e^{A_{L-1} * \omega_j + b_j}}{\sum_{k=1}^{K} e^{A_{L-1} * \omega_k + b_k}} \tag{3}$$

with $\hat{Y}_j$ denoting the probability of X having the class Y equal to class j out of K classes in the dataset. The set of weights w_j (and the corresponding bias b_j) for each class j are linked to each previous activation in layer l_{L-1}.

The weights in equations (2) and (3) should be learned automatically using an optimization algorithm that minimizes an objective cost function. In order to approximate the error of a certain given value of the weights, a differentiable cost (or loss) function that quantifies this error should be defined. The most used loss function in DNNs for the classification task is the categorical cross entropy as defined in the following equation:

$$L(X) = -\sum_{j=1}^{K} Y_j \log \hat{Y}_j \tag{4}$$

with L denoting the loss or cost when classifying the input time series X. Similarly, the average loss when classifying the whole training set of D can be defined using the following equation:

$$J(\Omega) = \frac{1}{N} \sum_{n=1}^{N} L(X_n) \tag{5}$$

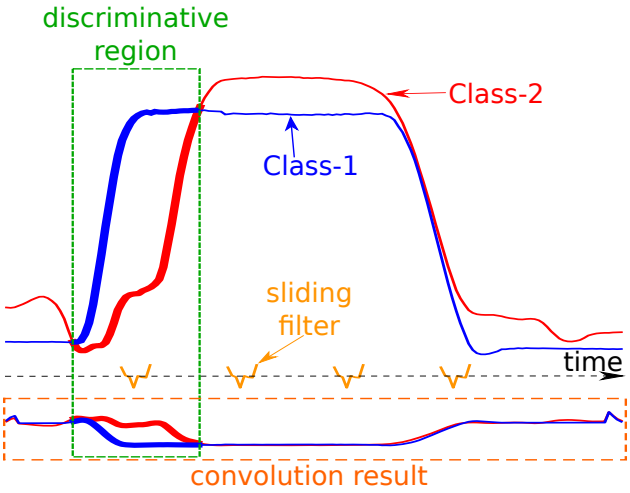


Fig. 2: The result of a applying a learned discriminative convolution on the GunPoint dataset

with Ω denoting the set of weights to be learned by the network (in this case the weights w from equations 2 and 3). The loss function is minimized to learn the weights in Ω using a gradient descent method which is defined using the following equation:

$$\omega = \omega - \alpha \frac{\partial J}{\partial \omega} \mid \forall \ \omega \in \Omega \tag{6}$$

with α denoting the learning rate of the optimization algorithm. By subtracting the partial derivative, the model is actually auto-tuning the parameters ω in order to reach a local minimum of J in case of a non-linear classifier (which is almost always the case for a DNN). We should note that when the partial derivative cannot be directly computed with respect to a certain parameter ω, the chain rule of derivative is employed which is in fact the main idea behind the backpropagation algorithm (LeCun et al., 1998b).

2.2.2 Convolutional Neural Networks

Since AlexNet (Krizhevsky et al., 2012) won the ImageNet competition in 2012, deep CNNs have seen a lot of successful applications in many different domains (LeCun et al., 2015) such as reaching human level performance in image recognition problems (Szegedy et al., 2015) as well as different natural language processing tasks (Sutskever et al., 2014; Bahdanau et al., 2015). Motivated by the success of these CNN architectures in these various domains, researchers have started adopting them for time series analysis (Cristian Borges Gamboa, 2017).

A convolution can be seen as applying and sliding a filter over the time series. Unlike images, the filters exhibit only one dimension (time) instead of two dimensions (width and height). The filter can also be seen as a generic non-linear transformation of a time series. Concretely, if we are convoluting (multiplying) a filter of length 3 with a univariate time series, by setting the filter values to be equal to $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$, the convolution will result in applying a moving average with a sliding window of length 3. A general form of applying the convolution for a centered time stamp t is given in the following equation:

$$C_t = f(\omega * X_{t-l/2:t+l/2} + b) \mid \forall \ t \in [1, T] \tag{7}$$

where C denotes the result of a convolution (dot product $*$) applied on a univariate time series X of length T with a filter ω of length l, a bias parameter b and a final non-linear function f such as

the Rectified Linear Unit (ReLU). The result of a convolution (one filter) on an input time series X can be considered as another univariate time series C that underwent a filtering process. Thus, applying several filters on a time series will result in a multivariate time series whose dimensions are equal to the number of filters used. An intuition behind applying several filters on an input time series would be to learn multiple discriminative features useful for the classification task.

Unlike MLPs, the same convolution (the same filter values w and b) will be used to find the result for all time stamps $t \in [1, T]$. This is a very powerful property (called weight sharing) of the CNNs which enables them to learn filters that are invariant across the time dimension.

When considering an MTS as input to a convolutional layer, the filter no longer has one dimension (time) but also has dimensions that are equal to the number of dimensions of the input MTS.

Finally, instead of setting manually the values of the filter ω, these values should be learned automatically since they depend highly on the targeted dataset. For example, one dataset would have the optimal filter to be equal to $[1, 2, 2]$ whereas another dataset would have an optimal filter equal to $[2, 0, -1]$. By *optimal* we mean a filter whose application will enable the classifier to easily discriminate between the dataset classes (see Figure 2). In order to learn automatically a discriminative filter, the convolution should be followed by a discriminative classifier, which is usually preceded by a *pooling* operation that can either be *local* or *global*.

Local pooling such as *average* or *max* pooling takes an input time series and reduces its length T by aggregating over a sliding window of the time series. For example if the sliding window's length is equal to 3 the resulting pooled time series will have a length equal to $\frac{T}{3}$ - this is only true if the stride is equal to the sliding window's length. With a global pooling operation, the time series will be aggregated over the whole time dimension resulting in a single real value. In other words, this is similar to applying a local pooling with a sliding window's length equal to the length of the input time series. Usually a global aggregation is adopted to reduce drastically the number of parameters in a model thus decreasing the risk of overfitting while enabling the use of CAM to explain the model's decision (Zhou et al., 2016).

In addition to pooling layers, some deep learning architectures include normalization layers to help the network converge quickly. For time series data, the batch normalization operation is performed over each channel therefore preventing the internal covariate shift across one mini-batch training of time series (Ioffe and Szegedy, 2015). Another type of normalization was proposed by Ulyanov et al. (2016) to normalize each instance instead of a per batch basis, thus learning the mean and standard deviation of each training instance for each layer via gradient descent. The latter approach is called instance normalization and mimics learning the z-normalization parameters for the time series training data.

The final discriminative layer takes the representation of the input time series (the result of the convolutions) and give a probability distribution over the class variables in the dataset. Usually, this layer is comprised of a softmax operation similarly to the MLPs. Note that for some approaches, we would have an additional non-linear FC layer before the final softmax layer which increases the number of parameters in a network. Finally in order to train and learn the parameters of a deep CNN, the process is identical to training an MLP: a feed-forward pass followed by backpropagation (LeCun et al., 1998b). An example of a CNN architecture for TSC with three convolutional layers is illustrated in Figure 3.

2.2.3 Echo State Networks

Another popular type of architectures for deep learning models is the Recurrent Neural Network (RNN). Apart from time series forecasting, we found that these neural networks were rarely applied

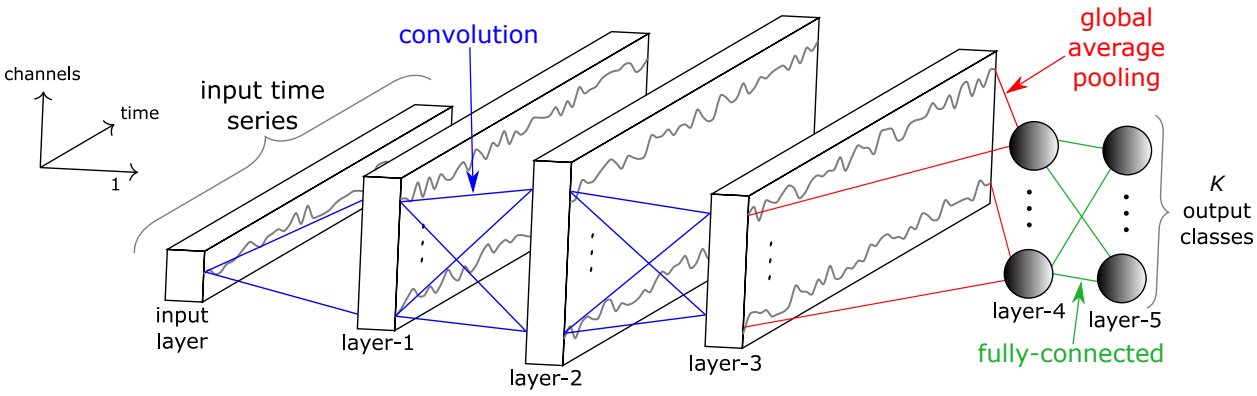


Fig. 3: Fully Convolutional Neural Network architecture

for time series classification which is mainly due to three factors: (1) the type of this architecture is designed mainly to predict an output for each element (time stamp) in the time series (Längkvist et al., 2014); (2) RNNs typically suffer from the vanishing gradient problem due to training on long time series (Pascanu et al., 2012); (3) RNNs are considered hard to train and parallelize which led the researchers to avoid using them for computational reasons (Pascanu et al., 2013).

Given the aforementioned limitations, a relatively recent type of recurrent architecture was proposed for time series: Echo State Networks (ESNs) (Gallicchio and Micheli, 2017). ESNs were first invented by Jaeger and Haas (2004) for time series prediction in wireless communication channels. They were designed to mitigate the challenges of RNNs by eliminating the need to compute the gradient for the hidden layers which reduces the training time of these neural networks thus avoiding the vanishing gradient problem. These hidden layers are initialized randomly and constitutes the *reservoir*: the core of an ESN which is a sparsely connected random RNN. Each neuron in the reservoir will create its own nonlinear activation of the incoming signal. The inter-connected weights inside the reservoir and the input weights are not learned via gradient descent, only the output weights are tuned using a learning algorithm such as logistic regression or Ridge classifier (Hoerl and Kennard, 1970).

To better understand the mechanism of these networks, consider an ESN with input dimensionality M, neurons in the reservoir N_r and an output dimensionality K equal to the number of classes in the dataset. Let $X(t) \in \mathbb{R}^M$, $I(t) \in \mathbb{R}^{N_r}$ and $\hat{Y}(t) \in \mathbb{R}^K$ denote the vectors of the input M-dimensional MTS, the internal (or hidden) state and the output unit activity for time t respectively. Further let $W_{in} \in \mathbb{R}^{N_r \times M}$ and $W \in \mathbb{R}^{N_r \times N_r}$ and $W_{out} \in \mathbb{R}^{C \times N_r}$ denote respectively the weight matrices for the input time series, the internal connections and the output connections as seen in Figure 4. The internal unit activity $I(t)$ at time t is updated using the internal state at time step $t-1$ and the input time series element at time t. Formally the hidden state can be computed using the following recurrence:

$$I(t) = f(W_{in}X(t) + WI(t-1)) \mid \forall\, t \in [1, T] \tag{8}$$

with f denoting an activation function of the neurons, a common choice is $tanh(\cdot)$ applied element-wise (Tanisaro and Heidemann, 2016). The output can be computed according to the following equation:

$$\hat{Y}(t) = W_{out}I(t) \tag{9}$$

thus classifying each time series element $X(t)$. Note that ESNs depend highly on the initial values of the reservoir that should satisfy a pre-determined hyperparameter: the spectral radius. Figure 4 shows an example of an ESN with a univariate input time series to be classified into K classes.

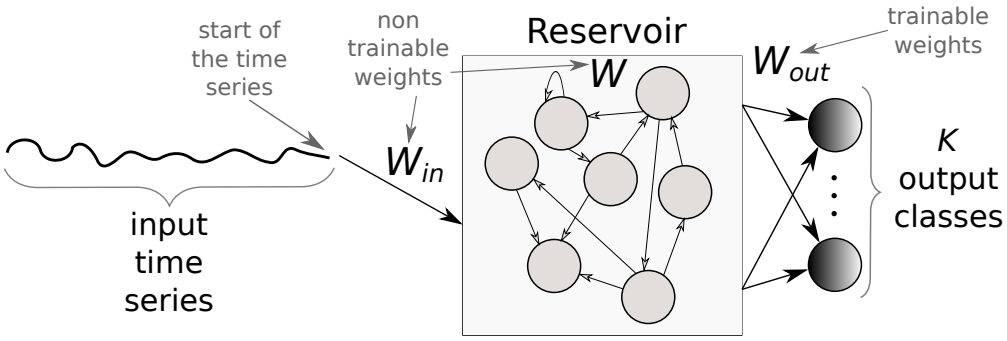


Fig. 4: An Echo State Network architecture for time series classification

Finally, we should note that for all types of DNNs, a set of techniques was proposed by the deep learning community to enhance neural networks' generalization capabilities. Regularization methods such as $l2$-norm weight decay (Bishop, 2006) or Dropout (Srivastava et al., 2014) aim at reducing overfitting by limiting the activation of the neurons. Another popular technique is data augmentation, which tackles the problem of overfitting a small dataset by increasing the number of training instances (Baird, 1992). This method consists in cropping, rotating and blurring images which have been shown to improve the DNNs' performance for computer vision tasks (Zhang et al., 2017). Although two approaches in this survey include a data augmentation technique, the study of its impact on TSC is currently limited (Ismail Fawaz et al., 2018a).

2.3 Generative or discriminative approaches

Deep learning approaches for TSC can be separated into two main categories: the *generative* and the *discriminative* models (as proposed in Längkvist et al. (2014)). We further separate these two groups into sub-groups which are detailed in the following subsections and illustrated in Figure 5.

2.3.1 Generative models

Generative models usually exhibit an unsupervised training step that precedes the learning phase of the classifier (Längkvist et al., 2014). This type of network has been referred to as *Model-based* classifiers in the TSC community (Bagnall et al., 2017). Some of these generative non deep learning approaches include auto-regressive models (Bagnall and Janacek, 2014), hidden Markov models (Kotsifakos and Papapetrou, 2014) and kernel models (Chen et al., 2013).

For all generative approaches, the goal is to find a good representation of time series prior to training a classifier (Längkvist et al., 2014). Usually, to model the time series, classifiers are preceded by an unsupervised pre-training phase such as stacked denoising auto-encoders (SDAEs) (Bengio et al., 2013; Hu et al., 2016). A generative CNN-based model was proposed in Wang et al. (2016b); Mittelman (2015) where the authors introduced a deconvolutional operation followed by an up-sampling technique that helps in reconstructing a multivariate time series. Deep Belief Networks (DBNs) were also used to model the latent features in an unsupervised manner which are then leveraged to classify univariate and multivariate time series (Wang et al., 2017a; Banerjee et al., 2017). In Mehdiyev et al. (2017); Malhotra et al. (2018); Rajan and Thiagarajan (2018), an RNN auto-encoder was designed to first generate the time series then using the learned latent representation, they trained a classifier (such as SVM or Random Forest) on top of these representations to predict the class of a given input time series.

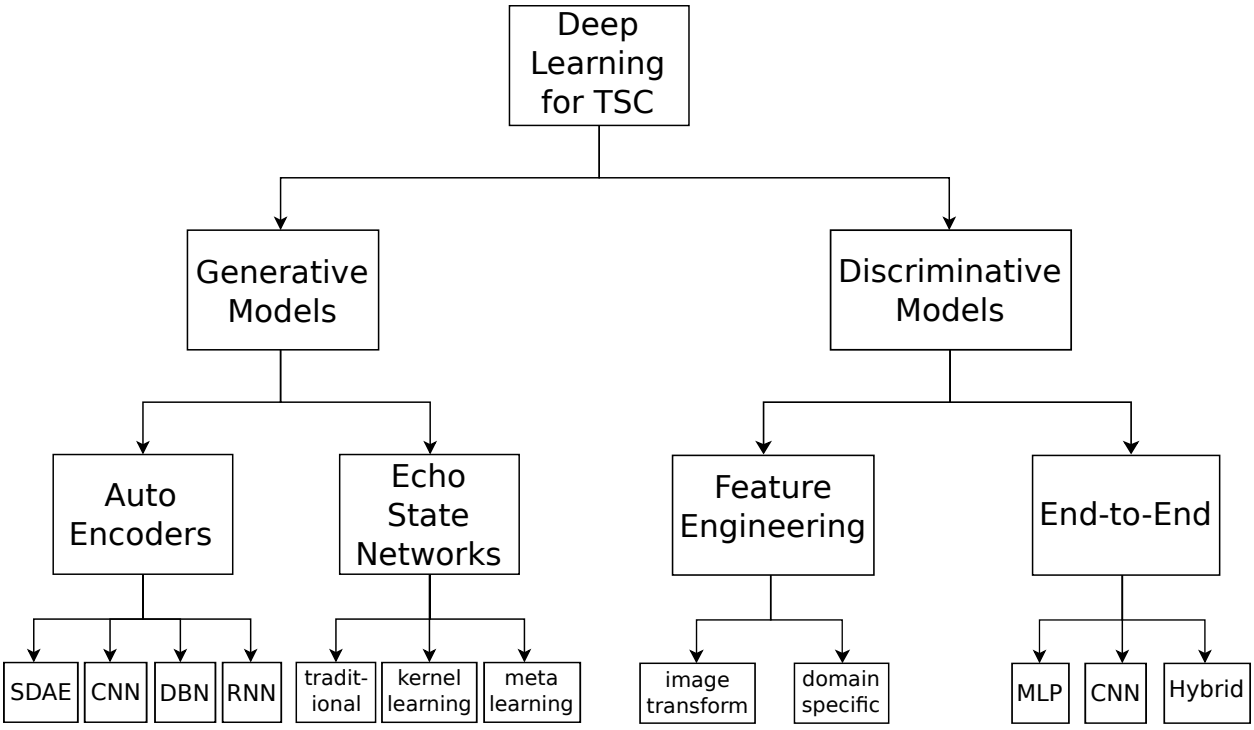


Fig. 5: An overview of the different deep learning approaches for time series classification

Other studies such as in Aswolinskiy et al. (2017); Bianchi et al. (2018); Chouikhi et al. (2018); Ma et al. (2016) used self-predict modeling for time series classification where ESNs were first used to re-construct the time series and then the learned representation in the reservoir space was utilized for classification. We refer to this type of architecture by traditional ESNs in Figure 5. Other ESN-based approaches (Chen et al., 2015a, 2013; Che et al., 2017b) define a kernel over the learned representation followed by an SVM or an MLP classifier. In Gong et al. (2018); Wang et al. (2016), a meta-learning evolutionary-based algorithm was proposed to construct an optimal ESN architecture for univariate and multivariate time series. For more details concerning generative ESN models for TSC, we refer the interested reader to a recent empirical study (Aswolinskiy et al., 2016) that compared classification in reservoir and model-space for both multivariate and univariate time series.

2.3.2 Discriminative models

A discriminative deep learning model is a classifier (or regressor) that directly learns the mapping between the raw input of a time series (or its hand engineered features) and outputs a probability distribution over the class variables in a dataset. Several discriminative deep learning architectures have been proposed to solve the TSC task, but we found that this type of model could be further sub-divided into two groups: (1) deep learning models with hand engineered features and (2) *end-to-end* deep learning models.

The most frequently encountered and computer vision inspired feature extraction method for hand engineering approaches is the transformation of time series into images using specific imaging methods such as Gramian fields (Wang and Oates, 2015b,a), recurrence plots (Hatami et al., 2017; Tripathy and Acharya, 2018) and Markov transition fields (Wang and Oates, 2015). Unlike image transformation, other feature extraction methods are not domain agnostic. These features are first

hand-engineered using some domain knowledge, then fed to a deep learning discriminative classifier. For example in Uemura et al. (2018), several features (such as the velocity) were extracted from sensor data placed on a surgeon's hand in order to determine the skill level during surgical training. In fact, most of the deep learning approaches for TSC with some hand engineered features are present in human activity recognition tasks (Ignatov, 2018). For more details on the different applications of deep learning for human motion detection using mobile and wearable sensor networks, we refer the interested reader to a recent survey (Nweke et al., 2018) where deep learning approaches (with or without hand engineered features) were thoroughly described specifically for the human activity recognition task.

In contrast to feature engineering, *end-to-end* deep learning aims to incorporate the feature learning process while fine-tuning the discriminative classifier (Nweke et al., 2018). Since this type of deep learning approach is domain agnostic and does not include any domain specific pre-processing steps, we decided to further separate these end-to-end approaches using their neural network architectures.

In Wang et al. (2017b); Geng and Luo (2018), an MLP was designed to learn from scratch a discriminative time series classifier. The problem with an MLP approach is that temporal information is lost and the features learned are no longer time-invariant. This is where CNNs are most useful, by learning spatially invariant filters (or features) from raw input time series (Wang et al., 2017b). During our study, we found that CNN is the most widely applied architecture for the TSC problem, which is probably due to their robustness and the relatively small amount of training time compared to complex architectures such as RNNs or MLPs. Several variants of CNNs have been proposed and validated on a subset of the UCR/UEA archive (Chen et al., 2015b; Bagnall et al., 2017) such as Residual Networks (ResNets) (Wang et al., 2017b; Geng and Luo, 2018) which add linear shortcut connections for the convolutional layers potentially enhancing the model's accuracy (He et al., 2016). In Le Guennec et al. (2016); Cui et al. (2016); Wang et al. (2017b); Zhao et al. (2017), traditional CNNs were also validated on the UCR/UEA archive. More recently in Wang et al. (2018), the architectures proposed in Wang et al. (2017b) were modified to leverage a filter initialization technique based on the Daubechies 4 Wavelet values (Rowe and Abbott, 1995). Outside of the UCR/UEA archive, deep learning has reached state-of-the-art performance on several datasets in different domains (Längkvist et al., 2014). For spatio-temporal series forecasting problems, such as meteorology and oceanography, DNNs were proposed in Ziat et al. (2017). Strodthoff and Strodthoff (2019) proposed to detect myocardial infractions from electrocardiography data using deep CNNs. For human activity recognition from wearable sensors, deep learning is replacing the feature engineering approaches (Nweke et al., 2018) where features are no longer hand-designed but rather learned by deep learning models trained through backpropagation. One other type of time series data is present in Electronic Health Records, where a recent generative adversarial network with a CNN (Che et al., 2017a) was trained for risk prediction based on patients historical medical records. In Ismail Fawaz et al. (2018b), CNNs were designed to reach state-of-the-art performance for surgical skills identification. Liu et al. (2018) leveraged a CNN model for multivariate and lag-feature characteristics in order to achieve state-of-the-art accuracy on the Prognostics and Health Management (PHM) 2015 challenge data. Finally, a recent review of deep learning for physiological signals classification revealed that CNNs were the most popular architecture (Faust et al., 2018) for the considered task. We mention one final type of hybrid architectures that showed promising results for the TSC task on the UCR/UEA archive datasets, where mainly CNNs were combined with other types of architectures such as Gated Recurrent Units (Lin and Runger, 2018) and the attention mechanism (Serrà et al., 2018). The reader may have noticed that CNNs appear under Auto Encoders as well as under End-to-End learning in Figure 5. This can be explained by the

fact that CNNs when trained as Auto Encoders have a complete different objective function than CNNs that are trained in an end-to-end fashion.

Now that we have presented the taxonomy for grouping DNNs for TSC, we introduce in the following section the different approaches that we have included in our experimental evaluation. We also explain the motivations behind the selection of these algorithms.

3 Approaches

In this section, we start by explaining the reasons behind choosing discriminative end-to-end approaches for this empirical evaluation. We then describe in detail the nine different deep learning architectures with their corresponding advantages and drawbacks.

3.1 Why discriminative end-to-end approaches ?

As previously mentioned in Section 2, the main characteristic of a generative model is fitting a time series self-predictor whose latent representation is later fed into an off-the-shelf classifier such as Random Forest or SVM. Although these models do sometimes capture the trend of a time series, we decided to leave these generative approaches out of our experimental evaluation for the following reasons:

- This type of method is mainly proposed for tasks other than classification or as part of a larger classification scheme (Bagnall et al., 2017);
- The informal consensus in the literature is that generative models are usually less accurate than direct discriminative models (Bagnall et al., 2017; Nguyen et al., 2017);
- The implementation of these models is usually more complicated than for discriminative models since it introduces an additional step of fitting a time series generator - this has been considered a barrier with most approaches whose code was not publicly available such as Gong et al. (2018); Che et al. (2017b); Chouikhi et al. (2018); Wang et al. (2017a);
- The accuracy of these models depends highly on the chosen off-the-shelf classifier which is sometimes not even a neural network classifier (Rajan and Thiagarajan, 2018).

Given the aforementioned limitations for generative models, we decided to limit our experimental evaluation to discriminative deep learning models for TSC. In addition of restricting the study to discriminative models, we decided to only consider end-to-end approaches, thus further leaving classifiers that incorporate feature engineering out of our empirical evaluation. We made this choice because we believe that the main goal of deep learning approaches is to remove the bias due to manually designed features (Ordóñez and Roggen, 2016), thus enabling the network to learn the most discriminant useful features for the classification task. This has also been the consensus in the human activity recognition literature, where the accuracy of deep learning methods depends highly on the quality of the extracted features (Nweke et al., 2018). Finally, since our goal is to provide an empirical study of domain agnostic deep learning approaches for any TSC task, we found that it is best to compare models that do not incorporate any domain knowledge into their approach.

As for why we chose the nine approaches (described in the next Section), it is first because among all the discriminative end-to-end deep learning models for TSC, we wanted to cover a wide range of architectures such as CNNs, Fully CNNs, MLPs, ResNets, ESNs, etc. Second, since we cannot cover an empirical study of all approaches validated in all TSC domains, we decided to only include approaches that were validated on the whole (or a subset of) the univariate time series

UCR/UEA archive (Chen et al., 2015b; Bagnall et al., 2017) and/or on the MTS archive (Baydogan, 2015). Finally, we chose to work with approaches that do not try to solve a sub task of the TSC problem such as in Geng and Luo (2018) where CNNs were modified to classify imbalanced time series datasets. To justify this choice, we emphasize that imbalanced TSC problems can be solved using several techniques such as data augmentation (Ismail Fawaz et al., 2018a) and modifying the class weights (Geng and Luo, 2018). However, any deep learning algorithm can benefit from this type of modification. Therefore if we did include modifications for solving imbalanced TSC tasks, it would be much harder to determine if it is the choice of the deep learning classifier or the modification itself that improved the accuracy of the model. Another sub task that has been at the center of recent studies is early time series classification (Wang et al., 2016a) where deep CNNs were modified to include an early classification of time series. More recently, a deep reinforcement learning approach was also proposed for the early TSC task (Martinez et al., 2018). For further details, we refer the interested reader to a recent survey on deep learning for *early* time series classification (Santos and Kern, 2017).

3.2 Compared approaches

After having presented an overview over the recent deep learning approaches for time series classification, we present the nine architectures that we have chosen to compare in this paper.

3.2.1 Multi Layer Perceptron

The MLP, which is the most traditional form of DNNs, was proposed in Wang et al. (2017b) as a baseline architecture for TSC. The network contains 4 layers in total where each one is fully connected to the output of its previous layer. The final layer is a softmax classifier, which is fully connected to its previous layer's output and contains a number of neurons equal to the number of classes in a dataset. All three hidden FC layers are composed of 500 neurons with ReLU as the activation function. Each layer is preceded by a dropout operation (Srivastava et al., 2014) with a rate equal to 0.1, 0.2, 0.2 and 0.3 for respectively the first, second, third and fourth layer. Dropout is one form of regularization that helps in preventing overfitting (Srivastava et al., 2014). The dropout rate indicates the percentage of neurons that are deactivated (set to zero) in a feed forward pass during training.

MLP does not have any layer whose number of parameters is invariant across time series of different lengths (denoted by $\#invar$ in Table 1) which means that the transferability of the network is not trivial: the number of parameters (weights) of the network depends directly on the length of the input time series.

3.2.2 Fully Convolutional Neural Network

Fully Convolutional Neural Networks (FCNs) were first proposed in Wang et al. (2017b) for classifying univariate time series and validated on 44 datasets from the UCR/UEA archive. FCNs are mainly convolutional networks that do not contain any local pooling layers which means that the length of a time series is kept unchanged throughout the convolutions. In addition, one of the main characteristics of this architecture is the replacement of the traditional final FC layer with a Global Average Pooling (GAP) layer which reduces drastically the number of parameters in a neural network while enabling the use of the CAM (Zhou et al., 2016) that highlights which parts of the input time series contributed the most to a certain classification.

The architecture proposed in Wang et al. (2017b) is first composed of three convolutional blocks where each block contains three operations: a convolution followed by a batch normalization (Ioffe and Szegedy, 2015) whose result is fed to a ReLU activation function. The result of the third convolutional block is averaged over the whole time dimension which corresponds to the GAP layer. Finally, a traditional softmax classifier is fully connected to the GAP layer's output.

All convolutions have a stride equal to 1 with a zero padding to preserve the exact length of the time series after the convolution. The first convolution contains 128 filters with a filter length equal to 8, followed by a second convolution of 256 filters with a filter length equal to 5 which in turn is fed to a third and final convolutional layer composed of 128 filters, each one with a length equal to 3.

We can see that FCN does not hold any pooling nor a regularization operation. In addition, one of the advantages of FCNs is the invariance (denoted by *#invar* in Table 1) in the number of parameters for 4 layers (out of 5) across time series of different lengths. This invariance (due to using GAP) enables the use of a transfer learning approach where one can train a model on a certain source dataset and then fine-tune it on the target dataset (Ismail Fawaz et al., 2018c).

3.2.3 Residual Network

The third and final proposed architecture in Wang et al. (2017b) is a relatively deep Residual Network (ResNet). For TSC, this is the deepest architecture with 11 layers of which the first 9 layers are convolutional followed by a GAP layer that averages the time series across the time dimension. The main characteristic of ResNets is the shortcut residual connection between consecutive convolutional layers. Actually, the difference with the usual convolutions (such as in FCNs) is that a linear shortcut is added to link the output of a residual block to its input thus enabling the flow of the gradient directly through these connections, which makes training a DNN much easier by reducing the vanishing gradient effect (He et al., 2016).

The network is composed of three residual blocks followed by a GAP layer and a final softmax classifier whose number of neurons is equal to the number of classes in a dataset. Each residual block is first composed of three convolutions whose output is added to the residual block's input and then fed to the next layer. The number of filters for all convolutions is fixed to 64, with the ReLU activation function that is preceded by a batch normalization operation. In each residual block, the filter's length is set to 8, 5 and 3 respectively for the first, second and third convolution.

Similarly to the FCN model, the layers (except the final one) in the ResNet architecture have an invariant number of parameters across different datasets. That being said, we can easily pre-train a model on a source dataset, then transfer and fine-tune it on a target dataset without having to modify the hidden layers of the network. As we have previously mentioned and since this type of transfer learning approach can give an advantage for certain types of architecture, we leave the exploration of this area of research for future work. The ResNet architecture proposed by Wang et al. (2017b) is depicted in Figure 6.

3.2.4 Encoder

Originally proposed by Serrà et al. (2018), Encoder is a hybrid deep CNN whose architecture is inspired by FCN (Wang et al., 2017b) with a main difference where the GAP layer is replaced with an attention layer. In Serrà et al. (2018), two variants of Encoder were proposed: the first approach was to train the model from scratch in an end-to-end fashion on a target dataset while the second one was to pre-train this same architecture on a source dataset and then fine-tune it on a target dataset. The latter approach reached higher accuracy thus benefiting from the transfer learning

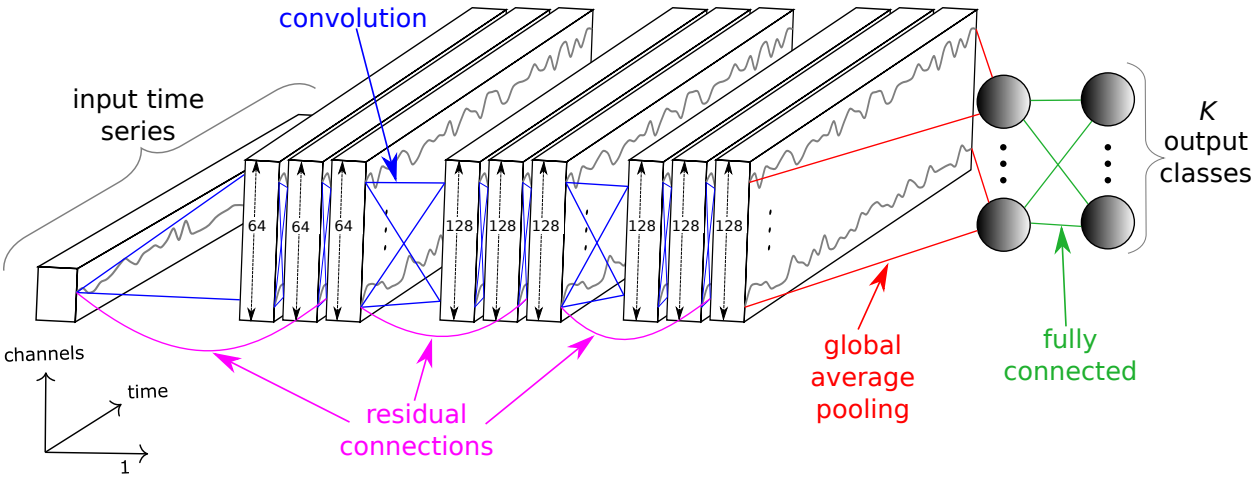


Fig. 6: The Residual Network's architecture for time series classification.

technique. On the other hand, since almost all approaches can benefit to certain degree from a transfer learning method, we decided to implement only the end-to-end approach (training from scratch) which already showed high performance in the author's original paper.

Similarly to FCN, the first three layers are convolutional with some relatively small modifications. The first convolution is composed of 128 filters of length 5; the second convolution is composed of 256 filters of length 11; the third convolution is composed of 512 filters of length 21. Each convolution is followed by an instance normalization operation (Ulyanov et al., 2016) whose output is fed to the Parametric Rectified Linear Unit (PReLU) (He et al., 2015) activation function. The output of PReLU is followed by a dropout operation (with a rate equal to 0.2) and a final max pooling of length 2. The third convolutional layer is fed to an attention mechanism (Bahdanau et al., 2015) that enables the network to learn which parts of the time series (in the time domain) are important for a certain classification. More precisely, to implement this technique, the input MTS is multiplied with a second MTS of the same length and number of channels, except that the latter has gone through the softmax function. Each element in the second MTS will act as a weight for the first MTS, thus enabling the network to learn the importance of each element (time stamp). Finally, a traditional softmax classifier is fully connected to the latter layer with a number of neurons equal to the number of classes in the dataset.

In addition to replacing the GAP layer with the attention layer, Encoder differs from FCN in three main core changes: (1) the PReLU activation function where an additional parameter is added for each filter to enable learning the slope of the function, (2) the dropout regularization technique and (3) the max pooling operation. One final note is that the careful design of Encoder's attention mechanism enabled the invariance across all layers which encouraged the authors to implement a transfer learning approach.

3.2.5 Multi-scale Convolutional Neural Network

Originally proposed by Cui et al. (2016), Multi-scale Convolutional Neural Network (MCNN) is the earliest approach to validate an end-to-end deep learning architecture on the UCR Archive. MCNN's architecture is very similar to a traditional CNN model: with two convolutions (and max pooling) followed by an FC layer and a final softmax layer. On the other hand, this approach is very complex with its heavy data pre-processing step. Cui et al. (2016) were the first to introduce the Window Slicing (WS) method as a data augmentation technique. WS slides a window over the

input time series and extract subsequences, thus training the network on the extracted subsequences instead of the raw input time series. Following the extraction of a subsequence from an input time series using the WS method, a transformation stage is used. More precisely, prior to any training, the subsequence will undergo three transformations: (1) identity mapping; (2) down-sampling and (3) smoothing; thus, transforming a univariate input time series into a multivariate input time series. This heavy pre-processing would question the end-to-end label of this approach, but since their method is generic enough we incorporated it into our developed framework.

For the first transformation, the input subsequence is left unchanged and the raw subsequence will be used as an input for an independent first convolution. The down-sampling technique (second transformation) will result in shorter subsequences with different lengths which will then undergo another independent convolutions in parallel to the first convolution. As for the smoothing technique (third transformation), the result is a smoothed subsequence whose length is equal to the input raw subsequence which will also be fed to an independent convolution in parallel to the first and the second convolutions.

The output of each convolution in the first convolutional stage is concatenated to form the input of the subsequent convolutional layer. Following this second layer, an FC layer is deployed with 256 neurons using the sigmoid activation function. Finally, the usual softmax classifier is used with a number of neurons equal to the number of classes in the dataset.

Note that each convolution in this network uses 256 filters with the sigmoid as an activation function, followed by a max pooling operation. Two architecture hyperparameters are cross-validated, using a grid search on an unseen split from the training set: the filter length and the pooling factor which determines the pooling size for the max pooling operation. The total number of layers in this network is 4, out of which only the first two convolutional layers are invariant (transferable). Finally, since the WS method is also used at test time, the class of an input time series is determined by a majority vote over the extracted subsequences' predicted labels.

3.2.6 Time Le-Net

Time Le-Net (t-LeNet) was originally proposed by Le Guennec et al. (2016) and inspired by the great performance of LeNet's architecture for the document recognition task (LeCun et al., 1998a). This model can be considered as a traditional CNN with two convolutions followed by an FC layer and a final softmax classifier. There are two main differences with the FCNs: (1) an FC layer and (2) local max-pooling operations. Unlike GAP, local pooling introduces invariance to small perturbations in the activation map (the result of a convolution) by taking the maximum value in a local pooling window. Therefore for a pool size equal to 2, the pooling operation will halve the length of a time series by taking the maximum value between each two time steps.

For both convolutions, the ReLU activation function is used with a filter length equal to 5. For the first convolution, 5 filters are used and followed by a max pooling of length equal to 2. The second convolution uses 20 filters followed by a max pooling of length equal to 4. Thus, for an input time series of length l, the resulting output of these two convolutions will divide the length of the time series by $8 = 4 \times 2$. The convolutional blocks are followed by a non-linear fully connected layer which is composed of 500 neurons, each one using the ReLU activation function. Finally, similarly to all previous architectures, the number of neurons in the final softmax classifier is equal to the number of classes in a dataset.

Unlike ResNet and FCN, this approach does not have much invariant layers (2 out of 4) due to the use of an FC layer instead of a GAP layer, thus increasing drastically the number of parameters needed to be trained which also depends on the length of the input time series. Thus, the

transferability of this network is limited to the first two convolutions whose number of parameters depends solely on the number and length of the chosen filters.

We should note that t-LeNet is one of the approaches adopting a data augmentation technique to prevent overfitting especially for the relatively small time series datasets in the UCR/UEA archive. Their approach uses two data augmentation techniques: WS and Window Warping (WW). The former method is identical to MCNN's data augmentation technique originally proposed in Cui et al. (2016). As for the second data augmentation technique, WW employs a warping technique that squeezes or dilates the time series. In order to deal with multi-length time series the WS method is adopted to ensure that subsequences of the same length are extracted for training the network. Therefore, a given input time series of length l is first dilated ($\times 2$) then squeezed ($\times \frac{1}{2}$) resulting in three time series of length l, $2l$ and $\frac{1}{2}l$ that are fed to WS to extract equal length subsequences for training. Not that in their original paper (Le Guennec et al., 2016), WS' length is set to $0.9l$. Finally similarly to MCNN, since the WS method is also used at test time, a majority vote over the extracted subsequences' predicted labels is applied.

3.2.7 Multi Channel Deep Convolutional Neural Network

Multi Channel Deep Convolutional Neural Network (MCDCNN) was originally proposed and validated on two multivariate time series datasets (Zheng et al., 2014, 2016). The proposed architecture is mainly a traditional deep CNN with one modification for MTS data: the convolutions are applied independently (in parallel) on each dimension (or channel) of the input MTS.

Each dimension for an input MTS will go through two convolutional stages with 8 filters of length 5 with ReLU as the activation function. Each convolution is followed by a max pooling operation of length 2. The output of the second convolutional stage for all dimensions is concatenated over the channels axis and then fed to an FC layer with 732 neurons with ReLU as the activation function. Finally, the softmax classifier is used with a number of neurons equal to the number of classes in the dataset. By using an FC layer before the softmax classifier, the transferability of this network is limited to the first and second convolutional layers.

3.2.8 Time Convolutional Neural Network

Time-CNN approach was originally proposed by Zhao et al. (2017) for both univariate and multivariate TSC. There are three main differences compared to the previously described networks. The first characteristic of Time-CNN is the use of the mean squared error (MSE) instead of the traditional categorical cross-entropy loss function, which has been used by all the deep learning approaches we have mentioned so far. Hence, instead of a softmax classifier, the final layer is a traditional FC layer with sigmoid as the activation function, which does not guarantee a sum of probabilities equal to 1. Another difference to traditional CNNs is the use of a local *average* pooling operation instead of local *max* pooling. In addition, unlike MCDCNN, for MTS data they apply one convolution for all the dimensions of a multivariate classification task. Another unique characteristic of this architecture is that the final classifier is fully connected directly to the output of the second convolution, which removes completely the GAP layer without replacing it with an FC non-linear layer.

The network is composed of two consecutive convolutional layers with respectively 6 and 12 filters followed by a local average pooling operation of length 3. The convolutions adopt the sigmoid as the activation function. The network's output consists of an FC layer with a number of neurons equal to the number of classes in the dataset.

3.2.9 Time Warping Invariant Echo State Network

Time Warping Invariant Echo State Network (TWIESN) (Tanisaro and Heidemann, 2016) is the only non-convolutional *recurrent* architecture tested and re-implemented in our study. Although ESNs were originally proposed for time series forecasting, Tanisaro and Heidemann (2016) proposed a variant of ESNs that uses directly the raw input time series and predicts a probability distribution over the class variables.

In fact, for each element (time stamp) in an input time series, the reservoir space is used to project this element into a higher dimensional space. Thus, for a univariate time series, the element is projected into a space whose dimensions are inferred from the size of the reservoir. Then for each element, a Ridge classifier (Hoerl and Kennard, 1970) is trained to predict the class of each time series element. During test time, for each element of an input test time series, the already trained Ridge classifier will output a probability distribution over the classes in a dataset. Then the a posteriori probability for each class is averaged over all time series elements, thus assigning for each input test time series the label for which the averaged probability is maximum. Following the original paper of Tanisaro and Heidemann (2016), using a grid-search on an unseen split (20%) from the training set, we optimized TWIESN's three hyperparameters: the reservoir's size, sparsity and spectral radius.

3.3 Hyperparameters

Tables 1 and 2 show respectively the architecture and the optimization hyperparameters for all the described approaches except for TWIESN, since its hyperparameters are not compatible with the eight other algorithms' hyperparameters. We should add that for all the other deep learning classifiers (with TWIESN omitted), a model checkpoint procedure was performed either on the training set or a validation set (split from the training set). Which means that if the model is trained for 1000 epochs, the best one on the validation set (or the train set) loss will be chosen for evaluation. This characteristic is included in Table 2 under the "valid" column. In addition to the model checkpoint procedure, we should note that all deep learning models in Table 1 were initialized randomly using Glorot's uniform initialization method (Glorot and Bengio, 2010). All models were optimized using a variant of Stochastic Gradient Descent (SGD) such as Adam (Kingma and Ba, 2015) and AdaDelta (Zeiler, 2012). We should add that for FCN, ResNet and MLP proposed in Wang et al. (2017b), the learning rate was reduced by a factor of 0.5 each time the model's training loss has not improved for 50 consecutive epochs (with a minimum value equal to 0.0001). One final note is that we have no way of controlling the fact that those described architectures might have been overfitted for the UCR/UEA archive and designed empirically to achieve a high performance, which is always a risk when comparing classifiers on a benchmark (Bagnall et al., 2017). We therefore think that challenges where only the training data is publicly available and the testing data are held by the challenge organizer for evaluation might help in mitigating this problem.

4 Experimental setup

We first start by presenting the datasets' properties we have adopted in this empirical study. We then describe in details our developed open-source framework of deep learning for time series classification.

Methods	Architecture							
	#Layers	#Conv	#Invar	Normalize	Pooling	Feature	Activate	Regularize
MLP	4	0	0	None	None	FC	ReLU	Dropout
FCN	5	3	4	Batch	None	GAP	ReLU	None
ResNet	11	9	10	Batch	None	GAP	ReLU	None
Encoder	5	3	4	Instance	Max	Att	PReLU	Dropout
MCNN	4	2	2	None	Max	FC	Sigmoid	None
t-LeNet	4	2	2	None	Max	FC	ReLU	None
MCDCNN	4	2	2	None	Max	FC	ReLU	None
Time-CNN	3	2	2	None	Avg	Conv	Sigmoid	None

Table 1: Architecture's hyperparameters for the deep learning approaches

Methods	Optimization						
	Algorithm	Valid	Loss	Epochs	Batch	Learning rate	Decay
MLP	AdaDelta	Train	Entropy	5000	16	1.0	0.0
FCN	Adam	Train	Entropy	2000	16	0.001	0.0
ResNet	Adam	Train	Entropy	1500	16	0.001	0.0
Encoder	Adam	Train	Entropy	100	12	0.00001	0.0
MCNN	Adam	$\text{Split}_{20\%}$	Entropy	200	256	0.1	0.0
t-LeNet	Adam	Train	Entropy	1000	256	0.01	0.005
MCDCNN	SGD	$\text{Split}_{33\%}$	Entropy	120	16	0.01	0.0005
Time-CNN	Adam	Train	MSE	2000	16	0.001	0.0

Table 2: Optimization's hyperparameters for the deep learning approaches

4.1 Datasets

4.1.1 Univariate archive

In order to have a thorough and fair experimental evaluation of all approaches, we tested each algorithm on the whole UCR/UEA archive (Chen et al., 2015b; Bagnall et al., 2017) which contains 85 univariate time series datasets. The datasets possess different varying characteristics such as the length of the series which has a minimum value of 24 for the ItalyPowerDemand dataset and a maximum equal to 2,709 for the HandOutLines dataset. One important characteristic that could impact the DNNs' accuracy is the size of the training set which varies between 16 and 8926 for respectively DiatomSizeReduction and ElectricDevices datasets. We should note that twenty datasets contains a relatively small training set (50 or fewer instances) which surprisingly was not an impediment for obtaining high accuracy when applying a very deep architecture such as ResNet. Furthermore, the number of classes varies between 2 (for 31 datasets) and 60 (for the ShapesAll dataset). Note that the time series in this archive are already z-normalized (Bagnall et al., 2017).

Other than the fact of being publicly available, the choice of validating on the UCR/UEA archive is motivated by having datasets from different domains which have been broken down into seven different categories (Image Outline, Sensor Readings, Motion Capture, Spectrographs, ECG, Electric Devices and Simulated Data) in Bagnall et al. (2017). Further statistics, which we do not repeat for brevity, were conducted on the UCR/UEA archive in Bagnall et al. (2017).

Dataset	Old length	New length	Classes	Dimensions	Train	Test
ArabicDigits	4-93	93	10	13	6600	2200
AUSLAN	45-136	136	95	22	1140	1425
CharacterTrajectories	109-205	205	20	3	300	2558
CMUsubject16	127-580	580	2	62	29	29
ECG	39-152	152	2	2	100	100
JapaneseVowels	7-29	29	9	12	270	370
KickVsPunch	274-841	841	2	62	16	10
Libras	45-45	45	15	2	180	180
Outflow	50-997	997	2	4	803	534
UWave	315-315	315	8	3	200	4278
Wafer	104-198	198	2	6	298	896
WalkVsRun	128-1918	1919	2	62	28	16

Table 3: The multivariate time series classification archive.

4.1.2 Multivariate archive

We also evaluated all deep learning models on Baydogan's archive (Baydogan, 2015) that contains 13 MTS classification datasets. For memory usage limitations over a single GPU, we left the MTS dataset Performance Measurement System (PeMS) out of our experimentations. This archive also exhibits datasets with different characteristics such as the length of the time series which, unlike the UCR/UEA archive, varies among the same dataset. This is due to the fact that the datasets in the UCR/UEA archive are already re-scaled to have an equal length among one dataset (Bagnall et al., 2017).

In order to solve the problem of unequal length time series in the MTS archive we decided to linearly interpolate the time series of each dimension for every given MTS, thus each time series will have a length equal to the longest time series' length. This form of pre-processing has also been used by Ratanamahatana and Keogh (2005) to show that the length of a time series is not an issue for TSC problems. This step is very important for deep learning models whose architecture depends on the length of the input time series (such as a MLP) and for parallel computation over the GPUs. We did not z-normalize any time series, but we emphasize that this traditional pre-processing step (Bagnall et al., 2017) should be further studied for univariate as well as multivariate data, especially since normalization is known to have a huge effect on DNNs' learning capabilities (Zhang et al., 2017). Note that this process is only true for the MTS datasets whereas for the univariate benchmark, the time series are already z-normalized. Since the data is pre-processed using the same technique for all nine classifiers, we can safely say, to some extent, that the accuracy improvement of certain models can be solely attributed to the model itself. Table 3 shows the different characteristics of each MTS dataset used in our experiments.

4.2 Experiments

For each dataset in both archives (97 datasets in total), we have trained the nine deep learning models (presented in the previous Section) with 10 different runs each. Each run uses the same original train/test split in the archive but with a different random weight initialization, which enables us to take the mean accuracy over the 10 runs in order to reduce the bias due to the weights' initial values. In total, we have performed 8730 experiments for the 85 univariate and 12 multivariate TSC datasets. Thus, given the huge number of models that needed to be trained, we

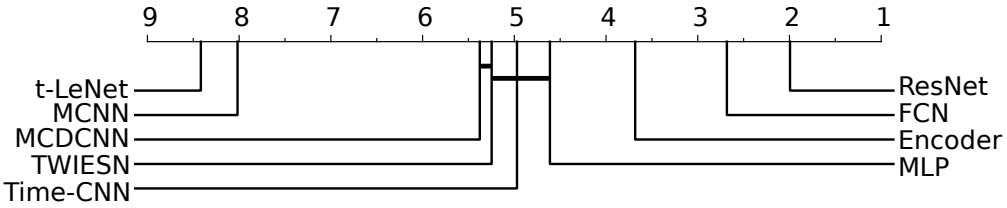


Fig. 7: Critical difference diagram showing pairwise statistical difference comparison of nine deep learning classifiers on the univariate UCR/UEA time series classification archive.

ran our experiments on a cluster of 60 GPUs. These GPUs were a mix of four types of Nvidia graphic cards: GTX 1080 Ti, Tesla K20, K40 and K80. The total sequential running time was approximately 100 days, that is if the computation has been done on a single GPU. However, by leveraging the cluster of 60 GPUs, we managed to obtain the results in less than one month. We implemented our framework using the open source deep learning library Keras (Chollet, 2015) with the Tensorflow (Abadi et al., 2015) back-end[1].

Following Lucas et al. (2018); Forestier et al. (2017); Petitjean et al. (2016); Grabocka et al. (2014) we used the mean accuracy measure averaged over the 10 runs on the test set. When comparing with the state-of-the-art results published in Bagnall et al. (2017) we averaged the accuracy using the median test error. Following the recommendation in Demšar (2006) we used the Friedman test (Friedman, 1940) to reject the null hypothesis. Then we performed the pairwise post-hoc analysis recommended by Benavoli et al. (2016) where the average rank comparison is replaced by a Wilcoxon signed-rank test (Wilcoxon, 1945) with Holm's alpha (5%) correction (Holm, 1979; Garcia and Herrera, 2008). To visualize this type of comparison we used a critical difference diagram proposed by Demšar (2006), where a thick horizontal line shows a group of classifiers (a clique) that are not-significantly different in terms of accuracy.

5 Results

In this section, we present the accuracies for each one of the nine approaches. All accuracies are absolute and not relative to each other that is if we claim algorithm A is 5% better than algorithm B, this means that the average accuracy is 0.05 higher for algorithm A than B.

5.1 Results for univariate time series

We provide on the companion GitHub repository the raw accuracies over the 10 runs for the nine deep learning models we have tested on the 85 univariate time series datasets: the UCR/UEA archive (Chen et al., 2015b; Bagnall et al., 2017). The corresponding critical difference diagram is shown in Figure 7. The ResNet significantly outperforms the other approaches with an average rank of almost 2. ResNet wins on 50 problems out of 85 and significantly outperforms the FCN architecture. This is in contrast to the original paper's results where FCN was found to outperform ResNet on 18 out of 44 datasets, which shows the importance of validating on a larger archive in order to have a robust statistical significance.

We believe that the success of ResNet is highly due to its deep flexible architecture. First of all, our findings are in agreement with the deep learning for computer vision literature where deeper

[1] The implementations are available on https://github.com/hfawaz/dl-4-tsc

neural networks are much more successful than shallower architectures (He et al., 2016). In fact, in a space of 4 years, neural networks went from 7 layers in AlexNet 2012 (Krizhevsky et al., 2012) to *1000* layers for ResNet 2016 (He et al., 2016). These types of deep architectures generally need a huge amount of data in order to generalize well on unseen examples (He et al., 2016). Although the datasets used in our experiments are relatively small compared to the billions of labeled images (such as ImageNet (Russakovsky et al., 2015) and OpenImages (Krasin et al., 2017) challenges), the deepest networks did reach competitive accuracies on the UCR/UEA archive benchmark.

We give two potential reasons for this high generalization capabilities of deep CNNs on the TSC tasks. First, having seen the success of convolutions in classification tasks that require learning features that are spatially invariant in a *two* dimensional space (such as width and height in images), it is only natural to think that discovering patterns in a *one* dimensional space (time) should be an easier task for CNNs thus requiring less data to learn from. The other more direct reason behind the high accuracies of deep CNNs on time series data is its success in other sequential data such as speech recognition (Hinton et al., 2012) and sentence classification (Kim, 2014) where text and audio, similarly to time series data, exhibit a natural temporal ordering.

The MCNN and t-LeNet architectures yielded very low accuracies with only one win for the Earthquakes dataset. The main common idea between both of these approaches is extracting subsequences to augment the training data. Therefore the model learns to classify a time series from a shorter subsequence instead of the whole one, then with a majority voting scheme the time series at test time are assigned a class label. The poor performances (worst average ranks) for these two approaches suggest that this ad-hoc method of slicing the time series does not guarantee that the discriminative information of a time series has not been lost. These two classifiers are similar to the phase dependent intervals TSC algorithms (Bagnall et al., 2017) where the classifiers derive features from intervals of each series. Similarly to the recent comparative study of TSC algorithms, this type of Window Slicing based approach yielded the lowest average ranks.

Although MCDCNN and Time-CNN were originally proposed to classify MTS datasets, we have evaluated them on the univariate UCR/UEA archive. The MCDCNN did not manage to beat any of the classifiers except for the ECG5000 dataset which is already a dataset where almost all approaches reached the highest accuracy. This low performance is probably due to the non-linear FC layer that replaces the GAP pooling of the best performing algorithms (FCN and ResNet). This FC layer reduces the effect of learning time invariant features which explains why MLP, Time-CNN and MCDCNN exhibit very similar performance.

One approach that shows relatively high accuracy is Encoder (Serrà et al., 2018). The statistical test indicates a significant difference between Encoder, FCN and ResNet. FCN wins on 36 datasets whereases Encoder wins only on 17 which suggests the superiority of the GAP layer compared to Encoder's attention mechanism.

5.2 Comparing with state-of-the-art approaches

In this section, we compared ResNet (the most accurate DNN of our study) with the current state-of-the-art classifiers evaluated on the UCR/UEA archive in the great time series classification bake off (Bagnall et al., 2017). Note that our empirical study strongly suggests to use ResNet instead of any other deep learning algorithm - it is the most accurate one with similar runtime to FCN (the second most accurate DNN). Finally, since ResNet's results were averaged over ten different random initializations, we chose to take one iteration of ResNet (the median) and compare it to other state-of-the-art algorithms that were executed once over the original train/test split provided by the UCR/UEA archive.

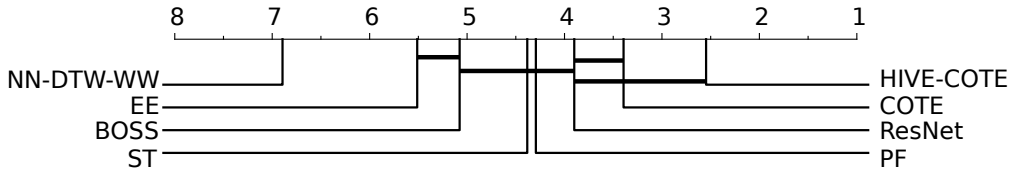


Fig. 8: Critical difference diagram showing pairwise statistical difference comparison of state-of-the-art classifiers on the univariate UCR/UEA time series classification archive.

Out of the 18 classifiers evaluated by Bagnall et al. (2017), we have chosen the four best performing algorithms: (1) Elastic Ensemble (EE) proposed by Lines and Bagnall (2015) is an ensemble of nearest neighbor classifiers with 11 different time series similarity measures; (2) Bag-of-SFA-Symbols (BOSS) published in Schäfer (2015) forms a discriminative bag of words by discretizing the time series using a Discrete Fourier Transform and then building a nearest neighbor classifier with a bespoke distance measure; (3) Shapelet Transform (ST) developed by Hills et al. (2014) extracts discriminative subsequences (shapelets) and builds a new representation of the time series that is fed to an ensemble of 8 classifiers; (4) Collective of Transformation-based Ensembles (COTE) proposed by Bagnall et al. (2017) is basically a weighted ensemble of 35 TSC algorithms including EE and ST. We also include the Hierarchical Vote Collective of Transformation-Based Ensembles (HIVE-COTE) proposed by Lines et al. (2018) which improves significantly COTE's performance by leveraging a hierarchical voting system as well as adding two new classifiers and two additional transformation domains. In addition to these five state-of-the-art classifiers, we have included the classic nearest neighbor coupled with DTW and a warping window (WW) set through cross-validation on the training set (denoted by NN-DTW-WW), since it is still one of the most popular methods for classifying time series data (Bagnall et al., 2017). Finally, we added a recent approach named Proximity Forest (PF) which is similar to Random Forest but replaces the attribute based splitting criteria by a random similarity measure chosen out of EE's elastic distances (Lucas et al., 2018). Note that we did not implement any of the non-deep TSC algorithms. We used the results provided by Bagnall et al. (2017) and the other corresponding papers to construct the critical difference diagram in Figure 8.

Figure 8 shows the critical difference diagram over the UEA benchmark with ResNet added to the pool of six classifiers. As we have previously mentioned, the state-of-the-art classifiers are compared to ResNet's median accuracy over the test set. Nevertheless, we generated the ten different average ranks for each iteration of ResNet and observed that the ranking of the compared classifiers is stable for the ten different random initializations of ResNet. The statistical test failed to find any significant difference between COTE/HIVE-COTE and ResNet which is the only TSC algorithm that was able to reach similar performance to COTE. Note that for the ten different random initializations of ResNet, the pairwise statistical test always failed to find any significance between ResNet and COTE/HIVE-COTE. PF, ST, BOSS and ResNet showed similar performances according to the Wilcoxon signed-rank test, but the fact that ResNet is not significantly different than COTE suggests that more datasets would give a better insight into these performances (Demšar, 2006). NN-DTW-WW and EE showed the lowest average rank suggesting that these methods are no longer competitive with current state-of-the-art algorithms for TSC. It is worthwhile noting that cliques formed by the Wilcoxon Signed Rank Test with Holm's alpha correction do not necessary reflect the rank order (Lines et al., 2018). For example, if we have three classifiers (C_1, C_2, C_3) with average ranks $(C_1 > C_2 > C_3)$, one can still encounter a case where C_1 is not significantly worse than C_2 and C_3 with C_2 and C_3 being significantly different. In our experiments, when comparing to state-of-the-art algorithms, we have encountered this problem with (ResNet>COTE>HIVE-

COTE). Therefore we should emphasize that HIVE-COTE and COTE are *significantly* different when performing the pairwise statistical test.

Although HIVE-COTE is still the most accurate classifier (when evaluated on the UCR/UEA archive) its use in a real data mining application is limited due to its huge training time complexity which is $O(N^2 \cdot T^4)$ corresponding to the training time of one of its individual classifiers ST. However, we should note that the recent work of Bostrom and Bagnall (2015) showed that it is possible to use a random sampling approach to decrease significantly the running time of ST (HIVE-COTE's choke-point) without any loss of accuracy. On the other hand, DNNs offer this type of scalability evidenced by its revolution in the field of computer vision when applied to images, which are thousand times larger than time series data (Russakovsky et al., 2015). In addition to the huge training time, HIVE-COTE's *classification* time is bounded by a linear scan of the training set due to employing a nearest neighbor classifier, whereas the trivial GPU parallelization of DNNs provides instant classification. Finally we should note that unlike HIVE-COTE, ResNet's hyperparameters were not tuned for each dataset but rather the same architecture was used for the whole benchmark suggesting further investigation of these hyperparameters should improve DNNs' accuracy for TSC. These results should give an insight of deep learning for TSC therefore encouraging researchers to consider the DNNs as robust real time classifiers for time series data.

5.2.1 The need of a fair comparison

In this section, we highlight the fairness of the comparison to other machine learning TSC algorithms. Since we did not train nor test any of the state-of-the-art non deep learning algorithms, it is possible that we allowed much more training time for the described DNNs. For example, for a lazy machine learning algorithm such as NN-DTW, training time is zero when allowing maximum warping whereas it has been shown that judicially setting the warping window Dau et al. (2017) can lead to a significant increase in accuracy. Therefore, we believe that allowing a much more thorough search of DTW's warping window would lead to a fairer comparison between deep learning approaches and other state-of-the-art TSC algorithms. In addition to cross-validating NN-DTW's hyper-parameters, we can imagine spending more time on data pre-processing and cleansing (e.g. smoothing the input time series) in order to improve the accuracy of NN-DTW (Höppner, 2016; Dau et al., 2018). Ultimately, in order to obtain a fair comparison between deep learning and current state-of-the-art algorithms for TSC, we think that the time spent on optimizing a network's weights should be also spent on optimizing non deep learning based classifiers especially lazy learning algorithms such as the K nearest neighbor coupled with any similarity measure.

5.3 Results for multivariate time series

We provide on our companion repository[2] the detailed performance of the nine deep learning classifiers for 10 different random initializations over the 12 MTS classification datasets (Baydogan, 2015). Although Time-CNN and MCDCNN are the only architectures originally proposed for MTS data, they were outperformed by the three deep CNNs (ResNet, FCN and Encoder), which shows the superiority of these approaches on the MTS classification task. The corresponding critical difference diagram is depicted in Figure 9, where the statistical test failed to find any significant difference between the nine classifiers which is mainly due to the small number of datasets compared to their univariate counterpart. Therefore, we illustrated in Figure 10 the critical difference diagram when both archives are combined (evaluation on 97 datasets in total). At first glance, we can notice that

[2] www.github.com/hfawaz/dl-4-tsc

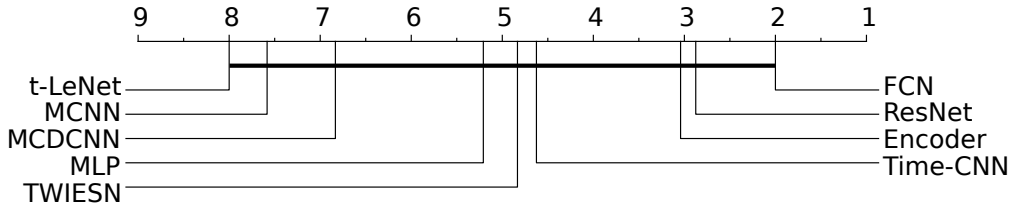


Fig. 9: Critical difference diagram showing pairwise statistical difference comparison of nine deep learning classifiers on the multivariate time series classification archive.

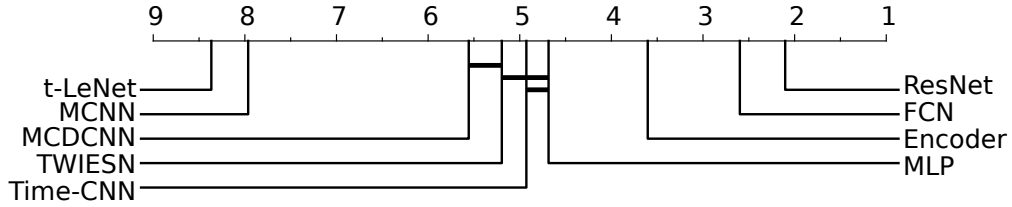


Fig. 10: Critical difference diagram showing pairwise statistical difference comparison of nine deep learning classifiers on both univariate and multivariate time series classification archives.

when adding the MTS datasets to the evaluation, the critical difference diagram in Figure 10 is not significantly different than the one in Figure 7 (where only the univariate UCR/UEA archive was taken into consideration). This is probably due to the fact that the algorithms' performance over the 12 MTS datasets is negligible to a certain degree when compared to the performance over the 85 univariate datasets. These observations reinforces the need to have an equally large MTS classification archive in order to evaluate hybrid univariate/multivariate time series classifiers. The rest of the analysis is dedicated to studying the effect of the datasets' characteristics on the algorithms' performance.

5.4 What can the dataset's characteristics tell us about the best architecture?

The first dataset characteristic we have investigated is the problem's domain. Table 4 shows the algorithms' performance with respect to the dataset's theme. These themes were first defined in Bagnall et al. (2017). Again, we can clearly see the dominance of ResNet as the best performing approach across different domains. One exception is the electrocardiography (ECG) datasets (7 in total) where ResNet was drastically beaten by the FCN model in 71.4% of ECG datasets. However, given the small sample size (only 7 datasets), we cannot conclude that FCN will almost always outperform the ResNet model for ECG datasets (Bagnall et al., 2017).

The second characteristic which we have studied is the time series length. Similar to the findings for non deep learning models in Bagnall et al. (2017), the time series length does not give information on deep learning approaches' performance. Table 5 shows the average rank of each DNN over the univariate datasets grouped by the datasets' lengths. One might expect that the relatively short filters (3) might affect the performance of ResNet and FCN since longer patterns cannot be captured by short filters. However, since increasing the number of convolutional layers will increase the path length viewed by the CNN model (Vaswani et al., 2017), ResNet and FCN managed to outperform other approaches whose filter length is longer (21) such as Encoder. For the recurrent TWIESN algorithm, we were expecting a poor accuracy for very long time series since a recurrent model may "forget" a useful information present in the early elements of a long time series. However,

Themes (#)	MLP	FCN	ResNet	Encoder	MCNN	t-LeNet	MCDCNN	Time-CNN	TWIESN
DEVICE (6)	0.0	50.0	**83.3**	0.0	0.0	0.0	0.0	0.0	0.0
ECG (7)	14.3	**71.4**	28.6	42.9	0.0	0.0	14.3	0.0	0.0
IMAGE (29)	6.9	34.5	**48.3**	10.3	0.0	0.0	6.9	10.3	0.0
MOTION (14)	14.3	28.6	**71.4**	21.4	0.0	0.0	0.0	0.0	0.0
SENSOR (16)	6.2	37.5	**75.0**	31.2	6.2	6.2	6.2	0.0	12.5
SIMULATED (6)	0.0	33.3	**100.0**	33.3	0.0	0.0	0.0	0.0	0.0
SPECTRO (7)	14.3	14.3	**71.4**	0.0	0.0	0.0	0.0	28.6	28.6

Table 4: Deep learning algorithms' performance grouped by themes. Each entry is the percentage of dataset themes an algorithm is most accurate for. Bold indicates the best model.

Length	MLP	FCN	ResNet	Encoder	MCNN	t-LeNet	MCDCNN	Time-CNN	TWIESN
<81	5.43	3.36	**2.43**	2.79	8.21	8.0	3.07	3.64	5.5
81-250	4.16	**1.63**	1.79	3.42	7.89	8.32	5.26	4.47	5.53
251-450	3.91	2.73	**1.64**	3.32	8.05	8.36	6.0	4.68	4.91
451-700	4.85	2.69	**1.92**	3.85	7.08	7.08	5.62	4.92	4.31
701-1000	4.6	1.9	**1.6**	3.8	7.4	8.5	5.2	6.0	4.5
>1000	3.29	2.71	**1.43**	3.43	7.29	8.43	4.86	5.71	6.0

Table 5: Deep learning algorithms' average ranks grouped by the datasets' length. Bold indicates the best model.

TWIESN did reach competitive accuracies on several long time series datasets such as reaching a 96.8% accuracy on Meat whose time series length is equal to 448. This would suggest that ESNs can solve the vanishing gradient problem especially when learning from long time series.

A third important characteristic is the training size of datasets and how it affects a DNN's performance. Table 6 shows the average rank for each classifier grouped by the train set's size. Again, ResNet and FCN still dominate with not much of a difference. However we found one very interesting dataset: DiatomSizeReduction. ResNet and FCN achieved the worst accuracy (30%) on this dataset while Time-CNN reached the best accuracy (95%). Interestingly, DiatomSizeReduction is the smallest datasets in the UCR/UEA archive (with 16 training instances), which suggests that ResNet and FCN are easily overfitting this dataset. This suggestion is also supported by the fact that Time-CNN is the smallest model: it contains a very small number of parameters by design with only 18 filters compared to the 512 filters of FCN. This simple architecture of Time-CNN renders overfitting the dataset much harder. Therefore, we conclude that the small number of filters in Time-CNN is the main reason behind its success on small datasets, however this shallow architecture is unable to capture the variability in larger time series datasets which is modeled efficiently by the FCN and ResNet architectures. One final observation that is in agreement with the deep learning literature is that in order to achieve high accuracies while training a DNN, a large training set is needed. Figure 11 shows the effect of the training size on ResNet's accuracy for the TwoPatterns dataset: the accuracy increases significantly when adding more training instances until it reaches 100% for 75% of the training data.

Finally, we should note that the number of classes in a dataset - although it yielded some variability in the results for the recent TSC experimental study conducted by Bagnall et al. (2017) - did not show any significance when comparing the classifiers based on this characteristic. In fact, most DNNs architectures, with the categorical cross-entropy as their cost function, employ mainly the same classifier: *softmax* which is basically designed for multi-class classification.

Train size	MLP	FCN	ResNet	Encoder	MCNN	t-LeNet	MCDCNN	Time-CNN	TWIESN
<100	4.3	2.03	**1.67**	4.13	7.67	7.73	6.1	4.37	4.77
100-399	4.85	2.76	**2.06**	3.24	7.71	8.12	4.59	4.97	4.5
400-799	3.62	2.38	**1.75**	3.5	8.0	8.62	4.38	5.0	5.88
>799	3.85	2.85	**1.62**	2.08	7.92	8.69	4.62	4.85	6.92

Table 6: Deep learning algorithms' average ranks grouped by the training sizes. Bold indicates the best model.

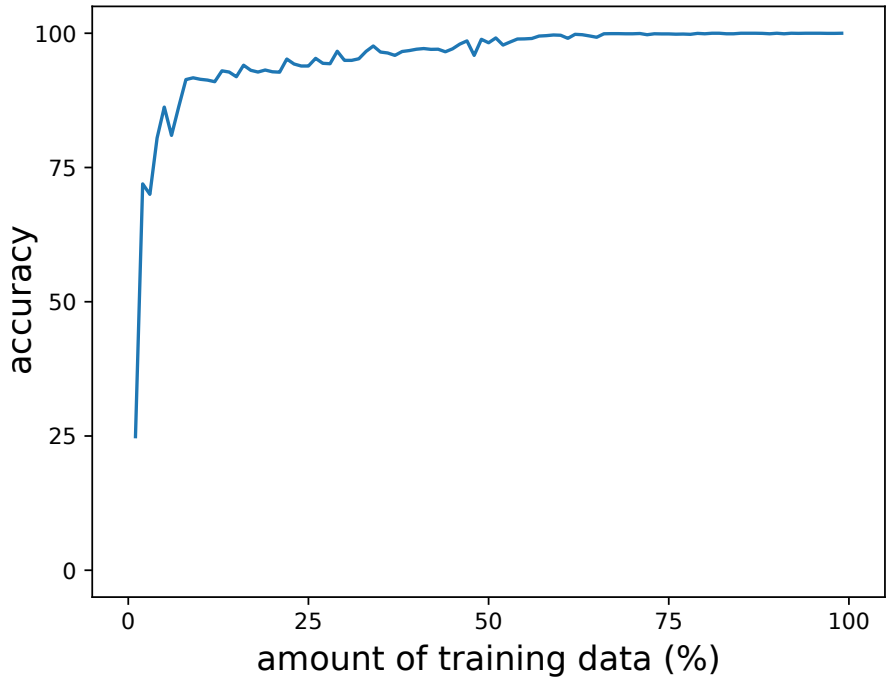


Fig. 11: ResNet's accuracy variation with respect to the amount of training instances in the TwoPatterns dataset.

Overall, our results show that, on average, ResNet is the best architecture with FCN and Encoder following as second and third respectively. ResNet performed very well in general except for the ECG datasets where it was outperformed by FCN. MCNN and t-LeNet, where time series were cropped into subsequences, were the worst on average. We found small variance between the approaches that replace the GAP layer with an FC dense layer (MCDCNN, CNN) which also showed similar performance to TWIESN and MLP.

5.5 Effect of random initializations

The initialization of deep neural networks has received a significant amount of interest from many researchers in the field (LeCun et al., 2015). These advancement have contributed to a better understanding and initialization of deep learning models in order to maximize the quality of non-optimal solutions found by the gradient descent algorithm (Glorot and Bengio, 2010). Nevertheless, we observed in our experiments, that DNNs for TSC suffer from a significant decrease (increase) in accuracy when initialized with bad (good) random weights. Therefore, we study in this section,

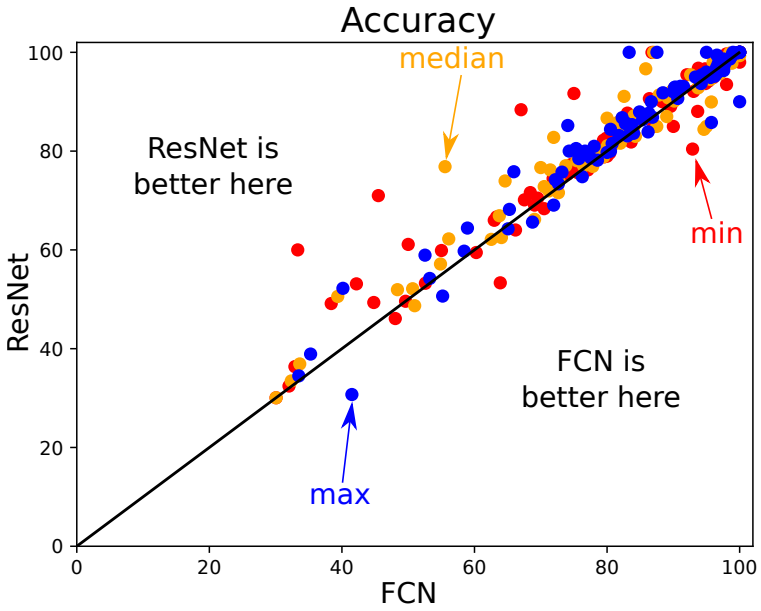


Fig. 12: Accuracy of ResNet versus FCN over the UCR/UEA archive when three different aggregations are taken: the minimum, median and maximum (Color figure online).

how random initializations can affect the performance of ResNet and FCN on the whole benchmark in a best and worst case scenario.

Figure 12 shows the accuracy plot of ResNet versus FCN on the 85 univariate time series datasets when aggregated over the 10 random initializations using three different functions: the minimum, median and maximum. When first observing Figure 12 one can easily conclude that ResNet has a better performance than FCN across most of the datasets regardless of the aggregation method. This is in agreement with the critical difference diagram as well as the analysis conducted in the previous subsections, where ResNet was shown to achieve higher performance on most datasets with different characteristics. A deeper look into the *minimum* aggregation (red points in Figure 12) shows that FCN's performance is less stable compared to ResNet's. In other words, the weight's initial value can easily decrease the accuracy of FCN whereas ResNet maintained a relatively high accuracy when taking the worst initial weight values. This is also in agreement with the average standard deviation of ResNet (1.48) which is less than FCN's (1.70). These observations would encourage a practitioner to avoid using a complex deep learning model since its accuracy may be unstable. Nevertheless we think that investigating different weight initialization techniques such as leveraging the weights of a pre-trained neural network would yield better and much more stable results (Ismail Fawaz et al., 2018c).

6 Visualization

In this section, we start by investigating the use of Class Activation Map to provide an interpretable feedback that highlights the reason for a certain decision taken by the classifier. We then propose another visualization technique which is based on Multi-Dimensional Scaling (Kruskal and Wish, 1978) to understand the latent representation that is learned by the DNNs.

6.1 Class Activation Map

We investigate the use of Class Activation Map (CAM) which was first introduced by Zhou et al. (2016) to highlight the parts of an image that contributed the most for a given class identification. Wang et al. (2017b) later introduced a one-dimensional CAM with an application to TSC. This method explains the classification of a certain deep learning model by highlighting the subsequences that contributed the most to a certain classification. Figure 13 and 14 show the results of applying CAM respectively on GunPoint and Meat datasets. Note that employing the CAM is only possible for the approaches with a GAP layer preceding the softmax classifier (Zhou et al., 2016). Therefore, we only considered in this section the ResNet and FCN models, who also achieved the best accuracies overall. Note that Wang et al. (2017b) was the only paper to propose an interpretable analysis of TSC with a DNN. We should emphasize that this is a very important research area which is usually neglected for the sake of improving accuracy: only 2 out of the 9 approaches provided a method that explains the decision taken by a deep learning model. In this section, we start by presenting the CAM method from a mathematical point of view and follow it with two interesting case studies on Meat and GunPoint datasets.

By employing a Global Average Pooling (GAP) layer, ResNet and FCN benefit from the CAM method (Zhou et al., 2016), which makes it possible to identify which regions of an input time series constitute the reason for a certain classification. Formally, let $A(t)$ be the result of the last convolutional layer which is an MTS with M variables. $A_m(t)$ is the univariate time series for the variable $m \in [1, M]$, which is in fact the result of applying the m^{th} filter. Now let w_m^c be the weight between the m^{th} filter and the output neuron of class c. Since a GAP layer is used then the input to the neuron of class c (z_c) can be computed by the following equation:

$$z_c = \sum_m w_m^c \sum_t A_m(t) \tag{10}$$

The second sum constitutes the averaged time series over the whole time dimension but with the denominator omitted for simplicity. The input z_c can be also written by the following equation:

$$z_c = \sum_t \sum_m w_m^c A_m(t) \tag{11}$$

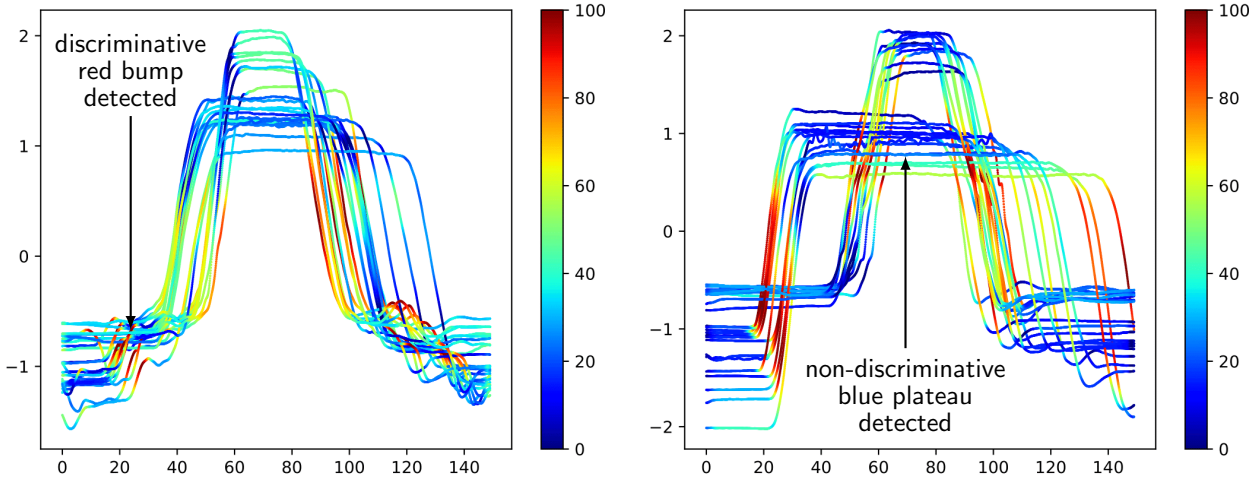
Finally the Class Activation Map (CAM_c) that explains the classification as label c is given in the following equation:

$$CAM_c(t) = \sum_m w_m^c A_m(t) \tag{12}$$

CAM is actually a univariate time series where each element (at time stamp $t \in [1, T]$) is equal to the weighted sum of the M data points at t, with the weights being learned by the neural network.

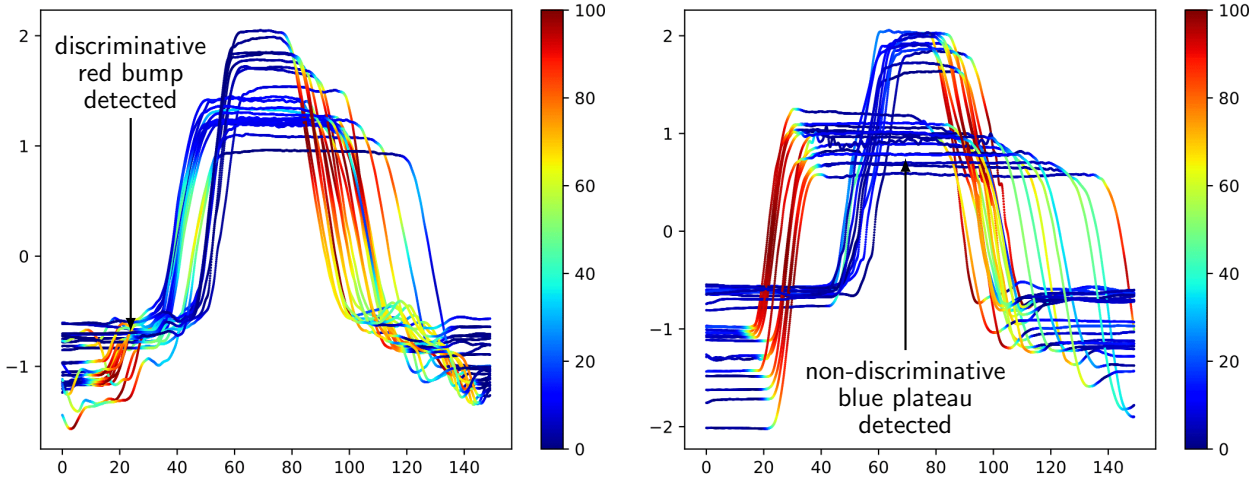
6.1.1 GunPoint dataset

The GunPoint dataset was first introduced by Ratanamahatana and Keogh (2005) as a TSC problem. This dataset involves one male and one female actor performing two actions (Gun-Draw and Point) which makes it a binary classification problem. For Gun-Draw (Class-1 in Figure 13), the actors have first their hands by their sides, then draw a replicate gun from hip-mounted holster, point it towards the target for one second, then finally place the gun in the holster and their hands to their initial position. Similarly to Gun-Draw, for Point (Class-2 in Figure 13) the actors follow the same steps but instead of pointing a gun they point their index finger. For each task, the



(a) FCN on GunPoint: Class-1

(b) FCN on GunPoint: Class-2



(c) ResNet on GunPoint: Class-1

(d) ResNet on GunPoint: Class-2

Fig. 13: Highlighting with the Class Activation Map the contribution of each time series region for both classes in GunPoint when using the FCN and ResNet classifiers. Red corresponds to high contribution and blue to almost no contribution to the correct class identification (smoothed for visual clarity and best viewed in color) (Color figure online).

centroid of the actor's right hands on both X and Y axes were tracked and seemed to be very correlated, therefore the dataset contains only one univariate time series: the X-axis.

We chose to start by visualizing the CAM for GunPoint for three main reasons. First, it is easy to visualize unlike other noisy datasets. Second, both FCN and ResNet models achieved almost 100% accuracy on this dataset which will help us to verify if both models are reaching the same decision for the same reasons. Finally, it contains only two classes which allow us to analyze the data much more easily.

Figure 13 shows the CAM's result when applied on each time series from both classes in the training set while classifying using the FCN model (Figure 13a and 13b) and the ResNet model

(Figure 13c and 13d). At first glance, we can clearly see how both DNNs are neglecting the plateau non-discriminative regions of the time series when taking the classification decision. It is depicted by the blue flat parts of the time series which indicates no contribution to the classifier's decision. As for the highly discriminative regions (the red and yellow regions) both models were able to select the same parts of the time series which correspond to the points with high derivatives. Actually, the first most distinctive part of class-1 discovered by both classifiers is almost the same: the little red bump in the bottom left of Figure 13a and 13c. Finally, another interesting observation is the ability of CNNs to localize a given discriminative shape regardless where it appears in the time series, which is evidence for CNNs' capability of learning time-invariant warped features.

An interesting observation would be to compare the discriminative regions identified by a deep learning model with the most discriminative shapelets extracted by other shapelet-based approaches. This observation would also be backed up by the mathematical proof provided by Cui et al. (2016), that showed how the learned filters in a CNN can be considered a generic form of shapelets extracted by the learning shapelets algorithm (Grabocka et al., 2014). Ye and Keogh (2011) identified that the most important shapelet for the Gun/NoGun classification occurs when the actor's arm is lowered (about 120 on the horizontal axis in Figure 13). Hills et al. (2014) introduced a shapelet transformation based approach that discovered shapelets that are similar to the ones identified by Ye and Keogh (2011). For ResNet and FCN, the part where the actor lowers his/her arm (bottom right of Figure 13) seems to be also identified as potential discriminative regions for some time series. On the other hand, the part where the actor raises his/her arm seems to be also a discriminative part of the data which suggests that the deep learning algorithms are identifying more "shapelets". We should note that this observation cannot confirm which classifier extracted the most discriminative subsequences especially because all algorithms achieved similar accuracy on GunPoint dataset. Perhaps a bigger dataset might provide a deeper insight into the interpretability of these machine learning models. Finally, we stress that the shapelet transformation classifier (Hills et al., 2014) is an ensemble approach, which makes unclear how the shapelets affect the decision taken by the individual classifiers whereas for an end-to-end deep learning model we can directly explain the classification by using the Class Activation Map.

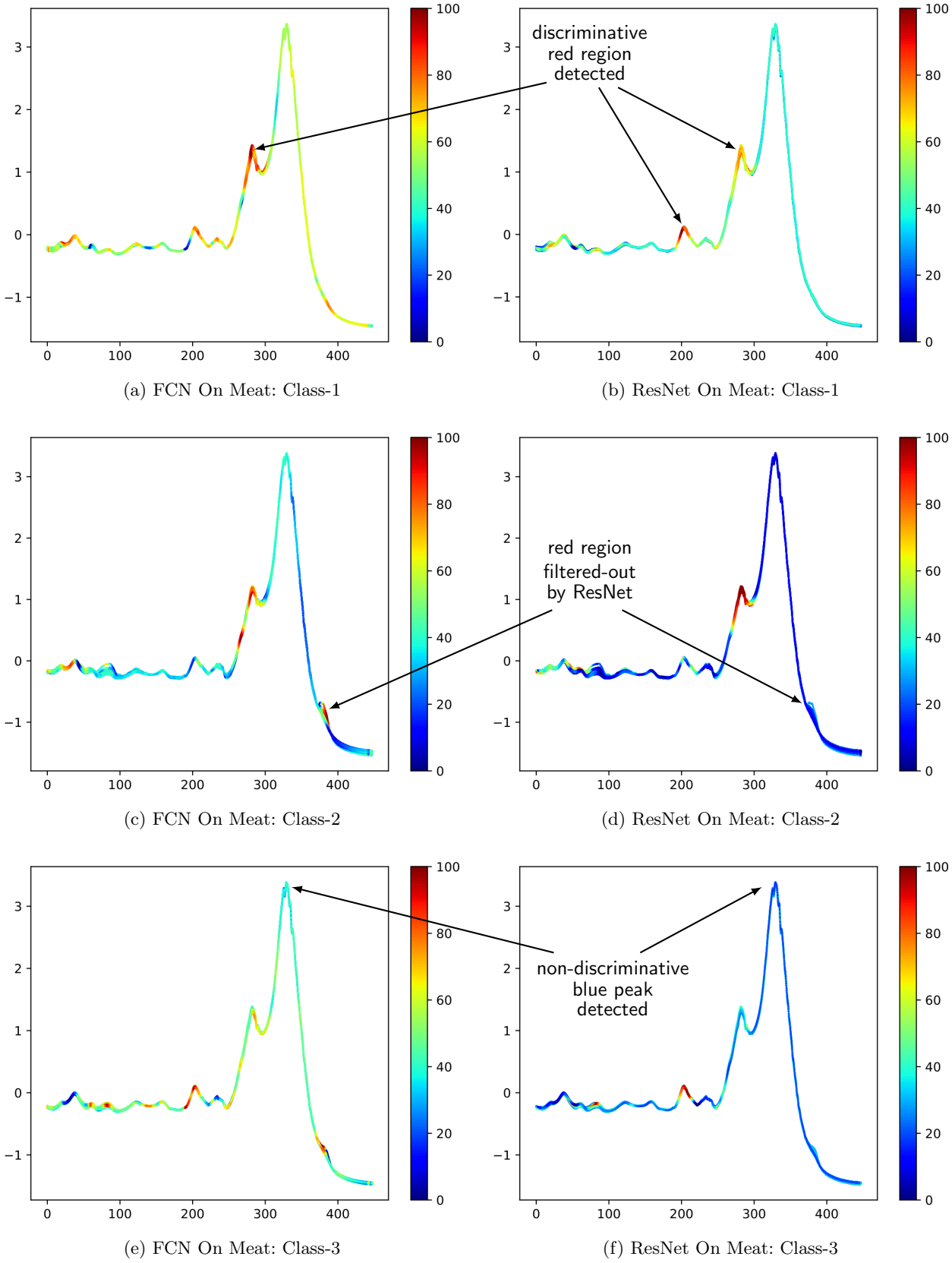
6.1.2 Meat dataset

Although the previous case study on GunPoint yielded interesting results in terms of showing that both models are localizing meaningful features, it failed to show the difference between the two most accurate deep learning classifiers: ResNet and FCN. Therefore we decided to further analyze the CAM's result for the two models on the Meat dataset.

Meat is a food spectrograph dataset which are usually used in chemometrics to classify food types, a task that has obvious applications in food safety and quality assurance. There are three classes in this dataset: Chicken, Pork and Turkey corresponding respectively to classes 1, 2 and 3 in Figure 14. Al-Jowder et al. (1997) described how the data is acquired from 60 independent samples using Fourier transform infrared (FTIR) spectroscopy with attenuated total reflectance (ATR) sampling.

Similarly to GunPoint, this dataset is easy to visualize and does not contain very noisy time series. In addition, with only three classes, the visualization is possible to understand and analyze. Finally, unlike for the GunPoint dataset, the two approaches ResNet and FCN reached significantly different results on Meat with respectively 97% and 83% accuracy.

Figure 14 enables the comparison between FCN's CAM (left) and ResNet's CAM (right). We first observe that ResNet is much more firm when it comes to highlighting the regions. In other words, FCN's CAM contains much more smoother regions with cyan, green and yellow regions,



(a) FCN On Meat: Class-1

(b) ResNet On Meat: Class-1

(c) FCN On Meat: Class-2

(d) ResNet On Meat: Class-2

(e) FCN On Meat: Class-3

(f) ResNet On Meat: Class-3

Fig. 14: Highlighting with the Class Activation Map the contribution of each time series region for the three classes in Meat when using the FCN and ResNet classifiers. Red corresponds to high contribution and blue to almost no contribution to the correct class identification (smoothed for visual clarity and best viewed in color) (Color figure online).

whereas ResNet's CAM contains more dark red and blue subsequences showing that ResNet can filter out non-discriminative and discriminative regions with a higher confidence than FCN, which probably explains why FCN is less accurate than ResNet on this dataset. Another interesting observation is related to the red subsequence highlighted by FCN's CAM for class 2 and 3 at the bottom right of Figure 14c and 14e. By visually investigating this part of the time series, we clearly see that it is a non-discriminative part since the time series of both classes exhibit this bump. This subsequence is therefore filtered-out by the ResNet model which can be seen by the blue color in the bottom right of Figure 14d and 14f. These results suggest that ResNet's superiority over FCN is mainly due to the former's ability to filter-out non-distinctive regions of the time series. We attribute this ability to the main characteristic of ResNet which is composed of the residual connections between the convolutional blocks that enable the model to *learn to skip* unnecessary convolutions by dint of its shortcut links (He et al., 2016).

6.2 Multi-Dimensional Scaling

We propose the use of Multi-Dimensional Scaling (MDS) (Kruskal and Wish, 1978) with the objective to gain some insights on the spatial distribution of the input time series belonging to different classes in the dataset. MDS uses a pairwise distance matrix as input and aims at placing each object in a N-dimensional space such as the between-object distances are preserved as well as possible. Using the Euclidean Distance (ED) on a set of input time series belonging to the test set, it is then possible to create a similarity matrix and apply MDS to display the set into a two dimensional space. This straightforward approach supposes that the ED is able to strongly separate the raw time series, which is usually not the case evident by the low accuracy of the nearest neighbor when coupled with ED (Bagnall et al., 2017).

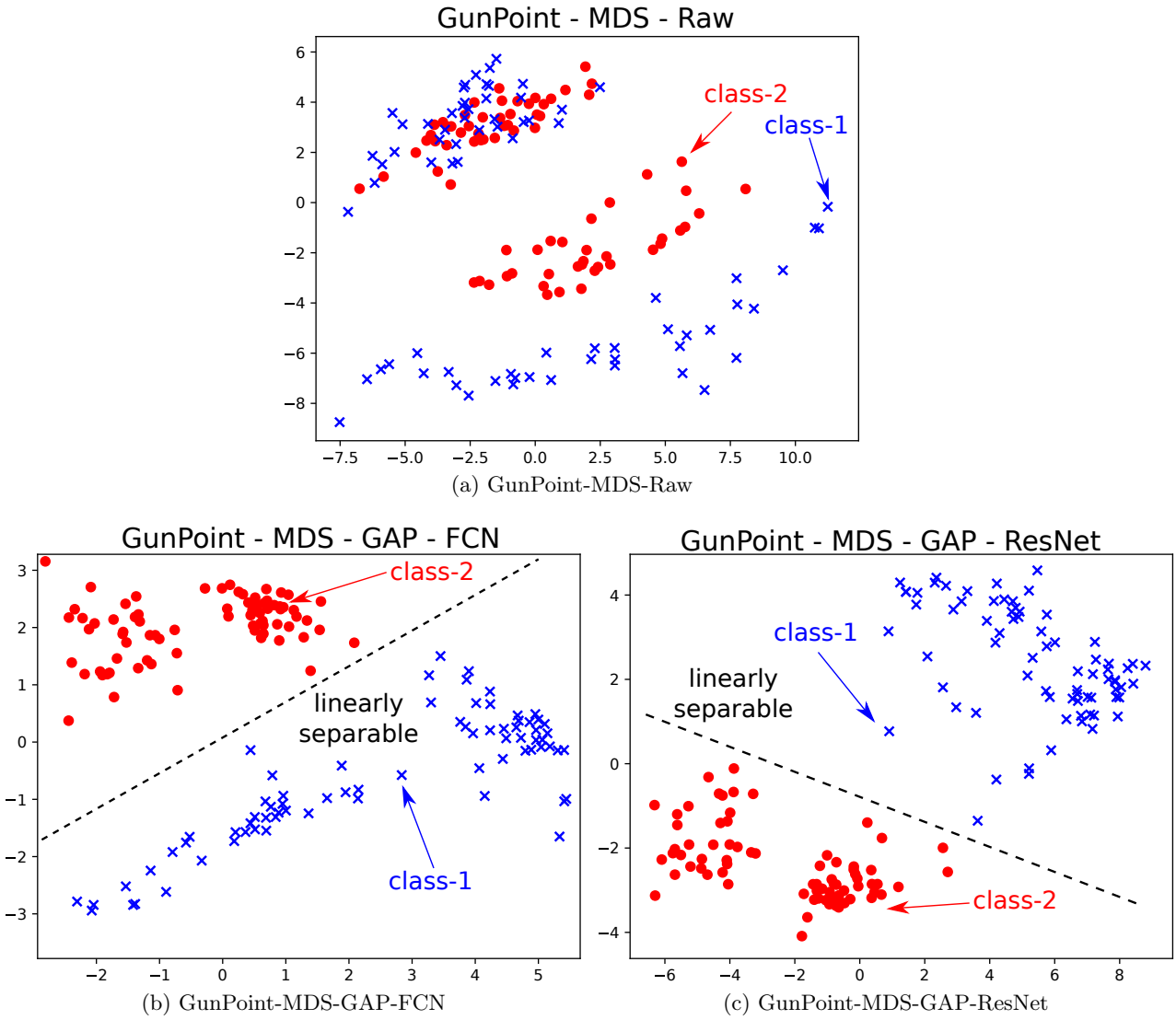
On the other hand, we propose to apply this MDS method to visualize the set of time series with its latent representation learned by the network. Usually in a deep neural network, we have several hidden layers and one can find several latent representation of the dataset. But since we are aiming at visualizing the class specific latent space, we chose to use the last latent representation of a DNN (the one directly before the softmax classifier), which is known to be a class specific layer (Yosinski et al., 2014). We decided to apply this method only on ResNet and FCN for two reasons: (1) when evaluated on the UCR/UEA archive they reached the highest ranks; (2) they both employ a GAP layer before the softmax layer making the number of latent features invariant to the time series length.

To better explain this process, for each input time series, the last convolution (for ResNet and FCN) outputs a multivariate time series whose dimensions are equal to the number of filters (128) in the last convolution, then the GAP layer averages the latter 128-dimensional multivariate time series over the time dimension resulting in a vector of 128 real values over which the ED is computed. As we worked with the ED, we used metric MDS (Kruskal and Wish, 1978) that minimizes a cost function called *Stress* which is a residual sum of squares:

$$Stress_D(X_1, \ldots, X_N) = \left(\frac{\sum_{i,j} (d_{ij} - \|x_i - x_j\|)^2}{\sum_{i,j} d_{ij}^2} \right)^{1/2} \tag{13}$$

where d_{ij} is the ED between the GAP vectors of time series X_i and X_j. Obviously, one has to be careful about the interpretation of MDS output, as the data space is highly simplified (each time series X_i is represented as a single data point x_i).

Figure 15 shows three MDS plots for the GunPoint dataset using: (1) the raw input time series (Figure 15a); (2) the learned latent features from the GAP layer for FCN (Figure 15b); and (3)



(a) GunPoint-MDS-Raw

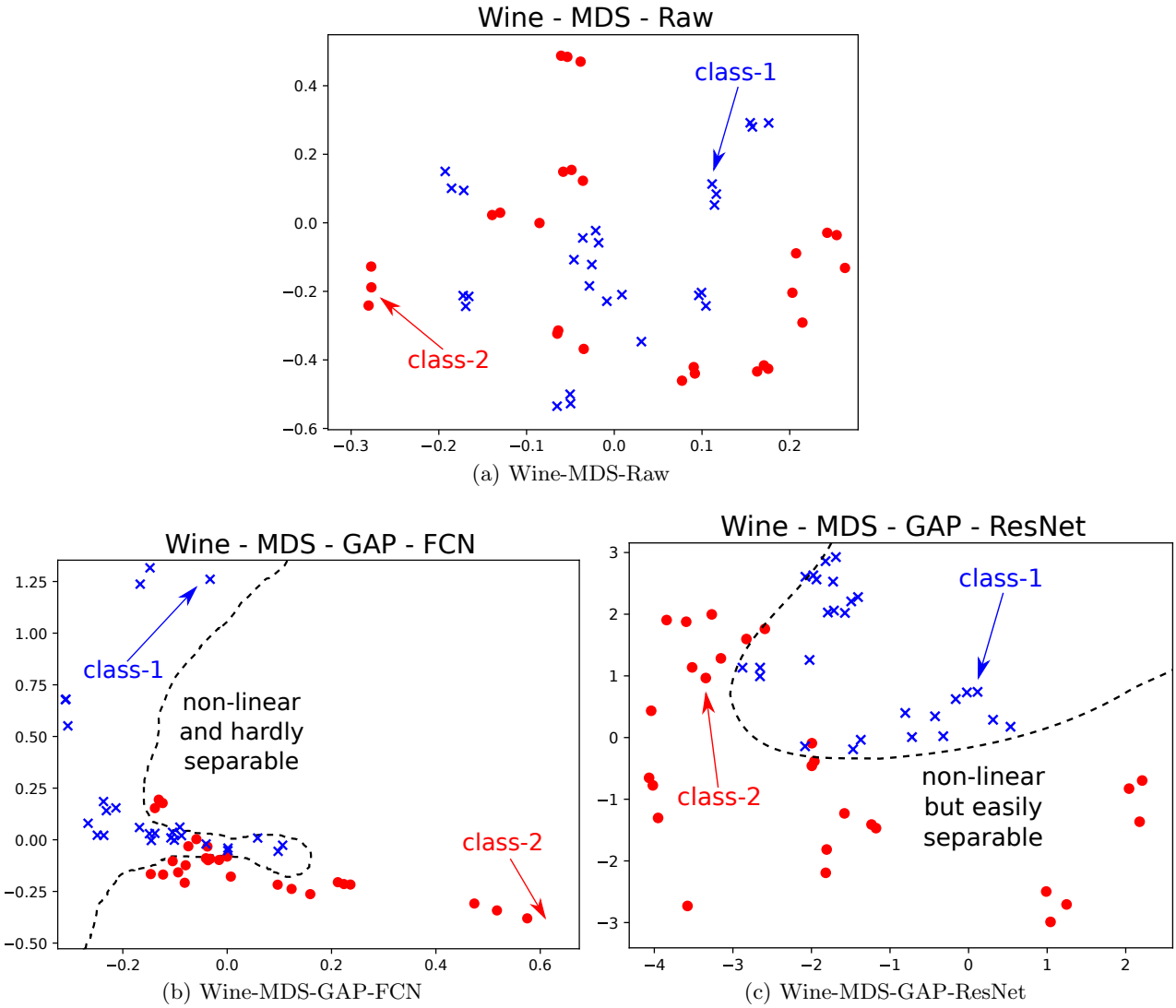
(b) GunPoint-MDS-GAP-FCN

(c) GunPoint-MDS-GAP-ResNet

Fig. 15: Multi-Dimensional Scaling (MDS) applied on GunPoint for: (top) the raw input time series; (bottom) the learned features from the Global Average Pooling (GAP) layer for FCN (left) and ResNet (right) - (best viewed in color). This figure shows how the ResNet and FCN are projecting the time series from a non-linearly separable 2D space (when using the raw input), into a linearly separable 2D space (when using the latent representation) (Color figure online).

the learned latent features from the GAP layer for ResNet (Figure 15c). We can easily observe in Figure 15a that when using the raw input data and projecting it into a 2D space, the two classes are not linearly separable. On the other hand, in both Figures 15b and 15c, by applying MDS on the latent representation learned by the network, one can easily separate the set of time series belonging to the two classes. We note that both deep learning models (FCN and ResNet) managed to project the data from GunPoint into a linearly separable space which explains why both models performed equally very well on this dataset with almost 100% accuracy.

Although the visualization of MDS on GunPoint yielded some interesting results, it failed to pinpoint the difference between the two deep learning models FCN and ResNet. Therefore we decided to analyze another dataset where the accuracy of both models differed by almost 15%. Figure 16



(a) Wine-MDS-Raw

(b) Wine-MDS-GAP-FCN

(c) Wine-MDS-GAP-ResNet

Fig. 16: Multi-Dimensional Scaling (MDS) applied on Wine for: (top) the raw input time series; (bottom) the learned features from the Global Average Pooling (GAP) layer for FCN (left) and ResNet (right) - (best viewed in color). This figure shows how ResNet, unlike FCN, is able to project the data into an easily separable space when using the learned features from the GAP layer (Color figure online).

shows three MDS plots for the Wine dataset using: (1) the raw input time series (Figure 16a); (2) the learned latent features from the GAP layer for FCN (Figure 16b); and (3) the learned latent features from the GAP layer for ResNet (Figure 16c). At first glimpse of Figure 16, the reader can conclude that all projections, even when using the learned representation, are not linearly separable which is evident by the relatively low accuracy of both models FCN and ResNet which is equal respectively to 58.7% and 74.4%. A thorough observation shows us that the learned hidden representation of ResNet (Figure 16c) separates the data from both classes in a much clearer way than the FCN (Figure 16b). In other words, FCN's learned representation has too many data points close to the decision boundary whereas ResNet's hidden features enables projecting data points

further away from the decision boundary. This observation could explain why ResNet achieves a better performance than FCN on the Wine dataset.

7 Conclusion

In this paper, we presented the largest empirical study of DNNs for TSC. We described the most recent successful deep learning approaches for TSC in many different domains such as human activity recognition and sleep stage identification. Under a unified taxonomy, we explained how DNNs are separated into two main categories of generative and discriminative models. We re-implemented nine recently published end-to-end deep learning classifiers in a unique framework which we make publicly available to the community. Our results show that end-to-end deep learning can achieve the current state-of-the-art performance for TSC with architectures such as Fully Convolutional Neural Networks and deep Residual Networks. Finally, we showed how the black-box effect of deep models which renders them uninterpretable, can be mitigated with a Class Activation Map visualization that highlights which parts of the input time series, contributed the most to a certain class identification.

Although we have conducted an extensive experimental evaluation, deep learning for time series classification, unlike for computer vision and NLP tasks, still lacks a thorough study of data augmentation (Ismail Fawaz et al., 2018a; Forestier et al., 2017) and transfer learning (Ismail Fawaz et al., 2018c; Serrà et al., 2018). In addition, the time series community would benefit from an extension of this empirical study that compares in addition to accuracy, the training and testing time of these deep learning models. Furthermore, we think that the effect of z-normalization (and other normalization methods) on the learning capabilities of DNNs should also be thoroughly explored. In our future work, we aim to investigate and answer the aforementioned limitations by conducting more extensive experiments especially on multivariate time series datasets. In order to achieve all of these goals, one important challenge for the TSC community is to provide one large generic *labeled* dataset similar to the large images database in computer vision such as ImageNet (Russakovsky et al., 2015) that contains 1000 classes.

In conclusion, with data mining repositories becoming more frequent, leveraging deeper architectures that can learn automatically from annotated data in an end-to-end fashion, makes deep learning a very enticing approach.

Acknowledgements The authors would like to thank the creators and providers of the datasets: Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, Eamonn Keogh and Mustafa Baydogan. The authors would also like to thank NVIDIA Corporation for the GPU Grant and the Mésocentre of Strasbourg for providing access to the cluster. The authors would also like to thank François Petitjean and Charlotte Pelletier for the fruitful discussions, their feedback and comments while writing this paper. This work was supported by the ANR TIMES project (grant ANR-17-CE23-0015) of the French Agence Nationale de la Recherche.

References

Abadi M, Agarwal A, Barham P, Brevdo E, Chen Z, Citro C, Corrado GS, Davis A, Dean J, Devin M, Ghemawat S, Goodfellow I, Harp A, Irving G, Isard M, Jia Y, Jozefowicz R, Kaiser L, Kudlur M, Levenberg J, Mané D, Monga R, Moore S, Murray D, Olah C, Schuster M, Shlens J, Steiner B, Sutskever I, Talwar K, Tucker P, Vanhoucke V, Vasudevan V, Viégas F, Vinyals O, Warden

P, Wattenberg M, Wicke M, Yu Y, Zheng X (2015) TensorFlow: Large-scale machine learning on heterogeneous systems. URL https://www.tensorflow.org/, Accessed 28 Feb 2019
Al-Jowder O, Kemsley E, Wilson R (1997) Mid-infrared spectroscopy and authenticity problems in selected meats: a feasibility study. Food Chemistry 59(2):195 – 201
Aswolinskiy W, Reinhart RF, Steil J (2016) Time series classification in reservoir- and model-space: A comparison. In: Artificial Neural Networks in Pattern Recognition, pp 197–208
Aswolinskiy W, Reinhart RF, Steil J (2017) Time series classification in reservoir- and model-space. Neural Processing Letters 48:789–809
Bagnall A, Janacek G (2014) A run length transformation for discriminating between auto regressive time series. Journal of Classification 31(2):154–178
Bagnall A, Lines J, Hills J, Bostrom A (2016) Time-series classification with COTE: The collective of transformation-based ensembles. In: International Conference on Data Engineering, pp 1548–1549
Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2017) The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. Data Mining and Knowledge Discovery 31(3):606–660
Bahdanau D, Cho K, Bengio Y (2015) Neural Machine Translation by Jointly Learning to Align and Translate. In: International Conference on Learning Representations
Baird HS (1992) Document Image Defect Models, Springer, Berlin, pp 546–556
Banerjee D, Islam K, Mei G, Xiao L, Zhang G, Xu R, Ji S, Li J (2017) A deep transfer learning approach for improved post-traumatic stress disorder diagnosis. In: IEEE International Conference on Data Mining, pp 11–20
Baydogan MG (2015) Multivariate time series classification datasets. http://www.mustafabaydogan.com, Accessed 28 Feb 2019
Baydogan MG, Runger G, Tuv E (2013) A bag-of-features framework to classify time series. IEEE Transactions on Pattern Analysis and Machine Intelligence 35(11):2796–2802
Bellman R (2010) Dynamic Programming. Princeton University Press
Benavoli A, Corani G, Mangili F (2016) Should we really use post-hoc tests based on mean-ranks? Machine Learning Research 17(1):152–161
Bengio Y, Yao L, Alain G, Vincent P (2013) Generalized denoising auto-encoders as generative models. In: International Conference on Neural Information Processing Systems, pp 899–907
Bianchi FM, Scardapane S, Løkse S, Jenssen R (2018) Reservoir computing approaches for representation and classification of multivariate time series. ArXiv 1803.07870
Bishop C (2006) Pattern Recognition and Machine Learning. Springer
Bostrom A, Bagnall A (2015) Binary shapelet transform for multiclass time series classification. In: Big Data Analytics and Knowledge Discovery, pp 257–269
Che Z, Cheng Y, Zhai S, Sun Z, Liu Y (2017a) Boosting deep learning risk prediction with generative adversarial networks for electronic health records. In: IEEE International Conference on Data Mining, pp 787–792
Che Z, He X, Xu K, Liu Y (2017b) DECADE: A deep metric learning model for multivariate time series. In: KDD Workshop on Mining and Learning from Time Series
Chen H, Tang F, Tino P, Yao X (2013) Model-based kernel for efficient time series analysis. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 392–400
Chen H, Tang F, Tiňo P, Cohn A, Yao X (2015a) Model metric co-learning for time series classification. In: International Joint Conference on Artificial Intelligence, pp 3387 – 3394
Chen Y, Keogh E, Hu B, Begum N, Bagnall A, Mueen A, Batista G (2015b) The UCR time series classification archive. www.cs.ucr.edu/~eamonn/time_series_data/, Accessed 28 Feb 2019
Chollet Fea (2015) Keras. https://keras.io, Accessed 28 Feb 2019

Chouikhi N, Ammar B, Alimi AM (2018) Genesis of basic and multi-layer echo state network recurrent autoencoders for efficient data representations. ArXiv 1804.08996

Cristian Borges Gamboa J (2017) Deep learning for time-series analysis. ArXiv 1701.01887

Cui Z, Chen W, Chen Y (2016) Multi-scale convolutional neural networks for time series classification. ArXiv 1603.06995

Dau HA, Silva DF, Petitjean F, Forestier G, Bagnall A, Keogh E (2017) Judicious setting of dynamic time warping's window width allows more accurate classification of time series. In: IEEE International Conference on Big Data, pp 917–922

Dau HA, Bagnall A, Kamgar K, Yeh CCM, Zhu Y, Gharghabi S, Ratanamahatana CA, Keogh E (2018) The UCR Time Series Archive. ArXiv 1810.07758

Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. Machine Learning Research 7:1–30

Deng H, Runger G, Tuv E, Vladimir M (2013) A time series forest for classification and feature extraction. Information Sciences 239:142 – 153

Esling P, Agon C (2012) Time-series data mining. ACM Computing Surveys 45(1):12:1–12:34

Faust O, Hagiwara Y, Hong TJ, Lih OS, Acharya UR (2018) Deep learning for healthcare applications based on physiological signals: A review. Computer Methods and Programs in Biomedicine 161:1 – 13

Forestier G, Petitjean F, Dau HA, Webb GI, Keogh E (2017) Generating synthetic time series to augment sparse datasets. In: IEEE International Conference on Data Mining, pp 865–870

Friedman M (1940) A comparison of alternative tests of significance for the problem of m rankings. The Annals of Mathematical Statistics 11(1):86–92

Gallicchio C, Micheli A (2017) Deep echo state network (DeepESN): A brief survey. ArXiv 1712.04323

Garcia S, Herrera F (2008) An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. Machine learning research 9:2677–2694

Geng Y, Luo X (2018) Cost-sensitive convolution based neural networks for imbalanced time-series classification. ArXiv 1801.04396

Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks. In: International Conference on Artificial Intelligence and Statistics, vol 9, pp 249–256

Goldberg Y (2016) A primer on neural network models for natural language processing. Artificial Intelligence Research 57(1):345–420

Gong Z, Chen H, Yuan B, Yao X (2018) Multiobjective learning in the model space for time series classification. IEEE Transactions on Cybernetics 99:1–15

Grabocka J, Schilling N, Wistuba M, Schmidt-Thieme L (2014) Learning time-series shapelets. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 392–401

Hatami N, Gavet Y, Debayle J (2017) Classification of time-series images using deep convolutional neural networks. In: International Conference on Machine Vision

He K, Zhang X, Ren S, Sun J (2015) Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: IEEE International Conference on Computer Vision, pp 1026–1034

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 770–778

Hills J, Lines J, Baranauskas E, Mapp J, Bagnall A (2014) Classification of time series by shapelet transformation. Data Mining and Knowledge Discovery 28(4):851–881

Hinton G, Deng L, Yu D, Dahl GE, Mohamed AR, Jaitly N, Senior A, Vanhoucke V, Nguyen P, Sainath TN, Kingsbury B (2012) Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. IEEE Signal Processing Magazine 29(6):82–97

Hoerl AE, Kennard RW (1970) Ridge regression: Applications to nonorthogonal problems. Technometrics 12(1):69–82

Holm S (1979) A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2):65–70

Höppner F (2016) Improving time series similarity measures by integrating preprocessing steps. Data Mining and Knowledge Discovery 31:851–878

Hu Q, Zhang R, Zhou Y (2016) Transfer learning for short-term wind speed prediction with deep neural networks. Renewable Energy 85:83 – 95

Ignatov A (2018) Real-time human activity recognition from accelerometer data using convolutional neural networks. Applied Soft Computing 62:915 – 922

Ioffe S, Szegedy C (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: International Conference on Machine Learning, vol 37, pp 448–456

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2018a) Data augmentation using synthetic data for time series classification with deep residual networks. In: International Workshop on Advanced Analytics and Learning on Temporal Data, ECML PKDD

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2018b) Evaluating surgical skills from kinematic data using convolutional neural networks. In: Medical Image Computing and Computer Assisted Intervention, pp 214–221

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2018c) Transfer learning for time series classification. In: IEEE International Conference on Big Data, pp 1367–1376

Jaeger H, Haas H (2004) Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. Science 304(5667):78–80

Kate RJ (2016) Using dynamic time warping distances as features for improved time series classification. Data Mining and Knowledge Discovery 30(2):283–312

Keogh E, Mueen A (2017) Curse of dimensionality. Encyclopedia of Machine Learning and Data Mining pp 314–315

Kim Y (2014) Convolutional neural networks for sentence classification. In: Empirical Methods in Natural Language Processing

Kingma DP, Ba J (2015) Adam: A method for stochastic optimization. In: International Conference on Learning Representations

Kotsifakos A, Papapetrou P (2014) Model-based time series classification. In: Advances in Intelligent Data Analysis, pp 179–191

Krasin I, Duerig T, Alldrin N, Ferrari V, Abu-El-Haija S, Kuznetsova A, Rom H, Uijlings J, Popov S, Kamali S, Malloci M, Pont-Tuset J, Veit A, Belongie S, Gomes V, Gupta A, Sun C, Chechik G, Cai D, Feng Z, Narayanan D, Murphy K (2017) OpenImages: A public dataset for large-scale multi-label and multi-class image classification. https://storage.googleapis.com/openimages/web/index.html, Accessed 28 Feb 2019

Krizhevsky A, Sutskever I, Hinton GE (2012) ImageNet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems 25, pp 1097–1105

Kruskal JB, Wish M (1978) Multidimensional scaling. number 07–011 in sage university paper series on quantitative applications in the social sciences

Längkvist M, Karlsson L, Loutfi A (2014) A review of unsupervised feature learning and deep learning for time-series modeling. Pattern Recognition Letters 42:11 – 24

Large J, Lines J, Bagnall A (2017) The Heterogeneous Ensembles of Standard Classification Algorithms (HESCA): the Whole is Greater than the Sum of its Parts. ArXiv 1710.09220

Le Q, Mikolov T (2014) Distributed representations of sentences and documents. In: International Conference on Machine Learning, vol 32, pp II–1188–II–1196

Le Guennec A, Malinowski S, Tavenard R (2016) Data augmentation for time series classification using convolutional neural networks. In: ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data

LeCun Y, Bottou L, Bengio Y, Haffner P (1998a) Gradient-based learning applied to document recognition. Proceedings of the IEEE 86(11):2278–2324

LeCun Y, Bottou L, Orr GB, Müller KR (1998b) Efficient backprop. In: Neural Networks: Tricks of the Trade, Springer, Berlin, pp 9–50

LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444

Lin S, Runger GC (2018) GCRNN: Group-constrained convolutional recurrent neural network. IEEE Transactions on Neural Networks and Learning Systems 99:1–10

Lines J, Bagnall A (2015) Time series classification with ensembles of elastic distance measures. Data Mining and Knowledge Discovery 29(3):565–592

Lines J, Taylor S, Bagnall A (2016) HIVE-COTE: The hierarchical vote collective of transformation-based ensembles for time series classification. In: IEEE International Conference on Data Mining, pp 1041–1046

Lines J, Taylor S, Bagnall A (2018) Time series classification with HIVE-COTE: The hierarchical vote collective of transformation-based ensembles. ACM Transactions on Knowledge Discovery from Data 12(5):52:1–52:35

Liu C, Hsaio W, Tu Y (2018) Time series classification with multivariate convolutional neural network. IEEE Transactions on Industrial Electronics 66:1–1

Lu J, Young S, Arel I, Holleman J (2015) A 1 tops/w analog deep machine-learning engine with floating-gate storage in 0.13 μm cmos. IEEE Journal of Solid-State Circuits 50(1):270–281

Lucas B, Shifaz A, Pelletier C, O'Neill L, Zaidi N, Goethals B, Petitjean F, Webb GI (2018) Proximity Forest: An effective and scalable distance-based classifier for time series. Data Mining and Knowledge Discovery 28:851–881

Ma Q, Shen L, Chen W, Wang J, Wei J, Yu Z (2016) Functional echo state network for time series classification. Information Sciences 373:1 – 20

Malhotra P, TV V, Vig L, Agarwal P, Shroff G (2018) TimeNet: Pre-trained deep recurrent neural network for time series classification. In: European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, pp 607–612

Martinez C, Perrin G, Ramasso E, Rombaut M (2018) A deep reinforcement learning approach for early classification of time series. In: European Signal Processing Conference

Mehdiyev N, Lahann J, Emrich A, Enke D, Fettke P, Loos P (2017) Time series classification using deep learning for process planning: A case from the process industry. Procedia Computer Science 114:242 – 249

Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient Estimation of Word Representations in Vector Space. In: International Conference on Learning Representations - Workshop

Mikolov T, Sutskever I, Chen K, Corrado G, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: Neural Information Processing Systems, pp 3111–3119

Mittelman R (2015) Time-series modeling with undecimated fully convolutional neural networks. ArXiv 1508.00317

Neamtu R, Ahsan R, Rundensteiner EA, Sarkozy G, Keogh E, Dau HA, Nguyen C, Lovering C (2018) Generalized dynamic time warping: Unleashing the warping power hidden in point-wise distances. In: IEEE International Conference on Data Engineering

Nguyen TL, Gsponer S, Ifrim G (2017) Time series classification by sequence learning in all-subsequence space. In: IEEE International Conference on Data Engineering, pp 947–958

Nwe TL, Dat TH, Ma B (2017) Convolutional neural network with multi-task learning scheme for acoustic scene classification. In: Asia-Pacific Signal and Information Processing Association

Annual Summit and Conference, pp 1347–1350
Nweke HF, Teh YW, Al-garadi MA, Alo UR (2018) Deep learning algorithms for human activity recognition using mobile and wearable sensor networks: State of the art and research challenges. Expert Systems with Applications 105:233 – 261
Ordóñez FJ, Roggen D (2016) Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. Sensors 16:115
Pan SJ, Yang Q (2010) A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering 22(10):1345–1359
Papernot N, McDaniel P (2018) Deep k-nearest neighbors: Towards confident, interpretable and robust deep learning. ArXiv 1803.04765
Pascanu R, Mikolov T, Bengio Y (2012) Understanding the exploding gradient problem. ArXiv 1211.5063
Pascanu R, Mikolov T, Bengio Y (2013) On the difficulty of training recurrent neural networks. In: International Conference on Machine Learning, vol 28, pp III–1310–III–1318
Petitjean F, Forestier G, Webb GI, Nicholson AE, Chen Y, Keogh E (2016) Faster and more accurate classification of time series by exploiting a novel dynamic time warping averaging algorithm. Knowledge Information Systems 47(1):1–26
Poggio T, Mhaskar H, Rosasco L, Miranda B, Liao Q (2017) Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review. International Journal of Automation and Computing 14(5):503–519
Rajan D, Thiagarajan J (2018) A generative modeling approach to limited channel ecg classification. In: IEEE Engineering in Medicine and Biology Society, vol 2018, p 2571
Rajkomar A, Oren E, Chen K, Dai AM, Hajaj N, Liu PJ, Liu X, Sun M, Sundberg P, Yee H, Zhang K, Duggan GE, Flores G, Hardt M, Irvine J, Le Q, Litsch K, Marcus J, Mossin A, Tansuwan J, Wang D, Wexler J, Wilson J, Ludwig D, Volchenboum SL, Chou K, Pearson M, Madabushi S, Shah NH, Butte AJ, Howell M, Cui C, Corrado G, Dean J (2018) Scalable and accurate deep learning for electronic health records. NPJ Digital Medicine 1:18
Ratanamahatana CA, Keogh E (2005) Three myths about dynamic time warping data mining. In: SIAM International Conference on Data Mining, pp 506–510
Rowe ACH, Abbott PC (1995) Daubechies wavelets and mathematica. Computers in Physics 9(6):635–648
Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L (2015) ImageNet large scale visual recognition challenge. International Journal of Computer Vision 115(3):211–252
Sainath TN, Mohamed AR, Kingsbury B, Ramabhadran B (2013) Deep convolutional neural networks for LVCSR. In: IEEE International Conference on Acoustics, Speech and Signal Processing, pp 8614–8618
Santos T, Kern R (2017) A literature survey of early time series classification and deep learning. In: International Conference on Knowledge Technologies and Data-driven Business
Schäfer P (2015) The BOSS is concerned with time series classification in the presence of noise. Data Mining and Knowledge Discovery 29(6):1505–1530
Serrà J, Pascual S, Karatzoglou A (2018) Towards a universal neural network encoder for time series. Artificial Intelligence Research and Development: Current Challenges, New Trends and Applications 308:120
Silva DF, Giusti R, Keogh E, Batista G (2018) Speeding up similarity search under dynamic time warping by pruning unpromising alignments. Data Mining and Knowledge Discovery 32(4):988–1016

Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R (2014) Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research 15:1929–1958

Strodthoff N, Strodthoff C (2019) Detecting and interpreting myocardial infarction using fully convolutional neural networks. Physiological Measurement 40(1):015001

Susto GA, Cenedese A, Terzi M (2018) Time-series classification methods: Review and applications to power systems data. In: Big Data Application in Power Systems, pp 179 – 220

Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. In: Neural Information Processing Systems, pp 3104–3112

Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 1–9

Tanisaro P, Heidemann G (2016) Time series classification using time warping invariant echo state networks. In: IEEE International Conference on Machine Learning and Applications, pp 831–836

Tripathy R, Acharya UR (2018) Use of features from RR-time series and EEG signals for automated classification of sleep stages in deep neural network framework. Biocybernetics and Biomedical Engineering 38:890–902

Uemura M, Tomikawa M, Miao T, Souzaki R, Ieiri S, Akahoshi T, Lefor AK, Hashizume M (2018) Feasibility of an AI-based measure of the hand motions of expert and novice surgeons. Computational and Mathematical Methods in Medicine 2018

Ulyanov D, Vedaldi A, Lempitsky V (2016) Instance normalization: The missing ingredient for fast stylization. ArXiv 1607.08022

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Lu, Polosukhin I (2017) Attention is all you need. In: Advances in Neural Information Processing Systems, pp 5998–6008

Wang J, Chen Y, Hao S, Peng X, Hu L (2018) Deep learning for sensor-based activity recognition: A survey. Pattern Recognition Letters

Wang J, Wang Z, Li J, Wu J (2018) Multilevel wavelet decomposition network for interpretable time series analysis. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 2437–2446

Wang L, Wang Z, Liu S (2016) An effective multivariate time series classification approach using echo state network and adaptive differential evolution algorithm. Expert Systems with Applications 43:237 – 249

Wang S, Hua G, Hao G, Xie C (2017a) A cycle deep belief network model for multivariate time series classification. Mathematical Problems in Engineering 2017:1–7

Wang W, Chen C, Wang W, Rai P, Carin L (2016a) Earliness-aware deep convolutional networks for early time series classification. ArXiv 1611.04578

Wang Z, Oates T (2015a) Encoding time series as images for visual inspection and classification using tiled convolutional neural networks. In: Workshops at AAAI Conference on Artificial Intelligence, pp 40–46

Wang Z, Oates T (2015b) Imaging time-series to improve classification and imputation. In: International Conference on Artificial Intelligence, pp 3939–3945

Wang Z, Oates T (2015) Spatially encoding temporal correlations to classify temporal data using convolutional neural networks. ArXiv 1509.07481

Wang Z, Song W, Liu L, Zhang F, Xue J, Ye Y, Fan M, Xu M (2016b) Representation learning with deconvolution for multivariate time series classification and visualization. ArXiv 1610.07258

Wang Z, Yan W, Oates T (2017b) Time series classification from scratch with deep neural networks: A strong baseline. In: International Joint Conference on Neural Networks, pp 1578–1585

Wilcoxon F (1945) Individual comparisons by ranking methods. Biometrics Bulletin 1(6):80–83

Yang Q, Wu X (2006) 10 challenging problems in data mining research. Information Technology & Decision Making 05(04):597–604

Ye L, Keogh E (2011) Time series shapelets: a novel technique that allows accurate, interpretable and fast classification. Data Mining and Knowledge Discovery 22(1):149–182

Yosinski J, Clune J, Bengio Y, Lipson H (2014) How transferable are features in deep neural networks? In: International Conference on Neural Information Processing Systems, vol 2, pp 3320–3328

Zeiler MD (2012) ADADELTA: An adaptive learning rate method. ArXiv 1212.5701

Zhang C, Bengio S, Hardt M, Recht B, Vinyals O (2017) Understanding deep learning requires rethinking generalization. In: International Conference on Learning Representations

Zhao B, Lu H, Chen S, Liu J, Wu D (2017) Convolutional neural networks for time series classification. Systems Engineering and Electronics 28(1):162–169

Zheng Y, Liu Q, Chen E, Ge Y, Zhao JL (2014) Time series classification using multi-channels deep convolutional neural networks. In: Web-Age Information Management, pp 298–310

Zheng Y, Liu Q, Chen E, Ge Y, Zhao JL (2016) Exploiting multi-channels deep convolutional neural networks for multivariate time series classification. Frontiers of Computer Science 10(1):96–112

Zhou B, Khosla A, Lapedriza A, Oliva A, Torralba A (2016) Learning deep features for discriminative localization. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 2921–2929

Ziat A, Delasalles E, Denoyer L, Gallinari P (2017) Spatio-temporal neural networks for space-time series forecasting and relations discovery. In: IEEE International Conference on Data Mining, pp 705–714

InceptionTime: Finding AlexNet for Time Series Classification

Hassan Ismail Fawaz[1] · Benjamin Lucas[2] · Germain Forestier[1,2] · Charlotte Pelletier[2,3] · Daniel F. Schmidt[2] · Jonathan Weber[1] · Geoffrey I. Webb[2] · Lhassane Idoumghar[1] · Pierre-Alain Muller[1] · François Petitjean[2]

Abstract This paper brings deep learning at the forefront of research into Time Series Classification (TSC). TSC is the area of machine learning tasked with the categorization (or labelling) of time series. The last few decades of work in this area have led to significant progress in the accuracy of classifiers, with the state of the art now represented by the HIVE-COTE algorithm. While extremely accurate, HIVE-COTE cannot be applied to many real-world datasets because of its high training time complexity in $O(N^2 \cdot T^4)$ for a dataset with N time series of length T. For example, it takes HIVE-COTE more than 8 days to learn from a small dataset with $N = 1500$ time series of short length $T = 46$. Meanwhile deep learning has received enormous attention because of its high accuracy and scalability. Recent approaches to deep learning for TSC have been scalable, but less accurate than HIVE-COTE. We introduce InceptionTime — an ensemble of deep Convolutional Neural Network (CNN) models, inspired by the Inception-v4 architecture. Our experiments show that InceptionTime is on par with HIVE-COTE in terms of accuracy while being much more scalable: not only can it learn from 1,500 time series in one hour but it can also learn from 8M time series in 13 hours, a quantity of data that is fully out of reach of HIVE-COTE.

Keywords time series classification · deep learning · scalable model · inception

1 Introduction

Recent times have seen an explosion in the magnitude and prevalence of time series data. Industries varying from health care (Forestier et al., 2018; Lee et al., 2018; Ismail Fawaz et al., 2019d) and social security (Yi et al., 2018) to human activity recognition (Yuan et al., 2018) and remote sensing (Pelletier et al., 2019), all now produce time series datasets of previously unseen scale — both in terms of time series length and quantity. This growth also means an increased dependence on automatic classification of time series data, and ideally, algorithms with the ability to do this at scale.

These problems, known as Time Series Classification (TSC), differ significantly to traditional supervised learning for structured data, in that the algorithms should be able to handle and harness

✉ H. Ismail Fawaz
E-mail: hassan.ismail-fawaz@uha.fr

[1]IRIMAS, Université Haute Alsace, Mulhouse, France
[2]Faculty of IT, Monash University, Melbourne, Australia
[3]IRISA, UMR CNRS 6074, Université Bretagne Sud, Vannes, France

the temporal information present in the signal (Bagnall et al., 2017). It is easy to draw parallels from this scenario to computer vision problems such as image classification and object localization, where successful algorithms learn from the spatial information contained in an image. Put simply, the time series problem is essentially the same class of problem, just with one less dimension. Yet despite this similarity, the current state-of-the-art algorithms from the two fields share little resemblance (Ismail Fawaz et al., 2019b).

Deep learning has a long history (in machine learning terms) in computer vision (LeCun et al., 1998) but its popularity exploded with AlexNet (Krizhevsky et al., 2012), after which it has been unquestionably the most successful class of algorithms (LeCun et al., 2015). Conversely, deep learning has only recently started to gain popularity amongst time series data mining researchers (Ismail Fawaz et al., 2019b). This is emphasized by the fact that the Residual Network (ResNet), which is currently considered the state-of-the-art neural network architecture for TSC when evaluated on the UCR archive (Dau et al., 2018), was originally proposed merely as a baseline model for the underlying task (Wang et al., 2017). Given the similarities in the data, it is easy to suggest that there is much potential improvement for deep learning in TSC.

In this paper, we take an important step towards finding the equivalent of 'AlexNet' for TSC by presenting InceptionTime — a novel deep learning ensemble for TSC. InceptionTime achieves state-of-the-art accuracy when evaluated on the UCR archive (currently the largest publicly available repository for TSC (Dau et al., 2018)) while also possessing ability to scale to a magnitude far beyond that of its strongest competitor.

InceptionTime is an ensemble of five deep learning models for TSC, each one created by cascading multiple Inception modules (Szegedy et al., 2015). Each individual classifier (model) will have exactly the same architecture but with different randomly initialized weight values. The core idea of an Inception module is to apply multiple filters simultaneously to an input time series. The module includes filters of varying lengths, which as we will show, allows the network to automatically extract relevant features from both long and short time series.

After presenting InceptionTime and its results, we perform an analysis of the architectural hyperparameters of deep neural networks — depth, filter length, number of filters — and the characteristics of the Inception module — the bottleneck and residual connection, in order to provide insight into why this model is so successful. In fact, we construct networks with filters larger than have ever been explored for computer vision tasks, taking direct advantage of the fact that time series exhibit one less dimension than images.

The remainder of this paper is structured as follows: first we start by presenting the background and related work in Section 2. We then proceed in Section 3 to explain the network architecture and its main building block — the Inception module. Section 4 contains the details of our experimental setup. In Section 5, we show that InceptionTime produces state-of-the-art accuracy on the UCR archive, the TSC benchmark, while also presenting a runtime comparison with its nearest competitor. In Section 6, we provide a detailed hyperparameter study that provides insight into the choices made when designing our proposed neural network. Finally we conclude the paper in Section 7 and give directions for further research on deep learning for TSC.

2 Related work

In this section, we start with some preliminary definitions for ease of understanding, before presenting the current state-of-the-art algorithms for TSC. We end by providing a deeper background for designing neural network architectures for domain-agnostic TSC problems.

2.1 Time series classification

Definition 1 A Multivariate Time Series (MTS) $X = [X_1, X_2, \ldots, X_T]$ with M dimensions, consists of T ordered elements $X_i \in \mathbb{R}^M$.

Definition 2 A *Univariate* time series X of length T is simply an MTS with $M = 1$.

Definition 3 $D = \{(X^1, Y^1), (X^2, Y^2), \ldots, (X^N, Y^N)\}$ is a dataset containing a collection of pairs (X^i, Y^i) where X^i could either be a univariate or multivariate time series with its corresponding label denoted by Y^i.

The task of classifying time series data consists of learning a classifier on D in order to map from the space of possible inputs X to a probability distribution over the classes Y.

The state-of-the-art for TSC has been organized by Bagnall et al. (2017) into the following main categories:

2.1.1 Whole series

This type of classifiers compares two series using a certain distance. For many years, the leading classifier for TSC was the nearest neighbor algorithm coupled with the Dynamic Time Warping similarity measure (NN-DTW) (Bagnall et al., 2017). Much research has subsequently focused on finding alternative similarity measures (Marteau, 2009; Stefan et al., 2013; Keogh and Pazzani, 2001; Vlachos et al., 2006), however none have been found to significantly outperform NN-DTW on the UCR Archive (Lines and Bagnall, 2015). Another research area focused on proposing global alignment kernels such as SoftDTW introduced by Cuturi and Blondel (2017), that can be further used in a nearest centroid classification scheme. This research informed one current state-of-the-art method, named Elastic Ensemble (EE), which is an ensemble of 11 nearest neighbor classifiers each coupled with a different similarity measure (Lines and Bagnall, 2015). While this algorithm produces state-of-the-art accuracy, its use on large datasets is limited by its training complexity, with some of its parameter searches being in $O(N^2 \cdot T^3)$. Following this line of research, all recent successful classification algorithms for time series data are all ensemble based models. Furthermore, to tackle EE's huge training time, Lucas et al. (2019) proposed a tree-based ensemble called Proximity Forest (PF) that uses EE's distances as a splitting criteria while replacing the parameter searches by a random sampling.

2.1.2 Dictionary based

This type of classifiers discriminate time series by the frequency of repetition of some sub-series. The most famous one being the Bag-of-SFA-Symbols (BOSS), which is based on an ensemble of NNs classifiers coupled with a bespoke Euclidean distance computed on the frequency histograms obtained from the Symbolic Fourier Approximation (SFA) discretization (Schäfer, 2015a). BOSS has a high training complexity of $O(N^2)$, which the authors identified as a shortcoming and attempted to address with subsequent scalable variations of the algorithm in Schäfer (2015b); Schäfer and Leser (2017), however neither of these reached state-of-the-art accuracy.

2.1.3 Shapelets

This family of algorithms focuses on finding relatively short repeated subsequences to identify a certain class. These patterns are time independent and are called shapelets. Another type of

ensemble classifiers is shapelet based algorithms, such as in Hills et al. (2014), where discriminative subsequences (shapelets) are extracted from the training set and fed to off-the-shelf classifiers such as Support Vector Machines and Random Forests. The shapelet transform has a training complexity of $O(N^2 \cdot T^4)$ and thus, again, has little potential to scale to large datasets.

2.1.4 Transformation ensembles

More recently, Bagnall et al. (2016) noted that there is no single time series transformation technique (such as shapelets or SFA) that significantly dominates the others, showing that constructing an ensemble of different classifiers over different time series representations, called COTE, will significantly improve the accuracy. Lines et al. (2016) extended COTE with a hierarchical voting scheme, which further improves the decision taken by the ensemble. Named the Hierarchical Vote Collective of Transformation-Based Ensembles (HIVE-COTE), it represents the current state-of-the-art accuracy when evaluated on the UCR archive, however its practicality is hindered by its huge training complexity of order $O(N^2 \cdot T^4)$. This is highlighted by the extensive experiments in Lucas et al. (2019) where PF showed competitive performance with COTE, while having a runtime that is orders of magnitudes lower. Deep learning models, which we will discuss in detail in the following subsection, also significantly beat the runtime of HIVE-COTE by trivially leveraging GPU parallel computation abilities. A comprehensive detailed review of recent methods for TSC can be found in Bagnall et al. (2017).

2.2 Deep learning for time series classification

Since the recent success of deep learning techniques in supervised learning such as image recognition (Zhang et al., 2018) and natural language processing (Guan et al., 2019), researchers started investigating these complex machine learning models for TSC (Wang et al., 2017; Cui et al., 2016; Ismail Fawaz et al., 2019a). Precisely, Convolutional Neural Networks (CNNs) have showed promising results for TSC. Given an input MTS, a convolutional layer consists of sliding one-dimensional filters over the time series, thus enabling the network to extract non-linear discriminant features that are time-invariant and useful for classification. By cascading multiple layers, the network is able to further extract hierarchical features that should in theory improve the network's prediction. Note that given an input univariate time series, by applying several one-dimensional filters, the outcome can be considered an MTS whose length is preserved and the number of dimensions M is equal the number of filters applied at this layer. More details on how deep CNNs are being adapted for one-dimensional time series data can be found in Ismail Fawaz et al. (2019b). The rest of this subsection is dedicated to describing what is currently being explored in deep learning for TSC.

Multi-scale Convolutional Neural Networks (MCNN) (Cui et al., 2016) and Time LeNet (Le Guennec et al., 2016) are considered among the first architectures to be validated on a domain-agnostic TSC benchmark such as the UCR archive. These models were inspired by image recognition modules, which hindered their accuracy, mainly because of the use of progressive pooling layers, that were mainly added for computational feasibility when dealing with image data (Sabour et al., 2017). Consequently, Fully Convolutional Neural Networks (FCNs) were shown to achieve great performance without the need to add pooling layers to reduce the input data's dimensionality (Wang et al., 2017). More recently, it has been shown that deeper CNN models coupled with residual connections such as ResNet can further improve the classification performance (Ismail Fawaz et al., 2019b). In essence, since time series data exhibit only one structuring dimension (i.e. time, as opposed to two spatial dimensions for images), it is possible to explore more complex

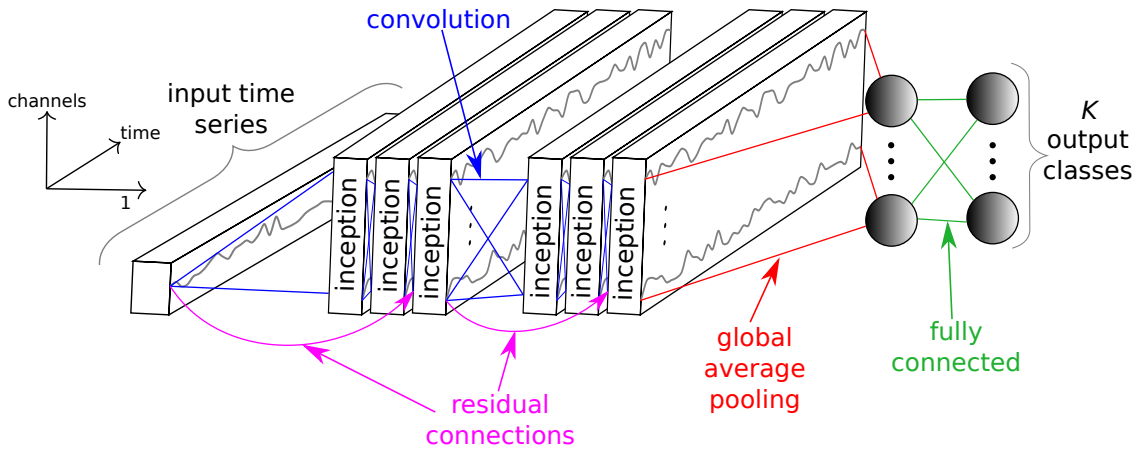


Fig. 1: Our Inception network for time series classification

models that are usually computationally infeasible for image recognition problems: for example removing the pooling layers that throw away valuable information in favour of reducing the model's complexity. In this paper, we propose an Inception based network that applies several convolutions with various filters lengths. In contrast to networks designed for images, we are able to explore filters 10 times longer than recent Inception variants for image recognition tasks (Szegedy et al., 2017).

Inception was first proposed by Szegedy et al. (2015) for end-to-end image classification. Now the network has evolved to become Inceptionv4, where Inception was coupled with residual connections to further improve the performance (Szegedy et al., 2017). As for TSC a relatively competitive Inception-based approach was proposed in Karimi-Bidhendi et al. (2018), where time series where transformed to images using Gramian Angular Difference Field (GADF), and finally fed to an Inception model that had been pre-trained for (standard) image recognition. Unlike this feature engineering approach, by adopting an end-to-end learning from raw time series data, a one-dimensional Inception model was used for Supernovae classification using the light flux of a region in space as an input MTS for the network (Brunel et al., 2019). However, the authors limited the conception of their Inception architecture to the one proposed by Google for ImageNet (Szegedy et al., 2017). In our work, we explore much larger filters than any previously proposed network for TSC in order to reach state-of-the-art performance on the UCR benchmark.

3 InceptionTime: an accurate and scalable time series classifier

In this section, we start by describing the proposed architecture we call InceptionTime for classifying time series data. Specifically, we detail the main component of our network: the Inception module. We then present our proposed model InceptionTime which consists of an ensemble of 5 different Inception networks initialized randomly. Finally, we adapt the concept of Receptive Field for time series data.

3.1 Inception Network: a novel architecture for TSC

The composition of an Inception network classifier contains *two* different residual blocks, as opposed to ResNet, which is comprised of *three*. For the Inception network, each block is comprised of three

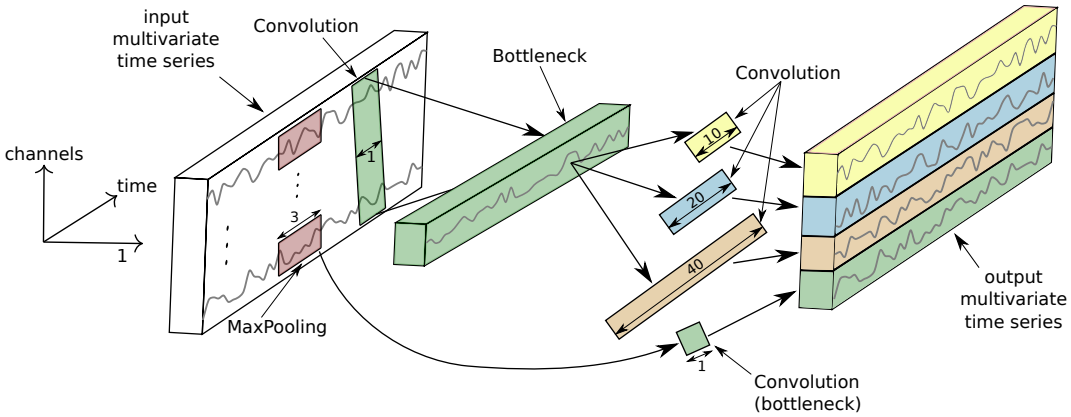


Fig. 2: Inside our Inception module for time series classification. For simplicity we illustrate a bottleneck layer of size $m = 1$.

Inception modules rather than traditional fully convolutional layers. Each residual block's input is transferred via a shortcut linear connection to be added to the next block's input, thus mitigating the vanishing gradient problem by allowing a direct flow of the gradient (He et al., 2016). Following these residual blocks, we employed a Global Average Pooling (GAP) layer that averages the output multivariate time series over the whole time dimension. At last, we used a final traditional fully-connected softmax layer with a number of neurons equal to the number of classes in the dataset. Fig. 1 depicts an Inception network's architecture showing 6 different Inception modules stacked one after the other.

As for the Inception module, Fig. 2 illustrates the inside details of this operation. Let us consider the input to be an MTS with M dimensions. The first major component of the Inception module is called the "bottleneck" layer. This layer performs an operation of sliding m filters of length 1 with a stride equal to 1. This will transform the time series from an MTS with M dimensions to an MTS with $m \ll M$ dimensions, thus reducing significantly the dimensionality of the time series as well as the model's complexity and mitigating overfitting problems for small datasets. Note that for visualization purposes, Fig. 2 illustrates a bottleneck layer with $m = 1$. Finally, we should mention that this bottleneck technique allows the Inception network to have much longer filters than ResNet (almost ten times) with roughly the same number of parameters to be learned, since without the bottleneck layer, the filters will have M dimensions compared to $m \ll M$ when using the bottleneck layer. The second major component of the Inception module is sliding multiple filters of different lengths simultaneously on the same input time series. For example in Fig. 2, three different convolutions with length $l \in \{10, 20, 40\}$ are applied to the input MTS, which is technically the output of the bottleneck layer. Additionally, in order to make our model invariant to small perturbations, we introduce another parallel MaxPooling operation, followed by a bottleneck layer to reduce the dimensionality. The output of sliding a MaxPooling window is computed by taking the maximum value in this given window of time series. Finally, the output of each independent parallel convolution/MaxPooling is concatenated to form the output MTS. The latter operations are repeated for each individual Inception module of the proposed network.

By stacking multiple Inception modules and training the weights (filters' values) via backpropagation, the network is able to extract latent hierarchical features of multiple resolutions thanks to the use of filters with various lengths. For completeness, we specify the exact number of filters for our proposed Inception module: 3 sets of filters each with 32 filters of length $l \in \{10, 20, 40\}$ with MaxPooling added to the mix, thus making the total number of filters per layer equal to

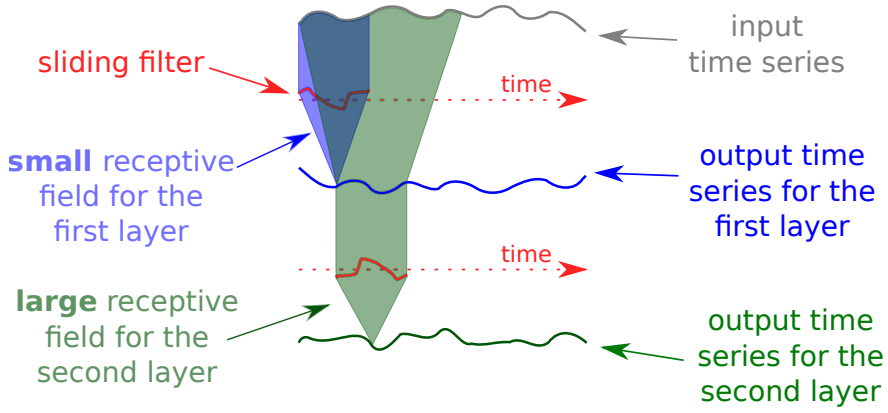


Fig. 3: Receptive field illustration for a two layers CNN

$32 \times 4 = 128 = M$ - the dimensionality of the output MTS. The default bottleneck size value was set to $m = 32$.

3.2 InceptionTime: a neural network ensemble for TSC

Our proposed state-of-the-art InceptionTime model is an ensemble of 5 Inception networks, with each prediction given an even weight. In fact, during our experimentation, we have noticed that a single Inception network exhibits high standard deviation in accuracy, which is very similar to ResNet's behavior (Ismail Fawaz et al., 2019c). We believe that this variability comes from both the randomly initialized weights and the stochastic optimization process itself. This was an important finding for us, previously observed in Scardapane and Wang (2017), as rather than training only one, potentially very good or very poor, instance of the Inception network, we decided to leverage this instability through ensembling, creating InceptionTime. The following equation explains the ensembling of predictions made by a network with different initializations:

$$\hat{y}_{i,c} = \frac{1}{n} \sum_{j=1}^{n} \sigma_c(x_i, \theta_j) \quad | \quad \forall c \in [1, C] \tag{1}$$

with $\hat{y}_{i,c}$ denoting the ensemble's output probability of having the input time series x_i belonging to class c, which is equal to the logistic output σ_c averaged over the n randomly initialized models. More details on ensembling neural networks for TSC can be found in Ismail Fawaz et al. (2019c). As for the proposed model in this paper, we chose the number of individual classifiers to be equal to 5, which is justified in Section 5. We should note that we have opted to a neural network ensemble given the small training size of the UCR archive datasets which are not well suited to deep learning approaches, thus allowing us to control and leverage the variance of the error, which is likely to reduce when increasing the training set's size.

3.3 Receptive field

The concept of Receptive Field (RF) is an essential tool to the understanding of deep CNNs (Luo et al., 2016). Unlike fully-connected networks or Multi-Layer Perceptrons, a neuron in a CNN depends only on a region of the input signal. This region in the input space is called the receptive field of that particular neuron. For computer vision problems this concept was extensively studied,

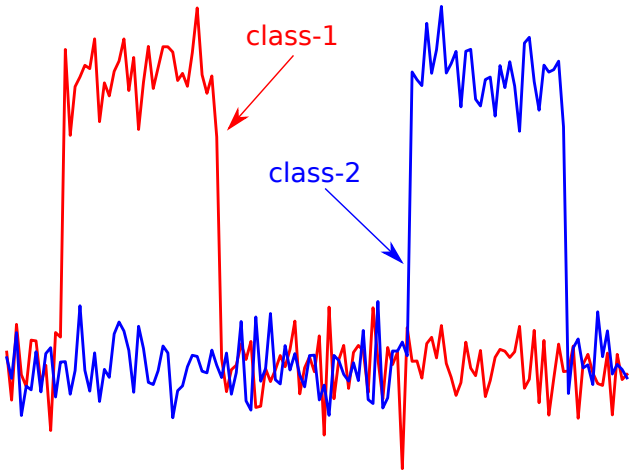


Fig. 4: Example of a synthetic binary time series classification problem

such as in Liu et al. (2018) where the authors compared the effective and theoretical receptive fields of a CNN for image segmentation.

For temporal data, the receptive field can be considered as a theoretical value that measures the maximum field of view of a neural network in a one-dimensional space: the larger it is, the better the network becomes (in theory) in detecting longer patterns. We now provide the definition of the RF for time series data, which is later used in our experiments. Suppose that we are sliding convolutions with a stride equal to 1. The formula to compute the RF for a network of depth d with each layer having a filter length equal to k_i with $i \in [1, d]$ is:

$$1 + \sum_{i=1}^{d} (k_i - 1) \tag{2}$$

By analyzing equation 2 we can clearly see that adding two layers to the initial set of d layers, will increase only slightly the value of RF. In fact in this case, if the old RF value is equal to $RF^{'}$, the new value RF will be equal to $RF^{'} + 2 \times (k - 1)$. Conversely, by increasing the filter length k_i, $\forall i \in [1, d]$ by 2, the new value RF will be equal to $RF^{'} + 2 \times d$. This is rather expected since by increasing the filter length for all layers, we are actually increasing the RF for each layer in the network. Fig. 3 illustrates the RF for a two layers CNN.

In this paper, we chose to focus on the RF concept since it has been known for computer vision problems, that larger RFs are required to capture more context for object recognition (Luo et al., 2016). Following the same line of thinking, we hypothesize that detecting larger patterns from very long one-dimensional time series data, requires larger receptive fields.

4 Experimental setup

First, we detail the method to generate our synthetic dataset, which is later used in our architecture and hyperparameter study. For testing our different deep learning methods, we created our own synthetic TSC dataset. The goal was to be able to control the length of the time series data as well as the number of classes and their distribution in time. To this end, we start by generating a univariate time series using uniformly distributed noise sampled between 0.0 and 0.1. Then in order to assign this synthetic random time series to a certain class, we inject a pattern with an

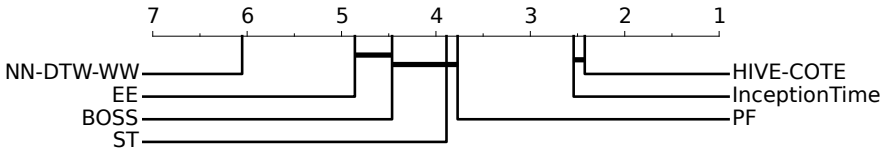


Fig. 5: Critical difference diagram showing the performance of InceptionTime compared to the current state-of-the-art classifiers of time series data.

amplitude equal to 1.0 in a pre-defined region of the time series. This region will be specific to a certain class, therefore by changing the placement of this pattern we can generate an unlimited amount of classes, whereas the random noise will allow us to generate an unlimited amount of time series instances per class. One final note is that we have fixed the length of the pattern to be equal to 10% the length of the synthetic time series. An example of a synthetic binary TSC problem is depicted in Fig. 4.

All deep neural networks were trained by leveraging the parallel computation of a remote cluster of more than 60 GPUs comprised of GTX 1080 Ti, Tesla K20, K40 and K80. Local testing and development was performed on an NVIDIA Quadro P6000. The latter graphics card was also used for computing the training time of a model. When evaluating global accuracy and computational complexity, we have used the UCR archive (Dau et al., 2018), which is the largest publicly available archive for TSC. The models were trained/tested using the original training/testing splits provided in the archive. To study the effect of different hyperparameters and architectural designs, we used in addition to the traditional UCR benchmark for TSC, the synthetic dataset whose generation is described in details in the previous paragraph. All time series data were z-normalized (including the synthetic series) to have a mean equal to zero and a standard deviation equal to one. This is considered a common best-practice before classifying time series data (Bagnall et al., 2017). Finally, we should note that all models are trained using the Adam optimization algorithm (Kingma and Ba, 2015) and all weights are initialized randomly using Glorot's uniform technique (Glorot and Bengio, 2010).

Similarly to Ismail Fawaz et al. (2019b), when comparing with the state-of-the-art results published in Bagnall et al. (2017) we used the deep learning model's median test accuracy over the different runs. Following the recommendations in Demšar (2006) we adopted the Friedman test (Friedman, 1940) in order to reject the null hypothesis. We then performed the pairwise post-hoc analysis recommended by Benavoli et al. (2016) where we replaced the average rank comparison by a Wilcoxon signed-rank test with Holm's alpha (5%) correction (Garcia and Herrera, 2008). To visualize this type of comparison we used a critical difference diagram proposed by Demšar (2006), where a thick horizontal line shows a cluster of classifiers (a clique) that are not-significantly different in terms of accuracy.

In order to allow for the time series community to build upon and verify our findings, the source code for all these experiments was made publicly available on our companion repository[1]. In addition, we provide the pre-trained deep learning models, thus allowing data mining practitioners to leverage these models in a transfer learning setting (Ismail Fawaz et al., 2018).

5 Experiments: InceptionTime

In this section, we present the results of our proposed novel classifier called InceptionTime, evalu-

[1] https://github.com/hfawaz/InceptionTime

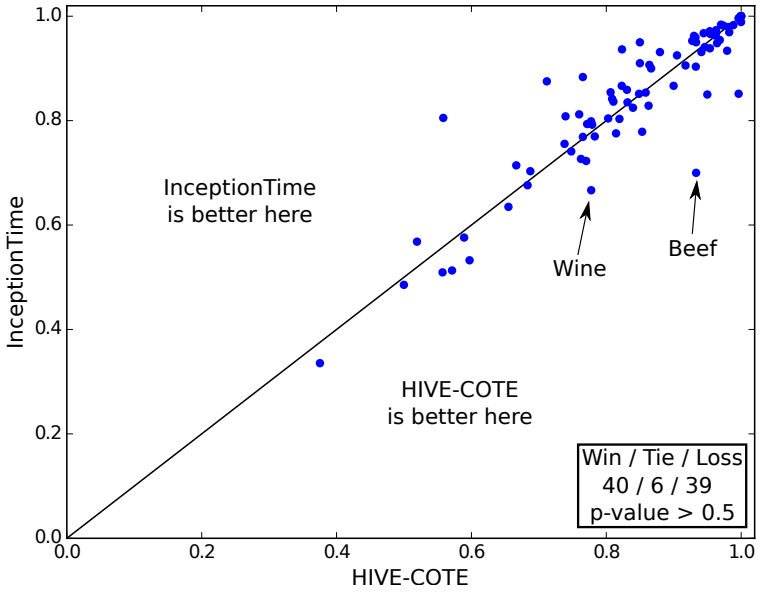


Fig. 6: Accuracy plot showing how our proposed InceptionTime model is not significantly different than HIVE-COTE.

ated on the 85 datasets of the UCR archive. We note that throughout the paper (unless specified otherwise) InceptionTime refers to an ensemble of 5 Inception networks, while the "InceptionTime(n)" notation is used to denote an ensemble of n Inception networks.

Fig. 5 illustrates the critical difference diagram with InceptionTime added to the mix of the current state-of-the-art classifiers for time series data, whose results were taken from Bagnall et al. (2017). We can see here that our InceptionTime ensemble reaches competitive accuracy with the class-leading algorithm HIVE-COTE, an ensemble of 37 TSC algorithms with a hierarchical voting scheme (Lines et al., 2016). While the two algorithms share the same clique on the critical difference diagram, the trivial GPU parallelization of deep learning models makes learning our InceptionTime model a substantially easier task than training the 37 different classifiers of HIVE-COTE, whose implementation does not trivially leverage the GPUs' computational power.

To further visualize the difference between the InceptionTime and HIVE-COTE, Fig. 6 depicts the accuracy plot of InceptionTime against HIVE-COTE for each of the 85 UCR datasets. The results show a Win/Tie/Loss of 40/6/39 in favor of InceptionTime, however the difference is not statistically significant as previously discussed. From Fig. 6, we can also easily spot the two datasets for which InceptionTime noticeably under-performs (in terms of accuracy) with respect to HIVE-COTE: Wine and Beef. These two datasets contain spectrography data from different types of beef/wine, with the goal being to determine the correct type of meat/wine using the recorded time series data. Recently, transfer learning has been shown to significantly increase the accuracy for these two datasets, especially when fine-tuning a dataset with similar time series data (Ismail Fawaz et al., 2018). Our results suggest that further potential improvements may be available for InceptionTime when applying a transfer learning approach, as recent discoveries in Kashiparekh et al. (2019) show that the various filter lengths of the Inception modules have been shown to benefit more from fine-tuning than networks with a static filter length.

Now that we have demonstrated that our proposed technique is able to reach the current state-of-the-art accuracy for TSC problems, we will further investigate the time complexity of our model. Note that during the following experiments, we ran our ensemble on a single Nvidia Quadro P6000

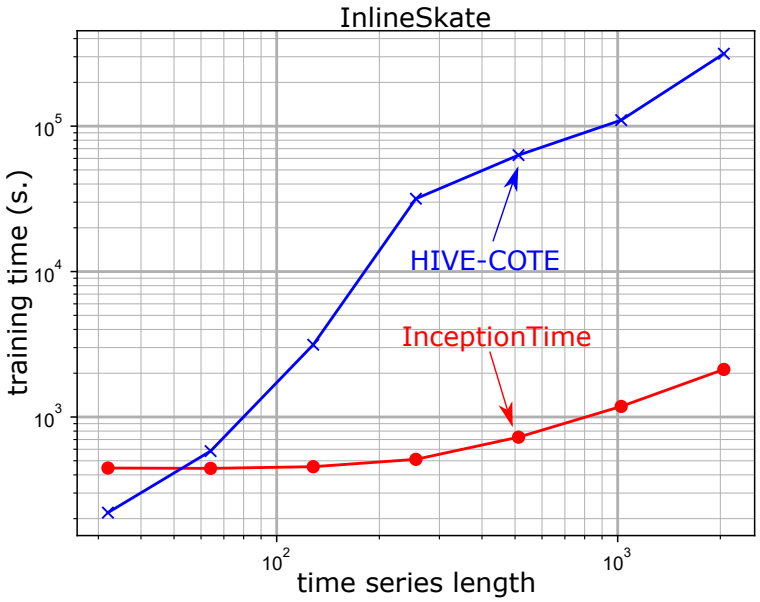


Fig. 7: Training time as a function of the series length for the InlineSkate dataset.

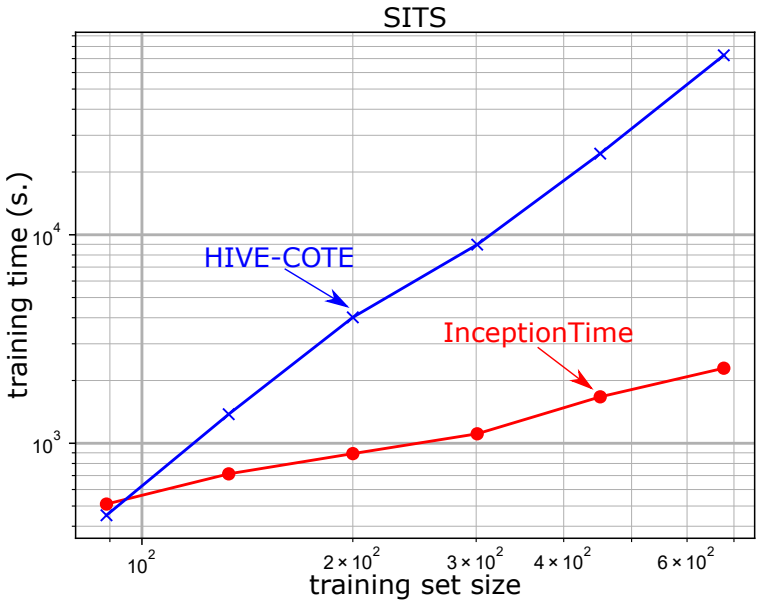


Fig. 8: Training time as a function of the training set size for the SITS dataset.

in a sequential manner, meaning that for InceptionTime, 5 different Inception networks were trained one after the other. Therefore we did not make use of our remote cluster of GPUs. First we start by investigating how our algorithm scales with respect to the length of the input time series. Fig. 7 shows the training time versus the length of the input time series. For this experiment, we used the InlineSkate dataset with an exponential re-sampling. We can clearly see that InceptionTime's complexity increases almost linearly with an increase in the time series' length, unlike HIVE-COTE, whose execution is almost two order of magnitudes slower. Having showed that InceptionTime is significantly faster when dealing with long time series, we now proceed to evaluating the training time with respect to the number of time series in a dataset. To this end, we used a Satellite Image

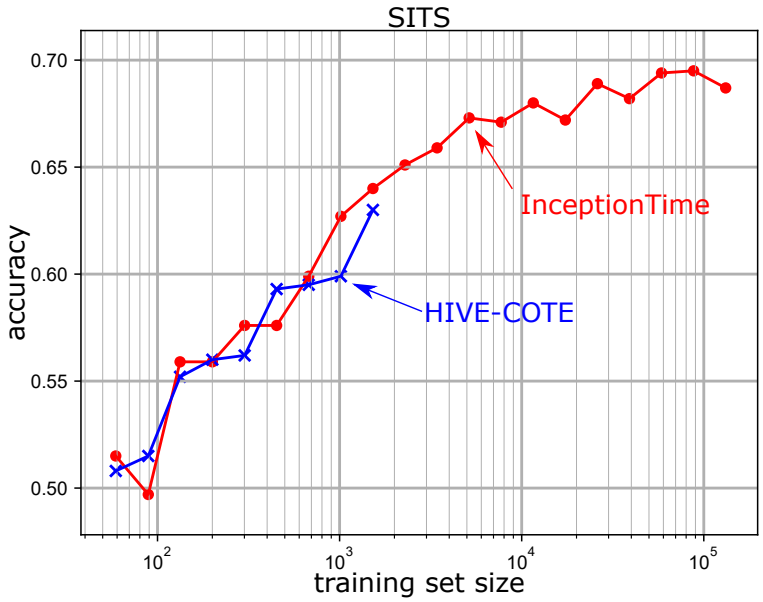


Fig. 9: Accuracy as a function of the training set size for the SITS dataset.

Time Series dataset (Tan et al., 2017). The data contain approximately one million time series, each of length 46 and labelled as one of 24 possible land-use classes (e.g. 'wheat', 'corn', 'plantation', 'urban'). From Fig. 8 we can easily see how our InceptionTime is an order of magnitude faster than HIVE-COTE, and the trend suggests that this difference will only continue to grow, rendering InceptionTime a clear favorite classifier in the Big Data era. Note that HIVE-COTE uses heuristics in its implementation, which explains why the complexity appears lower in the experiments than the expected $O(T^4)$. To summarize, we believe that InceptionTime should be considered as one of the top state-of-the-art methods for TSC, given that it demonstrates equal accuracy to that of HIVE-COTE (see Fig. 6) while being much faster (see Fig. 7 and 8).

In order to further demonstrate the capability of InceptionTime to handle efficiently a large amount of training samples unlike its counterpart HIVE-COTE, we show in Fig. 9 how the accuracy continues to increase with InceptionTime for larger training set sizes, where HIVE-COTE would take 100 times longer to run.

The pairwise accuracy plot in Fig. 10 compares InceptionTime to a model we call ResNet(5), which is an ensemble of 5 different ResNet networks (Ismail Fawaz et al., 2019c). We found that InceptionTime showed a significant improvement over its neural network competitor, the previous best deep learning ensemble for TSC. Specifically, our results show a Win/Tie/Loss of 54/8/23 in favor of InceptionTime against ResNet(5) with a p-value < 0.01, suggesting the significant gain in performance is mainly due to improvements in our proposed Inception network architecture. Additionally, in order to have a fair comparison between ResNet(5) and InceptionTime, we fixed the batch size of ResNet to 64 – equal to the default value used for InceptionTime. This would further highlight that the improvement is mainly due to the architectural design of our proposed network, and not due to some other optimization hyperparameter such as the batch size. Finally, we would like to note that when using the original batch size value proposed by Wang et al. (2017) for ResNet, we observed similar results: InceptionTime was significantly better than the original ResNet(5) with a Win/Tie/Loss of 53/7/25.

In order to better understand the effect of the randomness on the accuracy of our neural networks, we present in Fig. 11 the critical difference diagram of different InceptionTime(x) ensembles

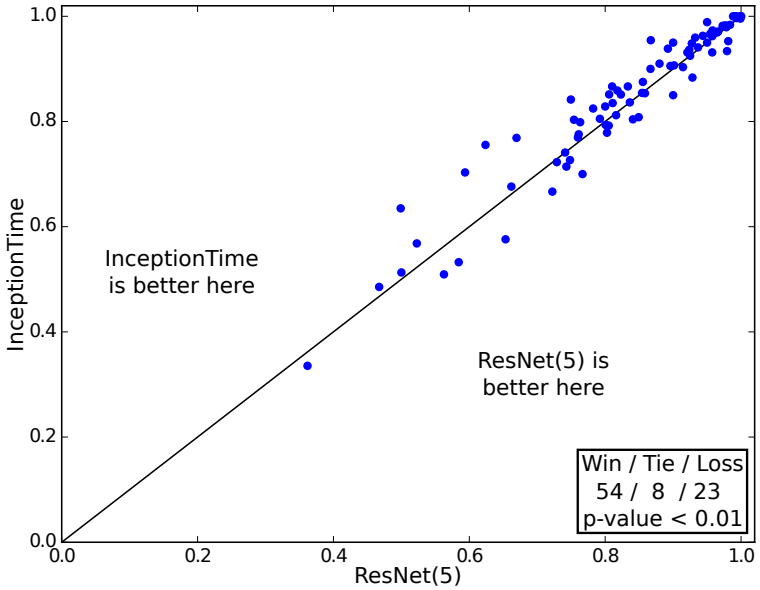


Fig. 10: Plot showing how InceptionTime significantly outperforms ResNet(5).

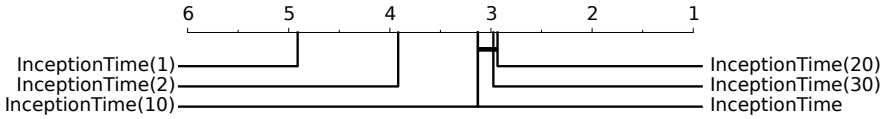


Fig. 11: Critical difference diagram showing the effect of the number of individual classifiers in the InceptionTime ensemble.

with $x \in \{1, 2, 5, 10, 20, 30\}$ denoting the number of individual networks in the ensemble. Note that InceptionTime(1) is equivalent to a single Inception network and InceptionTime is equivalent to InceptionTime(5). By observing Fig. 11 we notice how there is no significant improvement when $x \geq 5$, which is why we chose to use an ensemble of size 5, to minimize the classifiers' training time.

6 Architectural Hyperparameter study

In this section, we will further investigate the hyperparameters of our deep learning architecture and the characteristics of the Inception module in order to provide insight for practitioners looking at optimizing neural networks for TSC. First, we start by investigating the batch size hyperparameter, since this will greatly influence training time of all of our models. Then we investigate the effectiveness of residual and bottleneck connections, both of which are present in InceptionTime. After this we will experiment on model depth, filter length, and number of filters. In all experiments the default values for InceptionTime are: batch size 64; bottleneck size 32; depth 6; filter length {10,20,40}; and, number of filters 32. Finally, since the train/test split (provided in the archive) does not help in estimating the generalization ability of our approach, we have conducted a sensitivity analysis that evaluates the second best value for each of the network's hyperparameters (see subsection 6.6).

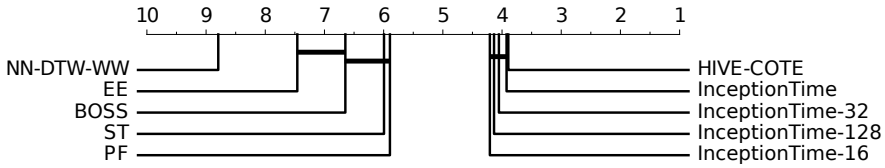


Fig. 12: Critical difference diagram showing the effect of the batch size hyperparameter value over InceptionTime's average rank.

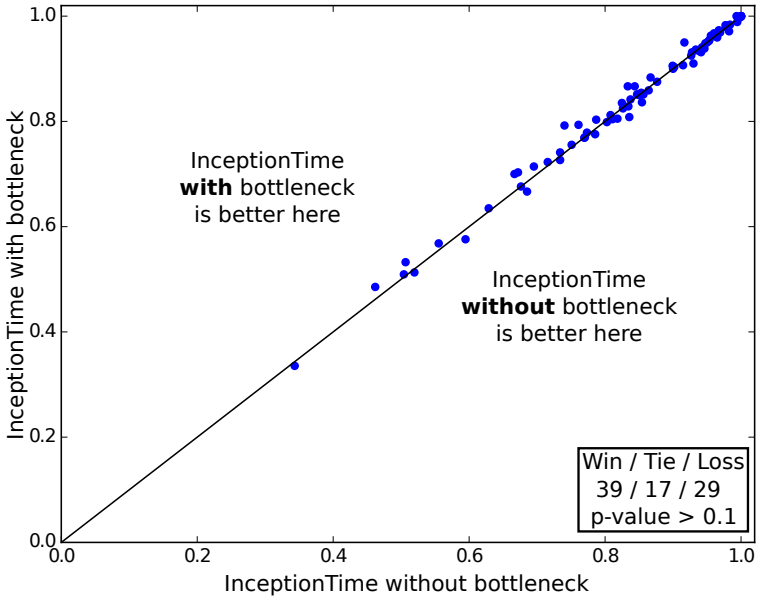


Fig. 13: Accuracy plot for InceptionTime with/without the bottleneck layer.

6.1 Batch size

We started by investigating the batch size hyperparameter on the UCR archive, since this will greatly influence the training time of our models. The critical difference diagram in Fig. 12 shows how the batch size affects the performance of InceptionTime. The horizontal thick line between the different models shows a non significant difference between them when evaluated on the 85 datasets, with a small superiority to InceptionTime (batch size equal to 64). Finally, we should note that as we did not observe any significant impact on accuracy we did not study the effect of this hyperparameter on the simulated dataset and we chose to fix the batch size to 64 (similarly to InceptionTime) when experimenting on the simulated dataset below.

6.2 Bottleneck and residual connections

In Ismail Fawaz et al. (2019b), compared to other deep learning classifiers, ResNet achieved the best classification accuracy when evaluated on the 85 datasets and as a result we chose to look at the specific characteristic of this architecture — its residual connections. Additionally, we tested one of the defining characteristics of Inception — the bottleneck feature. For the simulated dataset, we did not observe any significant impact of these two connections, we therefore proceed with experimenting on the 85 datasets from the UCR archive.

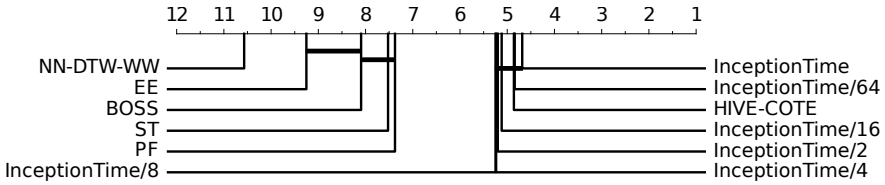


Fig. 14: Critical difference diagram showing how the network's bottleneck size affects InceptionTime' average rank.

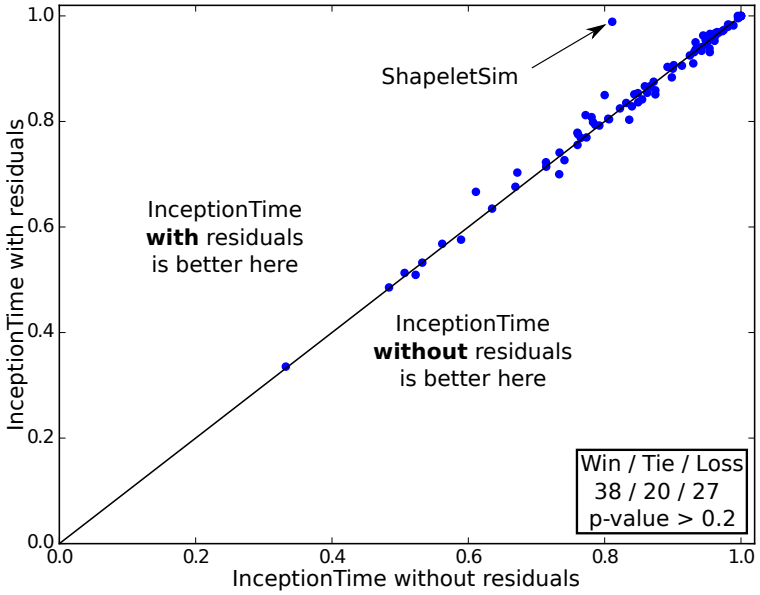


Fig. 15: Accuracy plot for InceptionTime with/without the residual connections.

Fig. 13 shows the pairwise accuracy plot comparing InceptionTime with/without the bottleneck. Similar to the experiments on the simulated dataset, we did not find any significant variation in accuracy when adding or removing the bottleneck layer.

In fact, using a Wilcoxon Signed-Rank test we found that InceptionTime with the bottleneck layer is only slightly better than removing the bottleneck layer (p-value > 0.1). In terms of accuracy, these results all suggest not to use a bottleneck layer, however we should note that the major benefit of this layer is to significantly decrease the number of parameters in the network. In this case, InceptionTime with the bottleneck contains almost half the number of parameters to be learned, and given that it does not significantly decrease accuracy, we chose to retain its usage. In a more general sense, these experiments suggest that choosing whether or not to use a bottleneck layer is actually a matter of finding a balance between a model's accuracy and its complexity. The latter observation is evident in Fig. 14 where choosing smaller bottleneck size in order to reduce InceptionTime's runtime will result in small yet insignificant decrease in accuracy.

To test the residual connections, we simply removed the residual connection from InceptionTime. Thus, without any shortcut connection, InceptionTime will simply become a deep convolutional neural network with stacked Inception modules. Fig. 15 shows how the residual connections have a minimal effect on accuracy when evaluated over the whole 85 datasets in the UCR archive with a p-value > 0.2.

This result was unsurprising given that for computer vision tasks residual connections are known to improve the convergence rate of the network but not alter its test accuracy (Szegedy et al., 2017).

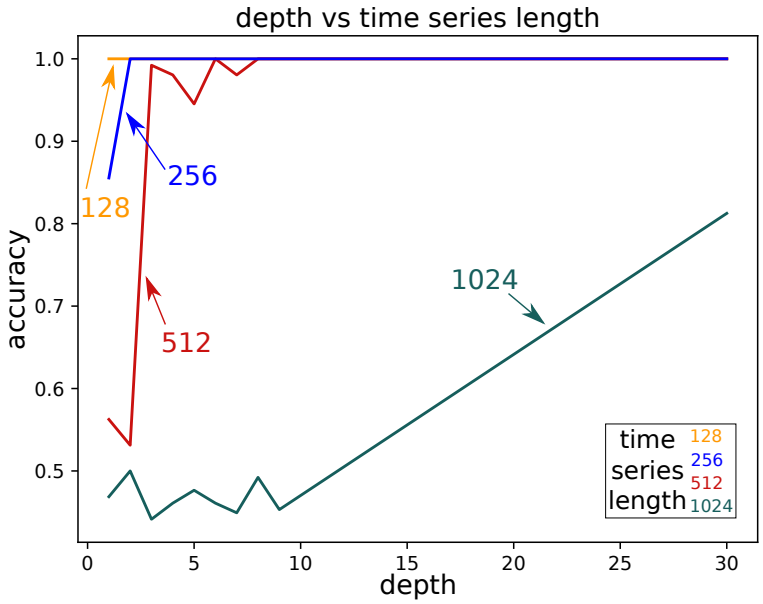


Fig. 16: Inception network's accuracy over the simulated dataset, with respect to the network's depth as well as the length of the input time series.

However, for some datasets in the archive, the residual connections did not show any improvement nor deterioration of the network's convergence either. This could be linked to other factors that are specific to these data, such as the complexity of the dataset.

One example of interest that we noticed was a significant decrease in InceptionTime's accuracy when removing the residual component for the ShapeletSim dataset. This is a synthetic dataset, designed specifically for shapelets discovery algorithms, with shapelets (discriminative subsequences) of different lengths (Hills et al., 2014). Further investigations on this dataset indicated that InceptionTime without the residual connections suffered from a severe overfitting.

While not the case here, some research has observed benefits of skip, dense or residual connections (Huang et al., 2017). Given this, and the small amount of labeled data available in TSC compared to computer vision problems, we believe that each case should be independently studied whether to include residual connections. The latter observation suggests that a large scale general purpose labeled dataset similar to ImageNet (Russakovsky et al., 2015) is needed for TSC. Finally, we should note that the residual connection has a minimal impact on the network's complexity (Szegedy et al., 2017).

6.3 Depth

Most of deep learning's success in image recognition tasks has been attributed to how 'deep' the architectures are (LeCun et al., 2015). Consequently, we decided to further investigate how the number of layers affects a network's accuracy. Unlike the previous hyperparameters, we present here the results on the simulated dataset. Apart from the depth parameter, we used the default values of InceptionTime. For this dataset we fixed the number of training instances to 256 and the number of classes to 2 (see Fig. 4 for an example). The only dataset parameter we varied was the length of the input time series.

Fig. 16 illustrates how the model's accuracy varies with respect to the network's depth when classifying datasets of time series with different lengths. Our initial hypothesis was that as longer

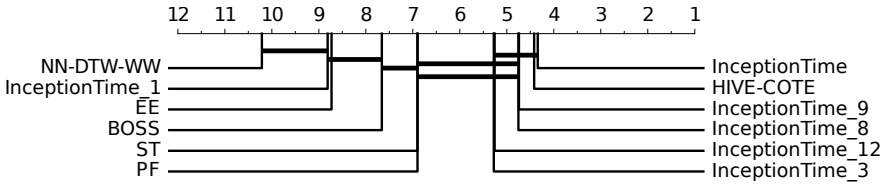


Fig. 17: Critical difference diagram showing how the network's depth affects InceptionTime' average rank.

time series can potentially contain longer patterns and thus should require longer receptive fields in order for the network to separate the classes in the dataset. In terms of depth, this means that longer input time series will garner better results with deeper networks. And indeed, when observing Fig. 16, one can easily spot this trend: deeper networks deliver better results for longer time series.

In order to further see how much effect the depth of a model has on real TSC datasets, we decided to implement deeper and shallower InceptionTime models, by varying the depth between 1 layer and 12 layers. In fact, compared with the original architecture proposed by Wang et al. (2017), the deeper (shallower) version of InceptionTime will contain one additional (fewer) residual blocks each one comprised of three inception modules. By adding these layers, the deeper (shallower) InceptionTime model will contain roughly double (half) the number of parameters to be learned. Fig. 17 depicts the critical difference diagram comparing the deeper and shallower InceptionTime models to the original InceptionTime.

Unlike the experiments on the simulated dataset, we did not manage to improve the network's performance by simply increasing its depth. This may be due to many reasons, however it is likely due to the fact that deeper networks need more data to achieve high generalization capabilities (LeCun et al., 2015), and since the UCR archive does not contain datasets with a huge number of training instances, the deeper version of InceptionTime was overfitting the majority of the datasets and exhibited a small insignificant decrease in performance. On the other hand, the shallower version of InceptionTime suffered from a significant decrease in accuracy (see InceptionTime_3 and InceptionTime_1 in Fig. 17). This suggests that a shallower architecture will contain a significantly smaller RF, thus achieving lower accuracy on the overall UCR archive.

From these experiments we can conclude that increasing the RF by adding more layers will not necessarily result in an improvement of the network's performance, particularly for datasets with a small training set. However, one benefit that we have observed from increasing the network's depth, is to choose an RF that is long enough to achieve good results without suffering from overfitting.

We therefore proceed by experimenting with varying the RF by varying the filter length.

6.4 Filter length

In order to test the effect of the filter length, we start by analyzing how the length of a time series influences the accuracy of the model when tuning this hyperparameter. In these experiments we fixed the number of training time series to 256 and the number of classes to 2. Fig. 18 illustrates the results of this experiment.

We can easily see that as the length of the time series increases, a longer filter is required to produce accurate results. This is explained by the fact that longer kernels are able to capture longer patterns, with higher probability, than shorter ones can. Thus, we can safely say that longer kernels almost always improve accuracy.

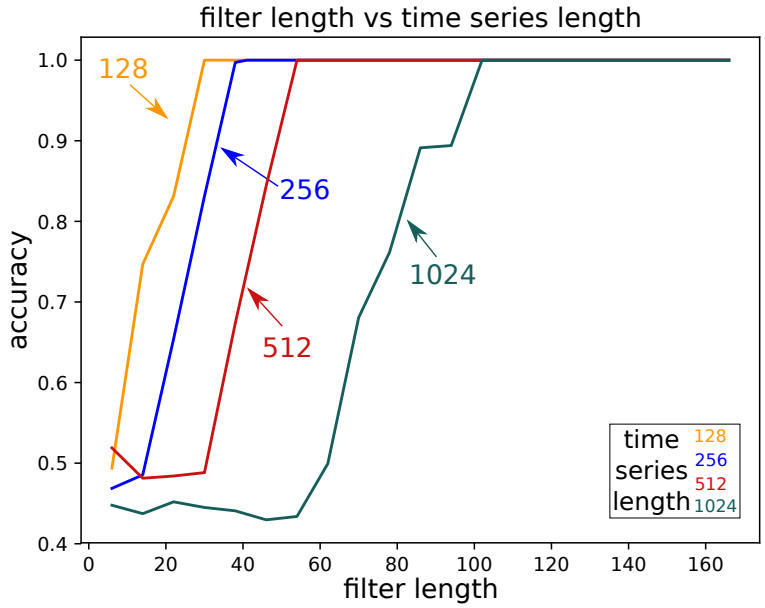


Fig. 18: Inception network's accuracy over the simulated dataset, with respect to the filter length as well as the input time series length.

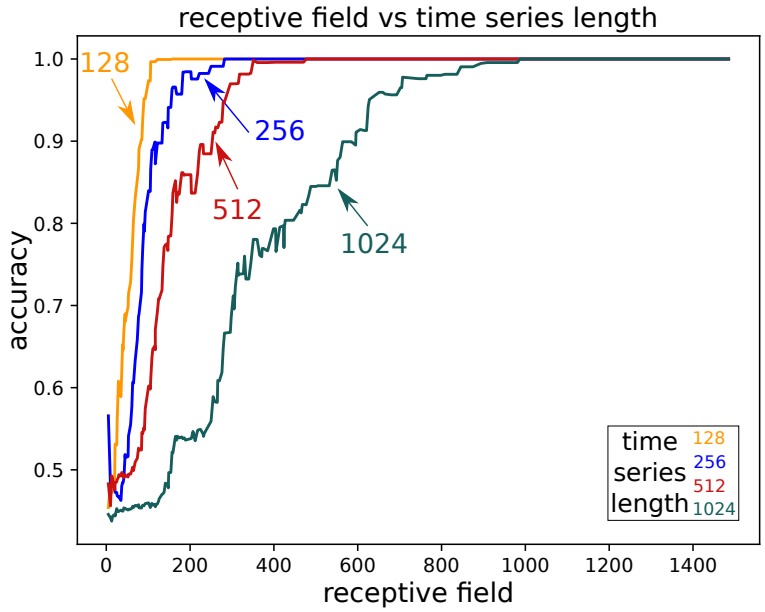


Fig. 19: Inception network's accuracy over the simulated dataset, with respect to the receptive field as well as the input time series length.

In addition to having visualized the accuracy as a function of both depth (Fig. 16) and filter length (Fig. 18), we proceed by plotting the accuracy as function of the RF for the simulated time series dataset with various lengths. By observing Fig. 19 we can confirm the previous observations that longer patterns require longer RFs, with length clearly having a higher impact on accuracy compared to the network's depth. Moreover, by using a large enough RF to cover the whole input time series, the usage of a GAP layer won't affect InceptionTime's ability to discriminate between the two patterns, because performing a GAP does not affect the model's RF.

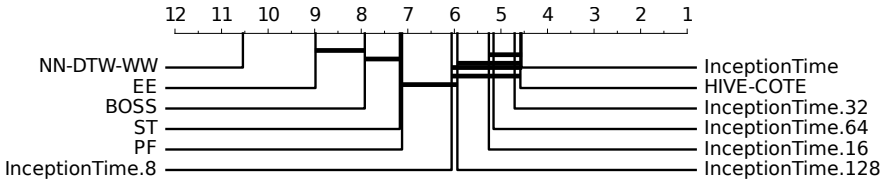


Fig. 20: Critical difference diagram showing the effect of the filter length hyperparameter value over InceptionTime' average rank.

Length	InceptionTime.8	InceptionTime.64	InceptionTime
<81	**1.71**	2.21	1.79
81-250	1.89	2.11	**1.42**
251-450	2.45	**1.32**	1.86
451-700	2.08	1.85	**1.62**
701-1000	**1.50**	2.60	1.80
>1000	2.14	2.00	**1.71**

Table 1: Filter length variants of InceptionTime with their corresponding average ranks grouped by the datasets' length. Bold indicates the best model.

There is a downside to longer filters however, in the potential for overfitting small datasets, as longer filters significantly increase the number of parameters in the network. To answer this question, we again extend our experiments to the real data from the UCR archive, allowing us to verify whether long kernels tend to overfit the datasets when a limited amount of training data is available. Therefore, we decided to train and evaluate InceptionTime versions containing both long and short filters on the UCR archive. Where the original InceptionTime contained filters of length {10,20,**40**}, the five models we are testing here contain filters of length {2,4,**8**}; {4,8,**16**}; {8,16,**32**}; {16,32,**64**}; {32,64,**128**}. Fig. 20 illustrates a critical difference diagram showing how InceptionTime with longer filters will slightly decrease the network's performance in terms of accurately classifying the time series datasets. We also investigate the relationship between the length of the time series and the length of the network's filter. Table 1 depicts the average rank of each variant of InceptionTime over the UCR archive grouped by the datasets' lengths (with about 15 datasets in each group). Similarly to Fig. 20, we observe that almost for all time series length, InceptionTime with its default filter length (32) achieves the best or the second best overall accuracy. We can therefore summarize that the results from the simulated dataset do generalize (to some extent) to real datasets: longer filters will improve the model's performance as long as there is enough training data to mitigate the overfitting phenomena.

In summary, we can confidently state that increasing the receptive field of a model by adopting longer filters will help the network in learning longer patterns present in longer time series. However there is an accompanying disclaimer that it may negatively impact the accuracy for some datasets due to overfitting.

6.5 Number of filters

To provide some directions on how the number of filters affects the performance of the network, we experimented with varying this hyperparameter with respect to the number of classes in the dataset. To generate new classes in the simulated data, we varied the position and length of the patterns; for example, to create data with three classes, we inject patterns of the same length at

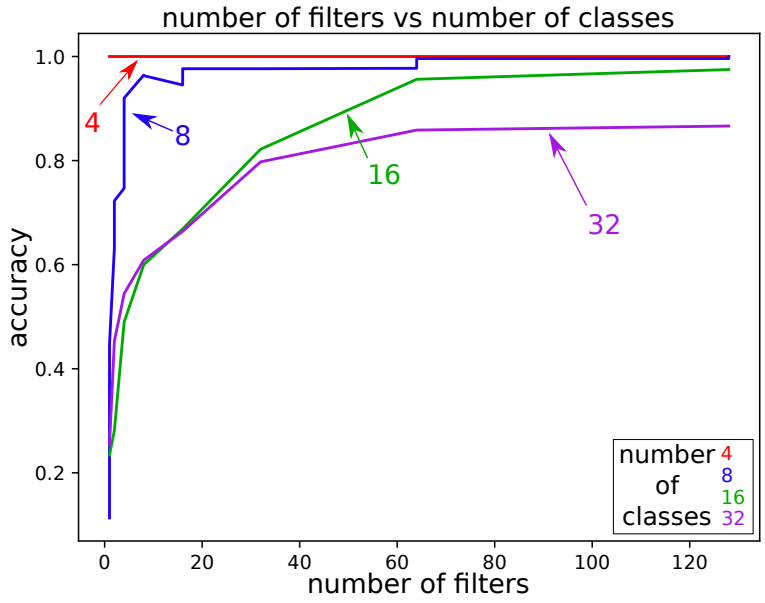


Fig. 21: Inception network's accuracy over the simulated dataset, with respect to the number of filters as well as the number of classes.

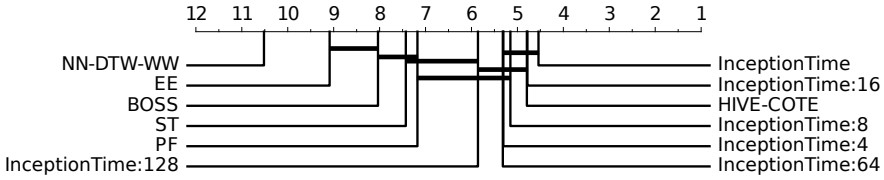


Fig. 22: Critical difference diagram showing how the network's width affects InceptionTime' average rank.

three different positions. For this series of experiments, we fixed the length of the time series to 256 and the number of training examples to 256.

Fig. 21 depicts the network's accuracy with respect to the number of filters for datasets with a differing number of classes. Our prior intuition was that the more classes, or variability, present in the training set, the more features are required to be extracted in order to discriminate the different classes, and this will necessitate a greater number of filters. This is confirmed by the trend displayed in Fig. 21, where the datasets with more classes require more filters to be learned in order to be able to accurately classify the input time series.

After observing on the synthetic dataset that the number of filters significantly affects the performance of the network, we asked ourselves if the current implementation of InceptionTime could benefit/lose from a naive increase/decrease in the number of filters per layer. Our proposed InceptionTime model contains 32 filters per Inception module's component, while for these experiments we tested six ensembles, by varying the hyperparameter with a power of two. Fig. 22 illustrates a critical difference diagram showing how increasing the number of filters per layer significantly deteriorated the accuracy of the network, whereas decreasing the number of filters did not significantly affect the accuracy. It appears that our InceptionTime model contains enough filters to separate the classes of the 85 UCR datasets, of which some have up to 60 classes (ShapesAll dataset).

Increasing the number of filters also has another side effect: it causes an explosion in the number of parameters in the network. The wider InceptionTime contains four times the number of parame-

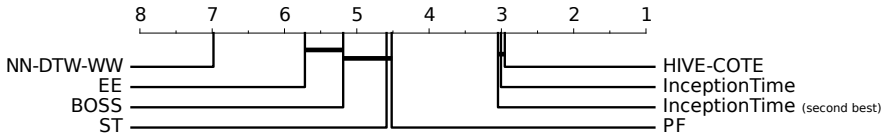


Fig. 23: Critical difference diagram showing how choosing the second best hyperparameters affects InceptionTime's average rank.

ters than the original implementation. We therefore conclude that naively increasing the number of filters is actually detrimental, as it will drastically increase the network's complexity and eventually cause overfitting.

6.6 Sensitivity analysis

Working with open benchmarks such as the UCR archive has pushed the community towards publishing high quality TSC algorithms. The UCR archive provides a train/test split for the data, which has allowed researchers to directly benchmark their works with the ones of others, as well as providing splits that were potentially more challenging and realistic than assuming that both train and test data were sampled from the same population. Having the train/test split available has however also led to the potential issue that the techniques designed on this benchmark archive might overfit it. This is especially true of deep learning classifiers that contain dozens of optimization and architectural hyperparameters (Ismail Fawaz et al., 2019b).

In an effort to give an idea of the sensitivity of InceptionTime to changes in its parameters, we have evaluated the performance of having chosen the second-best value of each of its parameters, that is the second-best value for the depth of the network (i.e. a value of 9 instead of the best value of 6), for its width (i.e. 16 instead of 32), for the length of the convolutions (final value of 32 instead of 40), for the batch size (i.e. 32 instead of 64), and for the bottleneck size (i.e. 64 instead of the default one 32). This gave us a new architecture — $\mathsf{InceptionTime}_{\text{(second best)}}$ — which we then compared with InceptionTime and also other algorithms. Fig. 23 depicts the average rank of current state-of-the-art TSC algorithms with both InceptionTime's default and second best hyperparameters added to the mix. We can clearly see that the effect is minimal: the ranking is a tiny bit lower but they are all well within the critical difference with HIVE-COTE (a post-hoc statistical test fails to reject the null hypothesis (p-value ≈ 0.71) making the difference between the default and second best hyperparameters non significant).

7 Conclusion

Deep learning for time series classification still lags behind neural networks for image recognition in terms of experimental studies and architectural designs. In this paper, we fill this gap by introducing InceptionTime, inspired by the recent success of Inception-based networks for various computer vision tasks. We ensemble these networks to produce new state-of-the-art results for TSC on the 85 datasets of the UCR archive. Our approach is highly scalable, two orders of magnitude faster than current state-of-the-art models such as HIVE-COTE. The magnitude of this speed up is consistent across both Big Data TSC repositories as well as longer time series with high sampling rate. We further investigate the effects on overall accuracy of various hyperparameters of the CNN architecture. For these, we go far beyond the standard practices for image data, and design networks with long filters. We look at these by using a simulated dataset and frame our investigation in terms

of the definition of the receptive field for a CNN for TSC. Finally, although InceptionTime can be extended straightforwardly to multivariate data (Ismail Fawaz et al., 2019b), we would like to further explore applying to multivariate TSC the many architectural advancements in deep neural networks that are being published each year for computer vision tasks.

Acknowledgements The authors would like to thank the creators and providers of the datasets. The authors would also like to thank NVIDIA Corporation for the GPU Grant and the Mésocentre of Strasbourg for providing access to the cluster. This work was supported by the ANR TIMES project (grant ANR-17-CE23-0015) of the French Agence Nationale de la Recherche. François Petitjean is the recipient of an Australian Research Council Discovery Early Career Award (project number DE170100037) funded by the Australian Government. This material is based upon work supported by the Air Force Office of Scientific Research, Asian Office of Aerospace Research and Development (AOARD) under award number FA2386-18-1-4030.

References

Bagnall A, Lines J, Hills J, Bostrom A (2016) Time-series classification with COTE: The collective of transformation-based ensembles. In: International Conference on Data Engineering, pp 1548–1549

Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2017) The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. Data Mining and Knowledge Discovery 31(3):606–660

Benavoli A, Corani G, Mangili F (2016) Should we really use post-hoc tests based on mean-ranks? Machine Learning Research 17(1):152–161

Brunel A, Pasquet J, Pasquet J, Rodriguez N, Comby F, Fouchez D, Chaumont M (2019) A CNN adapted to time series for the classification of Supernovae. In: Electronic Imaging

Cui Z, Chen W, Chen Y (2016) Multi-scale convolutional neural networks for time series classification. ArXiv 1603.06995

Cuturi M, Blondel M (2017) Soft-dtw: a differentiable loss function for time-series. In: International Conference on Machine Learnin, pp 894–903

Dau HA, Bagnall A, Kamgar K, Yeh CCM, Zhu Y, Gharghabi S, Ratanamahatana CA, Keogh E (2018) The ucr time series archive. ArXiv

Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. Machine Learning Research 7:1–30

Forestier G, Petitjean F, Senin P, Despinoy F, Huaulmé A, Ismail Fawaz H, Weber J, Idoumghar L, Muller PA, Jannin P (2018) Surgical motion analysis using discriminative interpretable patterns. Artificial Intelligence in Medicine 91:3 – 11

Friedman M (1940) A comparison of alternative tests of significance for the problem of m rankings. The Annals of Mathematical Statistics 11(1):86–92

Garcia S, Herrera F (2008) An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. Machine learning research 9:2677–2694

Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks. In: International Conference on Artificial Intelligence and Statistics, vol 9, pp 249–256

Guan C, Wang X, Zhang Q, Chen R, He D, Xie X (2019) Towards a deep and unified understanding of deep neural models in NLP. In: International Conference on Machine Learning, pp 2454–2463

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 770–778

Hills J, Lines J, Baranauskas E, Mapp J, Bagnall A (2014) Classification of time series by shapelet transformation. Data Mining and Knowledge Discovery 28(4):851–881

Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2017) Densely connected convolutional networks. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 4700–4708

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2018) Transfer learning for time series classification. In: IEEE International Conference on Big Data, pp 1367–1376

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2019a) Adversarial attacks on deep neural networks for time series classification. In: IEEE International Joint Conference on Neural Networks

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2019b) Deep learning for time series classification: a review. Data Mining and Knowledge Discovery

Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2019c) Deep neural network ensembles for time series classification. In: IEEE International Joint Conference on Neural Networks

Ismail Fawaz H, Forestier G, Weber J, Petitjean F, Idoumghar L, Muller PA (2019d) Automatic alignment of surgical videos using kinematic data. In: Artificial Intelligence in Medicine, pp 104–113

Karimi-Bidhendi S, Munshi F, Munshi A (2018) Scalable classification of univariate and multivariate time series. In: IEEE International Conference on Big Data, pp 1598–1605

Kashiparekh K, Narwariya J, Malhotra P, Vig L, Shroff G (2019) Convtimenet: A pre-trained deep convolutional neural network for time series classification. In: IEEE International Joint Conference on Neural Networks

Keogh EJ, Pazzani MJ (2001) Derivative dynamic time warping. In: Proceedings of the 2001 SIAM International Conference on Data Mining, SIAM, pp 1–11

Kingma DP, Ba J (2015) Adam: A method for stochastic optimization. In: International Conference on Learning Representations

Krizhevsky A, Sutskever I, Hinton GE (2012) ImageNet Classification with Deep Convolutional Neural Networks. In: Advances in Neural Information Processing Systems, pp 1097–1105

Le Guennec A, Malinowski S, Tavenard R (2016) Data augmentation for time series classification using convolutional neural networks. In: ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data

LeCun Y, Bottou L, Orr GB, Müller KR (1998) Efficient backprop. In: Neural Networks: Tricks of the Trade, This Book is an Outgrowth of a 1996 NIPS Workshop, pp 9–50

LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444

Lee W, Park S, Joo W, Moon IC (2018) Diagnosis prediction via medical context attention networks using deep generative modeling. In: IEEE International Conference on Data Mining, pp 1104–1109

Lines J, Bagnall A (2015) Time series classification with ensembles of elastic distance measures. Data Mining and Knowledge Discovery 29(3):565–592

Lines J, Taylor S, Bagnall A (2016) HIVE-COTE: The hierarchical vote collective of transformation-based ensembles for time series classification. In: IEEE International Conference on Data Mining, pp 1041–1046

Liu Y, Yu J, Han Y (2018) Understanding the effective receptive field in semantic image segmentation. Multimedia Tools and Applications 77(17):22159–22171

Lucas B, Shifaz A, Pelletier C, O'Neill L, Zaidi N, Goethals B, Petitjean F, Webb GI (2019) Proximity forest: an effective and scalable distance-based classifier for time series. Data Mining and Knowledge Discovery 33(3):607–635

Luo W, Li Y, Urtasun R, Zemel R (2016) Understanding the effective receptive field in deep convolutional neural networks. In: Advances in Neural Information Processing Systems, pp 4898–4906

Marteau P (2009) Time warp edit distance with stiffness adjustment for time series matching. IEEE Transactions on Pattern Analysis and Machine Intelligence 31(2):306–318

Pelletier C, Webb GI, Petitjean F (2019) Temporal convolutional neural network for the classification of satellite image time series. Remote Sensing 11(5):523

Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L (2015) ImageNet large scale visual recognition challenge. International Journal of Computer Vision 115(3):211–252

Sabour S, Frosst N, Hinton GE (2017) Dynamic routing between capsules. In: Advances in Neural Information Processing Systems, pp 3856–3866

Scardapane S, Wang D (2017) Randomness in neural networks: an overview. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery 7(2):e1200

Schäfer P (2015a) The boss is concerned with time series classification in the presence of noise. Data Mining and Knowledge Discovery 29(6):1505–1530

Schäfer P (2015b) Scalable time series classification. Data Mining and Knowledge Discovery pp 1–26

Schäfer P, Leser U (2017) Fast and accurate time series classification with WEASEL. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, ACM, pp 637–646

Stefan A, Athitsos V, Das G (2013) The move-split-merge metric for time series. IEEE Transactions on Knowledge and Data Engineering 25(6):1425–1438

Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 1–9

Szegedy C, Ioffe S, Vanhoucke V, Alemi AA (2017) Inception-v4, inception-resnet and the impact of residual connections on learning. In: AAAI Conference on Artificial Intelligence

Tan CW, Webb GI, Petitjean F (2017) Indexing and classifying gigabytes of time series under time warping. In: Proceedings of the 2017 SIAM International Conference on Data Mining, SIAM, pp 282–290

Vlachos M, Hadjieleftheriou M, Gunopulos D, Keogh E (2006) Indexing multidimensional time-series. The VLDB Journal—The International Journal on Very Large Data Bases 15(1):1–20

Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: A strong baseline. In: International Joint Conference on Neural Networks, pp 1578–1585

Yi F, Yu Z, Zhuang F, Zhang X, Xiong H (2018) An integrated model for crime prediction using temporal and spatial factors. In: IEEE International Conference on Data Mining, pp 1386–1391

Yuan Y, Xun G, Ma F, Wang Y, Du N, Jia K, Su L, Zhang A (2018) Muvan: A multi-view attention network for multivariate temporal data. In: IEEE International Conference on Data Mining, pp 717–726

Zhang C, Tavanapong W, Kijkul G, Wong J, de Groen PC, Oh J (2018) Similarity-based active learning for image classification under class imbalance. In: IEEE International Conference on Data Mining, pp 1422–1427

Noname manuscript No.
(will be inserted by the editor)

ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels

Angus Dempster · François Petitjean · Geoffrey I. Webb

Received: date / Accepted: date

Abstract Most methods for time series classification that attain state-of-the-art accuracy have high computational complexity, requiring significant training time even for smaller datasets, and are intractable for larger datasets. Additionally, many existing methods focus on a single type of feature such as shape or frequency. Building on the recent success of convolutional neural networks for time series classification, we show that simple linear classifiers using random convolutional kernels achieve state-of-the-art accuracy with a fraction of the computational expense of existing methods.

Keywords scalable · time series classification · random · convolution

1 Introduction

Most methods for time series classification that attain state-of-the-art accuracy have high computational complexity, requiring significant training time even for smaller datasets, and simply do not scale to large datasets. This has motivated the development of more scalable methods such as Proximity Forest (Lucas et al. 2019), TS-CHIEF (Shifaz et al. 2019), and InceptionTime (Ismail Fawaz et al. 2019c).

We show that state-of-the-art classification accuracy can be achieved using a fraction of the time required by even these recent, more scalable methods, by transforming time series using random convolutional kernels, and using the transformed features to train a linear classifier. We call this method Rocket (for **R**and**O**m **C**onvolutional **KE**rnel **T**ransform).

Angus Dempster · François Petitjean · Geoffrey I. Webb
Faculty of Information Technology, Monash University, Melbourne, Australia
E-mail: {angus.dempster1,francois.petitjean,geoff.webb}@monash.edu

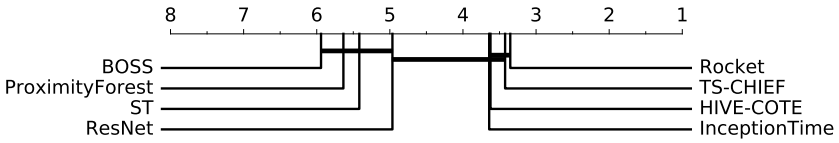


Fig. 1 Mean rank of Rocket versus state-of-the-art classifiers on the 85 'bake off' datasets.

Existing methods for time series classification typically focus on a single representation such as shape, frequency, or variance. Convolutional kernels constitute a single mechanism which can capture many of the features which have each previously required their own specialized techniques, and have been shown to be effective in convolutional neural networks for time series classification such as ResNet (Wang et al. 2017; Ismail Fawaz et al. 2019a), and InceptionTime.

In contrast to learned convolutional kernels as used in typical convolutional neural networks, we show that it is effective to generate a large number of random convolutional kernels which, in combination, capture features relevant for time series classification (even though, in isolation, a single random convolutional kernel may only very approximately capture a relevant feature in a given time series).

ROCKET achieves state-of-the-art classification accuracy on the datasets in the UCR archive (Dau et al. 2019), but requires only a fraction of the training time of existing methods. Figure 1 shows the mean rank of ROCKET versus several state-of-the-art methods for time series classification on the 85 'bake off' datasets from the UCR archive (Dau et al. 2019; Bagnall et al. 2017). Restricted to a single CPU core, the total training time for ROCKET is:

- 6 minutes for the 'bake off' dataset with the largest training set (*ElectricDevices*, with 8,926 training examples), compared to 1 hour 35 minutes for Proximity Forest, 2 hours 24 minutes for TS-CHIEF, and 7 hours 46 minutes for InceptionTime (trained on GPUs); and
- 4 minutes and 52 seconds for the 'bake off' dataset with the longest time series (*HandOutlines*, with time series of length 2,709), compared to 8 hours 10 minutes for InceptionTime (trained on GPUs), almost 3 days for Proximity Forest, and more than 4 days for TS-CHIEF.

The total compute time (training and test) for ROCKET on all 85 'bake off' datasets is 1 hour 50 minutes, compared to more than 6 days for InceptionTime (trained and tested using GPUs), and more than 11 days for each of Proximity Forest and TS-CHIEF. (Timings for ROCKET are averages over 10 runs, performed on a cluster using a mixture of Intel Xeon E5-2680 v3 and Intel Xeon Gold 6150 processors, restricted to a single CPU core per dataset per run.)

ROCKET is also more scalable for large datasets, with training complexity linear in both time series length and the number of training examples. ROCKET can learn from 1 million time series in 1 hour 15 minutes, to a similar accuracy as Proximity Forest, which requires more than 16 hours to train on the same quantity of data. A restricted variant of ROCKET can learn from the same 1 million time series in less than 1 minute, or approximately 100 times faster again, albeit to a slightly lower accuracy. ROCKET is naturally parallel, and can be made even faster by using multiple CPU cores (our implementation automatically parallelises the transform across multiple CPU cores where available) or GPUs.

The rest of this paper is structured as follows. In section 2, we review relevant related work. In section 3, we explain ROCKET in detail. In section 4, we present our experimental results, including a comparison of the accuracy of ROCKET against existing state-of-the-art classifiers on the datasets in the UCR archive, a scalability study, and a sensitivity analysis.

2 Related Work

2.1 State-of-the-Art Methods

The task of time series classification can be thought of as involving learning or detecting signals or patterns within time series associated with relevant classes. '[D]ifferent problems require different representations' (Bagnall et al. 2017, p. 647), and classes may be distinguished by multiple types of patterns: 'discriminatory features in multiple domains' (Bagnall et al. 2017, p. 645).

Different methods for time series classification represent different approaches for extracting useful features from time series (Bagnall et al. 2017). Existing approaches typically focus on a single type of feature, such as frequency or variance of the signal, or the presence of discriminative subseries (shapelets). Bagnall et al. (2017) identified COTE (since superseded by HIVE-COTE), Shapelet Transform (Hills et al. 2014; Bostrom and Bagnall 2015), and BOSS (Schäfer 2015) as the three most accurate classifiers on the UCR archive.

BOSS is one of several dictionary-based methods which use a representation based on the frequency of occurrence of patterns in time series (Bagnall et al. 2017). BOSS has a training complexity quadratic in both the number of training examples and time series length, $O(n^2 \cdot l^2)$. BOSS-VS is a more scalable variant of BOSS, but is less accurate (Schäfer 2016). Another related method, WEASEL, is more accurate than BOSS, but with a similar training complexity and high memory complexity (Schäfer and Leser 2017; see also Lucas et al. 2019).

Shapelet Transform is one of several methods based on finding discriminative subseries, so-called 'shapelets' (Bagnall et al. 2017). Shapelet Transform has a training complexity quadratic in the number of training examples, and quartic in time series length, $O(n^2 \cdot l^4)$. There are other, more scalable, shapelet methods, but these are less accurate (Bagnall et al. 2017).

HIVE-COTE is a large ensemble of other classifiers, including BOSS and Shapelet Transform, as well as classifiers based on elastic distance measures and frequency representations (Lines et al. 2018). Since Lines et al. (2018), HIVE-COTE has been considered the most accurate method for time series classification. The training complexity of HIVE-COTE is bound by the complexity of Shapelet Transform, $O(n^2 \cdot l^4)$, but its other components also have high computational complexity, such as the Elastic Ensemble with $O(n^2 \cdot l^2)$ (Lines et al. 2018).

2.2 More Scalable Methods

The high computational complexity of existing state-of-the-art methods for time series classification makes these methods slow, even for smaller datasets, and intractable for large datasets. This has motivated the development of more scalable methods, including Proximity Forest, TS-CHIEF, and InceptionTime.

Proximity Forest is an ensemble of decision trees, using elastic distance measures as splitting criteria (Lucas et al. 2019), with a training complexity quasilinear in the number of training examples, but quadratic in time series length.

TS-CHIEF builds on Proximity Forest, incorporating dictionary-based and interval-based splitting criteria (Shifaz et al. 2019). Like Proximity Forest, TS-

CHIEF has a training complexity quasilinear in the number of training examples, but quadratic in time series length.

Several methods for time series classification using convolutional neural networks have been proposed (see generally Ismail Fawaz et al. 2019a). More recently, InceptionTime (Ismail Fawaz et al. 2019c), an ensemble of five deep convolutional neural networks based on the Inception architecture, has been demonstrated to be competitive with HIVE-COTE on the UCR archive.

Convolutional neural networks are typically trained using stochastic gradient descent or closely related algorithms such as, for example, Adam (Kingma and Ba 2015). The training complexity of stochastic gradient descent is essentially linear with respect to the number of training examples, and training can be parallelised using GPUs (Goodfellow et al. 2016, pp. 147–149; Bottou et al. 2018).

2.3 Convolutional Neural Networks and Convolutional Kernels

Ismail Fawaz et al. (2019c, pp. 2–3) observe that the success of convolutional neural networks for image classification suggests that they should also be effective for time series classification, given that time series have essentially the same topology as images, with one less dimension (see also Bengio et al. 2013, p. 1820–1821).

Convolutional neural networks represent a different approach to time series classification than many other methods. Rather than approaching the problem with a preconceived representation, convolutional neural networks use convolutional kernels to detect patterns in the input. In learning the weights of the kernels, a convolutional neural network learns the features in time series associated with different classes (Ismail Fawaz et al. 2019a).

A kernel is convolved with an input time series through a sliding dot product operation, to produce a feature map which is, in turn, used as the basis for classification (see Ismail Fawaz et al. 2019a). The basic parameters of a kernel are its size (length), weights and bias, dilation, and padding (see generally Goodfellow et al. 2016, ch. 9). A kernel has the same structure as the input, but is typically much smaller. For time series, a kernel is a vector of weights, with a bias term which is added to the result of the convolution operation between an input time series and the weights of the given kernel. Dilation 'spreads' a kernel over the input such that with a dilation of two, for example, the weights in a kernel are convolved with every second element of an input time series (see Bai et al. 2018). Padding involves appending values (typically zero) to the start and end of input time series, typically such that the 'middle' weight of a given kernel aligns with the first element of an input time series at the start of the convolution operation.

Convolutional kernels can capture many of the types of features used in other methods. Kernels can capture basic patterns or shapes in time series, similar to shapelets: the convolution operation will produce large output values where the kernel matches the input. Further, dilation allows kernels to capture the same pattern at different scales (Yu and Koltun 2016). Multiple kernels in combination can capture complex patterns.

The feature maps produced in applying a kernel to a time series reflect the extent to which the pattern represented by the kernel is present in the time series. In a sense, this is not unlike dictionary methods, which are based on the frequency of occurrence of patterns in time series.

The kernels learned in convolutional neural networks often include filters for frequency (see, e.g., Krizhevsky et al. 2012; Yosinski et al. 2014; Zeiler and Fergus 2014). Saxe et al. (2011) demonstrate that even random kernels are frequency selective. Frequency information is also captured through dilation: larger dilations correspond to lower frequencies, smaller dilations to higher frequencies.

Kernels can detect patterns in time series despite warping. Pooling mechanisms make kernels invariant to the position of patterns in time series. Dilation allows kernels with similar weights to capture patterns at different scales, i.e., despite rescaling. Multiple kernels with different dilations can, in combination, capture discriminative patterns despite complex warping.

The success of convolutional neural networks for time series classification, such as ResNet and InceptionTime, demonstrates the effectiveness of convolutional kernels as the basis for time series classification.

2.4 Random Convolutional Kernels

The weights of convolutional kernels are typically learned. However, it is well established that random convolutional kernels can be effective (Jarrett et al. 2009; Pinto et al. 2009; Saxe et al. 2011; Cox and Pinto 2011).

Ismail Fawaz et al. (2019c) observe that individual convolutional neural networks exhibit high variance in classification accuracy on the UCR archive, motivating the use of ensembles of such architectures with a large number and variety of kernels (see Ismail Fawaz et al. 2019b). It may be that learning 'good' kernels is difficult on small datasets. Random convolutional kernels may have an advantage in this context (see Jarrett et al. 2009; Yosinski et al. 2014).

The idea of using convolutional kernels as a transform, and using the transformed features as the input to another classifier is well established (see, e.g., Bengio et al. 2013, 1803). Franceschi et al. (2019) present a method for unsupervised learning of convolutional kernels for a feature transform for time series input, based on a multilayer convolutional architecture with dilation increasing exponentially in each successive layer. The method is demonstrated using the output features as the input for a support vector machine.

Random convolutional kernels have been used as the basis of feature transformations. In Saxe et al. (2011), random convolutional layers are used as the basis of a feature transform (for images), used as the input for a support vector machine.

Here, there is a link between using random convolutional kernels as a transform for time series and work in relation to random transforms for kernel methods (as in support vector machines, not to be confused with convolutional kernels). Rahimi and Recht (2008) proposed a random transform for approximating kernels for kernel methods (see also Rahimi and Recht 2009). Morrow et al. (2017) propose a method for approximating a string kernel for DNA sequences, based on Rahimi and Recht (2008), which involves transforming input sequences using random convolutional kernels, and using the resulting features to train a linear classifier. Morrow et al. (2017, p. 1) describe their method as 'a 1 layer random convolutional neural network'. Also following Rahimi and Recht (2008), Jimenez and Raj (2019) propose a similar method for approximating a cross-correlation kernel for measuring similarity between time series, involving convolving input time series with random time series of the same length to produce what they

call 'random convolutional features', which can be used to train a linear classifier. Jimenez and Raj (2019) evaluate their method on a selection of binary classification datasets from the UCR archive. (In both cases, there are some differences with the convolution operation as used in typical convolutional neural networks.) Farahmand et al. (2017) propose a feature transformation based on convolving input time series with random autoregressive filters.

A number of things distinguish ROCKET from convolutional layers as used in typical convolutional neural networks, and from other methods using convolutional kernels (including random convolutional kernels) in relation to time series, set out in detail in section 3. We show that leveraging all aspects of kernel architecture—crucially, with a variety of random length, dilation, and padding (as well as weights and bias), and drawing an effective set of features from the output of the convolutions—provides for state-of-the-art accuracy with a fraction of the computational expense of existing state-of-the-art methods.

3 Method

ROCKET transforms time series using a large number of random convolutional kernels, i.e., kernels with random length, weights, bias, dilation, and padding. The transformed features are used to train a linear classifier. The combination of ROCKET and logistic regression forms, in effect, a single-layer convolutional neural network with random kernel weights, where the transformed features form the input for a trained softmax layer. However, in practice, for all but the largest datasets, we use a ridge regression classifier, which has the advantage of fast cross-validation for the regularization hyperparameter (and no other hyperparameters). Nonetheless, as logistic regression trained using stochastic gradient descent is more scalable for very large datasets, we use logistic regression when the number of training examples is substantially greater than the number of features.

Four things distinguish ROCKET from convolutional layers as used in typical convolutional neural networks, and from previous work using convolutional kernels (including random kernels) with time series:

1. ROCKET uses a very large number of kernels. As there is only a single 'layer' of kernels, and as the kernel weights are not learned, the computational cost of computing the convolutions is low, and it is possible to use a very large number of kernels with relatively little computational expense.
2. ROCKET uses a massive variety of kernels. In contrast to typical convolutional networks, where it is common for groups of kernels to share the same size, dilation, and padding, for ROCKET each kernel has random length, dilation, and padding, as well as random weights and bias.
3. In particular, ROCKET makes key use of kernel dilation. In contrast to the typical use of dilation in convolutional neural networks, where dilation increases exponentially with depth (e.g., Yu and Koltun 2016; Bai et al. 2018; Franceschi et al. 2019), we sample dilation randomly for each kernel, producing a huge variety of kernel dilation, capturing patterns at different frequencies and scales, which is critical to the performance of the method (see section 4.3.4, below).
4. As well as using the maximum value of the resulting feature maps (broadly speaking, similar to global max pooling), ROCKET uses an additional and, to

our knowledge, novel feature: the proportion of positive values (or *ppv*). This enables a classifier to weight the prevalence of a given pattern within a time series. This is the single element of the Rocket architecture that is most critical to its outstanding accuracy (see section 4.3.6).

In effect, the only hyperparameter for Rocket is the number of kernels, k. In setting k, there is a tradeoff between classification accuracy and computation time. Generally speaking, a larger value of k results in higher classification accuracy (see section 4.3.1), but at the expense of proportionally longer computation. (The complexity of the transform is linear with respect to k.) However, even with a very large number of kernels (we use 10,000 by default), Rocket is extremely fast.

We implement Rocket in Python, using just-in-time compilation via Numba (Lam et al. 2015). For the experiments on the datasets in the UCR archive, we use a ridge regression classifier from scikit-learn (Pedregosa et al. 2011). For the experiments studying scalability, we integrate Rocket with logistic regression and Adam, implemented using PyTorch (Paszke et al. 2017). Our code will be made available at `https://github.com/angus924/rocket`.

In developing Rocket, we have endeavoured to not overfit the entire UCR archive (see Bagnall et al. 2017, p. 608). At the same time, in order to develop the method, we required representative time series datasets. Accordingly, we chose to develop the method on a subset of 40 randomly-selected datasets from the 85 'bake off' datasets. We refer to these as the 'development' datasets. We provide a separate evaluation of the performance of Rocket on the 'development' datasets and the remaining 'holdout' datasets in Appendix B.

3.1 Kernels

Rocket transforms time series using convolutional kernels, as found in typical convolutional neural networks. Essentially all aspects of the kernels are random: length, weights, bias, dilation, and padding. For each kernel, these values are set as follows (as determined by experimentation to produce the highest classification accuracy on the 'development' datasets):

- **Length**. Length is selected randomly from $\{7, 9, 11\}$ with equal probability, making kernels considerably shorter than input time series in most cases.
- **Weights**. The weights are sampled from a normal distribution, $\forall w \in \boldsymbol{W}$, $w \sim \mathcal{N}(0, 1)$, and are mean centered after being set, $\omega = \boldsymbol{W} - \overline{\boldsymbol{W}}$. As such, most weights are relatively small, but can take on larger magnitudes.
- **Bias**. Bias is sampled from a uniform distribution, $b \sim \mathcal{U}(-1, 1)$. Only positive values in the feature maps are used (see section 3.2). Bias therefore has the effect that two otherwise similar kernels, but with different biases, can 'highlight' different aspects of the resulting feature maps by shifting the values in a feature map above or below zero by a fixed amount.
- **Dilation**. Dilation is sampled on an exponential scale $d = \lfloor 2^x \rfloor, x \sim \mathcal{U}(0, A)$, where $A = log_2 \frac{l_{\text{input}}-1}{l_{\text{kernel}}-1}$, which ensures that the effective length of the kernel, including dilation, is up to the length of the input time series, l_{input}. Dilation allows otherwise similar kernels but with different dilations to match the same or similar patterns at different frequencies and scales.

– **Padding**. When each kernel is generated, a decision is made (at random, with equal probability) whether or not padding will be used when applying the kernel. If padding is used, an amount of zero padding is appended to the start and end of each time series when applying the kernel, such that the 'middle' element of the kernel is centered on every point in the time series, i.e., $((l_{\text{kernel}} - 1) \times d)/2$. Without padding, kernels are not centered at the first and last $\lfloor l_{\text{kernel}}/2 \rfloor$ points of the time series, and 'focus' on patterns in the central regions of time series whereas with padding, kernels also match patterns at the start or end of time series (see also section 3.4.1).

Stride is always one. We do not apply a nonlinearity such as ReLU to the resulting feature maps (indeed both *ppv* and max are agnostic to ReLU). Note that the parameters for the weights and bias have been set based on the assumption that, as is standard practice, input time series have been normalized to have a mean of zero and a standard deviation of one (see generally Dau et al. 2019).

As noted above, these parameters were determined to produce the highest classification accuracy on the 'development' datasets. However, as demonstrated in section 4.3, below, there are several alternative configurations which produce similar classification accuracy. Overall, this suggests that our method is likely to generalise well to new problems, and that the kernel parameters are relatively 'uninformative' in the Bayesian sense of the word.

3.2 Transform

Each kernel is applied to each input time series, producing a feature map. The convolution operation involves a sliding dot product between a kernel and an input time series. The result of applying a kernel, ω, with dilation, d, to a given time series, X, from position i in X, is given by (see, e.g., Bai et al. 2018):

$$X_i * \omega = \sum_{j=0}^{l_{\text{kernel}}-1} X_{i+(j \times d)} \times \omega_j.$$

ROCKET computes two aggregate features from each feature map, producing two real-valued numbers as features per kernel, and composing our transform:

– the maximum value (broadly speaking, equivalent to global max pooling); and
– the proportion of positive values (or *ppv*).

Pooling, including global average pooling (Lin et al. 2014), and global max pooling (Oquab et al. 2015), is used in convolutional neural networks for dimensionality reduction and spatial (or temporal) invariance (Boureau et al. 2010).

The other feature computed by ROCKET on each feature map is *ppv*. The *ppv* directly captures the proportion of the input which matches a given pattern. We found that *ppv* produces meaningfully higher classification accuracy than other features, including the mean (broadly equivalent to global average pooling).

For k kernels, ROCKET produces $2k$ features per time series (i.e., *ppv* and max). For 10,000 kernels (the default), ROCKET produces 20,000 features. For smaller datasets (in fact, for all the datasets in the UCR archive), the number of

features is therefore possibly much larger than either the number of examples in the dataset or the number of elements in each time series.

Nevertheless, we find that the features produced by Rocket provide for high classification accuracy when used as the input for a linear classifier, even for datasets where the number of features dwarfs both the number of examples and the length of the time series.

3.3 Classifier

The transformed features are used to train a linear classifier. Rocket can, in principle, be used with any classifier. We have found that Rocket is very effective when used in conjunction with linear classifiers (which have the capacity to make use of a small amount of information from each of a large number of features).

Logistic regression. Rocket can be used with logistic regression and stochastic gradient descent. This is particularly suitable for very large datasets because it provides for fast training with a fixed memory cost (fixed by the size of each minibatch). The transform can be performed on each minibatch, or on larger tranches of the dataset which are then divided further into minibatches for training.

Ridge regression. However, for all of the datasets in the UCR archive we use a ridge regression classifier. (A ridge regression model is trained for each class in a 'one versus rest' fashion, with L_2 regularization.)

Regularization is critically important where the number of features is significantly greater than the number of training examples, allowing for the optimization of linear models, and preventing pathological behaviour in iterative optimisation, e.g., for logistic regression (see Goodfellow et al. 2016, pp. 232–233). The ridge regression classifier can exploit generalised cross-validation to determine an appropriate regularization parameter quickly (see Rifkin and Lippert 2007). We find that for smaller datasets, a ridge regression classifier is significantly faster in practice than logistic regression, while still achieving high classification accuracy.

3.4 Complexity Analysis

The computational complexity of Rocket has two aspects: (1) the complexity of the transform itself; and (2) the complexity of the linear classifier trained using the transformed features.

3.4.1 Transform

The transform itself is linear in relation to both: (a) the number of examples; and (b) the length of the time series in a given dataset. Formally, the computational complexity of the transform is $O(k \cdot n \cdot l_{\text{input}})$, where k is the number of kernels, n is the number of examples, and l_{input} is the length of the time series. The transform must be applied to both training and test sets.

The convolution operation can be implemented in more than one way, including as a matrix multiplication typical of implementations for convolutional neural

networks, and using the fast Fourier transform (see Goodfellow et al. 2016, ch. 9). We implement ROCKET simply, 'sliding' each kernel along each time series and computing the dot product at each location. This involves repeated elementwise multiplication and summation, the complexity of which is dictated by the length of the time series, and the length of the kernels (that is, the number of weights in the kernels). The length of the kernels for ROCKET is limited to, at most, 11. Accordingly, kernel length is a constant factor for the purpose of this analysis.

Dilation increases the effective size of a kernel. Accordingly, where no padding is used, dilation reduces computational complexity. Without padding, the convolution is computed with the first element of the kernel starting at the first element of the time series, and ends once the last element of the kernel reaches the last element of the time series. In an extreme case, for the largest values of dilation, the kernel will 'fill' the entire time series, and the number of computations will be the number of weights in the kernel. However, padding is applied randomly with equal probability, so the reduction in complexity is a constant factor.

Where padding is used, dilation has no effect on complexity: the same number of computations are required regardless of dilation or the effective size of the kernel. Regardless of dilation, the kernel is centered on the first element of the time series, and 'slides' the same number of elements along the time series.

Accordingly, for k kernels and n time series, each of length l_{input}, the complexity of the transform is $O(k \cdot n \cdot l_{\text{input}})$. For datasets with time series of different lengths, this could be taken to represent average complexity for an average length of l_{input}, or worst-case complexity for a maximum length of l_{input}.

3.4.2 Classifier

Logistic regression and stochastic gradient descent. The complexity of stochastic gradient descent is proportional to the number of parameters (dictated by the number of features and the number of classes), but is linear in relation to the number of training examples (Bottou et al. 2018). Further, the rate of convergence is not determined by the number of training examples. For large datasets, convergence may occur in a single pass of the data, or even without using all of the training data (Goodfellow et al. 2016, pp. 286–288; Bottou et al. 2018).

Ridge regression. In practice, the ridge regression classifier is significantly faster than logistic regression on smaller datasets because it can make use of so-called generalized cross-validation to determine appropriate regularization. The implementation used here employs eigen decomposition where there are more features than training examples, or singular value decomposition otherwise, with effective complexity of $O(n^2 \cdot f)$ and $O(n \cdot f^2)$ respectively (see Dongarra et al. 2018), where n is the number of training examples and f is the number of features.

This makes the ridge regression classifier less scalable for large datasets. This also requires the complete transform, and does not work incrementally. In practice, these limitations do not affect any of the datasets in the UCR archive. For larger datasets, where the importance of regularization decreases, and it is appropriate to perform the transform incrementally, the benefit of using the ridge regression classifier wanes, and training with stochastic gradient descent makes more sense.

4 Experiments

We evaluate Rocket on the UCR archive (section 4.1), demonstrating that Rocket is competitive with current state-of-the-art methods, obtaining the best mean rank over the 85 'bake off' datasets.

We evaluate scalability in terms of both training set size and time series length (section 4.2), demonstrating that Rocket is orders of magnitude faster than current methods. We also evaluate the effect of different kernel parameters (section 4.3), showing that several alternative configurations of Rocket perform similarly well, which is a good indication of the power of the idea, rather than of its fine-tuning. Unless otherwise stated, all experiments use 10,000 kernels.

The experiments on the datasets in the UCR archive are performed using Rocket in conjunction with a ridge regression classifier, and the experiment in relation to training set size is performed using Rocket integrated with logistic regression. The experiments on the UCR archive were conducted on a cluster (but using a single CPU core per experiment, not parallelised for speed). The experiments in relation to scalability (both time series length and training set size) were performed locally using an Intel Core i5-5200U dual-core processor.

4.1 UCR

4.1.1 'Bake Off' Datasets

We evaluate Rocket on the 85 'bake off' datasets from the UCR archive (on the original training/test split for each dataset). The results presented for Rocket are mean results over 10 runs (using a different set of random kernels for each run).

We compare Rocket to existing state-of-the-art methods for time series classification, namely, BOSS, Shapelet Transform, Proximity Forest, ResNet, and HIVE-COTE. We also compare Rocket with two more recent methods (with papers on arXiv), InceptionTime and TS-CHIEF, that have been demonstrated to be competitive with HIVE-COTE, while being more scalable. The results for BOSS, Shapelet Transform, and HIVE-COTE are taken from Bagnall et al. (2019).

For comparability with other published results, we compare Rocket to the other methods on all 85 'bake off' datasets. However, as noted above, Rocket was developed using a subset of 40 randomly-selected datasets, to make sure we didn't overfit the UCR archive. Separate rankings for the 40 'development' datasets, as well as the remaining 45 'holdout' datasets, are provided in Appendix B.

Figure 1 on page 1 gives the mean rank for each method included in the comparison. Classifiers for which the difference in pairwise classification accuracy is not statistically significant, as determined by a Wilcoxon signed-rank test with Holm correction (as a post hoc test to the Friedman test), are connected with a black line (see Demšar 2006; García and Herrera 2008; Benavoli et al. 2016). The relative accuracy of Rocket and each of the other methods included in the comparison is shown in Figure 13, Appendix A.

Figure 1 on page 1 shows that Rocket is competitive with (in fact, ranks slightly ahead of) HIVE-COTE, TS-CHIEF, and InceptionTime, although the difference in accuracy between Rocket, HIVE-COTE, InceptionTime, and TS-

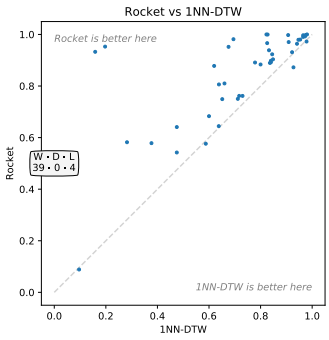


Fig. 2 Relative accuracy of ROCKET versus 1NN-DTW on the 43 additional 2018 datasets.

CHIEF is not statistically significant. TS-CHIEF ranks ahead of ROCKET on the 45 'holdout' datasets (Figure 14, Appendix B), but the difference is not significant.

4.1.2 Additional 2018 Datasets

We have also evaluated ROCKET on the 43 additional datasets in the UCR archive as of 2018, in order to: (1) show that our method is able to handle datasets with varying lengths; and (2) provide reference results for future research papers.

There are no published results for state-of-the-art methods on these datasets. Adapting these methods to work on variable-length time series is nontrivial, as the most appropriate method for handling variable lengths (1) depends on whether the variable lengths represent subsampling or variable sampling frequencies, and (2) is classifier dependent (Tan et al. 2019). Accordingly, we restrict our comparison to the available results for 1NN-DTW (Dau et al. 2019), where variable length time series have been padded with 'low amplitude random [noise]' to the same length as the longest time series (Dau et al. 2018, p. 16). Figure 2 shows the relative accuracy of ROCKET and 1NN-DTW on the 43 additional datasets. ROCKET is more accurate on all but four datasets, and substantially more so on most.

Following Dau et al. (2018), we have normalized each time series and interpolated missing values. Variable-length time series have been rescaled or used 'as is' (with their original lengths) as determined by 10-fold cross-validation.

4.2 Scalability

4.2.1 Training Set Size

Following Lucas et al. (2019), Shifaz et al. (2019), and Ismail Fawaz et al. (2019c), we evaluate scalability in terms of training set size on increasingly larger subsets (up to approximately 1 million time series) of the Satellite Image Time Series dataset (see Petitjean et al. 2012). The time series in this dataset represent a vegetation index, calculated from spectral data acquired by the Formosat-2 satellite, and the classes represent different land cover types. The aim in classifying these time series is to map different vegetation profiles to different types of crops and forested areas. Each time series has a length of 46.

Fig. 3 Accuracy (left) and training time (right) versus training set size.

For this purpose, we integrate Rocket with logistic regression. The transform is performed in tranches, which are further divided into minibatches for training. Each time series is normalised to have a zero mean and unit standard deviation.

We train the model for at least one epoch for each subset size. To prevent overfitting, we stop training (after the first epoch) if validation loss has failed to improve after 20 updates. In practice, while training may continue for 40 or 50 epochs for smaller subset sizes, training converges within a single pass for anything more than approximately 16,000 training examples. Validation loss is computed on a separate validation set (the same set of 2,048 examples for all subset sizes).

Optimization is performed using Adam (Kingma and Ba 2015). We perform a minimal search on the initial learning rate to ensure that training loss does not diverge. The learning rate is halved if training loss fails to improve after 100 updates (only relevant for larger subset sizes).

We have run Rocket in three guises: with 100, 1,000, and 10,000 kernels (the default). We compare Rocket against Proximity Forest and TS-CHIEF, which have already been demonstrated to be fundamentally more scalable than HIVE-COTE (Lucas et al. 2019; Shifaz et al. 2019). (Results for larger quantities of data are not yet available for InceptionTime.)

Figure 3 shows classification accuracy and training time versus training set size for Rocket, Proximity Forest, and TS-CHIEF. As expected, Rocket scales linearly with respect to both the number of training examples, and the number of kernels (see section 3.4). With 1,000 or 10,000 kernels, Rocket achieves similar classification accuracy to Proximity Forest and TS-CHIEF. With 100 kernels, Rocket achieves lower classification accuracy, but takes less than a minute to learn from more than 1 million time series. Even with 10,000 kernels, Rocket is an order of magnitude faster than Proximity Forest. (Training time for smaller subset sizes for Rocket is dominated by the cost of the transform for the validation set, which is why training time is 'flat' for smaller subset sizes.)

4.2.2 Time Series Length

Following Shifaz et al. (2019) and Ismail Fawaz et al. (2019c), we evaluate scalability in terms of time series length using the *InlineSkate* dataset from the UCR archive. We use Rocket in the same configuration as for the other datasets in

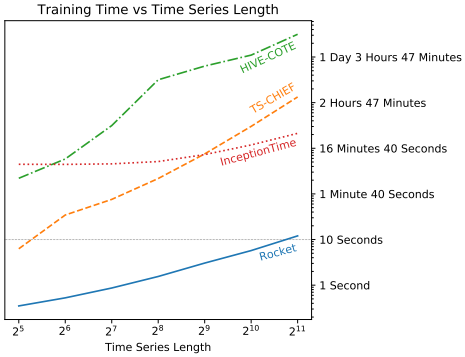


Fig. 4 Training time versus time series length.

the UCR archive (that is, using the ridge regression classifier and 10,000 kernels). Results for HIVE-COTE and TS-CHIEF are taken from Shifaz et al. (2019).

Figure 4 shows training time versus time series length for ROCKET, TS-CHIEF, HIVE-COTE, and InceptionTime. The difference in training time between ROCKET and TS-CHIEF is substantial. ROCKET takes approximately as long to train on time series of length 2,048 as TS-CHIEF takes for time series of length 32, and is approximately three orders of magnitude faster for the longest time series length.

ROCKET is considerably faster than InceptionTime as well. However, fundamental scalability is likely to be similar, given that both InceptionTime and ROCKET are based on convolutional architectures.

4.3 Sensitivity Analysis

We explore the effect of different kernel parameters on classification accuracy. We compare the accuracy of the default configuration (i.e., using the parameters specified in section 3) against different choices for the number of kernels, length, weights and bias, dilation, padding, and output features. In each case, only the given parameter (e.g., length) is varied, keeping all other parameters fixed at their default values. The comparison is made on the 'development' datasets. The results are mean results over 10 runs (using a different set of random kernels per run).

In most cases, alternative configurations represent a relatively subtle change from the default configuration. Unsurprisingly, therefore, in many cases one or more alternative choices for the relevant parameter produces similar accuracy to the baseline configuration. In other words, ROCKET is relatively robust to different choices for many parameters. However, it is clear that dilation and *ppv*, in particular, are two key aspects of the performance of the method.

4.3.1 Number of Kernels

We evaluate increasing numbers of kernels between 10 and 100,000. Figure 5 shows the effect of the number of kernels, k, on accuracy. Clearly, increasing the number of kernels improves accuracy. However, the actual difference in accuracy between, for example, $k = 5{,}000$ and $k = 10{,}000$, is relatively small, even if statistically

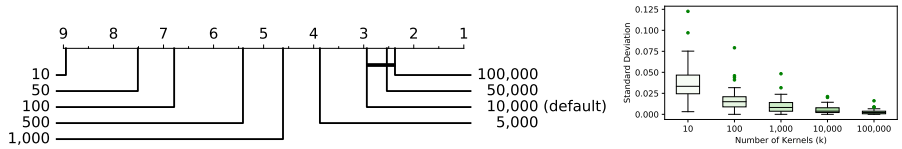


Fig. 5 Mean ranks (left), and variance in accuracy (right), versus k.

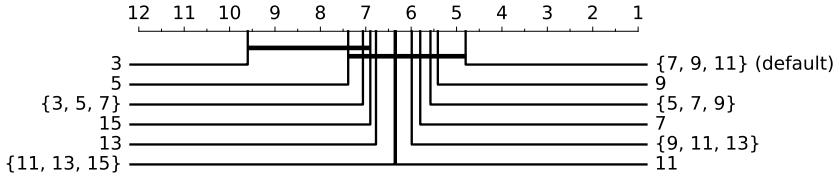


Fig. 6 Mean ranks for different choices in terms of kernel length.

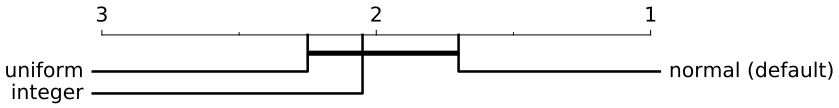


Fig. 7 Mean ranks for different choices in terms of the sampling distribution for the weights.

significant. Indeed, $k = 5{,}000$ produces higher accuracy on some datasets (Figure 16, Appendix C). Nevertheless, $k = 10{,}000$ is noticeably ahead in terms of win/draw/loss (29/3/8). The differences between $k = 10{,}000$, $k = 50{,}000$, and $k = 100{,}000$ are not statistically significant.

Even though Rocket is nondeterministic, the variability in accuracy is reasonably low for large numbers of kernels. Unsurprisingly, standard deviation diminishes as k increases. The median standard deviation across the 40 'development' datasets is 0.0038 for $k = 10{,}000$, and 0.0021 for $k = 100{,}000$.

4.3.2 Kernel Length

We vary kernel length, comparing the baseline (selecting length randomly from $\{7, 9, 11\}$) to:

- fixed lengths of 3, 5, 7, 9, 11, 13, and 15; and
- selecting length randomly from $\{3, 5, 7\}$, $\{5, 7, 9\}$, $\{9, 11, 13\}$, and $\{11, 13, 15\}$.

Figure 6 shows the effect of these choices on accuracy. Fixed lengths of 7, 9, and 11, as well as selecting length randomly from $\{5, 7, 9\}$ and $\{9, 11, 13\}$ result in similar accuracy to the default configuration, and the differences are not statistically significant (see also Figure 17, Appendix C). Shorter kernels are undesirable, being more strongly correlated with each other for a large number of kernels.

4.3.3 Weights (Including Centering) and Bias

Weights. We vary the distribution from which the weights are sampled, comparing the baseline (sampling from a normal distribution) to:

- sampling from a uniform distribution, $\forall w \in \boldsymbol{W}$, $w \sim \mathcal{U}(-1, 1)$; and

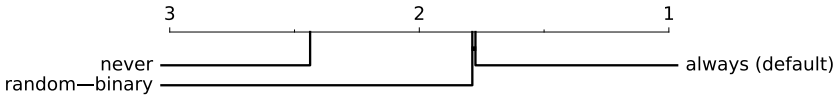


Fig. 8 Mean ranks for different choices in terms of centering.

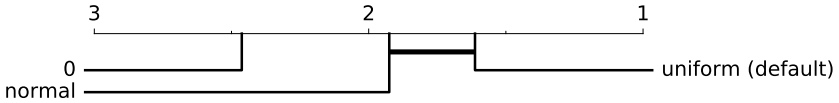


Fig. 9 Mean ranks for different choices in terms of bias.

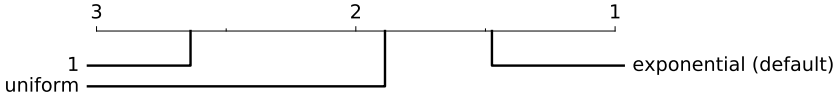


Fig. 10 Mean ranks for different choices in terms of dilation.

– sampling integer weights uniformly from $\{-1, 0, 1\}$.

Figure 7 shows the effect of these choices on accuracy. While sampling from a normal distribution produces higher accuracy, the actual difference in accuracy is small and not statistically significant (see also Figure 18, Appendix C). While it may seem surprising that weights sampled from only three integer values are so effective, note that kernels are still mean centered by default and have random bias, and there is still substantial variety in terms of length and dilation.

Centering. We vary centering, comparing the baseline (always centering) against:

– never centering the kernel weights; and
– centering or not centering at random with equal probability.

Figure 8 shows the effect of these choices on accuracy. It is clear that centering produces higher accuracy, but the difference between always centering and centering at random is very small and not statistically significant. Always centering, however, is noticeably more accurate on some datasets (Figure 19, Appendix C).

Bias. We vary bias, comparing the baseline (using a uniform distribution) against:

– using zero bias; and
– sampling bias from a normal distribution, $b \sim \mathcal{N}(0, 1)$.

Figure 9 shows the effect of these choices on accuracy. Using a bias term produces higher accuracy, but the difference between sampling bias from a uniform distribution or a normal distribution is relatively small and not statistically significant (see also Figure 20, Appendix C.)

4.3.4 Dilation

We vary dilation, comparing the baseline (sampling dilation on an exponential scale) against:

– no dilation (i.e., a fixed dilation of one); and

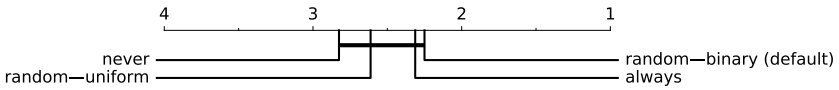


Fig. 11 Mean ranks for different choices in terms of padding.

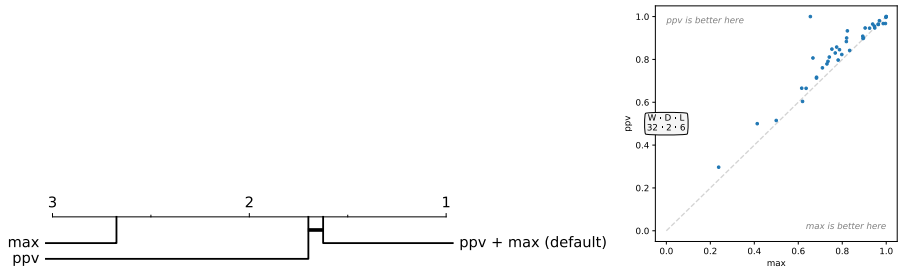


Fig. 12 Mean ranks (left), and relative accuracy (right), *ppv* and max.

– sampling dilation uniformly, $d = \lfloor x \rfloor$, $x \sim \mathcal{U}(1, \frac{l_{\text{input}}-1}{l_{\text{kernel}}-1})$.

Figure 10 shows the effect of these choices in terms of accuracy. It is clear that dilation is key to performance. Dilation produces obviously higher accuracy than no dilation. Exponential dilation produces higher accuracy than uniform dilation on most datasets (significantly higher on some datasets), and the difference is statistically significant (see also Figure 21, Appendix C).

4.3.5 Padding

We vary padding, comparing the baseline (applying padding at random) against:

– always padding, such that the 'middle' element of a given kernel is centered on the first element of the time series, $p = ((l_{\text{kernel}} - 1) \times d)/2$;
– sampling padding uniformly, $p \sim \mathcal{U}(0, ((l_{\text{kernel}} - 1) \times d)/2)$; and
– never padding.

Figure 11 shows the effect of these choices on accuracy. Padding is superior to not padding, but none of the differences are statistically significant. Different choices produce very similar results (Figure 22, Appendix C).

4.3.6 Features

We vary the output features, comparing the baseline, *ppv* and max, against using each in isolation. Figure 12 shows the effect of these choices on accuracy. It is clear that *ppv* is superior to max: *ppv* produces substantially higher classification accuracy for the majority of the 'development' datasets. In fact, *ppv* has the single biggest effect on accuracy of all the parameters. The combination of *ppv* and max is better again, although the difference between *ppv* and *ppv* plus max is small and not statistically significant (see also Figure 23, Appendix C).

5 Conclusion

Convolutional kernels are a single, powerful instrument which can capture many of the features used by existing methods for time series classification. We show that, rather than learning kernel weights, a large number of random kernels—while in isolation only approximating relevant patterns—in combination are extremely effective for capturing discriminative patterns in time series.

Further, random kernels have very low computational requirements, making learning and classification extremely fast. Our proposed method utilising random convolutional kernels for the purposes of transforming and classifying time series, ROCKET, achieves state-of-the-art accuracy with a fraction of the computational expense of existing methods. ROCKET also scales to millions of time series.

ROCKET makes key use of the proportion of positive values (or *ppv*) to summarise the output of feature maps, allowing a classifier to weight the prevalence of a pattern in a given time series. To our knowledge, *ppv* has not been used in this way before. We find that this is substantially more effective than a simple maximum as applied in a conventional max pooling operation. It is credible that *ppv* would also be effective for other data types such as images.

In future work, we propose to explore feature selection for ROCKET, the application of ROCKET to multivariate timeseries, the application of ROCKET beyond time series data, and the use of aspects of ROCKET with learned kernels.

Acknowledgements This material is based upon work supported by an Australian Government Research Training Program Scholarship; the Air Force Office of Scientific Research, Asian Office of Aerospace Research and Development (AOARD) under award number FA2386–18–1–4030; and the Australian Research Council under awards DE170100037 and DP190100017. The authors would like to thank Professor Eamonn Keogh and all the people who have contributed to the UCR time series classification archive. Figures showing the ranking of different classifiers and variants of ROCKET were produced using code from Ismail Fawaz et al. (2019a).

References

Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2017) The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. Data Mining and Knowledge Discovery 31(3):606–660

Bagnall A, Lines J, Vickers W, Keogh E (2019) The UEA & UCR time series classification repository. http://www.timeseriesclassification.com

Bai S, Kolter JZ, Koltun V (2018) An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:180301271

Benavoli A, Corani G, Mangili F (2016) Should we really use post-hoc tests based on mean-ranks? Journal of Machine Learning Research 17(5):1–10

Bengio Y, Courville A, Vincent P (2013) Representation learning: A review and new perspectives. IEEE Transactions on Pattern Analysis and Machine Intelligence 35(8):1798–1828

Bostrom A, Bagnall A (2015) Binary shapelet transform for multiclass time series classification. In: Madria S, Hara T (eds) Big Data Analytics and Knowledge Discovery, Springer, Cham, pp 257–269

Bottou L, Curtis FE, Nocedal J (2018) Optimization methods for large-scale machine learning. SIAM Review 60(2):223–311

Boureau YL, Ponce J, LeCun Y (2010) A theoretical analysis of feature pooling in visual recognition. In: Fürnkranz J, Joachims T (eds) Proceedings of the 27th International Conference on Machine Learning, Omnipress, USA, pp 111–118

Cox D, Pinto N (2011) Beyond simple features: A large-scale feature search approach to unconstrained face recognition. In: Face and Gesture 2011, pp 8–15

Dau HA, Keogh E, Kamgar K, et al. (2018) UCR time series classification archive (briefing document). https://www.cs.ucr.edu/~eamonn/time_series_data_2018/
Dau HA, Bagnall A, Kamgar K, Yeh CCM, Zhu Y, Gharghabi S, Ratanamahatana CA, Keogh E (2019) The UCR time series archive. arXiv:181007758
Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research 7:1–30
Dongarra J, Gates M, Haidar A, Kurzak J, Luszczek P, Tomov S, Yamazaki I (2018) The singular value decomposition: Anatomy of optimizing an algorithm for extreme scale. SIAM Review 60(4):808–865
Farahmand A, Pourazarm S, Nikovski D (2017) Random projection filter bank for time series data. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) Advances in Neural Information Processing Systems 30, pp 6562–6572
Franceschi J, Dieuleveut A, Jaggi M (2019) Unsupervised scalable representation learning for multivariate time series. In: Seventh International Conference on Learning Representations, Learning from Limited Labeled Data Workshop
García S, Herrera F (2008) An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. Journal of Machine Learning Research 9:2677–2694
Goodfellow I, Bengio Y, Courville A (2016) Deep Learning. MIT Press, Cambridge, MA
Hills J, Lines J, Baranauskas E, Mapp J, Bagnall A (2014) Classification of time series by shapelet transformation. Data Mining and Knowledge Discovery 28(4):851–881
Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller P (2019a) Deep learning for time series classification: a review. Data Mining and Knowledge Discovery 33(4):917–963
Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller P (2019b) Deep neural network ensembles for time series classification. arXiv:190306602
Ismail Fawaz H, Lucas B, Forestier G, Pelletier C, Schmidt DF, Weber J, Webb GI, Idoumghar L, Muller P, Petitjean F (2019c) InceptionTime: Finding AlexNet for time series classification. arXiv:190904939
Jarrett K, Kavukcuoglu K, Ranzato M, LeCun Y (2009) What is the best multi-stage architecture for object recognition? In: 2009 IEEE 12th International Conference on Computer Vision, pp 2146–2153
Jimenez A, Raj B (2019) Time signal classification using random convolutional features. In: 2019 IEEE International Conference on Acoustics, Speech and Signal Processing
Kingma DP, Ba JL (2015) Adam: A method for stochastic optimization. In: Third International Conference on Learning Representations, arXiv:1412.6980
Krizhevsky A, Sutskever I, Hinton G (2012) Imagenet classification with deep convolutional neural networks. In: Pereira F, Burges CJC, Bottou L, Weinberger KQ (eds) Advances in Neural Information Processing Systems 25, pp 1097–1105
Lam SK, Pitrou A, Seibert S (2015) Numba: A LLVM-based python JIT compiler. In: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, pp 1–6
Lin M, Chen Q, Yan S (2014) Network in network. In: Second International Conference on Learning Representations, arXiv:1312.4400
Lines J, Taylor S, Bagnall A (2018) Time series classification with HIVE-COTE: The hierarchical vote collective of transformation-based ensembles. ACM Transactions on Knowledge Discovery from Data 12(5):52:1–52:35
Lucas B, Shifaz A, Pelletier C, O'Neill L, Zaidi N, Goethals B, Petitjean F, Webb GI (2019) Proximity Forest: an effective and scalable distance-based classifier for time series. Data Mining and Knowledge Discovery 33(3):607–635
Morrow A, Shankar V, Petersohn D, Joseph A, Recht B, Yosef N (2017) Convolutional kitchen sinks for transcription factor binding site prediction. arXiv:170600125
Oquab M, Bottou L, Laptev I, Sivic J (2015) Is object localization for free? weakly-supervised learning with convolutional neural networks. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition, pp 685–694
Paszke A, Gross S, Chintala S, Chanan G, Yang E, DeVito Z, Lin Z, Desmaison A, Antiga L, Lerer A (2017) Automatic differentiation in PyTorch. In: NIPS Autodiff Workshop
Pedregosa F, Varoquaux G, Gramfort A, et al. (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12:2825–2830
Petitjean F, Inglada J, Gancarski P (2012) Satellite image time series analysis under time warping. IEEE Transactions on Geoscience and Remote Sensing 50(8):3081–3095
Pinto N, Doukhan D, DiCarlo JJ, Cox DD (2009) A high-throughput screening approach to discovering good forms of biologically inspired visual representation. PLOS Computational

Biology 5(11):1–12
Rahimi A, Recht B (2008) Random features for large-scale kernel machines. In: Platt JC, Koller D, Singer Y, Roweis ST (eds) Advances in Neural Information Processing Systems 20, pp 1177–1184
Rahimi A, Recht B (2009) Weighted sums of random kitchen sinks: Replacing minimization with randomization in learning. In: Koller D, Schuurmans D, Bengio Y, Bottou L (eds) Advances in Neural Information Processing Systems 21, pp 1313–1320
Rifkin RM, Lippert RA (2007) Notes on regularized least squares. Tech. rep., MIT
Saxe A, Koh PW, Chen Z, Bhand M, Suresh B, Ng A (2011) On random weights and unsupervised feature learning. In: Getoor L, Scheffer T (eds) Proceedings of the 28th International Conference on Machine Learning, Omnipress, USA, pp 1089–1096
Schäfer P (2015) The BOSS is concerned with time series classification in the presence of noise. Data Mining and Knowledge Discovery 29(6):1505–1530
Schäfer P (2016) Scalable time series classification. Data Mining and Knowledge Discovery 30(5):1273–1298
Schäfer P, Leser U (2017) Fast and accurate time series classification with WEASEL. In: Proceedings of the 2017 ACM Conference on Information and Knowledge Management, pp 637–646
Shifaz A, Pelletier C, Petitjean F, Webb GI (2019) TS-CHIEF: A scalable and accurate forest algorithm for time series classification. arXiv:190610329
Tan CW, Petitjean F, Keogh E, Webb GI (2019) Time series classification for varying length series. arXiv:191004341
Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: A strong baseline. In: 2017 International Joint Conference on Neural Networks, pp 1578–1585
Yosinski J, Clune J, Bengio Y, Lipson H (2014) How transferable are features in deep neural networks? In: Ghahramani Z, Welling M, Cortes C, Lawrence ND, Weinberger KQ (eds) Advances in Neural Information Processing Systems 27, pp 3320–3328
Yu F, Koltun V (2016) Multi-scale context aggregation by dilated convolutions. In: Fourth International Conference on Learning Representations, arXiv:1511.07122
Zeiler MD, Fergus R (2014) Visualizing and understanding convolutional networks. In: Fleet D, Pajdla T, Schiele B, Tuytelaars T (eds) European Conference on Computer Vision, Springer, Cham, pp 818–833

Appendices

A Relative Accuracy

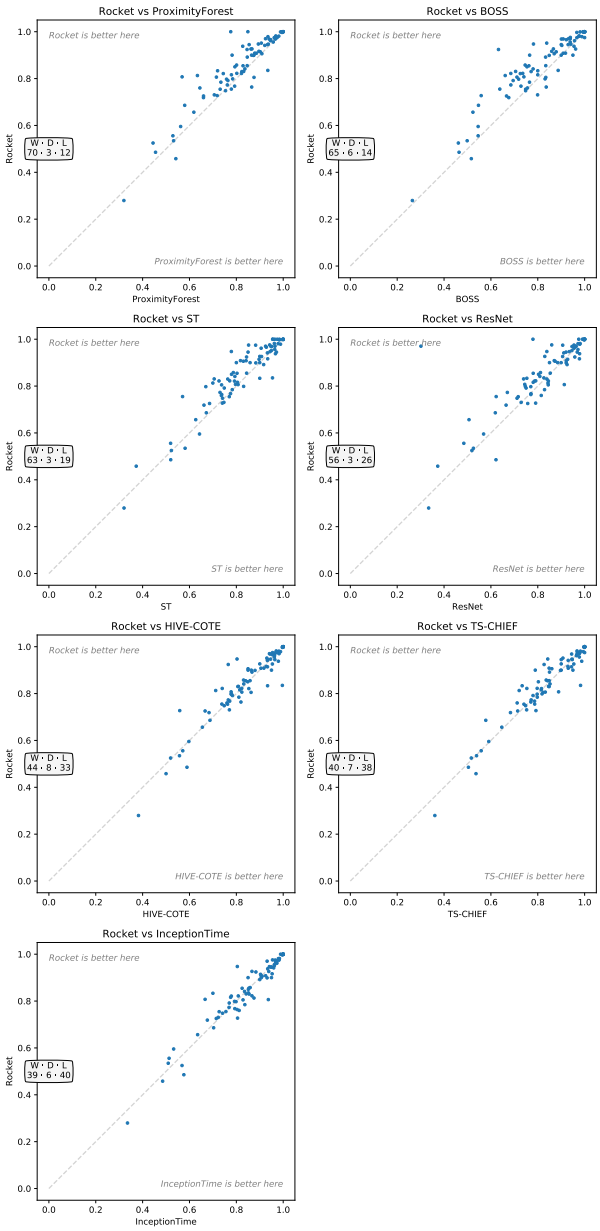


Fig. 13 Relative accuracy of Rocket vs state-of-the-art classifiers on the 'bake off' datasets.

B 'Development' and 'Holdout' Datasets

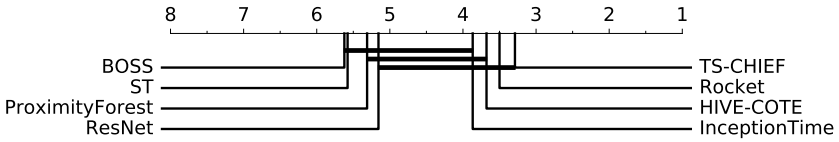


Fig. 14 Mean rank of ROCKET versus state-of-the-art classifiers on the 'holdout' datasets.

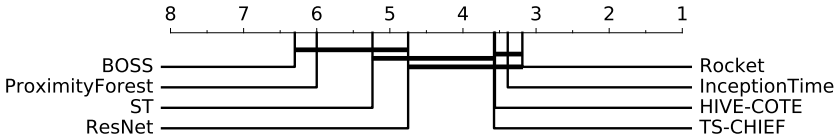


Fig. 15 Mean rank of ROCKET vs state-of-the-art classifiers on the 'development' datasets.

C Additional Plots

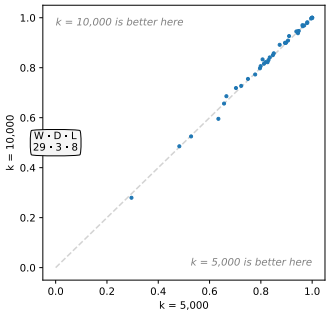


Fig. 16 Relative accuracy of $k = 10{,}000$ versus $k = 5{,}000$ on the 'development' datasets.

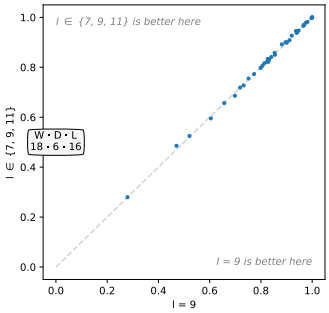


Fig. 17 Relative accuracy of $l \in \{7, 9, 11\}$ versus $l = 9$ on the 'development' datasets.

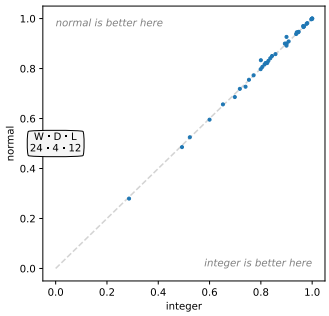


Fig. 18 Relative accuracy, normally-distributed vs integer weights, 'development' datasets.

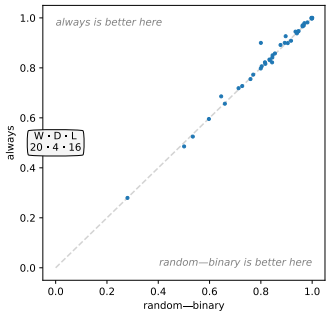


Fig. 19 Relative accuracy of always vs random centering on the 'development' datasets.

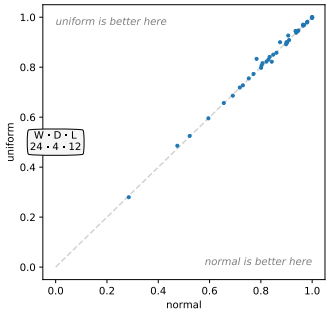


Fig. 20 Relative accuracy, uniformly versus normally-distributed bias, 'development' datasets.

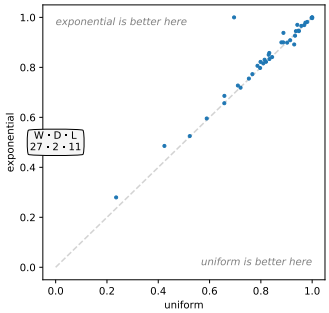


Fig. 21 Relative accuracy of exponential vs uniform dilation on the 'development' datasets.

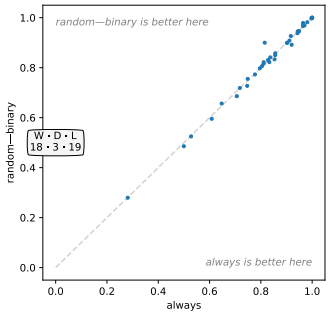


Fig. 22 Relative accuracy of random versus always padding on the 'development' datasets.

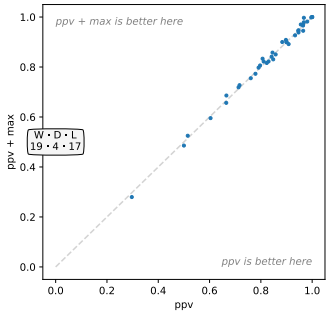


Fig. 23 Relative accuracy of *ppv* and max versus only *ppv* on the 'development' datasets.

D Results for 'Bake Off' Datasets

The 'development' datasets are marked with an asterisk.

Table 1: Classification Accuracy, 'Bake Off' Datasets

	Rocket	BOSS	ST	HCTE	ResNet	PF	CHIEF	ITime
Adiac	0.7847	0.7647	0.7826	0.8107	0.8289	0.7340	0.7980	0.8363
ArrowHead	0.8051	0.8343	0.7371	0.8629	0.8446	0.8754	0.8229	0.8286
*Beef	0.8333	0.8000	0.9000	0.9333	0.7533	0.7200	0.7333	0.7000
BeetleFly	0.9000	0.9000	0.9000	0.9500	0.8500	0.8750	0.9500	0.8500
*BirdChicken	0.9000	0.9500	0.8000	0.8500	0.8850	0.8650	0.9000	0.9500
CBF	0.9999	0.9978	0.9744	0.9989	0.9950	0.9933	0.9978	0.9989
*Car	0.8917	0.8333	0.9167	0.8667	0.9250	0.8467	0.8500	0.9000
ChlCon	0.8130	0.6609	0.6997	0.7120	0.8436	0.6339	0.7206	0.8753
CinCECGTorso	0.8349	0.8870	0.9543	0.9964	0.8261	0.9343	0.9826	0.8514
Coffee	1.0000	1.0000	0.9643	1.0000	1.0000	1.0000	1.0000	1.0000
Computers	0.7600	0.7560	0.7360	0.7600	0.8148	0.6444	0.7120	0.8120
*CricketX	0.8223	0.7359	0.7718	0.8231	0.7913	0.8021	0.7974	0.8667
*CricketY	0.8503	0.7538	0.7795	0.8487	0.8033	0.7938	0.8026	0.8513
*CricketZ	0.8577	0.7462	0.7872	0.8308	0.8115	0.8010	0.8359	0.8590
DiaSizRed	0.9703	0.9314	0.9248	0.9412	0.3013	0.9657	0.9771	0.9314
DisPhaOutAgeGro	0.7547	0.7482	0.7698	0.7626	0.7165	0.7309	0.7410	0.7266
DisPhaOutCor	0.7678	0.7283	0.7754	0.7717	0.7710	0.7928	0.7862	0.7935
*DisPhaTW	0.7187	0.6763	0.6619	0.6835	0.6647	0.6597	0.6835	0.6763
ECG200	0.9060	0.8700	0.8300	0.8500	0.8740	0.9090	0.8600	0.9100
*ECG5000	0.9470	0.9413	0.9438	0.9462	0.9342	0.9365	0.9458	0.9409
ECGFiveDays	1.0000	1.0000	0.9837	1.0000	0.9748	0.8492	1.0000	1.0000
Earthquakes	0.7482	0.7482	0.7410	0.7482	0.7115	0.7540	0.7482	0.7410
ElectricDevices	0.7305	0.7992	0.7470	0.7703	0.7291	0.7060	0.7524	0.7227
FaceAll	0.9475	0.7817	0.7787	0.8030	0.8388	0.8938	0.8426	0.8041
FaceFour	0.9750	1.0000	0.8523	0.9545	0.9545	0.9739	1.0000	0.9659
FacesUCR	0.9616	0.9571	0.9059	0.9629	0.9547	0.9459	0.9649	0.9732
*FiftyWords	0.8305	0.7055	0.7055	0.8088	0.7396	0.8314	0.8462	0.8418
*Fish	0.9789	0.9886	0.9886	0.9886	0.9794	0.9349	0.9943	0.9829
*FordA	0.9449	0.9295	0.9712	0.9644	0.9205	0.8546	0.9470	0.9483
*FordB	0.8063	0.7111	0.8074	0.8235	0.9131	0.7149	0.8321	0.9365
GunPoint	1.0000	1.0000	1.0000	1.0000	0.9907	0.9973	1.0000	1.0000
Ham	0.7257	0.6667	0.6857	0.6667	0.7571	0.6600	0.7143	0.7143
HandOutlines	0.9416	0.9027	0.9324	0.9324	0.9111	0.9214	0.9297	0.9595
*Haptics	0.5250	0.4610	0.5227	0.5195	0.5188	0.4445	0.5162	0.5682
*Herring	0.6859	0.5469	0.6719	0.6875	0.6188	0.5797	0.5781	0.7031
InlineSkate	0.4582	0.5164	0.3727	0.5000	0.3731	0.5418	0.5364	0.4855
*InsWinSou	0.6566	0.5232	0.6268	0.6551	0.5065	0.6187	0.6465	0.6348
*ItaPowDem	0.9691	0.9086	0.9475	0.9631	0.9630	0.9671	0.9718	0.9679
*LarKitApp	0.9000	0.7653	0.8587	0.8640	0.8997	0.7819	0.7893	0.9067
Lightning2	0.7639	0.8361	0.7377	0.8197	0.7705	0.8656	0.7705	0.8033
*Lightning7	0.8219	0.6849	0.7260	0.7397	0.8452	0.8219	0.7534	0.8082
Mallat	0.9560	0.9382	0.9642	0.9620	0.9716	0.9576	0.9774	0.9629
*Meat	0.9450	0.9000	0.8500	0.9333	0.9683	0.9333	0.9000	0.9500
*MedicalImages	0.7975	0.7184	0.6697	0.7776	0.7703	0.7582	0.7974	0.7987
*MidPhaOutAgeGro	0.5955	0.5455	0.6429	0.5974	0.5688	0.5623	0.5909	0.5325
*MidPhaOutCor	0.8412	0.7801	0.7938	0.8316	0.8089	0.8364	0.8522	0.8351
MiddlePhalanxTW	0.5558	0.5455	0.5195	0.5714	0.4844	0.5292	0.5584	0.5130
MoteStrain	0.9142	0.8786	0.8970	0.9329	0.9276	0.9024	0.9441	0.9034
NonInvFetECGTho1	0.9514	0.8382	0.9496	0.9303	0.9454	0.9066	0.9074	0.9623
NonInvFetECGTho2	0.9688	0.9008	0.9511	0.9445	0.9461	0.9399	0.9445	0.9674
*OSULeaf	0.9380	0.9545	0.9669	0.9793	0.9785	0.8273	0.9876	0.9339
*OliveOil	0.9267	0.8667	0.9000	0.9000	0.8300	0.8667	0.9000	0.8667

Table 1: Classification Accuracy, 'Bake Off' Datasets

	Rocket	BOSS	ST	HCTE	ResNet	PF	CHIEF	ITime
PhaOutCor	0.8300	0.7716	0.7634	0.8065	0.8390	0.8235	0.8485	0.8543
*Phoneme	0.2796	0.2648	0.3207	0.3824	0.3343	0.3201	0.3608	0.3354
*Plane	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
ProPhaOutAgeGro	0.8551	0.8341	0.8439	0.8585	0.8532	0.8463	0.8488	0.8537
*ProPhaOutCor	0.8990	0.8488	0.8832	0.8797	0.9213	0.8732	0.8969	0.9313
*ProPhaTW	0.8161	0.8000	0.8049	0.8146	0.7805	0.7790	0.8146	0.7756
RefDev	0.5347	0.4987	0.5813	0.5573	0.5253	0.5323	0.5387	0.5093
*ScreenType	0.4856	0.4640	0.5200	0.5893	0.6216	0.4552	0.5040	0.5760
*ShapeletSim	1.0000	1.0000	0.9556	1.0000	0.7794	0.7761	1.0000	0.9889
ShapesAll	0.9082	0.9083	0.8417	0.9050	0.9213	0.8858	0.9300	0.9250
SmaKitApp	0.8213	0.7253	0.7920	0.8533	0.7861	0.7443	0.8160	0.7787
SonAIBORobSur1	0.9241	0.6323	0.8436	0.7654	0.9581	0.8458	0.8270	0.8835
SonAIBORobSur2	0.9164	0.8594	0.9339	0.9276	0.9778	0.8963	0.9286	0.9528
StarLightCurves	0.9811	0.9778	0.9785	0.9815	0.9718	0.9813	0.9820	0.9792
*Strawberry	0.9819	0.9757	0.9622	0.9703	0.9805	0.9684	0.9676	0.9838
*SwedishLeaf	0.9659	0.9216	0.9280	0.9536	0.9563	0.9466	0.9664	0.9712
Symbols	0.9746	0.9668	0.8824	0.9739	0.9064	0.9616	0.9799	0.9819
*SynCon	0.9970	0.9667	0.9833	0.9967	0.9983	0.9953	1.0000	0.9967
*ToeSeg1	0.9702	0.9386	0.9649	0.9825	0.9627	0.9246	0.9693	0.9693
ToeSeg2	0.9262	0.9615	0.9077	0.9538	0.9062	0.8623	0.9538	0.9385
*Trace	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
TwoLeadECG	0.9991	0.9807	0.9974	0.9965	1.0000	0.9886	0.9965	0.9956
TwoPatterns	1.0000	0.9930	0.9550	1.0000	0.9999	0.9996	1.0000	1.0000
UWavGesLibAll	0.9757	0.9389	0.9422	0.9685	0.8595	0.9723	0.9687	0.9545
UWavGesLibX	0.8546	0.7621	0.8029	0.8398	0.7805	0.8286	0.8417	0.8247
*UWavGesLibY	0.7729	0.6851	0.7303	0.7655	0.6701	0.7615	0.7716	0.7688
UWavGesLibZ	0.7917	0.6949	0.7485	0.7831	0.7501	0.7640	0.7797	0.7697
*Wafer	0.9983	0.9948	1.0000	0.9994	0.9986	0.9955	0.9989	0.9987
Wine	0.8074	0.7407	0.7963	0.7778	0.7444	0.5685	0.8889	0.6667
*WordSynonyms	0.7552	0.6379	0.5705	0.7382	0.6224	0.7787	0.7868	0.7555
*Worms	0.7273	0.5584	0.7403	0.5584	0.7909	0.7182	0.7922	0.8052
WormsTwoClass	0.7987	0.8312	0.8312	0.7792	0.7468	0.7844	0.8182	0.7922
*Yoga	0.9085	0.9183	0.8177	0.9177	0.8702	0.8786	0.8483	0.9057

E Results for Additional 2018 Datasets

Table 2: Classification Accuracy, Additional 2018 Datasets

	Rocket	DTW
ACSF1	0.8780	0.6200
AllGestureWiimoteX	0.7619	0.7171
AllGestureWiimoteY	0.7617	0.7300
AllGestureWiimoteZ	0.7491	0.6514
BME	1.0000	0.9800
Chinatown	0.9802	0.9536
Crop	0.7502	0.7117
DodgerLoopDay	0.5762	0.5875
DodgerLoopGame	0.8725	0.9275
DodgerLoopWeekend	0.9725	0.9783
EOGHorizontalSignal	0.6409	0.4751
EOGVerticalSignal	0.5423	0.4751
EthanolLevel	0.5820	0.2820
FreezerRegularTrain	0.9976	0.9070
FreezerSmallTrain	0.9519	0.6758
Fungi	1.0000	0.8226
GestureMidAirD1	0.8062	0.6385
GestureMidAirD2	0.6831	0.6000
GestureMidAirD3	0.5785	0.3769
GesturePebbleZ1	0.9663	0.8256
GesturePebbleZ2	0.8911	0.7785
GunPointAgeSpan	0.9968	0.9652
GunPointMaleVersusFemale	0.9978	0.9747
GunPointOldVersusYoung	0.9905	0.9651
HouseTwenty	0.9639	0.9412
InsectEPGRegularTrain	0.9996	0.8273
InsectEPGSmallTrain	0.9815	0.6948
MelbournePedestrian	0.9035	0.8482
MixedShapesRegularTrain	0.9704	0.9089
MixedShapesSmallTrain	0.9386	0.8326
PLAID	0.8896	0.8361
PickupGestureWiimoteZ	0.8100	0.6600
PigAirwayPressure	0.0885	0.0962
PigArtPressure	0.9529	0.1971
PigCVP	0.9327	0.1587
PowerCons	0.9311	0.9222
Rock	0.8980	0.8400
SemgHandGenderCh2	0.9230	0.8450
SemgHandMovementCh2	0.6444	0.6378
SemgHandSubjectCh2	0.8836	0.8000
ShakeGestureWiimoteZ	0.8920	0.8400
SmoothSubspace	0.9793	0.9467
UMD	0.9924	0.9722

A Field Experience for a Vehicle Recognition System using Magnetic Sensors

Giovanni Burresi
Department of Information Engineering
University of Florence, Firenze, Italy
giovanni.burresi@unifi.it

Roberto Giorgi
Department of Information engineering and mathematics
University of Siena, Siena, Italy
giorgi@dii.unisi.it

Abstract—**This paper describes the development and testing of a vehicle recognition prototype based on magnetic sensors. The aim of this research is to design a low cost, low power consumption and simple hardware platform for vehicle recognition. The goal is to recognize four types of vehicles (car, bus, mini-bus or camper) as they run over a set of magnetic sensors. We describe all steps for correct vehicle presence detection, pattern pre-processing, speed and length detection using a combination of an empirical and an analytical method for signal alignment. We collected a set of data regarding this types of vehicles and explain how to differentiate them. Our classification tests reach a confidence factor greater than 91%.**

Keywords-component; classification; vehicle recognition; magnetic sensors; cyber-physical systems; traffic monitoring.

I. INTRODUCTION

Cities are getting bigger, roads increase their number and sizes and an increasing number of vehicles uses them daily. It is getting more and more important to know how they move, which fluxes they generate and their size, and also which types of vehicle compose these fluxes. Many methods are used to perform this task like cameras and image processing [9][10]. This work is focused on getting complementary traffic information from other types of sensors. In particular, we develop a vehicle monitoring and recognition system based on magnetic sensors, trying to achieve a contained hardware complexity. The rest of the paper describes the state of art of vehicle recognitions in section 2, the adopted hardware platform in sections 3, 4 and 5, all the implemented methods and mechanisms made to provide enoughaccuracy level finalized to vehicle recognition based on magnetic sensor (sections 7 and 8).

The main contributions of this paper are:

- Numerical field data for a traffic monitoring cyber-physical system,
- methodology for vehicle recognition based on magnetic sensors.

II. DETECTION AND CLASSIFICATION OF VEHICLES

A. State of the art

To perform vehicle detection different methods are used, depending on the requirements and the area where they are deployed. Most common technologies include: magnetic coils, pneumatic tubes, piezoelectric sensors, microwave radars, infrared sensors, video recognition, magnetic sensors [1]. Not all of them offer the same amount of information: magnetic sensor based detection is one of the technologies that allow us to gather information in a simple and accurate way [8]. The choice of magnetic sensors for this work was influenced by two other factors: low costs for installation and limited sensitivity to various types of noise (Table 1). In addition to detecting the passing of vehicles, we want to figure out which category they belong to, such as cars, camping van, buses and mini-buses.

B. Vehicle Recognition using magnetic sensors

Earth's magnetic field provides ways to build magnetic sensors that does not require magnetic materials on the vehicles. When magnetic flux lines condense or became widen because a vehicle is perturbing them, a magnetic sensor placed in proximity of vehicle is under the same magnetic perturbation created by the vehicle itself (Figure 1). Earth's magnetic field has a constant magnetic background for a fixed installation. Its intensity is about 0.5 G for a single axis and a 0.7 G variation from natural magnetic range [2][3]

TABLE I. VEHICLE RECOGNITION SYSTEM CAPABILITIES COMPARISON

Technology	Data Type				
	Count	Speed	Classification	Occupancy	Presence
Intrusive					
Inductive Loop	Y	Y	Y	Y	Y
pneumatic road tube	Y	Y	Y	N	N
piezoelectric cable	Y	Y	Y	N	N
Non-Intrusive					
WIM system	Y	Y	Y	N	N
Microwave Radar					
CW Doppler	Y	Y	Y	Y	N
FMCW	Y	Y	Y	Y	Y
Infrared					
Active	Y	Y	Y	N	N
Passive	Y	Y	Y	Y	Y
Video Image Processing	Y	Y	Y	Y	Y
Ultrasonic	Y	N	N	N	Y
Passive Acoustic	Y	Y	Y	Y	Y
Wireless Sensor Network					
Magnetometer	Y	Y	Y	Y	Y

Y: available, N: not available

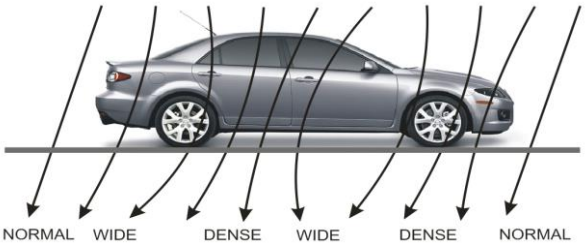


Figure 1:Earth magnetic field variations performed by a vehicle

III. PROJECT OVERVIEW

A. Project Phases

The project was divided into the following phases:

- Research in the literature about possible techniques for vehicles recognition.[6][7]
- Designing and assembling the sensor.
- Implementation of an analysis and post-processing software and data collection.
- Collection of samples in the field.
- Development of mechanisms for the correct vehicles segmentation, to ensure a proper construction of the input samples for the AI.
- Development of a MLP neural network suitable to recognize the four categories listed above.
- On-line and off-line testing phase

The last three steps were obviously not consecutive and clearly distinct. They were repeated and retraced several times, in order to find the right balance between the components involved.

B. Main components

The system consists of three main components:

- Sensors devices
- A Microcontroller and a CPU
- Recognizer algorithm (Java based)

IV. SENSORS DEVICE AND PLATFORM

A. Magnetic sensors hardware

We decided to use two Honeywell HMC 5843 (3-axis) magnetometer placed on a plastic support 1.2 m. (Figure 3) far one each other. We placed the two sensors with the same axis orientation.

Sensors orientation is very important. In theory, if a vehicle runs over two sensors with a constant speed and with direction parallel to the line passing through the two sensors, we expect to read the same output response on both sensors. This allows us to determine a time gap between the two signals and consequentially a distance.

B. Platform

To implement our prototypes we need the following features: a low power consumption platform, a CPU that runs a Java VM, a microcontroller interfacing with sensors and a wireless connection. We also want to reduce costs and simplify the hardware connections. As shown in Table 1 we choose the UDOO [12] as this reduces costs for our specification and its heterogeneous architecture also simplify the hardware implementation.

UDOO and Raspberry have similar price but we prefer UDOO for the higher performance CPU, for the flexibility and future upgrades[12].

The chosen microcontroller is an Arduino Due to simplify the interfacing with several magnetic sensors. Each sensor detects the following states:

- State 0: No vehicles over the sensors.
- State 1:The device hypothesize that a vehicle starts
- State 2: A vehicle is over the sensors.
- State 0-tail: A vehicle was over the sensors and now we suppose that is terminated. Therefore we are going on sending data to allow subsequent analysis on effective vehicle ending.

V. VEHICLE DETECTION SYSTEM

For some vehicles magnetic field signal tends to Earth's bias in random points of its length, with no speed dependency. It is an individual characteristic of each vehicle model. So it is not possible to avoid vehicle classification segmentation only using an amplitude threshold (Figure 2).

Now we will explain how the device makes a decision about the presence or absence of a vehicle on the sensors. The algorithm we developed uses the following sensitive points (Figure 4):

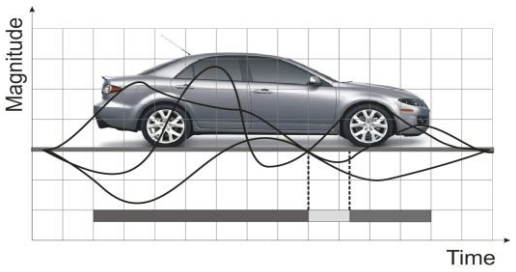


Figure 2: Vehicle with a convergent magnetic field in the middle of his lenght

A. up/down threshold: amplitude threshold is not a single value but we introduce an hysteresis cycle with two different up and down values.

B. under-threshold cycles: all 3 magnetic axis-components must be under this down threshold for n samples.

C. zero-crossing: magnetic axis-components must not change sign for m samples.

D. decreasing derivative: derivative of all magnetic axis-component modules must be negative for p samples.

E. tail: if all criteria are satisfied microcontroller goes on sending data for q samples. It allows to the recognizer to analyze data and determine if two sample set were wrongly subdivided.

VI. CLASSIFICATION METHODOLOGY

A. Data transmission and storage

Sensors data are transmitted on internal UART serial with a 115200 baud rate. All data are stored on database that includes: raw data, formatted data, pattern predictions, and other debug information used during development.

B. Preprocessing

Data is stored in a vector of "Magnetic Values", representative of the passing of a vehicle. The values contained in the vector cannot be used immediately as a feature for the neural network based recognizer. It is necessary to perform a pre-processing phase on the data, in order to make as homogeneous as possible the samples vector with each other.

Smoothing: A first step consists in smoothing pattern. To do this we scroll through all the vector of the samples, and for each step we apply a linear filter defined as follows:

$$S_t = 0.33\ V_{t-2} + 0.66\ V_{t-1} + V_t + 0.66\ V_{t+1} + 0.33\ V_{t+2}$$

Removing "dead zones": In our application context a vehicle may slow down or even stop. In a second step we try to remove those areas in which magnetic field values are stable within a certain threshold.

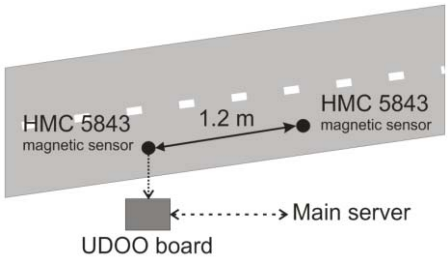


Figure 3: Hardware overview

C. Speed estimation

We can then proceed with one of the most delicate steps: estimating the speed of the vehicle. To do this it is necessary to find the distance in terms of time between the two sensors curves. Then we proceed to search for *significant points*, in order to make an attempt to alignment. First, we look for the maximum and minimum on all three axes.

Other notable points of the curve that we consider are the points of change of sign. In this case, however, we do not take all the changes of sign, but only those which occur very abruptly (a 100 mG gap).

Once we have the largest number of sensitive points on the two curves, we just have to look for alignment. For each point found in the curve of the first sensor, we look for the same point with the same type in the curve of the second sensor.

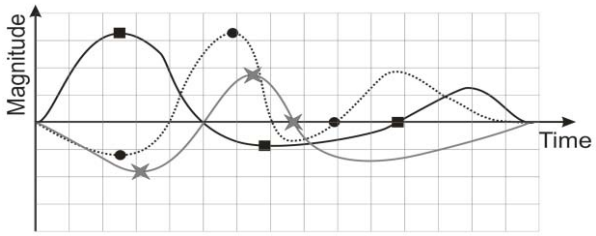


Figure 4: Vehicle sensitive points on 3-axis graph

The search is carried out within a certain window. Once this limit has been reached, if the maximum is not found, we consider that point as not alignable with the previous ones.

The problem with this method is that there is not any guarantee about the homogeneity of the points on which the alignment is attempted. So there is no certainty that the average velocity has been assessed during the entire passing of the vehicle (Figure 5).

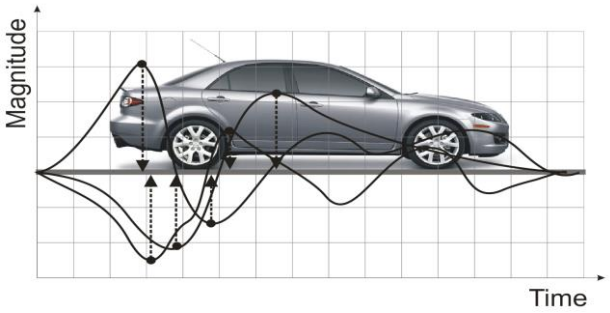


Figure 5: Uneven distribution of sensitive points in a vehicle

D. Dynamic Time Warping

The alignment method previously studied is definitely not analytical and deterministic, because it is mainly based on heuristics and empirical methods. So, we tried to identify and test a more analytical methodology, looking for any algorithm or more systematic methodology for the time sequences alignment. An interesting technique is represented by Dynamic Time Warping (DTW), originally developed in the context of speech recognition [4][5].

We have observed that in our case the estimate was not always consistent with the real case. The problem arises from the inherent nature of how our curves are constructed. First of all, in our case, the second curve will always be temporally shifted forward: the instants of beginning are never simultaneous, but the initial values of the second sensor will never have a correspondence with those of the first.

The second problem arises from the fact that ideally the DTW search for the alignment of a point of the first sensor at time T_i in the previous and successive instants. This implies that for some points of the second sensor the best alignment may happen with an apparently negative speed [6].

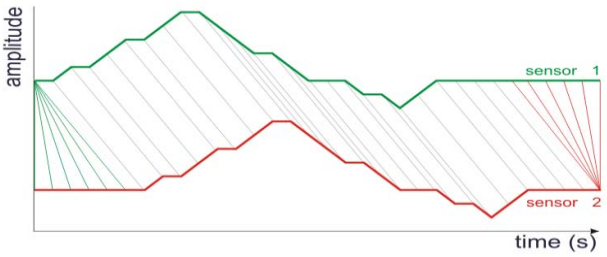


Figure 6: An ideal alignment of two magntic signals

After a series of tests we have observed that the use of the DTW does not provide significant improvements in the alignment of the curves. One possible development could be the customization of the algorithm DTW, by entering ad-hoc constraints to our case study.

E. Feature construction

Several combinations have been tested for the input features. As previously mentioned, the length of the vehicle is a feature that is easily detectable directly.

For other features we used the average values of the curves for each axis, in various portions of the curve.

For example, we can use the average value calculated on the Z axis in the first and in the second half of the curve. So an input feature can be described as follows:

$$F_1 = (Z^1_{AVG} ; Z^2_{AVG} ; length).$$

Similarly, we can use the average value of all axes:

$$F_2 = (X^1_{AVG}, X^2_{AVG}, Y^1_{AVG}, Y^2_{AVG}, Z^1_{AVG}, Z^2_{AVG}, length)$$

Otherwise we can divide each axis into a greater number of portions. Unfortunately, since there is no deterministic way to choose the most significant feature, it was necessary to repeat many configurations and multiple phases of training.

After that we choosed to use the average values of the Y and Z axes calculated of 5 portions of the curve, to which is added obviously the estimated length of the vehicle. Therefore:

$$F_3 = (Y^1_{t1}, Y^1_{t2}, Y^1_{t3}, Y^1_{t4}, Y^1_{t5}, Z^1_{t1}, Z^1_{t2}, Z^1_{t3}, Z^1_{t4}, Z^1_{t5}, length)$$

VII. Classification Test Results

Following our methodology, vehicle segmentation was reduced to about 2%. This allowed us to collect and use reliable correct data. It is possible to configure the system to perform predictions with different confidence ratio. Depending on user specifications it is possible to set different precision capabilities. Obviously a higher precision ratio involve more unpredicted vehicles.

In the last test performed on 332 samples were set two different confidence values.

A. Minimum confidence 30%

TABLE II. Confidence Matrix 30%

Output	*Car*	*Bus*	*Camper*	*Minibus*	
Car	88	0	0	1	99%
Bus	5	176	0	1	96,7%
Camper	6	1	9	5	43%
Minibus	3	3	3	31	77.5%
	86.3%	97.8%	75%	81.6%	91.6%

predicted: 332 unpredicted: 0 correct pred.:304 (91,6%) wrong pred.: 28 (8.4%)

B. Minimum confidence 50%

TABLE III. Confidence Matrix 50%

Output	*Car*	*Bus*	*Camper*	*Minibus*	
Car	87	0	0	0	100%
Bus	3	176	0	0	98.3%
Camper	2	0	6	0	75%
Minibus	2	2	3	23	82.1%
	92.6%	98.9%	85.7%	85.7%	96.7%

predicted: 302 unpredicted: 30 correct preds:292 (96.7%)wrong preds: 10 (3.3%)

C. Notes

Classification errors strongly decrease as we increase the minimum confidence percentage for assigning a pattern to a vehicle type. Accuracy increases up to 96,7%.

This is the configuration used for the Multi-Layer Perceptron (MLP) based machine learning classification:

- output layer neurons: 4 (output range 0 and 1)
- input layer neurons: 11
- hidden layer neurons: 4
- hidden layer's activation function: sigmoid

VIII. Conclusions

The prototype reaches good classification results greater than 90%. Better results can be reached with a large and more uniform dataset. The developed methodology allows us to use more complex and discriminant features. By adopting the Dynamic Time-Warping (DTW) algorithm introducing we improved vehicle length classification accuracy.

IX. Future works

We wish to add on the same hardware platform: camera streaming, a web server interface for visual traffic monitoring and control panel. We will evaluate multi-core embedded boards performance to find the best candidate for a large scale prototypation.

Acknowledgment

This project is partly founded by project EU AXIOM 6454916 and ERA 249059 [11].

References

[1] Sing-Yiu Cheung, Pravin Varaiya (2007) Trac Surveillance by Wireless Sensor Networks: Final Report, Institute of Transportation Studies University of California, Berckley.

[2] Kaewkamnerd, S., Pongthornseri, R., Chinrungrueng, J., Silawan, T. (2009) Automatic vehicle classification using wireless magnetic sensor, Nat. Electron. and Comput. Technol. Center, Pathumthani, Thailand.

[3] Jun Xing, Qi Zeng, (2007) A Model Based Vehicle Detection and Classication Using Magnetic Sensor Data, Department of Signals and Systems Chalmers University of Technology Goteborg, Sweden.

[4] Stan Salvador, Philip Chan, FastDTW: Toward Accurate Dynamic Time Warping in Linear Time and Space, Dept. of Computer Sciences Florida Institute of Technology, Melbourne.

[5] Hiroaki Sakoe, Seibi Chiba, (1978) Dynamic Programming Algorithm Optimization for Spoken Word Recognition.

[6] Coifman, Benjamin, and SeoungBum Kim. "Speed estimation and length based vehicle classification from freeway single-loop detectors." Transportation research part C: emerging technologies 17.4 (2009).

[7] Pushkar, Anna, Fred L. Hall, and Jorge A. Acha-Daza. "Estimation of speeds from single-loop freeway flow and occupancy data using cusp catastrophe theory model." *Transportation Research Record* 1457 (1994).

[8] Keawkamnerd, Saowaluck, Jatuporn Chinrungrueng, and Chaipat Jaruchart. "Vehicle classification with low computation magnetic sensor." *ITS Telecommunications, 2008. ITST 2008. 8th International Conference on*. IEEE, 2008.

[9] Lai, A.H.S.; Fung, G.S.K.; Yung, N.H.C., "Vehicle type classification from visual-based dimension estimation," *Intelligent Transportation Systems, 2001. Proceedings. 2001 IEEE* , vol., no., pp.201,206, 2001

[10] V Kastrinaki, M Zervakis, K Kalaitzakis, A survey of video processing techniques for traffic applications, Image and Vision Computing, Volume 21, Issue 4, 1 April 2003, Pages 359-381.

[11] Scionti, A., Kavvadias, S., Giorgi, R. Dynamic power reduction in self-Adaptive embedded systems through benchmark analysis (2014) Proc. MECO 2014 - Including ECyPS 2014, pp. 62-65

[12] Website: http://www.udoo.org

electronics

MDPI

Article

Methods for Magnetic Signature Comparison Evaluation in Vehicle Re-Identification Context

Juozas Balamutas [1], Dangirutis Navikas [1], Vytautas Markevicius [1], Mindaugas Cepenas [1], Algimantas Valinevicius [1], Mindaugas Zilys [1], Michal Prauzek [2], Jaromir Konecny [2], Michal Frivaldsky [3], Zhixiong Li [4] and Darius Andriukaitis [1,*]

[1] Department of Electronics Engineering, Faculty of Electrical and Electronics Engineering, Kaunas University of Technology, Studentu St. 50, LT-51368 Kaunas, Lithuania; juozas.balamutas@ktu.lt (J.B.); dangirutis.navikas@ktu.lt (D.N.); vytautas.markevicius@ktu.lt (V.M.); mindaugas.cepenas@ktu.lt (M.C.); algimantas.valinevicius@ktu.lt (A.V.); mindaugas.zilys@ktu.lt (M.Z.)
[2] Department of Cybernetics and Biomedical Engineering, VSB—Technical University of Ostrava, 708 00 Ostrava, Czech Republic; michal.prauzek@vsb.cz (M.P.); jaromir.konecny@vsb.cz (J.K.)
[3] Department of Electronics and Mechatronics, Faculty of Electrical Engineering and Information Technologies, University of Žilina, 010 26 Žilina, Slovakia; michal.frivaldsky@uniza.sk
[4] Department of Manufacturing Engineering and Automation Products, Opole University of Technology, 45-758 Opole, Poland; z.li@po.edu.pl
* Correspondence: darius.andriukaitis@ktu.lt; Tel.: +370-37-300-519

Abstract: Intelligent transportation systems represent innovative solutions for traffic congestion minimization, mobility improvements and safety enhancement. These systems require various inputs about vehicles and traffic state. Vehicle re-identification systems based on video cameras are most popular; however, more strict privacy policy necessitates depersonalized vehicle re-identification systems. Promising research for depersonalized vehicle re-identification systems involves leveraging the captured unique distortions induced in the Earth's magnetic field by passing vehicles. Employing anisotropic magneto-resistive sensors embedded in the road surface system captures vehicle magnetic signatures for similarity evaluation. A novel vehicle re-identification algorithm utilizing Euclidean distances and Pearson correlation coefficients is analyzed, and performance is evaluated. Initial processing is applied on registered magnetic signatures, useful features for decision making are extracted, different classification algorithms are applied and prediction accuracy is checked. The results demonstrate the effectiveness of our approach, achieving 97% accuracy in vehicle re-identification for a subset of 300 different vehicles passing the sensor a few times.

Keywords: intelligent transportation systems; vehicle re-identification; magnetic signature; signal classification

Citation: Balamutas, J.; Navikas, D.; Markevicius, V.; Cepenas, M.; Valinevicius, A.; Zilys, M.; Prauzek, M.; Konecny, J.; Frivaldsky, M.; Li, Z.; et al. Methods for Magnetic Signature Comparison Evaluation in Vehicle Re-Identification Context. *Electronics* **2024**, *13*, 2722. https://doi.org/10.3390/electronics13142722

Academic Editor: Felipe Jiménez

Received: 22 June 2024
Revised: 5 July 2024
Accepted: 10 July 2024
Published: 11 July 2024

1. Introduction and Related Work

Usage of magnetic sensing in intelligent transportation systems (ITS) is rapidly gaining popularity [1–3] compared to more traditional video processing [4,5]. Various systems for road traffic surveillance largely utilize MEMS (micro-electromechanical systems) magnetometers for the purpose of vehicle detection and classification [6,7]. Examples of such applications include vehicle speed calculation [8,9], length estimation [10], classification into defined categories [11], smart parking [12] and autonomous driving [13]. Wireless magnetic sensor networks are a popular choice for information acquisition. This solution has the advantages of low cost, low power consumption and miniaturization. Study [14] highlights the drawback of magnetic sensing for a parking application, which is the weak magnetic region under the vehicle. It is difficult to separate this magnetic signal from ambient noise since the magnetometers are mounted in the middle of the parking lane. To solve this problem, a UWB radio channel was proposed. The authors of [15] investigated anomaly detection for a moving magnetic target in controlled laboratory conditions. The

velocity–frequency relationship, direction angle and closest point of approach distance were highlighted in this study. The mathematical relationship between the detection platform and magnetic target was established to account for velocity differences [16].

Machine learning methods for magnetic signal analysis are gaining popularity. In [11], a method for vehicle detection and recognition using machine learning, input vectors composed from cestrum energy and Mel frequency cepstral coefficients was utilized. Vehicles were classified into five defined categories with high accuracy, although the sample size for training and testing was not very large. The authors of [17] defined vehicle classification as the process of assigning each vehicle to a pre-defined vehicle class based on some features extracted from its magnetic signature. The task of vehicle re-identification [18] was applied in determining segment travel times. Using an inductive loop detector, the authors implemented a decision tree using statistical features like lane speed and travel duration estimation. Classification into four different classes was performed in [19]. The authors extracted 10 different statistical features from the z axis of the triaxial magnetometer—such as mean, variance and local minimums and maximums. Classification was performed using different machine learning algorithms (BP NN, SVM, Random Forest) on a dataset containing more than 4500 vehicle entries. The classification result reached 80%. Real-time vehicle classification into nine classes was performed in [20] utilizing three-layer feedforward artificial neural networks (ANN). The authors established that using only time-domain features requires low computation and few memory resources, which enables easier implementation and real-time operation. A frequent problem in vehicle classification is an imbalanced dataset—certain types of vehicles like buses and motorcycles occur less often compared to sedans and hatchbacks. A solution called SMOTE was proposed by the authors of [21]. According to certain rules, the SMOTE algorithm randomly generates new minority sample points, which will be merged into the original. Utilizing this algorithm with a K-nearest neighbor (KNN) classifier, they managed to classify vehicles by magnetic signatures into four classes with 95% accuracy. Balid, in [22], used various statistical measurements to classify vehicles according to an FHWA scheme. Different classifiers were tested, all reaching almost 98% accuracy.

In [23], researchers explored the potential of estimating vehicle location using a magnetic sensor array. Although the array consisted of only two sensors, the mathematical principles can be extended to larger arrays. The vehicle, traveling parallel to the sensor array, is modeled as a magnetic dipole, generating an ideal dipole magnetic signature. While the overall accuracy was not specified, the study concluded that it is feasible to detect vehicle location using a magnetic sensor array. Similarly, Kwong [24] developed a system for the real-time estimation of travel time across the arterial segment of 1.5 km by identifying the same vehicles. The matching procedure is based on a statistical model of signature distance. The model can be used to predict rates of correct, incorrect and missed matches. As stated by [25], vehicle re-identification (identifying the same exact vehicle in traffic) can give essential information for traffic management about travel time and origin–destination matrices. Real-time traffic data are essential for optimizing traffic flow and reducing travel time and greenhouse gas emissions. The authors collected 261 temporal magnetic signatures from 25 different vehicles in the parking lot. A total of 10 sensors with 25 cm spacing and 200 Hz sampling frequency were used. Their findings included that the Euclidean distance and DTW (dynamic time warping) algorithms are suitable for same vehicle identification; however, performance is linked to the lateral position of the vehicle on the road.

Fourteen wireless magnetic sensors were installed in an array [26], and vehicle signature features consisting of a collection of peak values were collected. By comparing the upstream and downstream detectors' registered signatures and calculating different statistical features (mean, standard deviation), arrays were compared using distance metrics. The authors suggested that using their developed algorithm, the mismatch rate is around 8%, and, for congested traffic conditions, it reaches 14%. The authors of [27] utilized six magnetic sensors with 20 cm spacing for AVMR (automatic vehicle model recognition)—given

an unknown vehicle's signature and set of a database vehicle signatures, the goal is to associate the correct vehicle model. Re-identification using DTW and machine learning models (k-NN, LMD, Gaussian classifier) has shown promising results; however, there are no experiments indicating how the algorithms would perform with new, unseen data.

The advantages of wireless sensor networks utilizing magnetic sensing are summarized in [28]—they do not require an external power supply, they have a compact size, they minimally damage the road surface and they have resistance to interference from the urban environment. The authors emphasize as a disadvantage that magnetic sensors do not provide information about the direction of the vehicles—available systems cannot detect which gate the car entered the area through or left through. With our work, we try to challenge this statement and demonstrate that it is possible to determine specific vehicle traveling patterns in a specified area with great accuracy.

The available research is concentrated on single-magnetometer feature extraction for vehicle classification to strictly defined categories. Not much work is available on the vehicle re-identification problem—finding the same vehicle passing over sensors by comparing its magnetic signature with previously recorded signatures. Articles revolving around vehicle re-identification methods involved testing in restricted conditions or with a small data subset. In this work, a feature extraction method for a magnetometer array is proposed, and different machine learning methods are compared for same vehicle re-identification in real traffic conditions. Focus is shifted to define better input features and compare different classification algorithms.

2. Materials and Methods

Vehicles are registered using our system, presented in a previous work [29], updated to 15 sensors with 10 cm spacing between sensors. The system constantly measures Earth's magnetic field with tri-axis magnetic sensors, each sampled at 1 kHz frequency. Measured values are constantly detrended and register only change in the magnetic field. The magnetometer's measuring range is set to ± 400 μT, and practical experiments showed that the measured ambient noise is 1.5 μT when no vehicles or other metallic objects are near the sensors. The system starts to register magnetic signatures, then the center sensor of the array's calculated module exceeds 5 μT and waits for a decrease in amplitude or timeout condition. Collected signatures for all sensors (matrix consisting of 15 *x/y/z* arrays) are transmitted to a central server. Additionally, images from a temporarily installed camera with license plate recognition are collected and saved together with a .csv file.

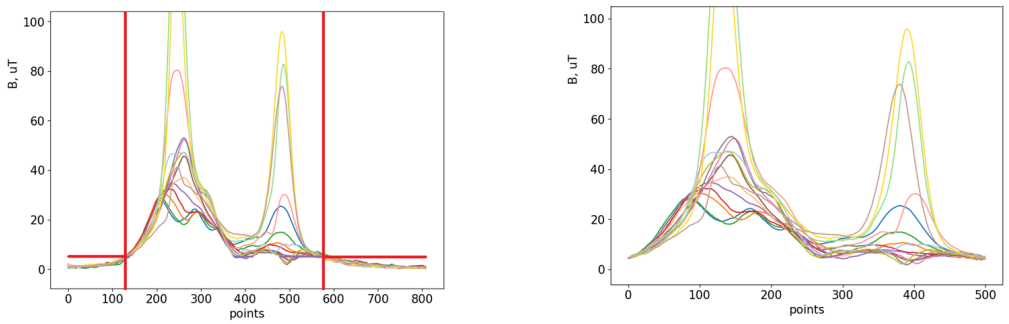
In the preprocessing stage, raw collected x/y/z values undergo low-pass filtering with a 100 Hz cutoff frequency in order to smooth the signal. Modules for every sensor are calculated according to the following formula:

$$mod_n = \sqrt{x^2{}_n + y^2{}_n + z^2{}_n} \quad (1)$$

where mod_n—calculated module, $x/y/z$—raw registered values, n—sensor number.

Signature length depends on vehicle speed and vehicle length. Many feature extraction methods and machine learning algorithms expect input data to be of uniform length. Signals with a different number of points might lead to inaccuracies and inconsistencies in the analysis. In the deployed signature collection system, vehicles travel with speeds ranging from 30 km/h to 60 km/h. Most of the time, they are light passenger cars with an average length of 4.5 m. The collected signatures sample count ranges from 600 to 1300 points. Based on these observations, it was decided to resample all signals to a fixed number of 500 points. This process involves interpolating or resampling the signals so they all have the same length regardless of the original number of data points. The vehicle signature sampling process is initiated by a single magnetometer located in the center of the array; other sensors values might be shifted with reference to this sensor depending on the angle at which the vehicle is passing through the sensors. To address these issues, all signature

values are cropped to a specified threshold—5 μT. The cropping and resampling process is illustrated in Figure 1.



(**a**) (**b**)

Figure 1. Registered signature cropping and resampling process. Original calculated modules (**a**) and cropped and resampled modules (**b**).

For each calculated module index, we identify points where the amplitude exceeds a pre-defined threshold. This search is performed from the beginning and the end of the signal. A new array is then formed based on these identified indices. The middle of the signal is not checked as the amplitude might drop below the threshold, but it still represents an important part of the signal.

The pipeline for the experiments is outlined in Figure 2. A record is defined as one unique vehicle passing once through the signature collection system and contains 15 *x*/*y*/*z* signal arrays of registered magnetic distortions. The collected database contains numerous different vehicles with multiple records. Information about vehicle make, model and driving trajectory is taken from the registered image. For experimentation, we select subsets from the database with the required number of vehicles n, each containing m passing entries from passing through the system. Constraints are added so that no vehicles of the same model should exist, and driving direction must be the same. One subset is selected for training samples and another subset for testing. Record comparison, pair generation, feature extraction, model training and evaluation are performed in parallel, applying the same actions.

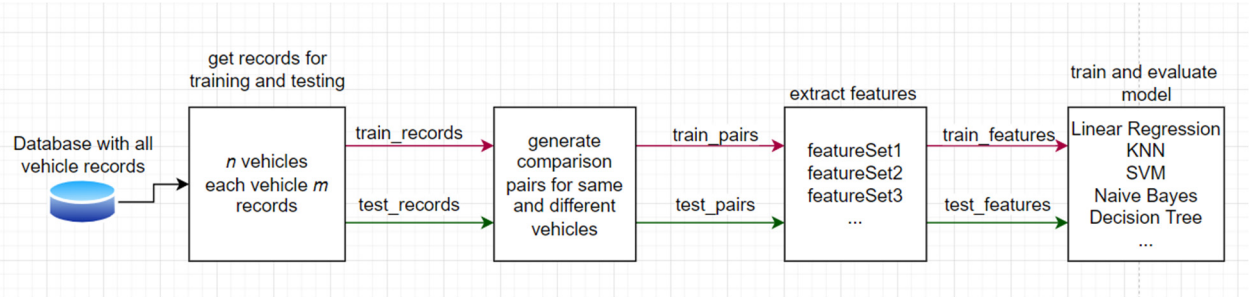


Figure 2. Pipeline for experiments.

Record pairs labeled "same" are defined as pairs of records that refer to the signatures from the same unique vehicle, while "different" records refer to pairs of records that refer to signatures from different vehicles. In our experiment, we used these definitions to evaluate the performance of our algorithm in distinguishing between matching (same) and non-matching (different) records. An example of the records is shown in Figure 3.

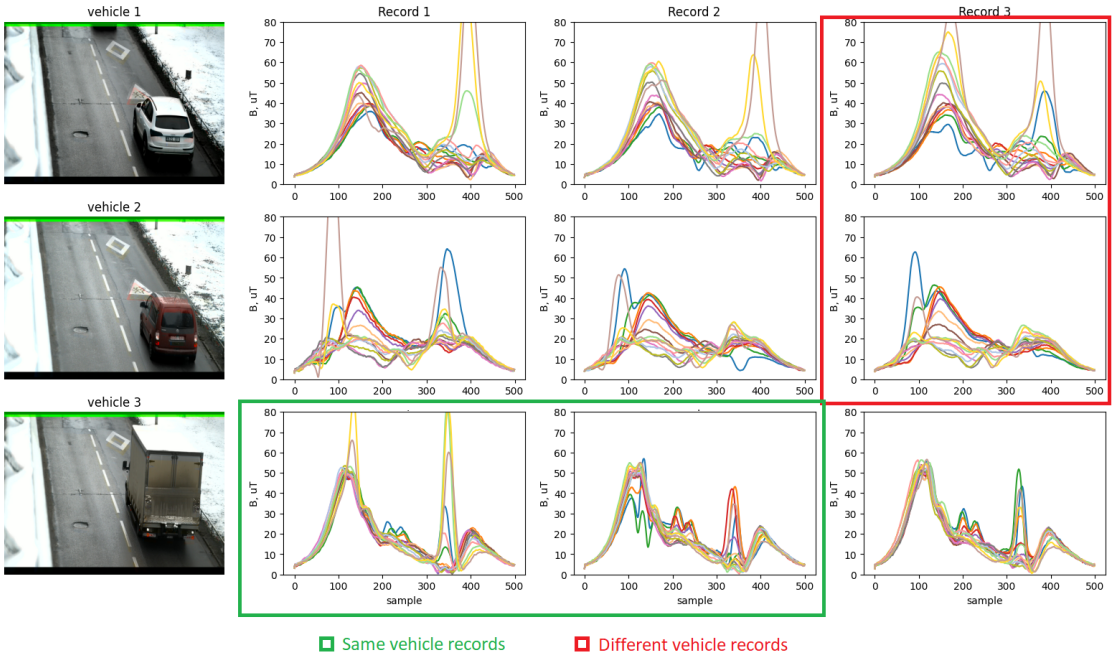


Figure 3. Explanation of same and different vehicle records with one "same" and one "different" pair marked.

The comparison pair generation algorithm (Algorithm 1) involves two steps—generating pairs for the same vehicles and, after that, pairs for different vehicles. Empty lists are initialized to store references to records. Each vehicle is iterated, and all record pairs are added to the list. Same records are not compared to themselves; in this way, the maximum number of "same" labels are created. In order to have a balanced dataset, an equal number of different record pairs should exist. For different record comparison pairs, two vehicles are randomly selected and checked to see whether they are different, and then a random record is selected for each vehicle. This process is repeated until different vehicle comparison pairs have an equal number to the same vehicle comparison pairs. The condition exists that randomly selected records cannot repeat.

Algorithm 1. Generate Comparison Pairs

Input: Database of n vehicles, each with m records
Output: Comparison pairs for each vehicle and between different vehicles
Initialize empty list sameVehiclePairs
Initialize empty list differentVehiclePairs
>Generate same-vehicle comparison pairs
for each vehicle $v \in \{1, 2, \ldots, n\}$ **do**
    for each record $r_i \in \{1, 2, \ldots, m-1\}$ **do**
        for each subsequent record $r_j \in \{r_i + 1, r_i + 2, \ldots, m\}$ **do**
            Add (v, r_i, r_j) to sameVehiclePairs
        end for
    end for
end for
>Generate different-vehicle comparison pairs
countSamePairs ← length of sameVehiclePairs
while length of differentVehiclePairs ¡ countSamePairs **do**
    Randomly select two different vehicles v_1 and v_2 from $\{1, 2, \ldots, n\}$
    if $v_1 \neq v_2$ **then**
        Randomly select one record r_i from v_1 and one record r_j from v_2
        Add (v_1, r_i, v_2, r_j) to differentVehiclePairs
    end if
end while

After extensive analysis of the magnetometers axis values and modules comparison, the decision was made to use the calculated signature modules processed as explained earlier. An example of the registered magnetic signature modules for the same vehicle driving through the system is shown in Figure 4. Based on the collected signal analysis, it is evident that at least one side of the vehicle's wheels is always presented in the single passing vehicle record, as shown in figure by the red dotted line. Sensors which are affected by the wheels register a distorted signature, usually with a much higher amplitude or unpredictable shape. Data of such sensors should be omitted from direct signal comparison. An example of different vehicles' signatures is presented in Figure 5. It is evident that the signatures' shape and amplitudes are significantly different between these two records.

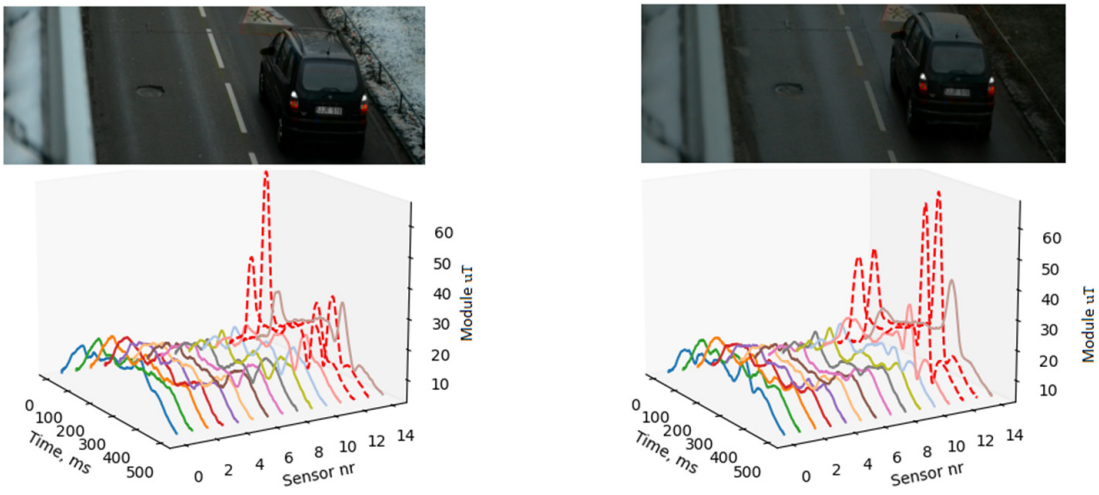


Figure 4. Two records (**a**,**b**) of the same vehicle driving through the registration system almost in the same trajectory. The red dotted line marks the wheel influence in the recorded signature.

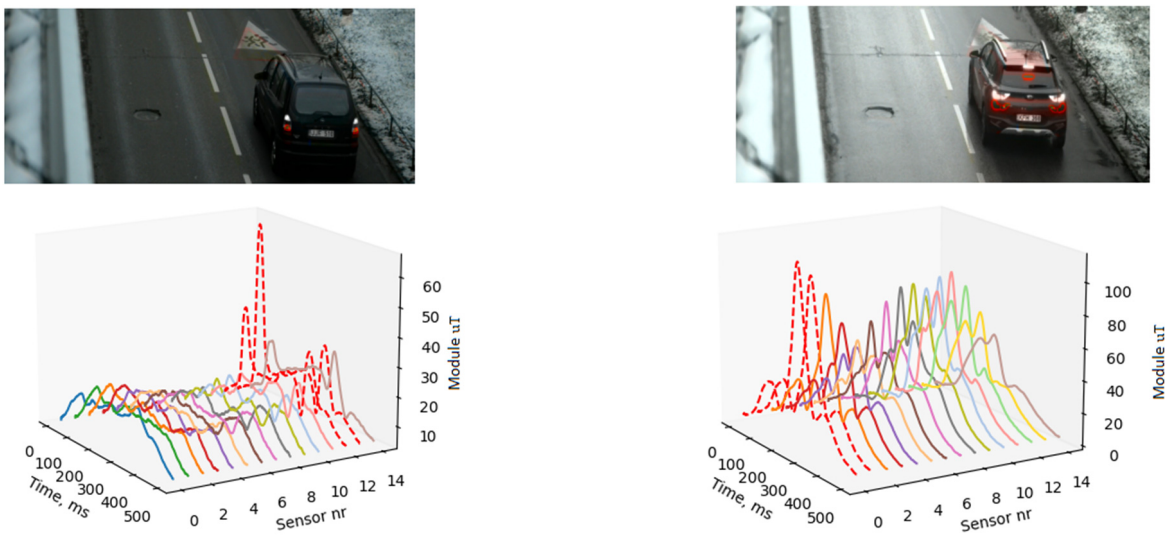


Figure 5. Record 1 registered signatures (**a**) and record 2 registered signatures (**b**) for different vehicles.

Vehicle re-identification is performed by searching for similarities between the registered magnetic signatures modules. The challenge lies in discovering how to effectively

evaluate these similarities. Different feature extraction ideas were tested. Every vehicle has a unique signature, although certain common attributes which are repeating between different vehicles can be observed. The shape of the module initially is always rising and reaches the maximum signal value or intermediate maximum. These ascents and descents can look like a parabola or an inverted parabola. Different vehicles have different numbers of intermediate maximum values in different locations. These amplitude values and location indices could be used as feature values. Signature shape approximation and single sensor values' relations to neighboring sensors' relations were investigated. These strategies were evaluated using stem plot diagrams but yielded unsatisfactory results that are not presented in this study. For vehicle re-identification, the following feature vectors were composed which showed less overlapping between same and different vehicle records:

1. Module amplitude maximum location differences;
2. Five maximum corr-coefficients from correlation matrix;
3. Ten corr-coefficients from best diagonal;
4. Five corr-coefficients calculated using vehicle center sensor;
5. Ten corr-coefficients from best diagonal and ten Euclidean distance values.

All feature vectors were calculated with normalized signature modules. Normalization was performed record-wise by taking all signatures of one record, finding the maximum amplitude and dividing all signatures by this value. This is the most reasonable approach because, if normalization were performed for every sensor separately, important information about amplitude differences between neighboring sensors would be lost. Without normalization, absolute amplitude difference information is presented which can better distinguish the same vehicles; however, in this case, the dynamic range can be high and unpredictable. To tackle this problem, it is feasible to perform value constraint in the feature extraction step so the machine learning algorithm obtains values defined in a specific range.

Feature vector 1 consists of 15 differences between module maximum indices. The idea is that same vehicle records will have smaller distance values compared to different vehicles. The vector is calculated using the following formula:

$$mind_k = loc(\max(sig1_k)) - loc(\max(sig2_k) \tag{2}$$

where $mind_k$ is the calculated distance value between indices for sensor k; $sig1$ corresponds to the signature module for record 1 and $sig2$ for record 2; function $\max()$ returns the maximum value of the array; function $loc()$ returns the array index.

Other feature vectors are calculated using correlation and Euclidean distance matrices. The matrices result in a 15×15 array while calculating values between all sensor pairs according to the following formula:

$$\rho_{XY}(k,j) = \frac{cov(X_k, Y_j)}{\sigma_{Xk}\sigma_{Yj}} \tag{3}$$

where $cov(X_k, Y_j)$ is the covariance between two time series—it measures the degree to which two variables change together—and $\sigma_{Xk}\sigma_{Yj}$ are standard deviations of the time series for every sensor $k \in \{1, 2, \ldots 15\}$, $j \in \{1, 2, \ldots 15\}$. Analogically, the Euclidean distance matrix is calculated using the following formula:

$$d_{XY}(k,j) = \sqrt{(X_k - Y_j)^2} \tag{4}$$

where d is the Euclidean distance value, and X_k , Y_j are signature modules for every sensor. An example of calculated matrices for same and different vehicles is shown in Figure 6.

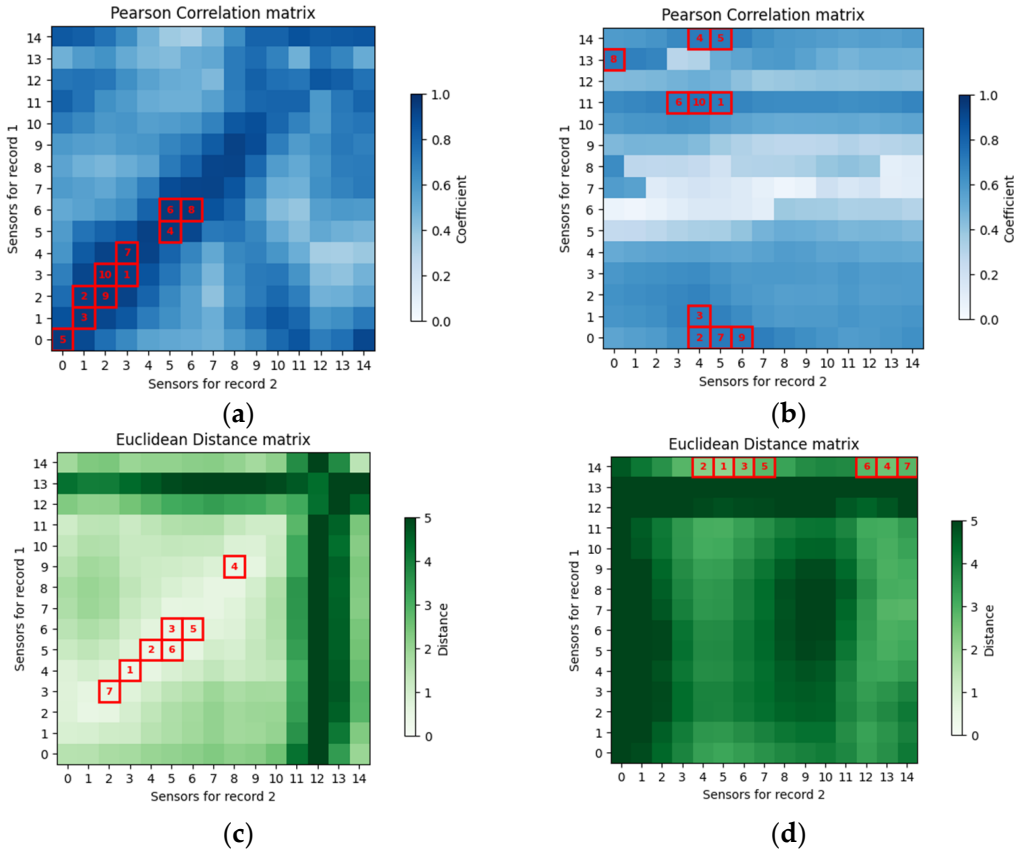


Figure 6. Matrices calculated from two records. Correlation coefficient matrix for same vehicle records (**a**) and different vehicle records (**b**). The maximum 10 values are marked in red boxes. Euclidean distance for same vehicle records (**c**), Euclidean distance for different vehicle records (**d**). The minimum 10 values are marked in red boxes.

The assumption is made that same vehicle records will have higher correlation coefficients and smaller Euclidean distance values compared to different vehicles' records. Feature vector 2 is based on this assumption and uses the five highest correlation matrix values as a feature. Additionally, as can be seen from Figure 6, for same vehicles, it is possible to distinguish a single diagonal for the largest correlation coefficients and smallest Euclidean distances just by looking at the first 10 maximum (minimum for Euclidean distance) values. In an ideal driving trajectory, when the same vehicle ideally passes sensors, this diagonal should appear between the same sensor pairs—(0, 0), (1, 1), (2, 2) and so on. However, depending on the vehicle trajectory, this diagonal could be shifted. For feature vector 3, ten correlation coefficients taken from the diagonal are used, so the diagonal can be offset by -5 to +5 indices from the center position. The algorithm for best diagonal selection (Algorithm 2) is as follows:

The algorithm identifies the most significant diagonal in a given matrix by evaluating the average values of diagonals shifted within a range from -5 to +5. For each possible shift, the resulting diagonal average is calculated. For correlation values, the diagonal with the highest average is selected, and, for Euclidean distance values, the diagonal with the lowest average is selected. The resulting diagonal is an array with 10 values extracted from the matrix. Feature vector 5 is similar; only values are taken from both matrices (correlation and Euclidean).

Feature vector 4 focuses around center sensor and two neighboring sensors from both sides. As explained in our previous work [25], it is complicated to decide which sensor is in the middle of the vehicle. The purpose of focusing on the center sensor is to mitigate the influence of the vehicle wheels on the magnetic signature, as described earlier. If the center sensor is correctly labeled, there is no need to calculate the matrix for all sensor pairs,

but only for corresponding sensors. This simplifies calculations and directly returns five element feature vectors. Records used for analysis were manually inspected, and the center sensor position was labeled. An example of two different vehicles' signatures revolving around the center sensor and without wheel influence is shown in Figure 7. Vehicle 1 is an Opel hatchback, and vehicle 2 is a VW panel van; however, the processed and resampled signature modules visually are similar because of the first rising edge (samples 0–130). A similar signature shape raises the correlation coefficients, which are around 0.94. Usually this high a correlation value suggests that the compared records originate from the same vehicles. Although the signature shape follows a similar pattern, the amplitude differs, and the Euclidean distance matrix can help separate it into different vehicles because maximum values are not concentrated on the diagonal but are chaotic. Based on this observation, feature vector 5 was composed containing ten maximum correlation values and ten minimum Euclidean distance values extracted from the best diagonal.

Algorithm 2. Find Best Diagonal in Matrix

```
procedure FindBestDiagonal(matrix)
        avg arr ← []
          for shift in [−5, 5] do
                diagonal ← GetDiagonal(matrix, shift)
                avg ← Mean(diagonal)
              avg arr.append(avg)
    end for
    ind ← ARGMAX(avg_arr)
    best_shift ← ind – 5
      best diagonal ← GetDiagonal(matrix, best shift)
      return best diagonal
end procedure

procedure GetDiagonal(matrix, shift)
        diagonal ← []
        for i in [0,15 − |shift|] do
              if shift > 0 then
                    diagonal.append(matrix[i, i + shift])
          else
                    diagonal.append(matrix[i + |shift|, i])
          end if
        end for
         return diagonal
end procedure
```

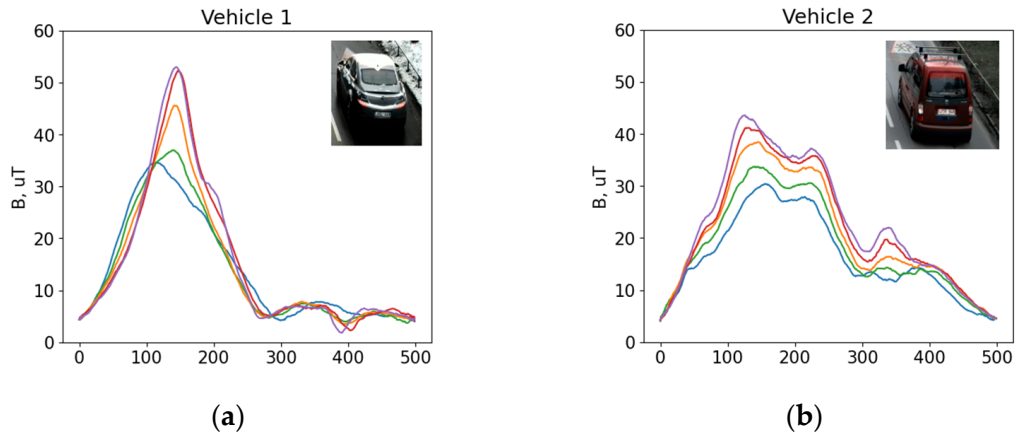


Figure 7. Two different vehicles' signatures around center sensor—vehicle 1 (**a**); vehicle 2 (**b**).

During the experiments, different methods for feature extraction were analyzed, and it was noticeable that the Pearson correlation coefficient and Euclidean distance emerged as the optimal parameters based on their computational efficiency, which allowed for the accurate comparison of records while minimizing processing time. Feature sets are composed of calculated feature vectors for generated comparison pairs.

In the results section, there is a comparison of accuracy, and calculation time is evaluated for the presented feature sets. Accuracy is evaluated by calculating the confusion matrix and accuracy metrics.

3. Results

The used dataset consists of 300 different vehicles passing the sensor in the same direction three times in real traffic conditions. Comparison pairs were generated, as explained earlier, resulting in 900 entries for same vehicle record pairs and 900 entries for different vehicles. Of all the pairs, 70% were used for training and 30% for testing. A feature boxplot of correlation coefficient features using the diagonal with the preselected center sensor (feature vector 4) is presented in Figure 8. For different vehicle records, correlation values had a wide range but were concentrated below 0.95. For the same vehicle records, the median was above 0.97; however, outliers existed. After inspecting the comparison pairs for the same vehicles with low correlation values, it was noted that incorrect center sensor selection generates more outliers in same vehicle comparison pair features. This suggests that, in order to reduce outliers, it is necessary to perform relabeling for the selected center sensor, oruse a magnetic sensor array with smaller spacing or a different similarity evaluation method which is not affected by wheel-distorted signatures.

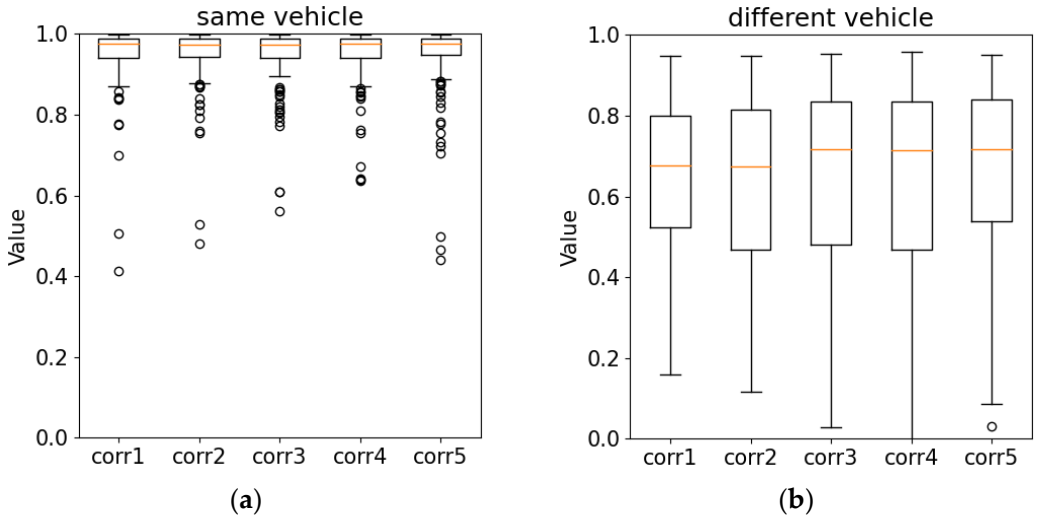


Figure 8. Box diagram showing feature set of five maximum correlations' distribution for same vehicle records (**a**) and different vehicle records (**b**). Round circles indicate outliers, orange line indicate median.

Seven different machine learning models for the classification problem were tested using the earlier-defined input feature sets. Linear regression was the simplest model. After training, model weights were calculated, which were used on input data, and summed together, and the activation function was applied. After training the model with simple feature sets (one value for one sensor pair) for correlation and amplitude differences and sorting calculated weights by importance, no significant sensor was noted—every time, a different sensor had the highest weight.

The nearest neighbor algorithm complexity is determined by value k. A higher value can give more accurate classification; however, the model might become overfitted and poorly generalize new unseen data. Value $k = 3$ was empirically selected after exploring small training/testing subsets. The KNN algorithm computes distances between input

features and trained samples and, based on majority vote, finds the closest neighbor sample. Intuitively, features for the same vehicle are closer together than for different vehicles.

The support vector machine model creates a hyperplane separating input features into two categories for same and different vehicles. Although the model is more aimed at high-dimensional feature space where the feature count exceeds sample count, it worked correctly in the vehicle re-identification task. Different kernel functions can be used to capture complex feature relationships. A linear kernel was selected.

The Naïve Bayes classifier works with probabilities based on Bayes' Theorem. The Gaussian Naïve Bayes classifier was used. The model was fitted by finding the mean and standard deviation of the two vehicle classes' input features.

A decision tree aims to create a tree-like model for decisions based on input features. Values are best split into homogeneous subsets with respect to the target variable. These models are transparent and intuitive, allowing us to understand and explore relationships within the data. By constructing multiple instances of decision trees during training time, a Random Forest classifier is achieved.

The Gradient Boosting Machine (GBM) is a powerful ensemble machine learning technique suitable for classification tasks. The boosting method converts weak learners into strong learners.

Models were compared using accuracy, precision and recall metrics, which are summarized in Table 1. It is notable that different methods yielded varying levels of accuracy across different feature sets. For instance, the Random Forest and Gradient Boosting Machine consistently demonstrated higher accuracy across most features compared to other methods. In this re-identification task, it was important to capture all positive indices (recall)—vehicle records which correspond to the same vehicle. While some methods exhibited high precision but lower recall (e.g., Naïve Bayes), others achieved a balance between precision and recall (e.g., Gradient Boosting Machine). The performance of each method varied across different features, indicating the importance of feature selection.

Table 1. Used features and machine learning method evaluation.

		Feature 1	**Feature 2**	**Feature 3**	**Feature 4**	**Feature 5**
Linear Regression	Accuracy	0.817	0.856	0.908	0.869	0.972
	Precision	0.774	0.776	0.845	0.812	0.983
	Recall	0.894	1.0	1.00	0.961	0.961
KNN	Accuracy	0.844	0.942	0.972	0.903	0.978
	Precision	0.888	0.904	0.947	0.876	0.989
	Recall	0.789	0.989	1.00	0.939	0.967
SVM	Accuracy	0.822	0.872	0.933	0.900	0.953
	Precision	0.776	0.796	0.882	0.860	0.96
	Recall	0.906	1.00	1.00	0.956	0.944
Naïve Bayes	Accuracy	0.819	0.947	0.958	0.869	0.969
	Precision	0.767	0.917	0.923	0.812	0.988
	Recall	0.917	0.983	1.00	0.961	0.950
Decision Tree	Accuracy	0.769	0.933	0.897	0.889	0.972
	Precision	0.765	0.882	0.839	0.857	0.967
	Recall	0.778	1.00	0.983	0.933	0.978
Random Forest	Accuracy	0.897	0.942	0.953	0.911	0.986
	Precision	0.874	0.896	0.914	0.889	0.973
	Recall	0.928	1.00	1.00	0.939	1.00
Gradient Boosting Machine	Accuracy	0.883	0.936	0.928	0.908	0.978
	Precision	0.856	0.887	0.874	0.889	0.967
	Recall	0.922	1.00	1.00	0.933	0.989

Different machine learning models were compared using a Receiver Operating Characteristic (ROC) curve. It illustrates the performance of a binary classification model across

different discrimination thresholds. The true-positive rate is plotted against false-positive rates at various thresholds. ROC curves for different analyzed feature sets are shown in Figure 9. Feature sets that only include correlation coefficients or amplitude evaluations tend to perform worse compared to feature sets that combine both types of features. This highlights the benefit of mixing signature amplitude and shape evaluation. These findings underscore the significance of feature engineering in signature re-identification tasks, emphasizing the critical role played by feature selection and combination strategies.

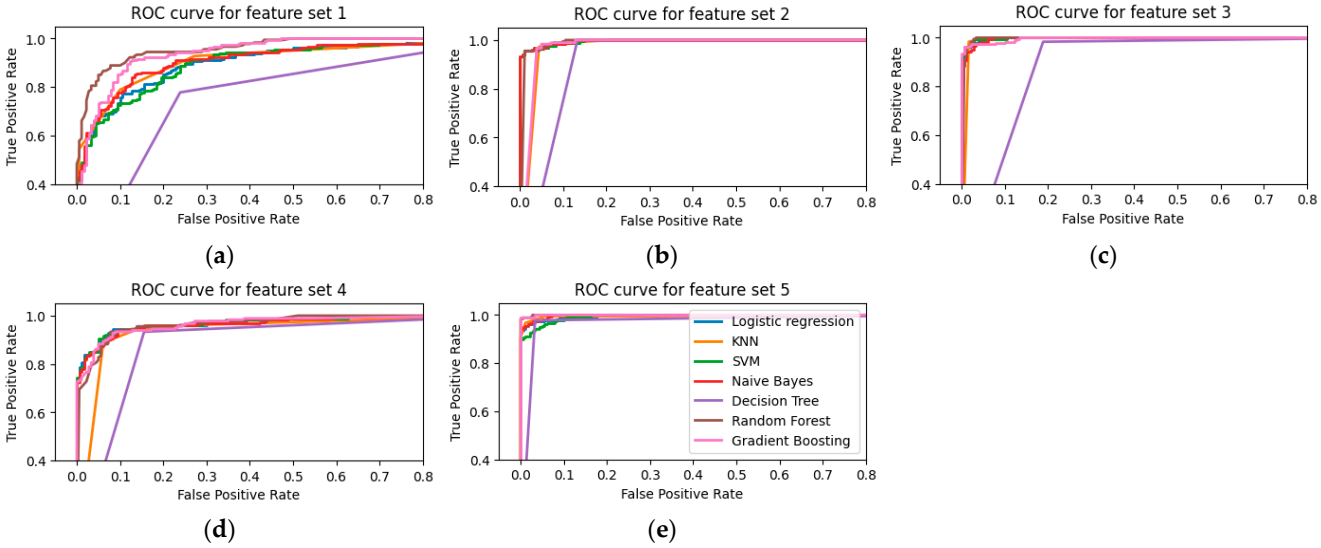


Figure 9. ROC curves for comparing different machine learning methods with different feature sets. (**a**) feature set1, (**b**) feature set2, (**c**) feature set3, (**d**) feature set4, (**e**) feature set5.

4. Discussion

In this paper, a novel methodology for feature extraction and signature comparison using a magnetometer array is presented. Raw registered signals were preprocessed and different feature sets extracted.

Using only correlation features and ignoring amplitudes gives bad re-identification accuracy because different vehicle signatures can have a similar shape. Alternatively, when using only amplitude difference, same vehicle records might appear as different because registered amplitude values fluctuate based on driving pattern.

Combining correlation coefficients with amplitudes for specific sensor pairs gives the best re-identification accuracy, which reaches 98% with the Random Forest algorithm.

Choosing a machine learning algorithm is not critical. With a good feature set, all tested algorithms perform well, with accuracy reaching 97%, while, with a bad feature set, the accuracy gap is more visible—achieving accuracy values of 76–93%. The best accuracy is reached with the Random Forest algorithm and KNN. In all cases, the traditional decision tree's performance is the worst.

The proposed method does not require the intensive computation associated with deep learning models, making it more suitable for real-time applications and deployment in resource-constrained environments. Unlike deep-learning-based methods, our approach ensures anonymity by not relying on image or video data, which often contain personally identifiable information.

5. Conclusions

Our findings can be summarized in the following conclusions:

1. Using only correlation coefficients or amplitude features for specific sensor pairs, the re-identification accuracy is about 93%.
2. Feature selection has a higher impact than the machine learning method used.

3. Same vehicle identification accuracy using correlation coefficients combined with Euclidean distances can reach 97%.

Author Contributions: Conceptualization: D.N., J.B. and D.A.; Methodology: V.M., A.V. and M.Z.; Software: J.B. and M.C.; Validation: D.A., A.V., M.Z., M.P., J.K., Z.L. and M.F.; Visualization: Z.L., J.B., M.F. and M.C.; Investigation: V.M., D.N., J.B., A.V., J.K., M.P. and M.Z.; Resources: D.N., M.Z., M.P., M.F. and M.C.; Data Curation: D.N., V.M., Z.L. and J.K.; Writing—Original Draft Preparation: D.N., J.B., D.A. and M.C.; Writing—Review and Editing: J.B., D.N., D.A. and A.V.; Supervision: D.N., V.M. and D.A.; Project Administration: D.A. and M.Z.; Funding Acquisition: D.A., A.V. and M.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research has received funding from the Research Council of Lithuania (LMTLT), agreement no. S-A-UEI-23-1 (22 December 2023).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Bikku, T.; Narayana, V.; Arepalli, G.; Khadherbhi, S. Sensors Systems for Traffic Congestion Reduction Methodologies. In Proceedings of the 2019 Third International conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), Palladam, India, 12–14 December 2019; pp. 452–455. [CrossRef]
2. Spandonidis, C.; Giannopoulos, F.; Elias, S.; Reppas, D.; Theodoropoulos, P. Development of a MEMS-Based IoV System for Augmenting Road Traffic Survey. *IEEE Trans. Instrum. Meas.* **2022**, *71*, 1–8. [CrossRef]
3. Yue, W.; Li, C.; Mao, G.; Cheng, N.; Zhou, D. Evolution of road traffic congestion control: A survey from erspective of sensing, communication, and computation. *China Commun.* **2021**, *18*, 151–177. [CrossRef]
4. Komolovaite, D.; Krisciunas, A.; Lagzdinyte-Budnike, I.; Budnikas, A.; Rentelis, D. Vehicle Make Detection Using the Transfer Learning Approach. *Elektron. Elektrotechnika* **2022**, *28*, 55–64. [CrossRef]
5. Surgailis, T.; Valinevicius, A.; Markevicius, V.; Navikas, D.; Andriukaitis, D. Avoiding Forward Car Collision using Stereo Vision System. *Elektron. Elektrotechnika* **2012**, *18*, 37–40. [CrossRef]
6. Hodoň, M.; Karpis, O.; Sevcík, P.; Kociánová, A. Which Digital-Output MEMS Magnetometer Meets the Requirements of Modern Road Traffic Survey? *Sensors* **2021**, *21*, 266. [CrossRef] [PubMed]
7. Alsayfi, M.S.; Dahab, M.Y.; Eassa, F.E.; Salama, R.; Haridii, S.; Al-Ghamdi, A.S. Big Data in Vehicular Cloud Computing: Review, Taxonomy, and Security Challenges. *Elektron. Elektrotechnika* **2022**, *28*, 59–71. [CrossRef]
8. Sanchez, R.O.; Christopher, F.; Rajagopal, R.; Varaiya, P. Arterial travel time estimation based on vehicle re-identification using magnetic sensors: Performance analysis. In Proceedings of the Conference Record-IEEE Conference on Intelligent Transportation Systems, Washington, DC, USA, 5–7 October 2011; Volume 10, pp. 997–1002. [CrossRef]
9. Balid, W.; Tafish, H.; Refai, H. Versatile real-time traffic monitoring system using wireless smart sensors networks. In Proceedings of the 2016 IEEE Wireless Communications and Networking Conference, Doha, Qatar, 3–6 April 2016; Volume 201, pp. 1–6. [CrossRef]
10. Miklusis, D.; Markevicius, V.; Navikas, D.; Cepenas, M.; Valinevicius, A.; Zilys, M.; Cuinas, I.; Klimenta, D.; Andriukaitis, D. Research of Distorted Vehicle Magnetic Signatures Recognitions, for Length Estimation in Real Traffic Conditions. *Sensors* **2021**, *21*, 7872. [CrossRef] [PubMed]
11. Chen, X.; Kong, X.; Xu, M.; Sandrasegaran, K.; Zheng, J. Road Vehicle Detection and Classification Using Magnetic Field Measurement. *IEEE Access* **2019**, *7*, 52622–52633. [CrossRef]
12. Arab, M.; Nadeem, T. MagnoPark-Locating On-Street Parking Spaces Using Magnetometer-Based Pedestrians' Smartphones. In Proceedings of the 2017 14th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON), San Diego, CA, USA, 12–14 June 2017; pp. 1–9. [CrossRef]
13. Bakirci, M. Simulation of Autonomous Driving for a Line-Following Robotic Vehicle: Determining the Optimal Manoeuvring Mode. *Elektron. Elektrotechnika* **2023**, *29*, 4–11. [CrossRef]
14. Zhang, Z.; He, X.; Yuan, H. An Antiinterference Traffic Speed Estimation System With Wireless Magnetic Sensor Network. *IEEE Trans. Ind. Inform.* **2020**, *16*, 2458–2468. [CrossRef]
15. Ou, J.; Qiu, J.; Dong, X.; Wang, Z.; Du, J.; Chang, Q. Research on the Moving Magnetic Object Recognition Method Based on Magnetic Signature Waveform. *IEEE Trans. Magn.* **2022**, *58*, 1–8. [CrossRef]
16. Wang, Z.; Zheng, E.; Liu, J.; Guo, T. Adaptive Orthogonal Basis Function Detection Method for Unknown Magnetic Target Motion State. *Appl. Sci.* **2024**, *14*, 902. [CrossRef]
17. Prateek, G.V.; Nijil, K.; Hari, K.V.S. Classification of Vehicles Using Magnetic Field Angle Model. In Proceedings of the 2013 4th International Conference on Intelligent Systems, Modelling and Simulation, Bangkok, Thailand, 29–31 January 2013; pp. 214–219. [CrossRef]

18. Tawfik, A.Y.; Abdulhai, B.; Peng, A.; Tabib, S.M. Using Decision Trees to Improve the Accuracy of Vehicle Signature Reidentification. *Transp. Res. Rec.* **2004**, *1886*, 24–33. [CrossRef]
19. Dong, H.; Wang, X.; Zhang, C.; He, R.; Jia, L.; Qin, Y. Improved Robust Vehicle Detection and Identification Based on Single Magnetic Sensor. *IEEE Access* **2018**, *6*, 5247–5255. [CrossRef]
20. Sarcevic, P.; Pletl, S.; Odry, A. Real-Time Vehicle Classification System Using a Single Magnetometer. *Sensors* **2022**, *22*, 9299. [CrossRef]
21. Xu, C.; Wang, Y.; Bao, X.; Li, F. Vehicle Classification Using an Imbalanced Dataset Based on a Single Magnetic Sensor. *Sensors* **2018**, *18*, 1690. [CrossRef] [PubMed]
22. Balid, W.; Tafish, H.; Refai, H.H. Intelligent Vehicle Counting and Classification Sensor for Real-Time Traffic Surveillance. *IEEE Trans. Intell. Transp. Syst.* **2018**, *19*, 1784–1794. [CrossRef]
23. Zhou, X. Vehicle location estimation based on a magnetic sensor array. In Proceedings of the 2013 IEEE Sensors Applications Symposium Proceedings, Galveston, TX, USA, 19–21 February 2013; pp. 80–83. [CrossRef]
24. Kwong, K.; Kavaler, R.; Rajagopal, R.; Varaiya, P. Arterial travel time estimation based on vehicle re-identification using wireless magnetic sensors. *Transp. Res. Part C Emerg. Technol.* **2009**, *17*, 586–606. [CrossRef]
25. Pitton, A.C.; Vassilev, A.; Charbonnier, S. Vehicle Re-Identification with Several Magnetic Sensors. In *Advanced Microsystems for Automotive Applications 2012: Smart Systems for Safe, Sustainable and Networked Vehicles*; Springer: Berlin/Heidelberg, Germany, 2012; pp. 281–290.
26. Sanchez, R.O.; Flores, C.; Horowitz, R.; Rajagopal, R.; Varaiya, P. Vehicle re-identification using wireless magnetic sensors: Algorithm revision, modifications and performance analysis. In Proceedings of the 2011 IEEE International Conference on Vehicular Electronics and Safety, Beijing, China, 10–12 July 2011; pp. 226–231. [CrossRef]
27. Amodio, A.; Ermidoro, M.; Savaresi, S.M.; Previdi, F. Automatic Vehicle Model Recognition and Lateral Position Estimation Based on Magnetic Sensors. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 2775–2785. [CrossRef]
28. Čulík, K.; Štefancová, V.; Hrudkay, K. Application of Wireless Magnetic Sensors in the Urban Environment and Their Accuracy Verification. *Sensors* **2023**, *23*, 5740. [CrossRef]
29. Balamutas, J.; Navikas, D.; Markevicius, V.; Cepenas, M.; Valinevicius, A.; Zilys, M.; Frivaldsky, M.; Li, Z.; Andriukaitis, D. Passing Vehicle Road Occupancy Detection Using the Magnetic Sensor Array. *IEEE Access* **2023**, *11*, 50984–50993. [CrossRef]

Vehicle Detection and Classification for Low-Speed Congested Traffic With Anisotropic Magnetoresistive Sensor

Bo Yang and Yiqun Lei

***Abstract*— A vehicle detection and classification system has been developed based on a low-cost triaxial anisotropic magnetoresistive sensor. Considering the characteristics of vehicle magnetic detection signals, especially the signals for low-speed congested traffic in large cities, a novel fixed threshold state machine algorithm based on signal variance is proposed to detect vehicles within a single lane and segment the vehicle signals effectively according to the time information of vehicles entering and leaving the sensor monitoring area. In our experiments, five signal features are extracted, including the signal duration, signal energy, average energy of the signal, ratio of positive and negative energy of *x*-axis signal, and ratio of positive and negative energy of *y*-axis signal. Furthermore, the detected vehicles are classified into motorcycles, two-box cars, saloon cars, buses, and Sport Utility Vehicle commercial vehicles based on a classification tree model. The experimental results have shown that the detection accuracy of the proposed algorithm can reach up to 99.05% and the average classification accuracy is 93.66%, which verify the effectiveness of our algorithm for low-speed congested traffic.**

***Index Terms*— Anisotropic magnetoresistive sensor (AMR), low-speed congested traffic, vehicle detection, vehicle classification.**

I. Introduction

VEHICLE detection and classification technologies play an important role in the Intelligent Transportation Systems (ITS). Most conventional traffic surveillance systems use inductive loop detectors and video cameras [1], [2]. For maximizing the benefits from these detection technologies, there must be a large scale deployment of traffic surveillance on all the major streets and freeways. As intrusive sensors, however, the installation and maintenance of the inductive loop detectors lead to a relative high cost. The video camera is also expensive for a large-scale deployment, and they are greatly affected by the environmental factors, such as shadow, snow and rain [3].

Magnetoresistive sensor has recently been utilized for vehicle surveillance owing to its advantages of easy installation, low cost, small size, strong anti-jamming capability [4], [5], etc. Scholars worldwide have investigated the application of magnetic sensor and wireless sensor network for vehicle detection and classification. They offer a very attractive alternative to inductive loops and video cameras. Currently, the most commonly used vehicle detection algorithms based on magnetic sensor are fixed threshold algorithm, the state machine algorithm and adaptive threshold algorithm [6]. Sing Y. C. et al. [7] proposed an efficient vehicle detection algorithm, Adaptive Threshold Detection Algorithm (ATDA), based on 3-axis Anisotropic Magnetoresistive (AMR) sensor with high precision of 99%. However the classification scheme was not good enough and overall detection rate was below 60%. Wireless sensor network based on AMR can detect and track moving vehicles to obtain more traffic information [8], [9]. Many classification algorithms using AMR sensors were proposed, including the Hill-Patterns algorithm, the Average-Bar algorithm, the BP Neural Network algorithm and the Support Vector Machine, etc. Most of them have conducted experimental tests and comprehensive study in normal driving situation and achieved good results. For example, a vehicle detection and classification system was proposed based on a wireless sensor network using the adaptive threshold algorithm and intelligent neuron classifier giving the promising classification rate at 90% [10].

However, little attention has been given to the vehicle detection and classification for low-speed congested traffic, which is prevalent in big cities. When vehicles driving at a low speed on a congested road, there will be serious waveform distortion in sensor signal output, and because vehicles are close to each other, the front and rear of the vehicle signals will interfere mutually. All these factors will have a negative impact on the vehicle detection and classification. Traffic surveillance for low speed congested vehicles will be a new direction for future research.

This paper focuses on the investigation of anisotropic magnetic sensor to detect the vehicle in a slow-moving and congested traffic zone. A simple and effective vehicle detection and classification system is developed based on low-cost three-axis AMR sensors. The characteristics of the vehicle magnetic detection signals are particularly analyzed and processed. A novel fixed threshold state machine algorithm is proposed based on variance weighting to detect vehicles within a single lane. The signal features are extracted accurately and the vehicles are classified by using a classification tree model.

Manuscript received August 16, 2014; revised September 9, 2014; accepted September 12, 2014. Date of publication September 18, 2014; date of current version November 26, 2014. This work was supported by the Aviation Science Foundation of China under Grant 2011ZD51051. The associate editor coordinating the review of this paper and approving it for publication was Dr. Stoyan Nihtianov.

The authors are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: boyang@buaa.edu.cn; leiyiqun@foxmail.com).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/JSEN.2014.2359014

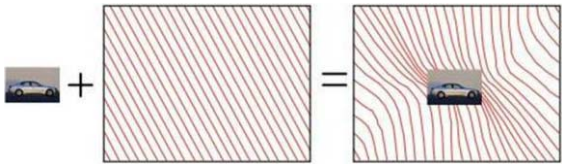


Fig. 1. The disturbance of of Earth's magnetic flux lines by a moving vehicle.

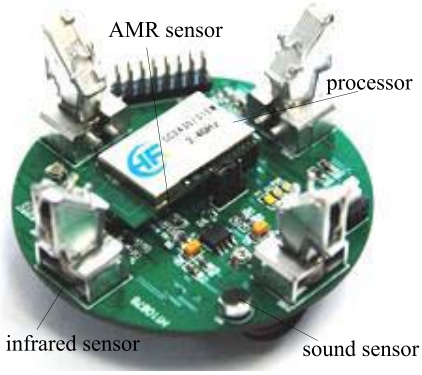


Fig. 2. Sensor node.

The experimental result has shown that the proposed algorithms can effectively detect the vehicles in the slow-moving and congested traffic zone and the accuracy of classification is improved correspondingly.

This paper is organized as follows. Section II introduces the foundation of vehicle detection using AMR sensor. Section III describes the principle of our novel signal variance based fixed threshold state machine algorithm. The algorithm for vehicle classification is given in Section IV. Section V presents the experimental results of vehicle detection and vehicle classification for low-speed congested traffic. Finally, Section VI contains some conclusions.

II. Foundation of Vehicle Detection Using AMR Sensor

The geomagnetic field provides a uniform magnetic field over a wide area and the magnetic sensor can detect the disturbances of the Earth's field caused by a large ferrous object like a vehicle. The disturbance depends on the ferrous material, its size and orientation (Figure 1).

The magnetic sensor is made of a nickel-iron (Permalloy) thin film deposited on a silicon wafer and patterned as a resistive strip. Typically, four resistors are connected in a Wheatstone bridge configuration to generate a differential output voltage [11].

Fig. 2 shows the sensor node we designed, which is composed of a CC2430 processor and an AMR sensor, etc. The AMR sensor we used is Honeywell triaxial HMC5883L, a fully digital sensor with built-in multiplexed ADC. The sensor has 12 bits ADC coupled with low noise AMR sensors and achieves 5 milli-gauss resolution in ±8 Gauss Fields. There are also 4 infrared sensors and a sound sensor in the sensor node to complete other measurement tasks. Only the AMR sensor is used in our experiment for the vehicle detection and classification with low power consumption. The average

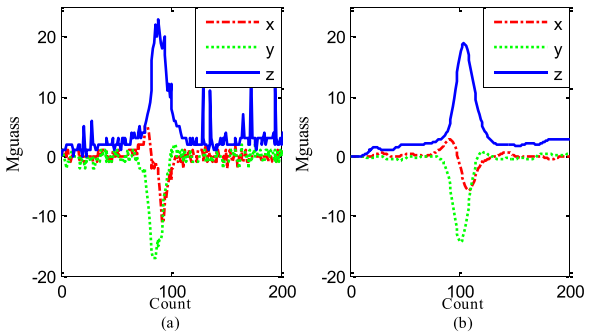


Fig. 3. Magnetic signals of a two-box vehicle: (a) original signal and (b) signal after filtering.

current drain of the sensor node is only about 100 μA at work. The AMR sensor senses disturbance of the Earth's magnetic field caused by a moving vehicle. This sensor can achieve magnetic field measurement for X, Y, and Z axis simultaneously with high accuracy and efficiency. Choosing a triaxial AMR sensor can obtain comprehensive information of magnetic field with high accuracy.

III. Vehicle Detection

When a vehicle passes over an AMR sensor, the sensor will detect the dipole moments of various parts of the vehicle, and the related field variation will reveal a very detailed magnetic signature of the vehicle. Figures 3(a) shows a 3-axis magnetometer output for a two-box vehicle passing directly over it. The three curves represent the X, Y, and Z-axis of the variation in the geomagnetic field respectively for the moving vehicle.

In order to facilitate the analysis for signals of the vehicle, a series of equally spaced sampling points are used to represent the three axis magnetic signals of vehicle. The sample rate of sensor is 75 Sa/s, so the horizontal axis is in Counts, and the vertical axis is in milli-gauss, which we mark as Mgauss in the following figures.

$$\begin{cases} X &= \{X(1), X(2), X(3), \ldots, X(n)\} \\ Y &= \{Y(1), Y(2), Y(3), \ldots, Y(n)\} \\ Z &= \{Z(1), Z(2), Z(3), \ldots, Z(n)\} \end{cases} \quad (1)$$

To get a smoother sensor signal, we use a digital filtering algorithm to eliminate noise. The filter combines fast Fourier transform and median filter, which can not only filter out background random noise well but also be able to suppress the big impulse noise. The filtering result are listed as follows and shown in Figure 3(b).

$$\begin{cases} filtered\_X(k) &= fftfilt(hn, medfilt1(X(k), N)) \\ filtered\_Y(k) &= fftfilt(hn, medfilt1(Y(k), N)) \\ filtered\_Z(k) &= fftfilt(hn, medfilt1(Z(k), N)) \end{cases} \quad (2)$$

where hn is the ideal low-pass filter window function and N is median filtering parameter.

The function *medfilt1* implements one-dimensional median filtering and the function *fftfil*t is a frequency domain filtering technique works on FIR filter using the efficient

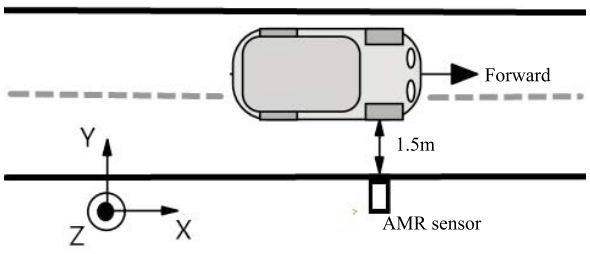


Fig. 4. Sensor installation diagram.

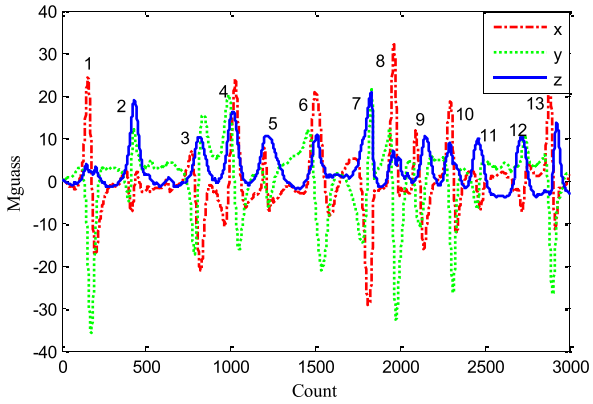


Fig. 5. Original magnetic signals of vehicles.

FFT-based method of overlap-add. {*filtered_X(k)*, *filtered_Y(k)*, *filtered_Z(k)*} are 3-axis vehicle magnetic signals after the filtering processing.

The vehicle density in most developed cities is centralized and huge, which cause a small amount of amplitude attenuation of output magnetic signal as the vehicles are driving at slow speed. Signals of adjacent vehicles will interfere with each other as two vehicles are crawling bumper to bumper. Furthermore, the output magnetic signal of sensor will drift when environment temperature changes [12]. All these factors will have a serious impact on the detection and segmentation of the vehicle signals.

An experiment is performed to analyze the signal characteristics of the vehicles. An AMR sensor node is placed on the side of lane and is about 1.5 m away from the vehicle. The system layout is illustrated in Fig. 4. X-axis is the direction parallel to the moving vehicle. Y-axis is the direction perpendicular to the direction of moving vehicle. Z-axis is the direction perpendicular to the ground.

Fig. 5 shows the sensor signal of 13 vehicles driving at about 10km/h. There is a slight drift in background signal over time.

We define $F(k)$ as the magnetic field intensity of measured signal.

$$F(k) = \sqrt{X(k)^2 + Y(k)^2 + Z(k)^2} \tag{3}$$

A new variance-based multi-state machine adaptive threshold detection algorithm was proposed in Ref. [10]. The algorithm calculated the real-time data fluctuation using historical data variance and then predicted average signal threshold in future. This algorithm is an efficient vehicle detection algorithm with high precision of 97% when the vehicle is driving normally. But for the vehicles crawling bumper to bumper with low speed in congested urban traffic, the detection result is not satisfactory.

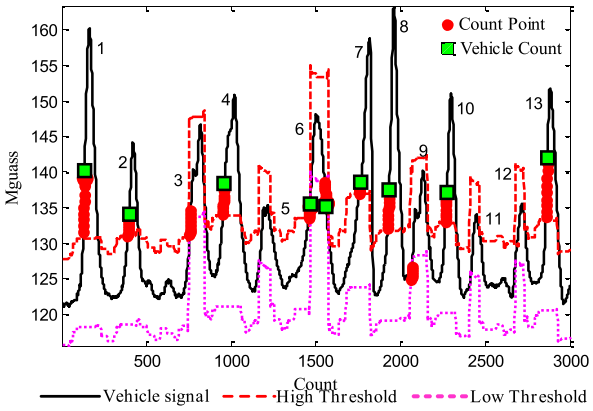


Fig. 6. Adaptive threshold detection algorithm [10].

Fig. 6 shows that the geomagnetic disturbances will be weakened when the vehicle is traveling at a low speed, i.e. the 5th, 11th, 12th vehicles in the figure. On the one hand, the interval between two vehicles is so short that the variance of the history signal does not reduce and the adaptive threshold increases, on the other hand, the amplitude of signal is small due to low-speed. Therefore, the algorithm cannot detect all these vehicles correctly. In addition, the adaptive threshold algorithm is not conducive to accurately determine the time information of vehicles entering and leaving the monitored area, which will increase the difficulty of effectively separating the vehicle signal.

In order to detect the low-speed congested vehicles, this paper presents an effective fixed threshold state machine algorithm based on the signal variance weighting. Firstly, we calculate the variance of magnetic field intensity signal, and then multiply it by the original signal value as a weighting factor. This operation can retain the characteristics of the magnetic field intensity signal; meanwhile it can also suppress the signal drift by the property of signal variance. The algorithm is given by

$$Processed\_F(k) = filtered\_F(k) \times variance\_F(k) \tag{4}$$

where $filtered\_F(k)$ is the filtered magnetic field intensity signal and $variance\_F(k)$ is the variance of the magnetic field intensity signal.

$$filtered\_F(k) = fftfilt(hn, medfilt1(F(k), N)) \tag{5}$$

In this paper, the variance of the signal value is calculated by the forward and backward 15 points. In the absence of a passing vehicle, the variance of signal will still be very low. The vehicle signals become more significant and easier to identify because of the obvious increase of the signal variance when the vehicle is passing.

When the vehicle is driving at a low speed and the vehicle-to-vehicle distance is relative small, the reference value will generate a large deviation due to the signal interference between certain vehicles. To improve the stability of the proposed algorithm, a state machine is introduced to judge

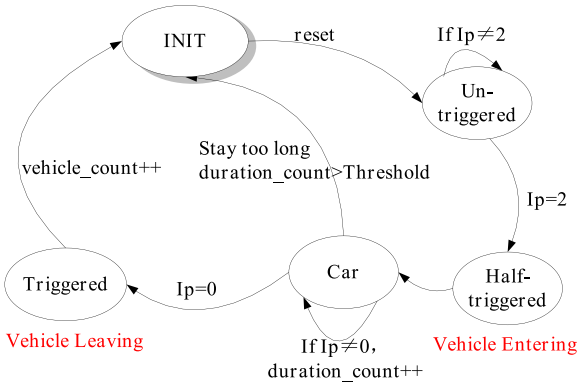


Fig. 7. Scheme of state machine algorithm.

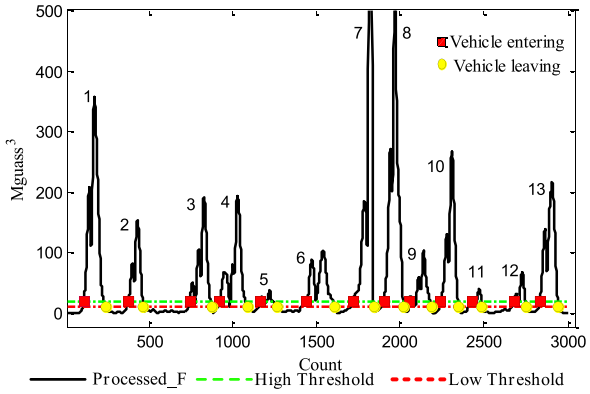


Fig. 8. The effect of vehicle identification.

the vehicles using two thresholds {*H-value, L-value*}which divided the input signal to three cases.

The vehicle detection algorithm uses the deviation of processed_F(k) from a baseline to drive a state machine, which makes the vehicle counting decisions (Fig. 7). The input of the decision state machine is defined as

$$Ip = \begin{cases} 0, & if \quad processed\_F(k) \leqslant \text{L-value} \\ 2, & if \quad processed\_F(k) \geqslant \text{H-value} \\ 1, & otherwise \end{cases} \tag{6}$$

where {*H-value, L-value*} are the high threshold and the low threshold.

At the initial state, reset all parameter to the initial value and the state machine to an Un-triggered state. *Duration_count* denotes the duration of vehicle signal and *vehicle_count* is the number of vehicles. If Ip changes to 2 (*processed_F(k)* ≥ *H-value*), the state machine will turn to a Half-triggered state and we consider the vehicle may enter the sensor monitoring area.

Normally, processed_F(k) increases first and then decreases. If it has exceeded the L-value for a long time, the algorithm will return to INIT. Otherwise, Ip will turn to 0, and the state machine will turn to a Triggered state. Then, we consider the vehicle is leaving the sensor monitoring area. Let the vehicle_count plus 1 and return to INIT waiting for another vehicle arriving.

The vehicle detection result is shown in Fig. 8. The variance of the signal can effectively prevent the reference value from drifting caused by the mutual interference between two

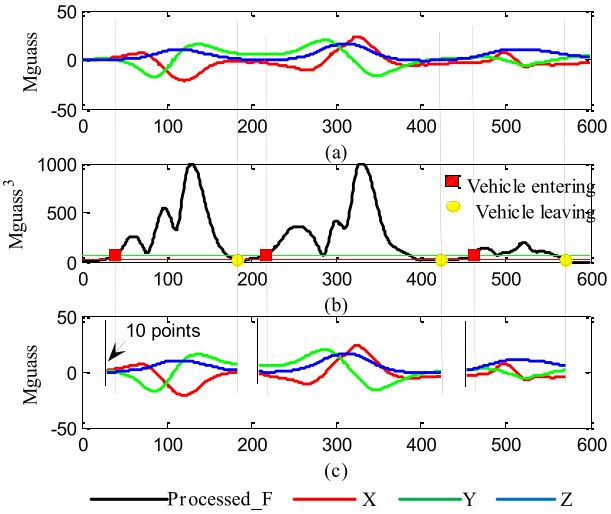


Fig. 9. Vehicle signal segmentation. (a) Original vehicle signal. (b) Vehicle identification. (c) Vehicle signal segmentation.

adjacent vehicles. The processed signal can be distinguished more clearly even though the vehicles are driving bumper to bumper. It can be concluded that our algorithm has higher detection rate both for low speed and congested vehicles.

IV. Vehicle Classification

We perform a simple classification based on a hierarchical tree methodology as in Ref. [13], which uses the above vehicle information to classify the vehicles into five types: Motorcycle, Two-box, Saloon, Bus and Sport Utility Vehicle (SUV).

A. Signal Segmentation

The first step of vehicle classification is to extract the vehicle signal from sensor signal. The selection of entering and leaving points of vehicles will have a great influence on vehicle classification. We assume the vehicle entering if the signal is just above H-value, and the vehicle leaving if lower than L-value in the above-mentioned detection algorithm. Then we segment the vehicle signals effectively according to the time information of vehicles entering and leaving the sensor monitoring area. There is slight loss about signal interception portions due to the high threshold, so that we compensate the time when the vehicle enters the sensor detection zone. In this paper, we extend 10 forward measuring points of the signal expansion to solve this problem (Figure 9).

Fig. 10 shows a schematic view of signal segmentation for above 13 vehicles in Section III. This result illustrates that vehicle X-axis signal and Y-axis signal have significant characteristics compared with Z-axis signal. The Z-axis signal is a single peak wave because the geomagnetic disturbances are consistent in this direction when vehicle passes over the AMR sensor.

B. Feature Extraction

In order to identify the vehicle type, some quantitative indices are required to extract the features of the magnetic signal. These indices should have high correlations with the signal waveform and can be easily computed. As above

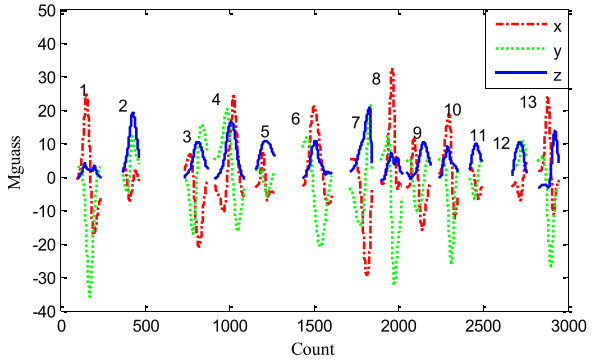


Fig. 10. The effect of vehicle signal segmentation.

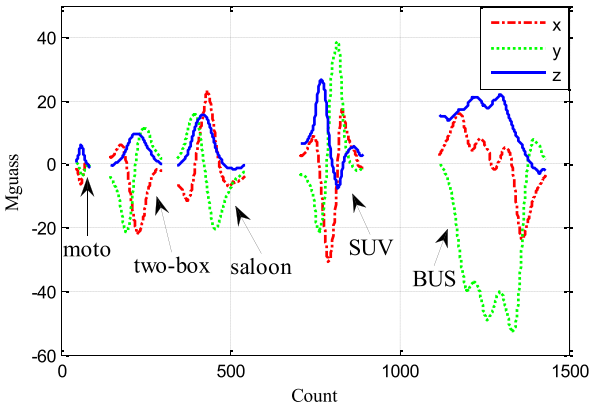


Fig. 11. Magnetic signals generated by different types of vehicles.

mentioned, the Z-axis signal characteristic of the vehicle has less influence in classification process. Thus, only X-axis and Y-axis vehicle signals are analyzed in the following process of feature extraction.

There are five types of vehicles to be analyzed. Each type of vehicle signal has its own unique characteristics due to the different structure of vehicle. In this paper, we will extract features through the time-domain waveform structure after data analysis. It should be noted that, the same type of vehicle signal waveform may be slightly different due to the impact of vehicle speed, vehicle equipment and other factors. But this does not have much impact for identifying the characteristics of vehicle signal. These five types vehicles signal characteristics are shown in Fig. 11.

We define that L is the vehicle signal duration; E and EV are the signal energy and the average energy of the signal.

$$E = \sum (X^2(k) + Y^2(k) + Z^2(k)) \tag{7}$$

$$EV = \frac{E}{L} \tag{8}$$

VX is the ratio of positive and negative energy of X-axis signal; VY is the ratio of positive and negative energy of Y-axis signal.

$$VX = \frac{\sum\limits_{X(k)>0} X^2(k)}{\sum\limits_{X(k)<0} X^2(k)} \quad VY = \frac{\sum\limits_{Y(k)>0} Y^2(k)}{\sum\limits_{Y(k)<0} Y^2(k)} \tag{9}$$

TABLE I
DIFFERENT TYPES OF VEHICLES SIGNAL CHARACTERISTICS

	Moto	*Two-box*	*Saloon*	*SUV*	*Bus*
					
L	53~102	65~129	73~139	109~154	243~445
E	125~331	1879~3421	3988~5534	>94650	---
EV	1.6~3.2	17.5~38.9	42.1~70.1	>720	---
VX	---	0.04~0.57	1.07~20.9	---	---
VY	---	0.23~0.65	0.78~3.21	---	---

A total of 100 vehicle signals (20 vehicles for each type) were collected for analysis of their characteristics under the same test condition. The speed of vehicles in our experiment is about 10km/h $\sim$ 30km/h. The analysis result is shown in Table I.

The number of vehicle sampling points is defined as the vehicle signal duration. The bus signals are sampled at about 200 to 400 points with a sampling frequency of 75 Hz, while sampling points of other types are generally between 50 and 100 and can even exceed 150 in particularly low speeds. The average speeds of vehicle we concerned in experiments are in a limited range (10 $\sim$ 15km/h for the low-speed congested traffic and 30 $\sim$ 40km/h for the normal traffic). Within this speed range, even if a bus drives faster and other vehicles drive slower, the signal duration of the bus is still significantly longer than others. Therefore, we can distinguish the bus from the other smaller cars by using the signal duration L.

The volume of vehicles is reflected in vehicle signal energy so that we can distinguish the motorcycles and SUV commercial vehicle according to the signal energy E and EV. In order to increase the stability of the proposed algorithm, the average energy of vehicle signal EV is extracted as an auxiliary characteristic value for vehicle classification.

Two-box car and saloon car are relatively similar in terms of signal length and energy, but the structure of the vehicle signal has more obvious characteristics. The value of X-axis signal of two-box car is essentially negative while value of X-axis signal of saloon car is positive. The ratio of positive and negative energy of X-axis signal is extracted as a characteristic for identification. In order to increase the stability of the proposed algorithm, the ratio of positive and negative energy of Y-axis signal is also extracted as an auxiliary characteristic to distinguish two-box car from saloon car.

C. Vehicle Classification Algorithm

The above features are used here to construct the hierarchical classification tree for classifying vehicles. The specific classification process is based on hierarchical tree shown in Fig. 12.

Vehicle Classification Algorithm

①if($L \geq L_0$); {bus}count, else
② if($E \leq E_a$&&$EV \leq EV_a$); {moto}count, else
③ if($E \geq E_b$&&$EV \geq EV_b$); {suv}count, else
④ if($VX \leq VX_0$&&$VY \leq VY_0$); {two-box}count else
⑤ {saloon}count, return ① cycle until the end of all vehicles judgment

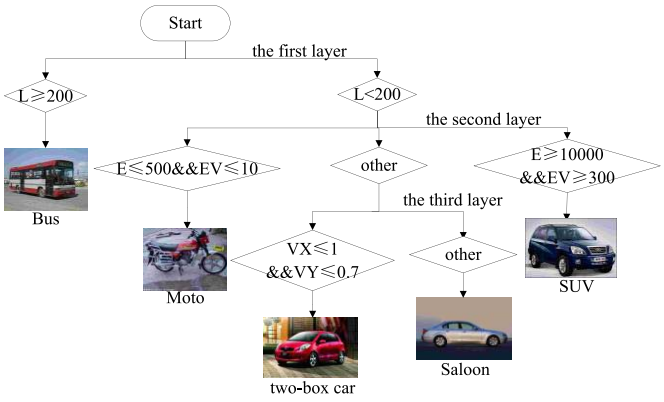


Fig. 12. Schematic of classification algorithm.

The classification tree model has three layers: the first layer of the classification criteria is the vehicle signal duration L; the second layer is the vehicle signal energy, including energy E and average energy EV; the third layer is the ratio of positive and negative energy of X-axis signal VX and Y-axis signal VY. The vehicle will be considered as a saloon car only when its characteristics do not meet any of the given conditions.

$\{L_0,\ E_a,\ EV_a,\ E_b,\ EV_b,\ VX_0,\ VY_0\}$ is a set of fixed threshold algorithm parameters. The thresholds are chosen as follows: $L_0 = 200$, Ea = 500, EVa = 10, Eb = 10000, EVb = 300, $VX_0 = 1$, $VY_0 = 0.7$. Various types of vehicles can be distinguished effectively using the vehicle classification algorithm. The advantage of this algorithm has small amount of calculation and high classification accuracy.

V. Experimental Results and Analysis

In this paper, the congested traffic condition is our main concern. The experimental test is performed for normal traffic and low speed congested traffic respectively at temperature about 21 ∼ 25 °C. The vehicle speed we tested is about 30 ∼ 40km/h for normal traffic, and 10 ∼ 15 km/h for low-speed congested traffic respectively. The sensor node is placed 7 cm above the ground and about 1.5 m away from the vehicle (Figure 13).

We focus on the vehicle detection and classification on a single lane in experiments. The AMR magnetic sensors work well for normal curbside sensing ranges of one to five feet (about 1.5m), while the width of lane is about 3.5m in the urban road of Beijing and the width of the vehicle is more than 2m in normal conditions, so the effect caused by the other lanes could be neglected.

Vehicle detection results are listed in Table II. For normal traffic, there are 147 vehicles being detected and all vehicles are identified correctly. For low-speed congested traffic, there are 105 vehicles being observed. But one of the vehicles is not detected as it approaching another one too close. Two vehicles are misjudged as four due to the low speed. Therefore, the total number of vehicle detection result is 106 and the detection accuracy is still over 99% for low speed congested traffic.

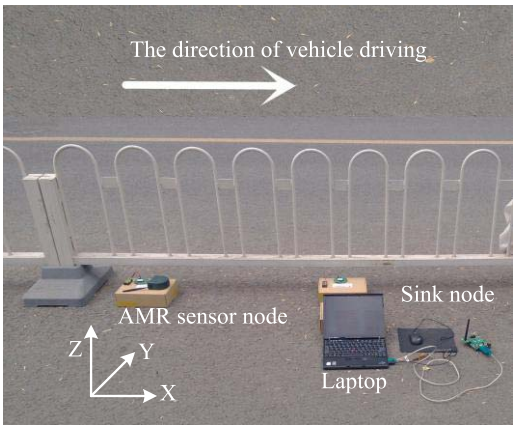


Fig. 13. Experimental scene.

TABLE II
Results of Vehicle Detection

Normal traffic (30 ~40km/h)			*Low-speed congested traffic (10 ~15km/h)*		
Observed Vehicles	Detected Vehicles	Correct%	Observed Vehicles	Detected Vehicles	Correct %
147	147	100	105	106	99.05

TABLE III
Results of Vehicle Classification

Class	*Normal traffic (30 ~40km/h)*			*Low-speed congested traffic (10 ~15km/h)*		
	Test Results	Actual results	Correct %	Test Results	Actual results	Correct %
Motorcycle	20	20	100	-	-	-
Two-box	31	30	96.7	53	49	92.45
Saloon	45	44	97.78	31	32	96.87
SUV	28	30	93.33	22	24	91.67
Bus	23	23	100	-	-	-

Vehicle classification results are listed in Table III.

The vehicles are tested on the main and side roads because the motorcycles and the buses cannot drive in the same lane. Most vehicles on the congested roads are small cars and SUV vehicles. During the rush hours, bus lanes are travel lanes restricted to buses. Therefore, the classification process in low-speed driving is given only for these three types of vehicles.

The classification accuracy is over 90% for low-speed congested traffic. Although it is slightly lower than the accuracy for normal traffic due to the amplitude attenuation of vehicle signal and the interference between adjacent vehicles, it is still good and promising.

VI. Conclusion

This paper presents the vehicle detection and classification methods for low-speed congested traffic with anisotropic

magnetoresistive sensor. A novel fixed threshold state machine algorithm based on signal variance is proposed to detect low-speed congested vehicles within a single lane. Five signal features are extracted to classify the detected vehicles into five types using the model classification algorithm based on hierarchical tree methodology. The experimental results obtained from the low-speed congested traffic have shown that the proposed algorithm has a high detection and classification accuracy.

The average speeds of vehicle we concern in the experiments are in a limited range (10 $\sim$ 15km/h for the low-speed congested traffic and 30 $\sim$ 40km/h for the normal traffic). Once the vehicle speed change exceeds the range, it should be introduced as a correction factor for the signal characteristics, which will be investigated in our future research.

REFERENCES

[1] R. Wang, L. Zhang, K. Xiao, R. Sun, and L. Cui, "EasiSee: Real-time vehicle classification and counting via low-cost collaborative sensing," *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 1, pp. 414–424, Feb. 2014.

[2] Y. Iwasaki, S. Kawata, and T. Nakamiya, "Robust vehicle detection even in poor visibility conditions using infrared thermal images and its application to road traffic flow monitoring," *Meas. Sci. Technol.*, vol. 22, no. 8, p. 085501, Aug. 2011.

[3] E. Sifuentes, O. Casas, and R. Pallas-Areny, "Wireless magnetic sensor node for vehicle detection with optical wake-up," *IEEE Sensors J.*, vol. 11, no. 8, pp. 1669–1676, Aug. 2011.

[4] S. Taghvaeeyan and R. Rajamani, "The development of vehicle position estimation algorithms based on the use of AMR sensors," *IEEE Trans. Intell. Transp. Syst.*, vol. 13, no. 4, pp. 1845–1854, Dec. 2012.

[5] M. H. Kang, B. W. Choi, K. C. Koh, J. H. Lee, and G. T. Park, "Experimental study of a vehicle detector with an AMR sensor," *Sens. Actuators A, Phys.*, vol. 118, no 2, pp. 278–284, Feb. 2005.

[6] P. Ni, L. Le, and W. Yu, "Review of vehicle detection algorithms based on magnetoresistive sensors," *Comput. Eng. Appl.*, vol. 45, no. 19, pp. 245–248, 2009.

[7] S. Y. Cheung, S. Coleri, B. Dundar, S. Ganesh, C.-W. Tan, and P. Varaiya, "Traffic measurement and vehicle classification with single magnetic sensor," *J. Transp. Res. Board*, vol. 1917, no. 1, pp. 173–181, Dec. 2005.

[8] B.-G. Lee and J.-H. Kim, "Algorithm for finding the moving direction of a vehicle using magnetic sensor," in *Proc. IEEE Symp. CICA*, Apr. 2011, pp. 74–79.

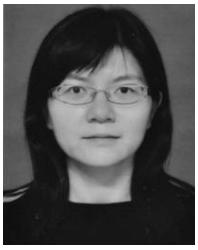
[9] S. Marshall, "Vehicle detection using a magnetic field sensor," *IEEE Trans. Veh. Technol.*, vol. 27, no. 2, pp. 65–68, May 1978.

[10] B. Yang and W. Nin, "Vehicle detection and classification algorithm based on anisotropic magnetoresistive sensor," *Chin. J. Sci. Instrum.*, vol. 34, no. 3, pp. 537–544, Mar. 2013.

[11] X. S. Li, R. Q. Kang, Y. X. Liu, Z. H. Wang, and X. Y. Shu, "High accuracy AMR magnetometer and its application to vehicle detection," *Adv. Mater. Res.*, vols. 443–444, pp. 150–155, Jan. 2012.

[12] P. Ripka, M. Butta, and A. Platil, "Temperature stability of AMR sensors," *Sensor Lett.*, vol. 11, no. 1, pp. 74–77, Jan. 2013.

[13] S. Kaewkamnerd, R. Pongthornseri, J. Chinrungrueng, and T. Silawan, "Automatic vehicle classification using wireless magnetic sensor," in *Proc. IEEE Int. Workshop IDAACS*, Sep. 2009, pp. 420–424.



Bo Yang was born in Sichuan, China, in 1972. She received the B.S. and M.S. degrees from Beihang University, Beijing, China, in 1993 and 1996, respectively, and the Ph.D. degree from the University of Paderborn, Paderborn, Germany, in 2004, all in electrical engineering.

She has been with the School of Automation Science and Electrical Engineering, Beihang University, since 2004, where she is currently an Associate Professor. Her main research interests include intelligent measurement and control.

Yiqun Lei was born in Shaanxi, China, in 1991. She received the B.S. degree in automation science and electric engineering from Beihang University, Beijing, China, in 2013, where she is currently pursuing the M.S. degree at the School of Automation Science and Electrical Engineering. Her research interest includes wireless sensor networks.